CÁC MÔ HÌNH PHI THAM SỐ

Định hướng: Dự đoán và ước lượng bằng cách "để dữ liệu tự nói", không giả định trước dạng hàm

1. Từ mô hình tham số đến mô hình phi tham số

Ba bài trước đã đi một chặng đường rõ ràng. **Perceptron** biến điểm số tuyến tính thành nhãn cứng bằng hàm dấu. **Logistic Regression** làm "mềm" hàm dấu thành hàm sigmoid để có xác suất và một hàm mất mát trơn, lồi. **Softmax Regression** mở rộng từ một xác suất lên K xác suất cộng lại bằng 1, và ở mục 6.3 của bài đó ta còn thấy nó là lớp cuối của mạng nơ-ron. Mỗi bước chỉ thay đổi *một* thành phần của khung mô hình, nhưng có một điều **chưa bao giờ thay đổi**: ta luôn *chọn trước* một công thức có hình dạng cố định cho điểm số, rồi dùng dữ liệu để tìm các con số nằm trong công thức đó.

Bài này đặt câu hỏi ngược lại: **có thể dự đoán mà không cần chọn trước công thức nào không?** Câu trả lời là có, bằng cách "để dữ liệu tự nói". Cách làm này nghe lạ, nhưng nền tảng của nó rất quen thuộc với đời sống: để đoán một người mới sẽ làm gì, ta nhìn những người *giống* họ đã làm gì. Bài này xây dựng bộ công cụ cho ý tưởng đó, gọi là **mô hình phi tham số**.

Trong mục 1, ta trả lời ba câu hỏi:

- Các mô hình đã học (Perceptron, Logistic, Softmax, mạng nơ-ron) có chung cấu trúc nào, và việc **chọn trước dạng hàm** làm ta được gì, mất gì?
- Nếu không chọn dạng hàm, ta có thể dự đoán dựa trên điều gì? Ý tưởng **"dự đoán bằng các mẫu lân cận"** được viết thành công thức ra sao?
- **Mô hình phi tham số** được định nghĩa thế nào, nó khác mô hình tham số ở những điểm nào, và ta phải trả giá gì cho sự linh hoạt đó?

Quy ước ký hiệu

Ta giữ nguyên quy ước của các bài trước: chữ thường in nghiêng là đại lượng vô hướng, **chữ thường in đậm là vector**, **chữ hoa in đậm là ma trận**. Vector đặc trưng là **vector hàng**, các vector đặc trưng của n mẫu xếp chồng thành các hàng của ma trận đặc trưng. Có một điểm khác nhỏ so với bài Softmax: các mô hình phi tham số dựa trên khoảng cách, mà một cột hằng số 1 không làm thay đổi khoảng cách giữa hai mẫu, nên **ta không thêm cột** 1 vào đặc trưng. Khi cần nhắc lại mô hình tham số, ta viết $\tilde{\mathbf{x}}$ và $\tilde{\mathbf{X}}$ cho dạng đã thêm cột 1 như trước.

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Số mẫu, số đặc trưng, số lớp	n, d, K	vô hướng	n quan sát, d đặc trưng, K lớp

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Vector đặc trưng (hàng)	$\mathbf{x}_i$	$1 \times d$	Đặc trưng của mẫu huấn luyện thứ i (không có cột 1)
Ma trận đặc trưng	$\mathbf{X}$	$n \times d$	Hàng thứ i của $\mathbf{X}$ là $\mathbf{x}_i$
Dạng đã thêm cột 1	$\tilde{\mathbf{x}}_i$, $\tilde{\mathbf{X}}$	$1 \times (d+1)$, $n \times (d+1)$	Dùng cho mô hình tham số (bài Softmax)
Điểm truy vấn	$\mathbf{x}_0$	$1 \times d$	Mẫu mới cần dự đoán
Ma trận one-hot	$\mathbf{Y}$	$n \times K$	$y_{ik} = 1$ nếu mẫu i thuộc lớp k, ngược lại bằng 0 (như bài Softmax)
Tham số của mô hình	$\boldsymbol{\theta}$	$p \times 1$	Vector gồm mọi tham số cần học (với Softmax, $\boldsymbol{\theta}$ gồm các phần tử của $\mathbf{W}$)
Khoảng cách	$d(\mathbf{x}_0, \mathbf{x}_i)$	vô hướng	Độ "xa" giữa điểm truy vấn và mẫu thứ i
Tập láng giềng	$\mathcal{N}_k(\mathbf{x}_0)$	tập chỉ số	Chỉ số của k mẫu huấn luyện gần $\mathbf{x}_0$ nhất
Ma trận láng giềng	$\mathbf{N}$	$m \times n$	Dùng khi có m điểm truy vấn (mục 1.2)

Hai ma trận quen thuộc từ bài Softmax vẫn giữ vai trò cũ: $\mathbf{Y}$ là "xác suất mục tiêu" và đầu ra của mô hình vẫn là một ma trận xác suất có K cột, mỗi hàng là một phân phối trên K lớp. Chỉ có **cách tạo ra ma trận xác suất đó** là thay đổi.

1.1. Hạn chế của các mô hình trước: luôn phải chọn trước dạng hàm

Một cấu trúc chung: $f(\mathbf{x}; \boldsymbol{\theta})$

Đặt các mô hình đã học cạnh nhau, ta thấy chúng chỉ khác nhau ở "công thức" và "cách học":

Mô hình	Dạng hàm được chọn trước	Cái cần học	Cách học
Hồi quy tuyến tính	$\hat{y} = \tilde{\mathbf{x}}\mathbf{w}$	$\mathbf{w}$	Cực tiểu sai số bình phương
Perceptron	$\hat{y} = \text{sign}(\tilde{\mathbf{x}}\mathbf{w})$	$\mathbf{w}$	Cập nhật theo mẫu bị sai
Logistic Regression	$p = \sigma(\tilde{\mathbf{x}}\mathbf{w})$	$\mathbf{w}$	Cực tiểu cross-entropy nhị phân
Softmax Regression	$\mathbf{P} = \text{softmax}(\tilde{\mathbf{X}}\mathbf{W})$	$\mathbf{W}$	Cực tiểu cross-entropy nhiều lớp
Mạng nơ-ron một lớp ẩn	$\mathbf{P} = \text{softmax}(\tilde{\mathbf{H}}\mathbf{W}_2)$, $\mathbf{H} = \tanh(\tilde{\mathbf{X}}\mathbf{W}_1)$	$\mathbf{W}_1, \mathbf{W}_2$	Lan truyền ngược

Cả năm dòng cùng một khuôn. Gọi $\boldsymbol{\theta}$ là vector gom *mọi* con số cần học (với Softmax, đó là $(d+1)K$ phần tử của $\mathbf{W}$). Mô hình có dạng

$$\hat{\mathbf{p}}(\mathbf{x}) = f(\mathbf{x}; \boldsymbol{\theta}), \qquad \boldsymbol{\theta} \in \mathbb{R}^p,$$

và việc học là tìm $\boldsymbol{\theta}$ làm hàm mất mát nhỏ nhất:

$$\widehat{\boldsymbol{\theta}} = \operatorname*{argmin}_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \qquad J(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \ell\left(\mathbf{y}_i, f(\mathbf{x}_i; \boldsymbol{\theta})\right),$$

trong đó ℓ là mất mát của một mẫu (cross-entropy ở bài trước). Khuôn này có ba đặc điểm, và chúng giải thích cả điểm mạnh lẫn điểm yếu của các mô hình tham số.

- **Dạng hàm do con người chọn.** Việc f là "tuyến tính rồi qua softmax" là quyết định của ta *trước khi nhìn dữ liệu*. Dữ liệu chỉ quyết định các con số $\widehat{\boldsymbol{\theta}}$.
- **Số tham số p cố định, không phụ thuộc n.** Với Softmax, $p = (d+1)K$. Có 100 hay 100 000 mẫu huấn luyện thì p vẫn vậy. Quá trình huấn luyện **nén** toàn bộ dữ liệu vào p con số; sau đó ta có thể vứt dữ liệu đi mà vẫn dự đoán được, chỉ cần $\widehat{\boldsymbol{\theta}}$.
- **Học là giải một bài toán tối ưu.** Ta cần hàm mất mát, gradient, tốc độ học, điều kiện dừng (mục 3 của bài Softmax).

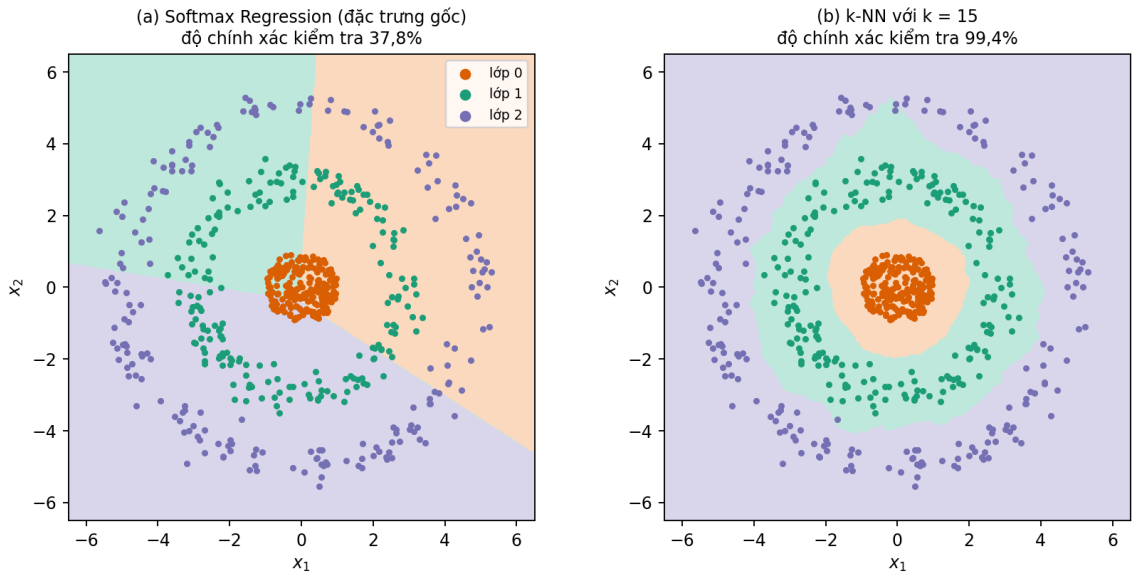
Cái được: vì sao các mô hình này vẫn rất hữu ích

Ba đặc điểm trên mang lại những lợi ích đã thấy ở mục 6.1 của bài Softmax: dự đoán **nhanh** (chỉ cần $O(dK)$ phép tính cho một mẫu, không phụ thuộc n), **gọn** (mô hình là một ma trận **W**), **diễn giải được** (tỉ lệ odds giữa hai lớp), và với Softmax còn có bài toán tối ưu **lồi** nên huấn luyện ổn định. Khi dạng hàm chọn đúng, một mô hình tham số cần rất ít dữ liệu để học tốt. Vì vậy cần nhớ rằng bài này không nhằm chứng minh mô hình tham số "kém"; ta đang xem xét một **đánh đổi**.

Cái giá: thiên lệch do chọn sai dạng hàm

Cái giá xuất hiện khi dạng hàm không khớp với thực tế. Ở mục 4.5 của bài Softmax, ta đã thấy vùng quyết định của Softmax Regression luôn **lồi**, ranh giới giữa hai lớp luôn là đường thẳng (siêu phẳng khi $d > 2$). Mô hình không thể phân loại đúng khi một lớp có dạng vòng nhẫn, lưỡi liềm, hay gồm hai mảnh tách rời.

Ta minh họa lại bằng một thí nghiệm mô phỏng cùng kiểu: 600 điểm trong mặt phẳng, 200 điểm mỗi lớp. Lớp 0 là hình tròn đặc bán kính 1 ở giữa, lớp 1 và lớp 2 là hai vòng tròn bán kính 3 và 5 (nhiễu Gauss với độ lệch chuẩn 0,3). Ta chia 70% huấn luyện, 30% kiểm tra và chuẩn hóa đặc trưng như các bài trước. Softmax Regression trên hai đặc trưng gốc x_1, x_2 chỉ đạt 43,3% **trên tập huấn luyện và** 37,8% **trên tập kiểm tra**, chỉ hơn mức đoán mò (33,3%) một chút (Hình 1a). Hai con số này **đều thấp**, nghĩa là lỗi không đến từ việc huấn luyện chưa đủ hay dữ liệu ít, mà đến từ chính dạng hàm: dù có thêm bao nhiêu mẫu, các đường biên vẫn là đường thẳng và không thể ôm lấy các vòng tròn. Đây là **thiên lệch** (bias) do mô hình không đủ "hình dạng".



Hình 1. Ba vòng tròn đồng tâm: (a) Softmax Regression trên đặc trưng gốc chỉ cho các vùng bị chia bằng đường thẳng, độ chính xác kiểm tra 37,8%. (b) k-NN với $k = 15$, một cách làm không giả định dạng hàm (mục 1.2), uốn theo hình dạng của dữ liệu, độ chính xác kiểm tra 99,4%.

Bài Softmax đã đưa ra hai cách chữa: thêm **đặc trưng đa thức** (mục 4.5 của bài đó) hoặc dùng **mạng nơ-ron** để học đặc trưng (mục 6.3 của bài đó). Cả hai đều hiệu quả, nhưng cần nhận ra chúng chỉ *dời* quyết định đi chỗ khác chứ không loại bỏ nó:

- Với đặc trưng đa thức, ta phải đoán **bậc** và **loại** đặc trưng. Ba vòng tròn đồng tâm cần bậc hai vì ta *biết* chúng là vòng tròn. Nếu dữ liệu có hình dạng ta không nhìn thấy được thì khó đoán.
- Với mạng nơ-ron, ta phải chọn **số lớp ẩn, số nơ-ron m, hàm kích hoạt**. Đó vẫn là chọn trước một *họ* hàm.

Vấn đề nặng hơn khi số chiều lớn. Ảnh chữ số viết tay 8×8 có $d = 64$ đặc trưng, ta không thể vẽ dữ liệu ra để nhìn hình dạng các lớp rồi mới chọn dạng hàm. Chọn sai thì chất lượng giảm, mà ta khó biết mình đã chọn sai ở đâu.

Câu hỏi dẫn đường cho phần còn lại

Hình 1b gợi ý một hướng khác: nhìn một điểm mới trên Hình 1, ta không cần phương trình nào mà vẫn đoán được lớp của nó, chỉ cần **nhìn những điểm đã biết nằm gần nó**. Nếu xung quanh toàn điểm lớp 1 thì điểm mới nhiều khả năng cũng thuộc lớp 1. Mục 1.2 biến trực giác này thành một mô hình hoàn chỉnh.

1.2. Ý tưởng: dự đoán dựa trên các mẫu lân cận, "học" là ghi nhớ dữ liệu

Trực giác: hỏi những người giống mình

Giả sử một sinh viên mới học 8 giờ mỗi tuần và vắng 2 buổi, và ta muốn đoán kết quả học tập của bạn ấy. Cách "tham số" là ước lượng một bộ trọng số **W** từ toàn bộ dữ liệu rồi tính điểm số $\tilde{\mathbf{x}}\mathbf{W}$. Cách "lân cận" đơn giản hơn nhiều: **tìm vài sinh viên trước đây có hồ sơ giống bạn ấy nhất, xem họ đã đạt kết quả gì**. Bác sĩ chẩn đoán dựa trên các ca bệnh tương tự, người mua nhà định giá dựa trên các căn tương tự trong khu phố: đó đều là suy luận từ các trường hợp gần giống.

Giả thiết duy nhất: "gần thì giống"

Cách làm này chỉ dựa trên một giả thiết, yếu hơn nhiều so với việc chọn công thức: nếu hai đầu vào gần nhau thì phân phối đầu ra của chúng gần nhau,

$$\mathbf{x}' \text{ gần } \mathbf{x} \;\Rightarrow\; P(y = k \mid \mathbf{x}') \approx P(y = k \mid \mathbf{x}), \qquad k = 0,1,\dots,K-1.$$

Giả thiết này **không nói** xác suất phụ thuộc vào **x** theo dạng nào (tuyến tính, đa thức hay gì khác), chỉ nói nó *thay đổi từ từ*. Để dùng nó ta cần hai thứ: một cách đo "gần" và một cách tận dụng các điểm gần. Cách đo thông dụng nhất là khoảng cách Euclid:

$$d(\mathbf{x}_0, \mathbf{x}_i) = \| \mathbf{x}_0 - \mathbf{x}_i \|_2 = \sqrt{\sum_{j=1}^{d} (x_{0j} - x_{ij})^2}.$$

(Ở mục 2.1 ta sẽ xét thêm Manhattan và Minkowski, và mục 2.5 giải thích vì sao phải chuẩn hóa đặc trưng trước khi đo khoảng cách.)

Từ giả thiết đến công thức: tần suất trong lân cận

Cách tận dụng các điểm gần là **đếm**. Quanh $\mathbf{x}_0$, vẽ một quả cầu bán kính r và gọi $\mathcal{I}_r$ là tập chỉ số các mẫu huấn luyện nằm trong quả cầu đó. Vì trong quả cầu nhỏ, xác suất gần như không đổi (theo giả thiết trên), ta ước lượng xác suất của lớp k bằng **tỉ lệ các mẫu lớp k trong quả cầu**:

$$\hat{P}(y = k \mid \mathbf{x}_0) = \frac{1}{|\mathcal{I}_r|} \sum_{i \in \mathcal{I}_r} y_{ik}.$$

Nhờ mã hóa one-hot, tổng $\sum_i y_{ik}$ chính là số mẫu lớp k trong lân cận. Có một bất tiện: số mẫu rơi vào quả cầu thay đổi tùy vùng (vùng thưa có thể rỗng). Cách khắc phục tự nhiên là **cố định số mẫu thay vì cố định bán kính**: luôn lấy đúng k mẫu gần $\mathbf{x}_0$ nhất. Gọi $\mathcal{N}_k(\mathbf{x}_0)$ là tập chỉ số của k mẫu đó, ta được mô hình k **láng giềng gần nhất (k-NN)**:

$$\hat{p}_k(\mathbf{x}_0) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x}_0)} y_{ik}, \qquad \hat{y}_0 = \operatorname*{argmax}_k \hat{p}_k(\mathbf{x}_0).$$

Quy tắc chọn lớp argmax giống hệt bài Softmax; chỉ khác ở chỗ xác suất không đến từ công thức softmax mà từ phép đếm.

Dạng ma trận: so sánh với Softmax

Giả sử có m điểm truy vấn xếp thành các hàng của ma trận $\mathbf{X}_0$ kích thước $m \times d$. Với mỗi điểm, ta đánh dấu k láng giềng của nó trong một ma trận $\mathbf{N}$ kích thước $m \times n$, với $N_{ji} = 1$ nếu mẫu huấn luyện i thuộc $\mathcal{N}_k$ của điểm truy vấn j, ngược lại bằng 0 (mỗi hàng có đúng k số 1). Khi đó toàn bộ ma trận xác suất dự đoán là

$$\widehat{\mathbf{P}} = \frac{1}{k}\,\mathbf{NY}, \qquad \widehat{\mathbf{P}} \in \mathbb{R}^{m \times K}.$$

Công thức này đáng đặt cạnh mô hình Softmax:

$$\text{Softmax:}\quad \mathbf{P} = \operatorname{softmax}\big(\widetilde{\mathbf{X}}\mathbf{W}\big), \qquad \text{k-NN:}\quad \widehat{\mathbf{P}} = 1/k\ \mathbf{NY}.$$

Cả hai đều là "một phép nhân ma trận rồi chuẩn hóa thành xác suất". Khác biệt cốt lõi: Softmax nhân dữ liệu với một ma trận **W được học**; k-NN nhân một ma trận chọn lọc **N** với **chính nhãn dữ liệu Y**. Không có **W** nào cần tìm. Mỗi hàng của $\widehat{\mathbf{P}}$ không âm và có tổng bằng 1 (vì mỗi hàng của **N** có k số 1 và mỗi hàng của **Y** có tổng 1), nên đây là một phân phối xác suất thật sự, giống Softmax.

Ví dụ: dự đoán cho một sinh viên mới

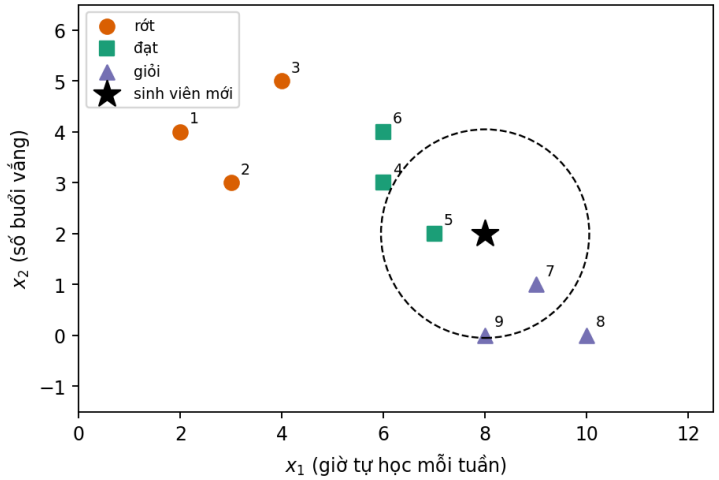
Ta dùng lại 9 sinh viên ở ví dụ xuyên suốt của bài Softmax (rớt $= 0$, đạt $= 1$, giỏi $= 2$) và dự đoán cho sinh viên mới $\mathbf{x}_0 = [8,2]$ (học 8 giờ, vắng 2 buổi). Bước đầu tiên là tính khoảng cách tới mọi sinh viên:

Sinh viên	(x_1, x_2)	Kết quả	$d(\mathbf{x}_0, \mathbf{x}_i)$	Hạng
1	(2,4)	rớt	$\sqrt{40} \approx 6{,}325$	9
2	(3,3)	rớt	$\sqrt{26} \approx 5{,}099$	8
3	(4,5)	rớt	$\sqrt{25} = 5{,}000$	7
4	(6,3)	đạt	$\sqrt{5} \approx 2{,}236$	4
5	(7,2)	đạt	$\sqrt{1} = 1{,}000$	**1**
6	(6,4)	đạt	$\sqrt{8} \approx 2{,}828$	5 (hòa)
7	(9,1)	giỏi	$\sqrt{2} \approx 1{,}414$	**2**
8	(10,0)	giỏi	$\sqrt{8} \approx 2{,}828$	5 (hòa)
9	(8,0)	giỏi	$\sqrt{4} = 2{,}000$	**3**

Với $k = 3$, ba láng giềng gần nhất là sinh viên 5 (đạt), 7 (giỏi) và 9 (giỏi) (Hình 2). Hàng tương ứng của **N** có số 1 ở vị trí 5,7,9:

$$\mathbf{n}_0 = [\,0\ \ 0\ \ 0\ \ 0\ \ 1\ \ 0\ \ 1\ \ 0\ \ 1\,], \qquad \widehat{\mathbf{p}}_0 = \frac{1}{3}\,\mathbf{n}_0\mathbf{Y} = \frac{1}{3}\,[\,0\ \ 1\ \ 2\,] = [\,0\ \ \ 1/3\ \ \ 2/3\,].$$

Mô hình dự đoán **giỏi** với xác suất $2/3$, đạt với xác suất $1/3$ và không có khả năng rớt, một kết quả hợp lý vì sinh viên mới học nhiều và gần như không vắng. Nếu lấy $k = 1$ thì chỉ nhìn sinh viên 5 và dự đoán **đạt**. Vậy **kết quả phụ thuộc vào** k, và việc chọn k sẽ là chủ đề của mục 2.4.



Hình 2. Sinh viên mới (ngôi sao) tại $(8,2)$ *và* 9 *sinh viên cũ. Vòng tròn nét đứt bao ba láng giềng gần nhất (sinh viên 5, 7 và 9).*

"Học" nghĩa là ghi nhớ

Hãy so sánh với những gì Softmax cần làm trước khi dự đoán được một sinh viên: khởi tạo $\mathbf{W}$, tính gradient, cập nhật hàng trăm bước, chọn tốc độ học, chuẩn hóa đặc trưng, theo dõi mất mát. Ở k-NN, **giai đoạn huấn luyện chỉ là lưu** $(\mathbf{X}, \mathbf{Y})$. Không có hàm mất mát để cực tiểu, không có gradient, không có tham số nào được cập nhật. Toàn bộ công việc dồn sang lúc dự đoán: mỗi lần có điểm truy vấn mới, ta phải tính khoảng cách tới các mẫu đã lưu và chọn ra láng giềng. Vì thế k-NN còn được gọi là **học lười** (lazy learning) hay **học dựa trên ví dụ** (instance-based learning): mô hình *là* dữ liệu.

Hai lưu ý để không hiểu nhầm

Thứ nhất, "không chọn dạng hàm" không có nghĩa là "không có giả định". Giả thiết "gần thì giống" là một giả thiết thật, và nó chỉ đúng khi ta đo khoảng cách hợp lý: nếu một đặc trưng có thang đo lớn gấp nghìn lần các đặc trưng khác thì nó sẽ lấn át khoảng cách, và nếu có nhiều đặc trưng vô nghĩa thì "gần" cũng mất ý nghĩa. Hai điểm này được phân tích ở mục 2.5 và 6.3.

Thứ hai, hai con số được ta chọn bằng tay là cách đo khoảng cách và kích thước lân cận k. Chúng không phải là tham số *học từ dữ liệu* theo nghĩa của $\boldsymbol{\theta}$, mà là **siêu tham số** điều khiển mức "mượt" của mô hình, và ta chọn chúng bằng kiểm định chéo như đã làm với tốc độ học và hệ số điều chuẩn λ.

1.3. Mô hình phi tham số là gì: độ phức tạp tăng theo n, không có bước tối ưu

Định nghĩa

Từ hai mục trước, ta rút ra cách phân biệt:

- **Mô hình tham số**: tồn tại một vector $\boldsymbol{\theta} \in \mathbb{R}^p$ với p **cố định**, được chọn trước và **không phụ thuộc** n, sao cho mô hình hoàn toàn được xác định bởi $\boldsymbol{\theta}$. Softmax Regression có $p = (d+1)K$; mạng nơ-ron một lớp ẩn có $p = (d+1)m + (m+1)K$.
- **Mô hình phi tham số**: không giả định dạng hàm gồm hữu hạn tham số cố định. Lượng thông tin mô hình cần lưu để dự đoán **tăng theo** n, nên mô hình có thể trở nên phức tạp tùy ý khi có thêm dữ liệu. Với k-NN, mô hình là toàn bộ n mẫu đã lưu.

Chữ "phi tham số" dễ gây hiểu nhầm. Nó **không** có nghĩa là mô hình không có con số nào cần chọn: ta vẫn có k, băng thông h (mục 3 và 4), độ đo khoảng cách. Chính xác hơn, đó là **các siêu tham số điều khiển độ mượt**, còn số "tham số học được" không bị đóng khung trước.

Dạng chung: trung bình có trọng số của dữ liệu

Các phương pháp trong bài này có chung một dạng. Gọi $\mathbf{y}_i$ là hàng thứ i của $\mathbf{Y}$ (với bài toán hồi quy, y_i là một giá trị thực), thì dự đoán tại điểm truy vấn $\mathbf{x}_0$ là **trung bình có trọng số** của các giá trị đã biết:

$$\hat{\mathbf{p}}(\mathbf{x}_0) = \sum_{i=1}^{n} w_i(\mathbf{x}_0)\, \mathbf{y}_i, \qquad w_i(\mathbf{x}_0) \geq 0, \qquad \sum_{i=1}^{n} w_i(\mathbf{x}_0) = 1.$$

Trọng số $w_i(\mathbf{x}_0)$ cho biết mẫu i "được nghe" nhiều hay ít khi dự đoán tại $\mathbf{x}_0$, và nó phụ thuộc vào khoảng cách giữa $\mathbf{x}_i$ và $\mathbf{x}_0$. Các phương pháp ở những mục sau chỉ khác nhau ở cách đặt trọng số này:

Phương pháp	**Trọng số $w_i(\mathbf{x}_0)$**	**Mục**
k-NN (bỏ phiếu đa số)	$\frac{1}{k}$ nếu $i \in \mathcal{N}_k(\mathbf{x}_0)$, ngược lại 0	2
k-NN có trọng số khoảng cách	giảm dần khi $d(\mathbf{x}_0, \mathbf{x}_i)$ tăng, chỉ xét k mẫu gần nhất	2.2
Nadaraya–Watson	tỉ lệ với hàm nhân $\kappa\left(\frac{\mathbf{x}_0 - \mathbf{x}_i}{h}\right)$, chuẩn hóa để tổng bằng 1	3.2

Cùng tinh thần đó, ở mục 4 ta sẽ ước lượng một hàm mật độ bằng cách đặt một "cái gò" nhỏ lên mỗi mẫu rồi lấy trung bình; mục 5 dùng các hàm mật độ này để phân loại.

So sánh trực tiếp hai họ mô hình

Tiêu chí	Mô hình tham số (ví dụ Softmax)	Mô hình phi tham số (ví dụ k-NN)
Giả định về dạng hàm	Chọn trước (log-odds tuyến tính theo đặc trưng)	Không chọn; chỉ giả định "gần thì giống" và một độ đo khoảng cách
Cái mô hình lưu	$\widehat{\mathbf{W}}$, gồm $(d+1)K$ số	Toàn bộ dữ liệu $(\mathbf{X}, \mathbf{Y})$, cỡ $O(nd)$
Khi n tăng	Kích thước mô hình **không đổi**	Kích thước mô hình **tăng tuyến tính** theo n
Huấn luyện	Giải bài toán tối ưu, lặp nhiều epoch, mỗi epoch $O(ndK)$	**Không có bước tối ưu**; chỉ lưu dữ liệu
Dự đoán một mẫu	$O(dK)$, rất nhanh	$O(nd)$ nếu tính khoảng cách tới mọi mẫu (mục 2.7)
Thiên lệch và phương sai	Thiên lệch cao nếu dạng hàm sai; phương sai thấp	Thiên lệch thấp; phương sai cao khi dữ liệu ít
Số chiều d lớn	Khá ổn định	Khoảng cách mất ý nghĩa (lời nguyền chiều)
Diễn giải	Trọng số, tỉ lệ odds	Chỉ giải thích bằng các ví dụ láng giềng
"Nút chỉnh" độ phức tạp	λ, bậc đa thức, số nơ-ron m	k, băng thông h

Ví dụ bằng số cho dòng thứ ba: với ảnh chữ số viết tay 8×8 ($d = 64$, $K = 10$), Softmax chỉ cần $(64 + 1) \times 10 = 650$ tham số dù dữ liệu lớn hay nhỏ. Còn k-NN trên bộ 1 797 ảnh phải giữ $1\,797 \times 64 = 115\,008$ giá trị đặc trưng, và nếu số ảnh tăng gấp đôi thì lượng cần lưu cũng gấp đôi (230 016 giá trị).

Đánh đổi: linh hoạt đi kèm với phương sai

Sự linh hoạt của mô hình phi tham số có cái giá. Khi dữ liệu là một mẫu ngẫu nhiên hữu hạn, sai số kiểm tra gồm ba phần (sẽ được phân tích kỹ ở mục 6.1):

$$\text{sai số kiểm tra} = \text{thiên lệch}^2 + \text{phương sai} + \text{nhiễu}.$$

Mô hình tham số đơn giản (như Softmax trên đặc trưng gốc) có **thiên lệch cao, phương sai thấp**: nó "cứng", ít bị lung lay bởi từng mẫu nhưng không bám được hình dạng phức tạp. Mô hình phi tham số có **thiên lệch thấp, phương sai cao**: nó bám sát dữ liệu nên uốn theo mọi hình dạng, nhưng cũng bám cả nhiễu. Ví dụ, với $k = 1$ một mẫu bị gán nhãn sai sẽ kéo cả vùng quanh nó sang lớp sai; với k lớn thì các chi tiết nhỏ bị làm mờ. Siêu tham số k (hay h) chính là nút điều chỉnh giữa hai thái cực, giống vai trò của λ trong điều chuẩn hóa L2 ở bài Softmax.

Còn một cái giá nữa là **lời nguyền chiều**. Để "nghe" được các mẫu lân cận thì phải *có* mẫu nằm gần. Chia mỗi trục của hình lập phương đơn vị $[0,1]^d$ thành các đoạn 0,1, ta được số ô cần có ít nhất một mẫu:

$$\#\text{ô} = 10^d: \qquad d = 2 \Rightarrow 10^2, \qquad d = 10 \Rightarrow 10^{10}, \qquad d = 64 \Rightarrow 10^{64}.$$

Số mẫu cần thiết tăng theo cấp số nhân với d. Vì vậy mô hình phi tham số hoạt động tốt khi số đặc trưng vừa phải hoặc khi dữ liệu thực chất nằm trong không gian có chiều thấp; mục 2.5 và 6.3 sẽ làm rõ hơn điều này.

Lộ trình của bài

Bài này đi theo mạch "từ dự đoán đến mật độ rồi quay lại dự đoán":

Mục	Nội dung	Câu hỏi trả lời
2	k-NN cho phân loại	Dùng láng giềng để chọn lớp thế nào? Chọn k ra sao?
3	k-NN hồi quy, Nadaraya–Watson, LOESS	Cùng ý tưởng cho biến mục tiêu liên tục?
4	Ước lượng mật độ (histogram, KDE, k láng giềng)	Ước lượng được cả hàm mật độ $p(\mathbf{x})$ mà không giả định dạng không?
5	Bayes ngây thơ	Dùng mật độ để phân loại: mô hình **sinh** so với mô hình **phân biệt** như Softmax
6	Thiên lệch, phương sai, lời nguyền chiều, chi phí	Vì sao các mô hình này hoạt động và khi nào thất bại?
7	Đánh giá và so sánh trên chữ số viết tay	Chúng so với Logistic và Softmax ra sao?
8	Tổng kết và mở rộng	Chọn họ mô hình nào cho bài toán nào?

Mục 5 cũng cho thấy rằng ranh giới "tham số" và "phi tham số" không hoàn toàn tách bạch: Bayes ngây thơ với phân phối Gauss là mô hình tham số, còn khi thay phân phối Gauss bằng ước lượng mật độ nhân thì thành mô hình phi tham số.

Mối liên hệ với các bài trước

Mạch các bài trước đi theo cách mỗi bài thay một thành phần của khung mô hình. Bài này thay đổi một thành phần ở tầng sâu hơn: **bỏ việc chọn trước dạng hàm**.

Perceptron $\xrightarrow{\text{làm mềm hàm dấu}}$ Logistic $\xrightarrow{\text{một điểm số}\to K\text{ điểm số}}$ Softmax $\xrightarrow{\text{thay dạng hàm bằng dữ liệu}}$ Phi tham số

Phần **giữ nguyên**: vẫn là bài toán ước lượng $P(y = k \mid \mathbf{x})$ với $\sum_k p_k = 1$; vẫn dùng ma trận one-hot $\mathbf{Y}$ làm "xác suất mục tiêu"; vẫn chọn lớp bằng argmax; vẫn đánh giá bằng accuracy, ma trận nhầm lẫn, log-loss (mục 7). Phần **thay đổi**: $\mathbf{P}$ không còn là $\text{softmax}(\tilde{\mathbf{X}}\mathbf{W})$ với $\mathbf{W}$ học bằng Gradient Descent, mà là kết quả của việc **nhìn vào các mẫu lân cận**. Ở chiều ngược lại, những khái niệm ta đã học về thiên lệch, phương sai, điều chuẩn hóa và kiểm định chéo vẫn cần thiết, chỉ đổi cái được điều chỉnh từ λ sang k và h.

Tóm tắt mục 1

- Các mô hình đã học (hồi quy tuyến tính, Perceptron, Logistic, Softmax, mạng nơ-ron) đều có dạng $f(\mathbf{x}; \boldsymbol{\theta})$ với $\boldsymbol{\theta} \in \mathbb{R}^p$, trong đó dạng hàm f được **chọn trước** và p **cố định**. Học là giải bài toán $\widehat{\boldsymbol{\theta}} = \text{argmin} J(\boldsymbol{\theta})$.
- Ưu điểm: dự đoán nhanh, mô hình gọn, dễ diễn giải, ít dữ liệu vẫn học được **nếu dạng hàm đúng**. Nhược điểm: nếu dạng hàm sai thì có **thiên lệch** mà thêm dữ liệu không sửa được. Trên ba vòng tròn đồng tâm, Softmax chỉ đạt 37,8% trên tập kiểm tra.
- Ý tưởng thay thế: dựa trên giả thiết "gần thì giống", ước lượng $\hat{P}(y = k \mid \mathbf{x}_0)$ bằng **tỉ lệ các mẫu lớp** k **trong lân cận** của $\mathbf{x}_0$. Với k láng giềng, $\widehat{\mathbf{P}} = \frac{1}{k}\mathbf{NY}$; trên cùng bộ dữ liệu ba vòng tròn, k-NN đạt 99,4%.
- Với mô hình phi tham số, "học" là **ghi nhớ** $(\mathbf{X}, \mathbf{Y})$; không có hàm mất mát, không có gradient, mọi tính toán nằm ở lúc dự đoán.
- **Phi tham số** nghĩa là kích thước mô hình **tăng theo** n và dạng hàm không bị khống chế; nó không có nghĩa là không còn siêu tham số (k, h, độ đo khoảng cách). Dạng chung là trung bình có trọng số $\hat{\mathbf{p}}(\mathbf{x}_0) = \sum_i w_i(\mathbf{x}_0)\mathbf{y}_i$.
- Đánh đổi: phi tham số có thiên lệch thấp nhưng phương sai cao, tốn bộ nhớ và thời gian dự đoán, và chịu **lời nguyền chiều**. Mô hình tham số ngược lại.

Câu hỏi kiểm tra nhanh

1. Một mô hình Softmax Regression có $d = 20$ đặc trưng và $K = 5$ lớp. Có bao nhiêu tham số cần học? Số tham số thay đổi thế nào khi số mẫu huấn luyện tăng từ 1 000 lên 100 000?
2. Vì sao việc thêm đặc trưng đa thức hay dùng mạng nơ-ron vẫn chưa thoát hẳn khỏi việc "chọn trước dạng hàm"?
3. Cho 5 mẫu một chiều $x = 1,2,4,7,8$ với nhãn lần lượt $0,0,1,1,1$. Dùng k-NN với $k = 3$ và khoảng cách Euclid, hãy tính $\hat{\mathbf{p}}$ và dự đoán lớp cho điểm truy vấn $x_0 = 3$.
4. "Mô hình phi tham số là mô hình không có tham số nào cần chọn." Nhận định này đúng hay sai? Giải thích.
5. Vì sao k-NN gần như không tốn thời gian huấn luyện nhưng dự đoán chậm, trong khi Softmax thì ngược lại?

Gợi ý đáp án

1. Số tham số là $(d+1)K = 21 \times 5 = 105$. Khi n tăng từ $1\,000$ lên $100\,000$, số tham số **không đổi** (105); chỉ thời gian huấn luyện tăng. Đây là đặc điểm "p cố định" của mô hình tham số.

2. Vì ta vẫn phải chọn trước một *họ* hàm. Với đặc trưng đa thức, ta chọn bậc và loại đặc trưng; với mạng nơ-ron, ta chọn kiến trúc (số lớp ẩn, số nơ-ron, hàm kích hoạt). Hai cách này làm họ hàm rộng hơn, nhưng nếu dữ liệu nằm ngoài họ hàm đã chọn thì thiên lệch vẫn còn, và số tham số vẫn cố định khi n tăng.

3. Khoảng cách từ $x_0 = 3$ tới các mẫu là 2,1,1,4,5. Ba mẫu gần nhất là $x = 2$ (nhãn 0), $x = 4$ (nhãn 1) và $x = 1$ (nhãn 0). Hàng của **N** là $[1 \quad 1 \quad 1 \quad 0 \quad 0]$, nên số mẫu từng lớp là $(2,1)$ và $\hat{\mathbf{p}} = (2/3\,,\,1/3)$. Dự đoán là **lớp** 0.

4. **Sai.** Mô hình phi tham số vẫn có siêu tham số như k, băng thông h và độ đo khoảng cách. "Phi tham số" nghĩa là dạng hàm không bị gói trong một vector $\boldsymbol{\theta}$ có số chiều cố định, và kích thước mô hình tăng theo n.

5. k-NN chỉ lưu dữ liệu khi huấn luyện, nhưng mỗi lần dự đoán phải tính khoảng cách tới n mẫu, tốn $O(nd)$. Softmax tốn nhiều công sức huấn luyện (tối ưu **W**), nhưng sau đó dự đoán chỉ cần nhân với ma trận **W**, tốn $O(dK)$ cho mỗi mẫu.

2. k láng giềng gần nhất (k-NN) cho phân loại

Ở mục 1, ta đã đi từ một nhận xét về mô hình tham số (dạng hàm chọn trước, số tham số cố định) đến một ý tưởng khác hẳn: **dự đoán bằng cách nhìn các mẫu lân cận**. Ta đã viết được công thức cốt lõi của mô hình k láng giềng gần nhất, đã tính tay cho một sinh viên mới, và thấy k-NN đạt 99,4% trên bộ ba vòng tròn đồng tâm, nơi Softmax chỉ đạt 37,8%. Nhưng mục 1 mới chỉ phác thảo. Để dùng k-NN một cách nghiêm túc, còn nhiều chi tiết bị bỏ ngỏ: "gần" cụ thể là gần theo thước đo nào, các láng giềng "bỏ phiếu" ra sao, đường biên quyết định trông thế nào, chọn k thế nào, và vì sao một mô hình đơn giản như vậy vẫn có thể thất bại.

Trong mục 2, ta lần lượt trả lời các câu hỏi sau:

- **"Gần" được đo bằng gì?** Các độ đo khoảng cách Euclid, Manhattan, Minkowski, và cách tính khoảng cách từ mọi điểm truy vấn tới mọi mẫu huấn luyện chỉ bằng vài phép nhân ma trận (mục 2.1).
- **Các láng giềng quyết định lớp như thế nào?** Bỏ phiếu đa số, bỏ phiếu theo trọng số khoảng cách, và cách đọc kết quả thành ma trận xác suất $\widehat{\mathbf{P}}$ (mục 2.2).
- **Đường biên quyết định có hình dạng gì, và k ảnh hưởng ra sao?** (mục 2.3), **chọn k bằng cách nào?** (mục 2.4)
- **Vì sao phải chuẩn hóa đặc trưng, và điều gì xảy ra khi số chiều lớn?** (mục 2.5)
- Cuối cùng, ta tính tay một ví dụ đầy đủ (mục 2.6) rồi cài đặt k-NN từ đầu bằng Python và bàn về chi phí tính toán (mục 2.7).

Quy ước ký hiệu

Ta giữ các quy ước của mục 1: chữ thường in nghiêng là đại lượng vô hướng, **chữ thường in đậm là vector**, **chữ hoa in đậm là ma trận**, vector đặc trưng là vector hàng và **không có cột** 1. Ngoài các ký hiệu đã dùng (n, d, K, $\mathbf{X}$, $\mathbf{Y}$, $\mathbf{x}_0$, $\mathcal{N}_k(\mathbf{x}_0)$, $\mathbf{N}$), mục này dùng thêm:

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Số điểm truy vấn	m	vô hướng	Số mẫu cần dự đoán cùng lúc
Ma trận truy vấn	$\mathbf{X}_0$	$m \times d$	Hàng thứ j là điểm truy vấn $\mathbf{x}_{0,j}$
Bậc Minkowski	q	vô hướng	$q = 1$: Manhattan; $q = 2$: Euclid
Ma trận khoảng cách	$\mathbf{D}$	$m \times n$	$D_{ji} = d(\mathbf{x}_{0,j}, \mathbf{x}_i)$
Ma trận bình phương khoảng cách	$\mathbf{D}^{(2)}$	$m \times n$	$D_{ji}^{(2)} = D_{ji}^2$ (bình phương từng phần tử)
Vector bình phương chuẩn	$\mathbf{a}$, $\mathbf{b}$	$m \times 1$, $n \times 1$	$a_j = \| \mathbf{x}_{0,j} \|^2$, $b_i = \| \mathbf{x}_i \|^2$

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Vector gồm toàn số 1	$\mathbf{1}_n$	$n \times 1$	Dùng để lặp một vector thành ma trận
Ma trận láng giềng có trọng số	$\mathbf{N}_w$	$m \times n$	Mỗi hàng có k phần tử khác 0, tổng bằng 1
Số phần chia của kiểm định chéo	V	vô hướng	Dùng chữ V để không trùng với K (số lớp)
Trung bình và độ lệch chuẩn đặc trưng	$\boldsymbol{\mu}$, $\boldsymbol{\sigma}$	$1 \times d$	Thống kê của tập huấn luyện (mục 2.5)

2.1. Ý tưởng, các độ đo khoảng cách và tính khoảng cách bằng ma trận

Ý tưởng: ba bước

Từ mục 1.2, thuật toán k-NN gồm ba bước cho mỗi điểm truy vấn $\mathbf{x}_0$:

1. **Đo**: tính khoảng cách $d(\mathbf{x}_0, \mathbf{x}_i)$ từ $\mathbf{x}_0$ tới mọi mẫu huấn luyện $i = 1, \dots, n$.
2. **Chọn**: lấy k mẫu có khoảng cách nhỏ nhất, tạo thành tập chỉ số $\mathcal{N}_k(\mathbf{x}_0)$.
3. **Quyết định**: tổng hợp nhãn của k mẫu đó thành một phân phối xác suất trên K lớp và chọn lớp có xác suất lớn nhất.

Bước 3 đã có từ mục 1.2 và sẽ được mở rộng ở mục 2.2. Bước 2 là một phép sắp xếp (hoặc chọn k phần tử nhỏ nhất). Mọi sự "thông minh" của k-NN nằm ở bước 1: nếu thước đo khoảng cách không phản ánh đúng mức độ "giống nhau" thì các láng giềng được chọn sẽ không còn ý nghĩa.

Các độ đo khoảng cách

Với hai vector hàng $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{1\times d}$, họ **khoảng cách Minkowski** bậc $q \geq 1$ là

$$d_q(\mathbf{u}, \mathbf{v}) = \left(\sum_{j=1}^{d} |u_j - v_j|^q \right)^{1/q}.$$

Ba trường hợp thường gặp:

$$d_1(\mathbf{u}, \mathbf{v}) = \sum_{j=1}^{d} |u_j - v_j| \quad \text{(Manhattan)}, \qquad d_2(\mathbf{u}, \mathbf{v})$$

$$= \sqrt{\sum_{j=1}^{d} (u_j - v_j)^2} \quad \text{(Euclid)}, \qquad d_\infty(\mathbf{u}, \mathbf{v}) = \max_j |u_j - v_j| \quad \text{(Chebyshev)}.$$

- **Euclid** ($q = 2$) là khoảng cách "đường chim bay", lựa chọn mặc định vì cho kết quả không đổi khi xoay hệ trục và có dạng bình phương dễ tính bằng ma trận (xem bên dưới).
- **Manhattan** ($q = 1$) cộng các chênh lệch theo từng đặc trưng, giống quãng đường đi trong một thành phố có các con phố vuông góc. Nó ít bị một đặc trưng lệch quá lớn chi phối hơn Euclid, vì chênh lệch không bị bình phương.
- **Chebyshev** ($q \to \infty$) chỉ giữ lại chênh lệch lớn nhất trên một đặc trưng.

Ví dụ, lấy sinh viên mới $\mathbf{x}_0 = [8,2]$ ở mục 1.2 và sinh viên 1 có $\mathbf{x}_1 = [2,4]$, hai vector khác nhau [6,2] (theo giá trị tuyệt đối). Khi đó $d_1 = 6 + 2 = 8$, $d_2 = \sqrt{36 + 4} \approx 6{,}325$, $d_3 = (216 + 8)^{1/3} \approx 6{,}073$ và $d_\infty = 6$. Bậc q càng lớn thì kết quả càng gần chênh lệch lớn nhất, và khoảng cách giảm dần khi q tăng. Các độ đo khác nhau có thể chọn ra tập láng giềng khác nhau, nên q cũng là một siêu tham số (mục 2.4). Trong bài này ta dùng Euclid trừ khi nói khác.

Tính khoảng cách bằng ma trận

Cài đặt trực tiếp hai vòng lặp (qua m điểm truy vấn và qua n mẫu) rất chậm trong Python. Với khoảng cách Euclid, ta tránh vòng lặp bằng cách khai triển bình phương:

$$\| \mathbf{x}_{0,j} - \mathbf{x}_i \|^2 = \| \mathbf{x}_{0,j} \|^2 - 2\, \mathbf{x}_{0,j} \mathbf{x}_i^\top + \| \mathbf{x}_i \|^2.$$

Gom cho mọi cặp (j, i) vào một ma trận $m \times n$:

$$\mathbf{D}^{(2)} = \mathbf{a}\, \mathbf{1}_n^\top \; - \; 2\, \mathbf{X}_0 \mathbf{X}^\top \; + \; \mathbf{1}_m\, \mathbf{b}^\top, \qquad a_j = \| \mathbf{x}_{0,j} \|^2, \quad b_i = \| \mathbf{x}_i \|^2.$$

Trong công thức này, $\mathbf{a}\mathbf{1}_n^\top$ lặp vector $\mathbf{a}$ thành n cột, còn $\mathbf{1}_m \mathbf{b}^\top$ lặp vector $\mathbf{b}^\top$ thành m hàng. Thành phần chính là tích $\mathbf{X}_0 \mathbf{X}^\top$, một phép nhân ma trận $m \times d$ với $d \times n$ mà thư viện đại số tuyến tính tính rất nhanh. Cách này cũng giống cách ta viết $\mathbf{S} = \mathbf{XW}$ ở bài Softmax: gom nhiều phép tính thành một phép nhân ma trận. Lưu ý rằng do sai số làm tròn, một phần tử có thể ra giá trị âm rất nhỏ; khi cài đặt ta cắt về 0 trước khi lấy căn.

Sau khi có $\mathbf{D}^{(2)}$, ta cần $\mathbf{D} = \sqrt{\mathbf{D}^{(2)}}$ (lấy căn từng phần tử) chỉ khi dùng đến giá trị khoảng cách. Để **chọn k láng giềng gần nhất** thì không cần lấy căn, vì căn bậc hai là hàm tăng nên giữ nguyên thứ tự.

2.2. Bỏ phiếu đa số, bỏ phiếu theo trọng số khoảng cách và xác suất lớp

Bỏ phiếu đa số

Sau bước chọn, ta có k láng giềng với các nhãn one-hot $\mathbf{y}_i$. Số phiếu của lớp c là

$$v_c(\mathbf{x}_0) = \sum_{i \in \mathcal{N}_k(\mathbf{x}_0)} y_{ic}\,, \qquad \hat{y}_0 = \operatorname*{argmax}_c \, v_c(\mathbf{x}_0), \qquad \hat{p}_c(\mathbf{x}_0) = \frac{v_c(\mathbf{x}_0)}{k}.$$

(Ta dùng chỉ số c cho lớp để không trùng với k là số láng giềng.) Đây chính là công thức ở mục 1.2, nay gắn thêm tên gọi: mỗi láng giềng có đúng một phiếu, bất kể gần hay xa. Xếp kết quả cho m điểm truy vấn vào ma trận, ta có

$$\widehat{\mathbf{P}} = \frac{1}{k}\,\mathbf{N}\mathbf{Y}, \qquad N_{ji} = \begin{cases} 1 & \text{nếu } i \in \mathcal{N}_k(\mathbf{x}_{0,j}), \\ 0 & \text{ngược lại.} \end{cases}$$

Mỗi hàng của $\widehat{\mathbf{P}}$ không âm và có tổng bằng 1. Vì vậy ta có thể dùng nó giống như ma trận $\mathbf{P}$ của Softmax: lấy argmax theo hàng để dự đoán nhãn, hoặc đưa vào các độ đo như log-loss (mục 7). Một điểm khác với Softmax cần nhớ: $\hat{p}_c$ chỉ nhận các giá trị dạng v/k, tức **chỉ có $k+1$ mức xác suất khả dĩ cho mỗi lớp**, và có thể bằng đúng 0. Xác suất 0 cho lớp thật sẽ làm log-loss bằng vô cùng, nên khi cần đánh giá bằng log-loss ta làm trơn $\tilde{p}_c = (v_c + \alpha)/(k + K\alpha)$ với $\alpha > 0$ nhỏ (xem mục 7.2).

Xử lý khi hòa phiếu

Khi hai lớp có số phiếu bằng nhau thì argmax không xác định duy nhất. Với bài toán hai lớp, chọn k lẻ là đủ để tránh hòa. Với $K \geq 3$ lớp, hòa vẫn có thể xảy ra ngay cả khi k lẻ (ví dụ $k = 3$ và ba láng giềng thuộc ba lớp khác nhau). Các cách xử lý thông dụng là:

- chọn lớp của láng giềng **gần nhất** trong số các lớp hòa;
- chọn lớp có **tổng khoảng cách** tới $\mathbf{x}_0$ nhỏ nhất;
- hoặc chọn theo thứ tự cố định (ví dụ lớp có chỉ số nhỏ hơn), cách đơn giản mà nhiều thư viện dùng.

Điều quan trọng là phải chọn một quy tắc *xác định* để kết quả lặp lại được. Bỏ phiếu theo trọng số, ngay dưới đây, cũng là một cách giảm hòa phiếu một cách tự nhiên.

Bỏ phiếu theo trọng số khoảng cách

Trong bỏ phiếu đa số, láng giềng thứ k (xa nhất trong $\mathcal{N}_k$) có tiếng nói bằng láng giềng gần nhất. Điều này không hợp lý: nếu láng giềng gần nhất nằm sát $\mathbf{x}_0$, còn các láng giềng khác khá xa, ta muốn tin vào láng giềng gần nhất hơn. Cách làm là gán cho láng giềng i một trọng số w_i giảm dần theo khoảng cách $d_i = d(\mathbf{x}_0, \mathbf{x}_i)$:

$$\hat{p}_c(\mathbf{x}_0) = \frac{\sum_{i \in \mathcal{N}_k(\mathbf{x}_0)} w_i \; y_{ic}}{\sum_{i \in \mathcal{N}_k(\mathbf{x}_0)} w_i}, \qquad w_i = \frac{1}{d_i^2 + \varepsilon}.$$

Hằng số nhỏ $\varepsilon > 0$ chặn trường hợp $d_i = 0$ (điểm truy vấn trùng một mẫu huấn luyện). Cũng có thể dùng $w_i = 1/d_i$ hoặc hàm Gauss $w_i = \exp(-d_i^2/2h^2)$ (mục 3.2 sẽ dùng kiểu trọng số này với tên gọi hàm nhân). Đặt $\tilde{w}_i = w_i / \sum_{l \in \mathcal{N}_k} w_l$ (các trọng số đã chuẩn hóa, tổng bằng 1 trên k láng giềng) và gom vào ma trận $\mathbf{N}_w$ có $(\mathbf{N}_w)_{ji} = \tilde{w}_i$ nếu $i \in \mathcal{N}_k(\mathbf{x}_{0,j})$, ngược lại bằng 0, ta được cùng một dạng như trước:

$$\widehat{\mathbf{P}} = \mathbf{N}_w \mathbf{Y}.$$

Bỏ phiếu đa số chính là trường hợp riêng $\mathbf{N}_w = \frac{1}{k}\mathbf{N}$. Cả hai đều nằm trong dạng "trung bình có trọng số" $\hat{\mathbf{p}}(\mathbf{x}_0) = \sum_i w_i(\mathbf{x}_0)\mathbf{y}_i$ mà ta đã nêu ở mục 1.3, chỉ khác cách đặt $w_i(\mathbf{x}_0)$. Bỏ phiếu có trọng số làm kết quả **bớt nhạy với** k: láng giềng xa có trọng số nhỏ nên việc thêm hay bớt chúng ít làm thay đổi dự đoán. Ví dụ ở mục 2.6 sẽ cho thấy trọng số có thể đổi cả kết luận.

2.3. Đường biên quyết định (ô Voronoi khi $k = 1$) và ảnh hưởng của k

Trường hợp $k = 1$: các ô Voronoi

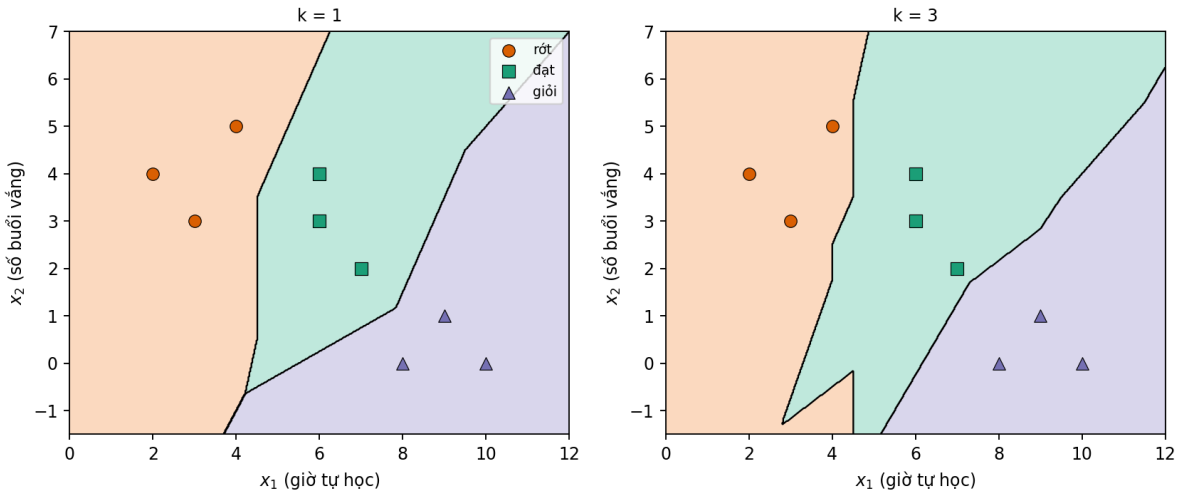
Với $k = 1$, điểm $\mathbf{x}$ được gán nhãn của mẫu gần nó nhất. Mỗi mẫu huấn luyện $\mathbf{x}_i$ vì vậy "sở hữu" một vùng gồm mọi điểm gần nó hơn mọi mẫu khác, gọi là **ô Voronoi**:

$$\mathcal{R}_i = \{\mathbf{x}: \| \mathbf{x} - \mathbf{x}_i \| \leq \| \mathbf{x} - \mathbf{x}_j \| \quad \text{với mọi } j \neq i\}, \qquad \hat{y}(\mathbf{x}) = y_i \quad \text{nếu } \mathbf{x} \in \mathcal{R}_i.$$

Khai triển bình phương hai vế của một bất đẳng thức trong định nghĩa, các số hạng $\| \mathbf{x} \|^2$ triệt tiêu, còn lại

$$2\,\mathbf{x}\,(\mathbf{x}_j - \mathbf{x}_i)^{\mathsf{T}} \leq \| \mathbf{x}_j \|^2 - \| \mathbf{x}_i \|^2,$$

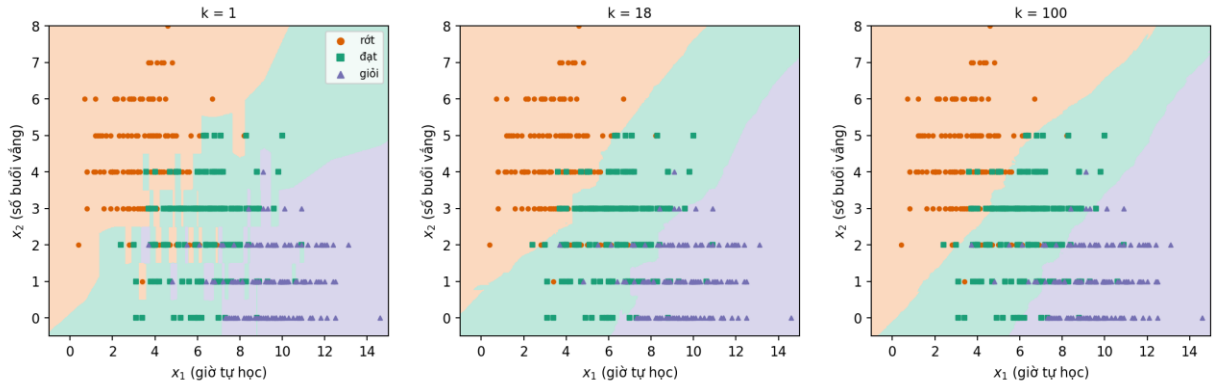
là một **nửa không gian** (nửa mặt phẳng khi $d = 2$). Do đó mỗi ô $\mathcal{R}_i$ là giao của $n - 1$ nửa không gian, tức một **đa giác lồi**, và biên giữa hai ô là một đoạn của đường trung trực nối hai mẫu. Điểm đáng chú ý là so với Softmax Regression: ở bài Softmax, *vùng của mỗi lớp* là lồi; ở k-NN, *mỗi ô của một mẫu* là lồi, nhưng vùng của một lớp là **hợp của nhiều ô**, nên có thể có hình dạng bất kỳ, lõm, hay gồm nhiều mảnh rời. Đó là lý do k-NN uốn theo các vòng tròn đồng tâm ở Hình 1b. Hình 3 minh họa trên 9 sinh viên: với $k = 1$, đường biên gồm các đoạn thẳng gấp khúc, mỗi đoạn nằm trên một đường trung trực.



Hình 3. Vùng quyết định của k-NN trên 9 sinh viên (rớt, đạt, giỏi) với $k = 1$ (trái) và $k = 3$ (phải). Vùng của mỗi lớp là hợp các ô Voronoi tương ứng; biên là các đoạn gấp khúc. Khi tăng k, biên bớt gấp khúc hơn.

Ảnh hưởng của k: từ bám nhiễu đến quá mượt

Khi k tăng, mỗi quyết định dựa trên một vùng rộng hơn và biên trở nên mượt hơn. Hình 4 cho thấy điều này trên bộ dữ liệu mô phỏng 600 sinh viên (200 mỗi lớp; hai đặc trưng là số giờ tự học và số buổi vắng). Đây là dữ liệu do chúng ta tự sinh ra cho mục này (ba lớp là ba đám mây Gauss chồng lấn vừa phải, đặc trưng được chuẩn hóa như ở mục 2.5), chia 70% huấn luyện và 30% kiểm tra.



Hình 4. Vùng quyết định của k-NN trên tập huấn luyện của 600 sinh viên mô phỏng, với $k = 1$ (trái), $k = 18$ (giữa) và $k = 100$ (phải). Với $k = 1$, vùng quyết định vỡ thành nhiều mảnh nhỏ quanh từng điểm; với $k = 100$, biên gần như một đường trơn.

- k **nhỏ ($k = 1$)**: mô hình *học thuộc*. Mỗi điểm, kể cả điểm nhiễu hoặc bị gán nhãn sai, tạo ra một "đảo" nhỏ quanh nó (Hình 4, trái). Độ chính xác trên tập huấn luyện rất cao (92,9%; không phải 100% vì dữ liệu đã làm tròn nên có những mẫu trùng tọa độ nhưng khác nhãn), nhưng trên tập kiểm tra chỉ còn 72,2%. Đây là hiện tượng **phương sai cao** (quá khớp).
- k **lớn**: mô hình *quá mượt*. Ở cực đoan $k = n_{\text{huấn luyện}}$, mọi điểm đều có cùng một tập láng giềng là toàn bộ dữ liệu, nên mô hình dự đoán cùng một lớp ở mọi nơi (lớp phổ biến nhất, hoặc theo quy tắc hòa phiếu nếu các lớp cân bằng). Trong thí nghiệm này, với $k = 420$ độ chính xác kiểm tra chỉ còn 33,3%, bằng mức đoán mò. Đây là hiện tượng **thiên lệch cao** (thiếu khớp).

Như vậy k đóng vai trò "nút chỉnh độ phức tạp" của mô hình, tương tự bậc đa thức hay hệ số điều chuẩn λ ở bài Softmax: k nhỏ thì mô hình linh hoạt nhưng bám nhiễu, k lớn thì mô hình ổn định nhưng bỏ lỡ chi tiết. Ta cần một cách có hệ thống để chọn k ở giữa, chủ đề của mục 2.4.

2.4. Chọn k bằng kiểm định chéo

Vì sao không chọn k bằng độ chính xác huấn luyện

Độ chính xác trên tập huấn luyện **không dùng được** để chọn k: với $k = 1$, mỗi mẫu huấn luyện là láng giềng gần nhất của chính nó, nên mô hình "thuộc lòng" tập huấn luyện và gần như luôn có độ chính xác cao nhất (Hình 5, đường cam). Ta phải đo trên dữ liệu mô hình *chưa nhìn thấy*.

Nhưng cũng không nên dùng tập kiểm tra cuối cùng để chọn k, vì khi đó tập kiểm tra không còn là phép thử khách quan. Giải pháp là **kiểm định chéo** (cross-validation), quen thuộc từ các bài trước, trong đó siêu tham số được chọn trên *một phần tách ra từ tập huấn luyện*.

Kiểm định chéo V phần

Chia ngẫu nhiên tập huấn luyện thành V phần xấp xỉ bằng nhau (với phân loại, nên chia sao cho tỉ lệ các lớp ở mỗi phần giống nhau). Với mỗi giá trị k ứng viên và mỗi phần $v = 1, \dots, V$: huấn luyện (tức là **lưu** dữ liệu) trên $V - 1$ phần còn lại, rồi đo độ chính xác $A_v(k)$ trên phần v. Độ chính xác kiểm định chéo là trung bình:

$$\mathrm{CV}(k) = \frac{1}{V}\sum_{v=1}^{V} A_v\,(k), \qquad \hat{k} = \operatorname*{argmax}_k \mathrm{CV}(k).$$

Có hai lưu ý thực hành. **Thứ nhất**, chuẩn hóa đặc trưng (mục 2.5) phải được tính lại **bên trong mỗi phần**, chỉ từ $V - 1$ phần huấn luyện, nếu không thông tin từ phần kiểm định sẽ "rò" vào bước chuẩn hóa và làm kết quả lạc quan. **Thứ hai**, danh sách ứng viên thường là các số nguyên từ 1 đến vài chục hoặc đến khoảng $\sqrt{n}$; $\sqrt{n}$ chỉ là quy tắc kinh nghiệm để có điểm xuất phát chứ không có đảm bảo. Với V phần, k lớn nhất hợp lý còn bị giới hạn bởi kích thước của lớp nhỏ nhất, như ví dụ ngay dưới đây cho thấy.

Ví dụ: kiểm định chéo trên 9 sinh viên

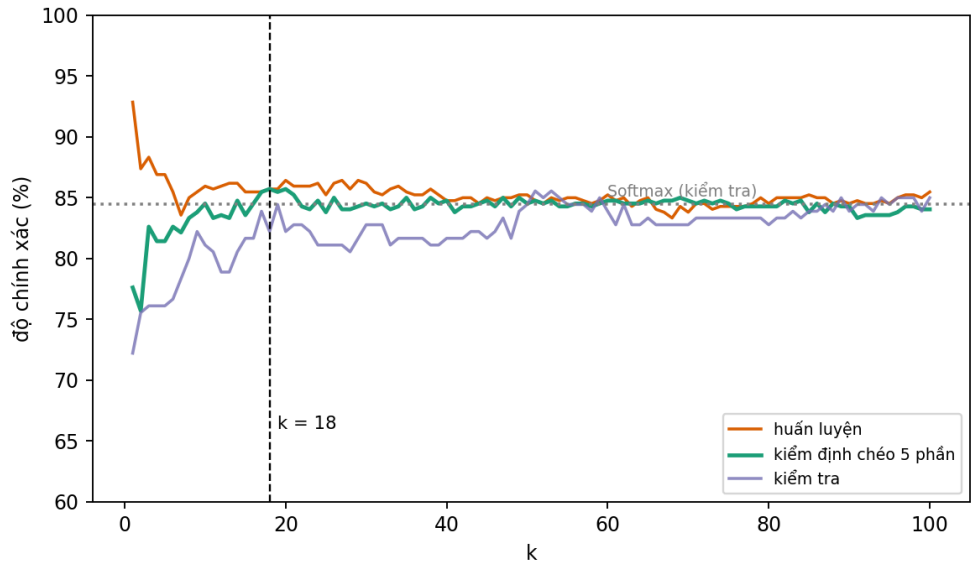
Với 9 sinh viên, ta dùng trường hợp cực đoan $V = n = 9$, gọi là **bỏ-một-ra** (leave-one-out): lần lượt bỏ từng sinh viên ra, dự đoán bằng 8 sinh viên còn lại. Kết quả (khoảng cách Euclid, bỏ phiếu đa số):

k	Số sinh viên dự đoán đúng	Độ chính xác bỏ-một-ra
1	9 / 9	100% (có một lần hòa khoảng cách, xem ghi chú)
3	9 / 9	100%
5	0 / 9	0%

Ghi chú: khi bỏ sinh viên 3 ra, hai sinh viên 2 (rớt) và 6 (đạt) cùng cách sinh viên 3 một khoảng $\sqrt{5}$; với $k = 1$ ta chọn theo thứ tự chỉ số nên chọn sinh viên 2 và dự đoán đúng. Kết quả $k = 5$ là bài học đáng nhớ: mỗi lớp chỉ có 3 sinh viên, khi bỏ một em ra thì lớp của em đó chỉ còn 2 đại diện, mà 5 láng giềng buộc phải lấy thêm ít nhất 3 sinh viên từ lớp khác, nên lớp đúng chỉ có tối đa 2 phiếu trong 5 và thường thua; ở bộ dữ liệu nhỏ này nó thua cả 9 lần. k **không được lớn hơn (hoặc quá gần) kích thước của lớp nhỏ nhất.** Từ đây ta chọn $k = 3$ hoặc $k = 1$ cho bộ dữ liệu nhỏ này, và ưu tiên $k = 3$ vì mượt hơn (Hình 3).

Kết quả trên 600 sinh viên mô phỏng

Ta chạy kiểm định chéo $V = 5$ phần (chuẩn hóa trong từng phần) cho $k = 1,2, \dots, 100$ trên 420 sinh viên huấn luyện. Hình 5 vẽ độ chính xác huấn luyện, kiểm định chéo và kiểm tra theo k; Bảng dưới trích một số giá trị.



Hình 5. Độ chính xác theo k trên dữ liệu 600 sinh viên mô phỏng: huấn luyện (cam), kiểm định chéo 5 phần (xanh lục) và kiểm tra (tím). Đường nét đứt đánh dấu $\hat{k} = 18$; đường chấm là độ chính xác kiểm tra của Softmax Regression.

k	Huấn luyện	Kiểm định chéo	Kiểm tra
1	92,9%	77,6%	72,2%
5	86,9%	81,4%	76,1%
15	85,5%	83,6%	81,7%
18 ($\hat{k}$)	85,7%	85,7%	82,2%
50	85,2%	84,5%	84,4%
100	85,5%	84,0%	85,0%
Softmax Regression	86,0%		84,4%

Có ba nhận xét.

- **Hình chữ "U ngược" rõ ở phía k nhỏ.** Từ $k = 1$ đến khoảng $k = 15$, độ chính xác kiểm định chéo tăng nhanh (77,6% → 83,6%) khi mô hình bớt bám nhiễu. Khoảng cách giữa độ chính xác huấn luyện và kiểm định chéo (rất lớn ở $k = 1$) thu hẹp dần, dấu hiệu quá khớp giảm.
- **Đường cong khá phẳng từ $k \approx 15$ trở đi.** Giá trị lớn nhất của kiểm định chéo là 85,7% tại $\hat{k} = 18$, nhưng sai số chuẩn của ước lượng này vào khoảng 2 điểm phần trăm, nên các k từ 15 đến 100 thực chất không phân biệt được với nhau. Tập kiểm tra có 180 sinh viên, mỗi sinh viên ứng với 0,56 điểm phần trăm, nên chênh lệch kiểu 82,2% so với 85,0% giữa $k = 18$ và $k = 100$ chỉ là vài sinh viên và

chưa đủ để kết luận $k = 100$ tốt hơn. Khi đường kiểm định chéo phẳng, nên chọn k lớn hơn trong vùng phẳng cho mô hình ổn định.

- **k-NN không thắng Softmax trên dữ liệu này.** Với $\hat{k} = 18$, độ chính xác kiểm tra là 82,2%, gần với 84,4% của Softmax. Điều đó không bất ngờ: dữ liệu mô phỏng gồm các đám Gauss chồng lấn, ranh giới tốt nhất gần như tuyến tính, đúng loại dạng hàm mà Softmax giả định. Khi dạng hàm *đúng* thì mô hình tham số rất hiệu quả, còn mô hình phi tham số tốn thêm phương sai để "học lại" cái hình dạng đơn giản đó. Ngược lại với ba vòng tròn ở mục 1.1, nơi dạng hàm *sai*. Đây là đúng đánh đổi đã nêu ở mục 1.3.

2.5. Chuẩn hóa đặc trưng và lời nguyền chiều

Vì sao phải chuẩn hóa: khoảng cách phụ thuộc đơn vị đo

k-NN chỉ dựa trên khoảng cách, nên đơn vị đo của từng đặc trưng ảnh hưởng trực tiếp đến kết quả. Giả sử số giờ tự học được đo bằng **phút** thay vì giờ, tức $x_1 \to 60\,x_1$, còn số buổi vắng x_2 giữ nguyên. Khi đó chênh lệch trên x_1 thường lớn hơn chênh lệch trên x_2 hàng chục lần; trong tổng $(x_{01} - x_{i1})^2 + (x_{02} - x_{i2})^2$, số hạng thứ nhất lấn át hoàn toàn số hạng thứ hai, và k-NN gần như bỏ qua số buổi vắng. Nguyên nhân giống hệt với việc Gradient Descent hội tụ chậm khi các đặc trưng khác thang đo (đã gặp ở bài Softmax), nhưng ở k-NN nó còn nặng hơn, vì không có tham số nào học được để "bù" lại.

Cách chữa quen thuộc là **chuẩn hóa z-score**: với mỗi đặc trưng j, dùng trung bình và độ lệch chuẩn của **tập huấn luyện**:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}, \qquad \mu_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}\,, \qquad \sigma_j = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_{ij} - \mu_j)^2}.$$

Điểm truy vấn cũng phải được chuẩn hóa bằng đúng $\boldsymbol{\mu}$ và $\boldsymbol{\sigma}$ này (**không** tính lại trên tập kiểm tra), theo dạng ma trận $\mathbf{Z}_0 = (\mathbf{X}_0 - \mathbf{1}_m\boldsymbol{\mu})\,\mathrm{diag}(\boldsymbol{\sigma})^{-1}$. Một lựa chọn khác là chuẩn hóa min-max đưa mỗi đặc trưng về $[0,1]$, nhưng nó nhạy với ngoại lai hơn. Sau chuẩn hóa z-score, kết quả không còn phụ thuộc đơn vị đo.

Trên dữ liệu 600 sinh viên mô phỏng, với $k = 18$ và đo số giờ bằng phút:

Cách xử lý đặc trưng	Độ chính xác kiểm tra
Đo bằng giờ, không chuẩn hóa	82,8%
Đo bằng phút, không chuẩn hóa	77,2%
Chuẩn hóa z-score (giờ hay phút đều cho cùng kết quả)	82,2%

Điều quan trọng không phải là chuẩn hóa luôn nâng điểm (ở đây đo bằng giờ không chuẩn hóa tình cờ nhỉnh hơn 0,6 điểm, tương ứng 1 mẫu), mà là **kết quả sau chuẩn hóa không bị đổi theo đơn vị đo tùy ý**, trong khi không chuẩn hóa thì chỉ cần đổi giờ sang phút là mất 5,6 điểm phần

trăm. (Cũng cần nhớ rằng việc chuẩn hóa ngầm coi mọi đặc trưng đều quan trọng ngang nhau; nếu ta biết đặc trưng nào quan trọng hơn thì có thể nhân thêm hệ số riêng cho từng đặc trưng.)

Lời nguyền chiều

Giả thiết "gần thì giống" (mục 1.2) cần có điểm *thật sự gần* để dựa vào. Khi số chiều d tăng, điều này ngày càng khó, vì hai lý do.

Thứ nhất, dữ liệu trở nên thưa. Như đã tính ở mục 1.3, số ô cần có mẫu tăng như 10^d khi d tăng, còn số mẫu n thì không thể tăng theo kịp. Vùng lân cận "đủ nhỏ để xác suất gần như không đổi" sẽ chứa rất ít mẫu, hoặc phải lớn đến mức không còn "địa phương" nữa.

Thứ hai, khoảng cách mất khả năng phân biệt. Lấy $n = 1\,000$ điểm phân bố ngẫu nhiên đều trong hình lập phương $[0,1]^d$ và một điểm truy vấn ngẫu nhiên. Gọi $d_{\min}$ và $d_{\max}$ là khoảng cách tới điểm gần nhất và xa nhất. Ta đo độ chênh tương đối:

$$\rho(d) = \frac{d_{\max} - d_{\min}}{d_{\min}}.$$

Số chiều d	2	10	100	1 000
$\rho(d)$	269,1	2,95	0,55	0,15

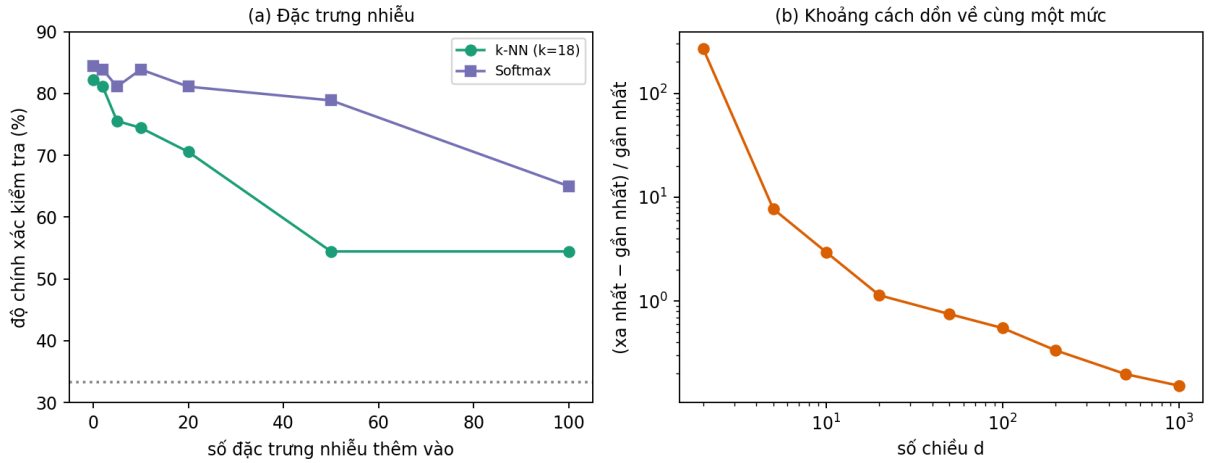
Ở $d = 2$, điểm xa nhất cách gấp hàng trăm lần điểm gần nhất, nên "gần nhất" có ý nghĩa rõ rệt. Ở $d = 1\,000$, điểm gần nhất cách khoảng 12,0 còn điểm xa nhất khoảng 13,8, chỉ chênh 15%: mọi điểm đều "xấp xỉ cùng xa", và việc chọn k láng giềng gần nhất gần như tùy tiện (Hình 6b). Đây là một trường hợp điển hình của hiện tượng **tập trung khoảng cách**.

Đặc trưng nhiễu phá k-NN

Một cách thấy tác hại trực tiếp: thêm vào dữ liệu 600 sinh viên các đặc trưng **nhiễu thuần túy** (số ngẫu nhiên chuẩn tắc, không liên quan đến nhãn). Mỗi đặc trưng nhiễu đóng góp vào khoảng cách một lượng ngẫu nhiên có độ lớn ngang với đặc trưng thật (do ta đã chuẩn hóa), và khi có nhiều đặc trưng nhiễu thì phần nhiễu lấn át phần tín hiệu, làm các láng giềng được chọn gần như ngẫu nhiên. Hình 6a so sánh k-NN ($k = 18$) với Softmax khi số đặc trưng nhiễu tăng (mỗi điểm là một lần chạy với một lần chia dữ liệu, nên có dao động nhỏ):

Số đặc trưng nhiễu thêm vào	0	5	20	50	100
k-NN ($k = 18$)	82,2%	75,6%	70,6%	54,4%	54,4%
Softmax Regression	84,4%	81,1%	81,1%	78,9%	65,0%

Với 50 đặc trưng nhiễu (so với 2 đặc trưng thật), k-NN rơi từ 82,2% xuống 54,4%, còn Softmax chỉ giảm nhẹ. Lý do là Softmax *học được trọng số*: nó gán trọng số gần 0 cho các đặc trưng vô ích, còn k-NN coi mọi đặc trưng như nhau trong khoảng cách. Đây là một điểm yếu quan trọng của mô hình dựa trên khoảng cách; ở mục 6.3 ta sẽ so sánh với Bayes ngây thơ (ít nhạy với đặc trưng nhiễu hơn). Trong thực hành, ta cần **chọn lọc đặc trưng** hoặc **giảm chiều** trước khi dùng k-NN.



Hình 6. (a) Độ chính xác kiểm tra khi thêm các đặc trưng nhiễu thuần túy: k-NN suy giảm nhanh hơn Softmax. Đường chấm là mức đoán mò (33,3%). (b) Với 1 000 điểm ngẫu nhiên đều trong $[0,1]^d$, độ chênh tương đối giữa điểm xa nhất và điểm gần nhất giảm mạnh khi d tăng.

2.6. Ví dụ tính tay trên sinh viên ba mức kết quả

Ta tính trọn vẹn một lần dự đoán cho nhiều sinh viên mới cùng lúc bằng đúng các công thức ở mục 2.1 và 2.2, với 9 sinh viên huấn luyện (rớt $= 0$, đạt $= 1$, giỏi $= 2$) như ở mục 1.2:

$$\mathbf{X} = \begin{pmatrix} 2 & 4 \\ 3 & 3 \\ 4 & 5 \\ 6 & 3 \\ 7 & 2 \\ 6 & 4 \\ 9 & 1 \\ 10 & 0 \\ 8 & 0 \end{pmatrix}, \qquad \mathbf{Y} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}.$$

Ba sinh viên mới (các hàng của $\mathbf{X}_0$), dùng $k = 3$ và khoảng cách Euclid (không chuẩn hóa vì hai đặc trưng cùng cỡ):

$$\mathbf{X}_0 = \begin{pmatrix} 5 & 3 \\ 8 & 2 \\ 3 & 2 \end{pmatrix}.$$

Bước 1: vector bình phương chuẩn. $a_j = \| \mathbf{x}_{0,j} \|^2$ và $b_i = \| \mathbf{x}_i \|^2$:

$$\mathbf{a} = \begin{pmatrix} 34 \\ 68 \\ 13 \end{pmatrix}, \qquad \mathbf{b}^\top = (20 \quad 18 \quad 41 \quad 45 \quad 53 \quad 52 \quad 82 \quad 100 \quad 64).$$

Bước 2: ma trận bình phương khoảng cách $\mathbf{D}^{(2)} = \mathbf{a}\mathbf{1}^\top - 2\mathbf{X}_0\mathbf{X}^\top + \mathbf{1}\mathbf{b}^\top$. Kiểm tra một phần tử: điểm truy vấn thứ nhất và sinh viên 4 có $34 - 2 \cdot (5 \cdot 6 + 3 \cdot 3) + 45 = 34 - 78 + 45 = 1$. Toàn bộ ma trận (cột i ứng với sinh viên i):

$$\mathbf{D}^{(2)} = \begin{pmatrix} 10 & 4 & 5 & 1 & 5 & 2 & 20 & 34 & 18 \\ 40 & 26 & 25 & 5 & 1 & 8 & 2 & 8 & 4 \\ 5 & 1 & 10 & 10 & 16 & 13 & 37 & 53 & 29 \end{pmatrix}.$$

Hàng thứ hai khớp với bảng khoảng cách ở mục 1.2 (ví dụ $d^2 = 40$ cho sinh viên 1 nghĩa là $d = \sqrt{40} \approx 6{,}325$).

Bước 3: chọn $k = 3$ láng giềng gần nhất (ba giá trị nhỏ nhất trong mỗi hàng của $\mathbf{D}^{(2)}$). Hàng 1: 1 (sinh viên 4), 2 (sinh viên 6), 4 (sinh viên 2). Hàng 2: 1, 2, 4 (sinh viên 5,7,9). Hàng 3: 1, 5, 10 (sinh viên 2,1,3). Từ đó ta được ma trận láng giềng $\mathbf{N}$ (ba hàng, chín cột):

$$\mathbf{N} = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Bước 4: bỏ phiếu đa số $\widehat{\mathbf{P}} = \frac{1}{3}\mathbf{NY}$:

$$\mathbf{NY} = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 2 \\ 3 & 0 & 0 \end{pmatrix}, \qquad \widehat{\mathbf{P}} = \frac{1}{3}\mathbf{NY} = \begin{pmatrix} 1/3 & 2/3 & 0 \\ 0 & 1/3 & 2/3 \\ 1 & 0 & 0 \end{pmatrix}.$$

Lấy argmax theo hàng, dự đoán lần lượt là **đạt**, **giỏi**, **rớt**. Mỗi hàng của $\widehat{\mathbf{P}}$ cộng lại bằng 1. Sinh viên thứ ba cực kỳ chắc chắn (cả 3 láng giềng là sinh viên rớt, $\hat{p} = 1$), còn hai sinh viên đầu thì thuộc vùng biên giữa hai lớp ($\hat{p} = 2/3$), đúng với vị trí của họ trong Hình 2.

Bước 5: bỏ phiếu theo trọng số $w_i = 1/d_i^2$ (với bình phương khoảng cách đã có sẵn trong $\mathbf{D}^{(2)}$). Hàng 1: các trọng số $1, 1/2, 1/4$ ứng với sinh viên 4 (đạt), 6 (đạt), 2 (rớt), tổng bằng 1,75. Hàng 2: các trọng số $1, 1/2, 1/4$ ứng với sinh viên 5 (đạt), 7 (giỏi), 9 (giỏi). Hàng 3: $1, 1/5, 1/10$, cùng một lớp (rớt):

$$\widehat{\mathbf{P}}_w = \begin{pmatrix} 0{,}25/1{,}75 & 1{,}5/1{,}75 & 0 \\ 0 & 1/1{,}75 & 0{,}75/1{,}75 \\ 1 & 0 & 0 \end{pmatrix} \approx \begin{pmatrix} 0{,}143 & 0{,}857 & 0 \\ 0 & 0{,}571 & 0{,}429 \\ 1 & 0 & 0 \end{pmatrix}.$$

Nhận xét. Với sinh viên mới thứ hai $[8,2]$, bỏ phiếu đa số dự đoán *giỏi* $(2/3)$, còn bỏ phiếu có trọng số dự đoán *đạt* $(0{,}571)$. Lý do là sinh viên 5 (đạt) nằm *sát* điểm truy vấn ($d = 1$) trong khi hai sinh viên giỏi xa hơn ($d = \sqrt{2}$ và $d = 2$), nên một láng giềng rất gần có thể thắng hai láng giềng xa hơn. Hai cách cho kết luận khác nhau chính vì điểm này nằm gần vùng biên đạt–giỏi; với điểm xa biên (hai điểm còn lại) cả hai cách cho cùng kết quả. Cũng chính nhờ vậy ta thấy đầu ra của k-NN không chỉ là nhãn mà còn có "độ tin cậy" đọc được từ $\widehat{\mathbf{P}}$.

2.7. Mã hóa k-NN từ đầu bằng Python, chi phí tính toán và KD-tree ở mức trực giác

Cài đặt từ đầu

Phần cài đặt chỉ gồm vài hàm NumPy bám sát các công thức ở trên: tính $\mathbf{D}^{(2)}$ bằng ma trận (mục 2.1), chọn k láng giềng để dựng $\mathbf{N}$ hoặc $\mathbf{N}_w$, và nhân với $\mathbf{Y}$ (mục 2.2).

```
import numpy as np

def khoang_cach_binh_phuong(X0, X):
    """Ma trận bình phương khoảng cách Euclid, kích thước m x n."""
    a = np.sum(X0**2, axis=1, keepdims=True)      # m x 1
    b = np.sum(X**2, axis=1)[None, :]             # 1 x n
    D2 = a - 2.0 * X0 @ X.T + b
    return np.maximum(D2, 0.0)                    # tránh số âm nhỏ do sai số
làm tròn

def knn_xac_suat(X0, X, Y, k=3, co_trong_so=False, eps=1e-12):
    """Ma trận xác suất lớp P_hat (m x K) của k-NN."""
    m, n = X0.shape[0], X.shape[0]
    D2 = khoang_cach_binh_phuong(X0, X)
    idx = np.argsort(D2, axis=1, kind="stable")[:, :k]    # chỉ số k láng giề
ng gần nhất
    N = np.zeros((m, n))
    hang = np.arange(m)[:, None]
    if co_trong_so:
        N[hang, idx] = 1.0 / (D2[hang, idx] + eps)         # w_i tỉ lệ 1/d^2
    else:
        N[hang, idx] = 1.0                                 # bỏ phiếu đa số
    return (N @ Y) / N.sum(axis=1, keepdims=True)          # (1/k) N Y, hoặc
có chuẩn hóa

def knn_du_doan(X0, X, Y, k=3, co_trong_so=False):
    return np.argmax(knn_xac_suat(X0, X, Y, k, co_trong_so), axis=1)

class ChuanHoa:
    def fit(self, X):
        self.mu, self.sigma = X.mean(axis=0), X.std(axis=0)
        return self
    def transform(self, X):
        return (X - self.mu) / self.sigma
```

Dòng cuối của *knn_xac_suat* chia cho tổng từng hàng của $\mathbf{N}$ (hoặc $\mathbf{N}_w$) nên một đoạn mã dùng chung cho cả bỏ phiếu đa số (tổng hàng bằng k) và có trọng số (tổng hàng bằng $\sum w_i$). Hàm *ChuanHoa* chỉ lưu $\boldsymbol{\mu}, \boldsymbol{\sigma}$ của tập huấn luyện; khi dự đoán, ta gọi *transform* cho cả tập kiểm tra.

Kiểm tra trên 9 sinh viên

```
S = np.array([[2,4,0],[3,3,0],[4,5,0],[6,3,1],[7,2,1],[6,4,1],[9,1,2],[10,0,2
],[8,0,2]], float)
X9, Y9 = S[:, :2], np.eye(3)[S[:, 2].astype(int)]
X0 = np.array([[5, 3], [8, 2], [3, 2]], float)

print(np.round(knn_xac_suat(X0, X9, Y9, k=3), 3))
print(np.round(knn_xac_suat(X0, X9, Y9, k=3, co_trong_so=True), 3))
```

```
[[0.333 0.667 0.   ]
 [0.    0.333 0.667]
```

```
 [1.    0.    0.   ]]
[[0.143 0.857 0.   ]
 [0.    0.571 0.429]
 [1.    0.    0.   ]]
```

Hai ma trận in ra trùng khớp với $\widehat{\mathbf{P}}$ và $\widehat{\mathbf{P}}_w$ ta đã tính tay ở mục 2.6.

Kiểm tra trên 600 sinh viên mô phỏng và so với scikit-learn

```
sc = ChuanHoa().fit(Xtr)
Ztr, Zte = sc.transform(Xtr), sc.transform(Xte)
Ytr = np.eye(3)[ytr]

for k in [1, 5, 18, 50]:
    mine = knn_du_doan(Zte, Ztr, Ytr, k)
    ref = KNeighborsClassifier(k).fit(Ztr, ytr).predict(Zte)
    print(k, (mine == yte).mean().round(4), (mine == ref).mean().round(4))
```

```
1  0.7333 0.95
5  0.7611 1.0
18 0.8167 0.9944
50 0.8389 0.9944
```

Cột thứ hai là độ chính xác kiểm tra của bản cài đặt này, cột thứ ba là tỉ lệ dự đoán trùng với `KNeighborsClassifier` của scikit-learn. Với $k = 5$ hai bên trùng hoàn toàn; ở các k khác có một vài mẫu khác nhau (tối đa 5% với $k = 1$), nguyên nhân là **hòa khoảng cách và hòa phiếu**: dữ liệu đã làm tròn nên có nhiều điểm trùng tọa độ hoặc cách đều, và mỗi thư viện chọn một quy tắc phá hòa khác nhau. Đây không phải lỗi mà là một chi tiết cần biết khi so sánh kết quả giữa hai cài đặt. Ma trận $\widehat{\mathbf{P}}$ trả về luôn có các hàng tổng bằng 1, ví dụ ba hàng đầu của tập kiểm tra ($k = 18$) là $[1{,}0{,}0]$, $[0{,}056{,}0{,}611{,}0{,}333]$ và $[0{,}0{,}1]$.

Chi phí tính toán

	Huấn luyện	**Dự đoán m điểm**	**Bộ nhớ**
k-NN vét cạn	$O(1)$ (chỉ lưu dữ liệu)	$O(mnd)$ cho khoảng cách, cộng chọn k phần tử nhỏ nhất trên mỗi hàng	$O(nd)$ cho dữ liệu, $O(mn)$ cho $\mathbf{D}^{(2)}$
Softmax Regression (so sánh)	$O(T\,ndK)$ cho T epoch	$O(mdK)$	$O\big((d+1)K\big)$

Hai điểm đáng lưu ý. **Thứ nhất**, chi phí dự đoán của k-NN tăng tuyến tính theo n, ngược với Softmax (không phụ thuộc n), nên k-NN khó áp dụng trực tiếp cho dữ liệu hàng triệu mẫu mà phải có cấu trúc tăng tốc. **Thứ hai**, ma trận $\mathbf{D}^{(2)}$ có kích thước $m \times n$; với m và n lớn nó không vừa bộ nhớ, nên trong thực hành ta chia các điểm truy vấn thành từng lô nhỏ. Việc sắp xếp toàn bộ mỗi hàng (như `argsort` ở trên, tốn $O(n\log n)$) cũng có thể thay bằng một phép chọn từng phần như `np.argpartition` (tốn $O(n)$); ta dùng `argsort` ổn định ở đây để kết quả lặp lại được và dễ đọc.

KD-tree ở mức trực giác

KD-tree là cấu trúc dữ liệu giúp tìm láng giềng mà không phải so với mọi mẫu. Ý tưởng: chia đệ quy không gian bằng các mặt phẳng song song với trục tọa độ. Ở mỗi bước, chọn một đặc trưng (thường lần lượt hoặc chọn đặc trưng có độ trải lớn nhất), lấy **trung vị** của đặc trưng đó làm ngưỡng và chia các điểm thành hai nửa; lặp lại cho từng nửa cho đến khi mỗi nhóm chỉ còn vài điểm. Khi có điểm truy vấn, ta đi xuống nhánh chứa nó để có ứng viên đầu tiên, rồi quay lui: một nhánh khác chỉ cần xét nếu vùng của nó *có thể* chứa điểm gần hơn k-láng giềng tốt nhất hiện có; nếu khoảng cách từ điểm truy vấn tới mặt cắt đã lớn hơn khoảng cách đó thì **bỏ cả nhánh**. Khi d nhỏ, hầu hết các nhánh bị loại và thời gian truy vấn trung bình chỉ cỡ $O(\log n)$; việc dựng cây tốn $O(n \log n)$.

Nhưng lời nguyền chiều lại xuất hiện: khi d lớn, như ở mục 2.5, mọi điểm đều "xấp xỉ cùng xa", nên rất ít nhánh bị loại và KD-tree phải duyệt gần hết, thậm chí chậm hơn vét cạn do chi phí duyệt cây. Ta kiểm tra bằng đo thời gian dự đoán 500 điểm truy vấn trên $n = 50\,000$ điểm phân bố đều ngẫu nhiên ($k = 5$, bộ cài đặt của scikit-learn, thời gian phụ thuộc máy nên chỉ có ý nghĩa tương đối):

Số chiều d	Vét cạn	KD-tree
2	0,108 s	0,003 s
10	0,096 s	0,103 s
50	0,196 s	5,903 s

Với $d = 2$ KD-tree nhanh hơn khoảng 36 lần; ở $d = 10$ hai cách tương đương; ở $d = 50$ KD-tree chậm hơn khoảng 30 lần. Quy tắc thực hành: KD-tree hữu ích khi d nhỏ (cỡ dưới 10 đến 20); với d lớn, người ta dùng vét cạn bằng phép nhân ma trận (như ở trên) hoặc các phương pháp tìm láng giềng **xấp xỉ**, đánh đổi một chút độ chính xác để lấy tốc độ.

Mối liên hệ với mục trước

Mục 1 trả lời câu hỏi "*vì sao*" dùng mô hình phi tham số và phác thảo k-NN bằng một công thức; mục 2 biến phác thảo đó thành một mô hình dùng được. Điều đã đổi so với mục 1: từ "nhìn các mẫu lân cận" thành một quy trình cụ thể với các lựa chọn rõ ràng (độ đo khoảng cách, cách bỏ phiếu, k, chuẩn hóa). Điều vẫn giữ nguyên: đầu ra là ma trận xác suất $\widehat{\mathbf{P}}$ với mỗi hàng là một phân phối, nhãn dự đoán lấy bằng argmax, và dạng chung "trung bình có trọng số $\widehat{\mathbf{p}} = \sum_i w_i(\mathbf{x}_0)\,\mathbf{y}_i$" của mục 1.3. Ba lời tiên đoán ở mục 1.3 cũng đã được kiểm nghiệm bằng số: k nhỏ thì phương sai cao (mục 2.3, 2.4), dạng hàm đúng thì mô hình tham số không thua (mục 2.4), và số chiều lớn làm phương pháp dựa trên khoảng cách suy giảm (mục 2.5). Mục 3 mang cùng ý tưởng sang bài toán **hồi quy**, nơi "bỏ phiếu" được thay bằng "lấy trung bình", và trọng số kiểu hàm Gauss ở mục 2.2 sẽ được gọi chính thức là hàm nhân.

Tóm tắt mục 2

- k-NN gồm ba bước: **đo** khoảng cách, **chọn** k láng giềng gần nhất, **quyết định** bằng bỏ phiếu. Huấn luyện chỉ là lưu dữ liệu.

- Khoảng cách Minkowski d_q gồm Manhattan ($q = 1$), Euclid ($q = 2$) và Chebyshev ($q \to \infty$). Với Euclid, ma trận bình phương khoảng cách tính được bằng một phép nhân ma trận: $\mathbf{D}^{(2)} = \mathbf{a}\mathbf{1}_n^\top - 2\mathbf{X}_0\mathbf{X}^\top + \mathbf{1}_m\mathbf{b}^\top$.
- Bỏ phiếu đa số cho $\widehat{\mathbf{P}} = \frac{1}{k}\mathbf{NY}$; bỏ phiếu theo trọng số khoảng cách cho $\widehat{\mathbf{P}} = \mathbf{N}_w\mathbf{Y}$ và có thể đổi kết luận khi các láng giềng ở xa nhau. Xác suất của k-NN chỉ có $k + 1$ mức nên cần làm trơn khi đánh giá bằng log-loss.
- Với $k = 1$, vùng quyết định là hợp các **ô Voronoi** (mỗi ô lồi, biên nằm trên các đường trung trực), nhưng vùng của một lớp không cần lồi. k nhỏ gây quá khớp (phương sai cao), k lớn gây thiếu khớp (thiên lệch cao).
- Chọn k bằng **kiểm định chéo** V phần (chuẩn hóa trong từng phần), không dùng độ chính xác huấn luyện. Trên dữ liệu mô phỏng, $\hat{k} = 18$ cho $85{,}7\%$ (kiểm định chéo) và $82{,}2\%$ (kiểm tra); k không được lớn hơn kích thước lớp nhỏ nhất.
- **Chuẩn hóa z-score** bằng thống kê của tập huấn luyện làm kết quả không phụ thuộc đơn vị đo. Khi d lớn, khoảng cách tập trung và đặc trưng nhiễu làm k-NN suy giảm mạnh (từ $82{,}2\%$ còn $54{,}4\%$ với 50 đặc trưng nhiễu), trong khi Softmax chỉ giảm nhẹ.
- Dự đoán vét cạn tốn $O(nd)$ cho mỗi điểm. KD-tree giúp ích khi d nhỏ nhưng mất tác dụng khi d lớn.

Câu hỏi kiểm tra nhanh

1. Tính d_1, d_2 và d_∞ giữa $\mathbf{u} = [1,2,3]$ và $\mathbf{v} = [4,6,3]$.
2. Ba láng giềng của một điểm truy vấn có khoảng cách $1,2,2$ và nhãn lần lượt là A, B, B. Bỏ phiếu đa số và bỏ phiếu theo trọng số $w_i = 1/d_i^2$ cho kết quả gì?
3. Vì sao việc chọn k bằng độ chính xác trên tập huấn luyện luôn dẫn đến $k = 1$ (hoặc rất gần 1)?
4. Một bộ dữ liệu có 3 lớp, mỗi lớp 10 mẫu. Vì sao $k = 25$ là lựa chọn tồi, ngay cả khi kiểm định chéo cho độ chính xác cao ở một vài k khác?
5. Một đặc trưng được đo bằng mét (0 đến 2), một đặc trưng khác đo bằng gam (0 đến $2\,000$). Điều gì xảy ra với khoảng cách Euclid nếu không chuẩn hóa, và cần chuẩn hóa bằng thống kê của tập nào?
6. Vì sao k-NN suy giảm khi thêm nhiều đặc trưng nhiễu còn Softmax Regression thì ít hơn?

Gợi ý đáp án

1. Hiệu theo từng đặc trưng là $|1-4|=3$, $|2-6|=4$, $|3-3|=0$. Vậy $d_1 = 3+4+0=7$, $d_2 = \sqrt{9+16+0} = 5$, $d_\infty = \max(3,4,0) = 4$.

2. Bỏ phiếu đa số: B được 2 phiếu, A được 1 phiếu nên chọn B ($\hat{p}_B = 2/3$). Bỏ phiếu theo trọng số: $w = 1, 1/4, 1/4$ nên tổng trọng số A là 1, của B là $1/2$; $\hat{p}_A = 1/1{,}5 \approx 0{,}667$ nên chọn A. Hai cách cho kết quả ngược nhau vì láng giềng của A gần hơn rất nhiều.

3. Với $k = 1$, mỗi mẫu huấn luyện có chính nó là láng giềng gần nhất (khoảng cách 0), nên mô hình luôn dự đoán đúng nhãn của nó (trừ khi có mẫu trùng tọa độ khác nhãn). Độ chính xác huấn luyện vì thế gần 100% và không phản ánh khả năng tổng quát hóa. Cần dùng dữ liệu mô hình chưa thấy, tức kiểm định chéo.

4. Mỗi lớp chỉ có 10 mẫu nên với $k = 25$ mọi điểm truy vấn nhận ít nhất 15 láng giềng từ các lớp khác; lớp đúng có tối đa 10 phiếu trên 25. Vùng lân cận lúc này gần như là toàn bộ dữ liệu, nên số phiếu của ba lớp xấp xỉ nhau ở mọi nơi, biên quyết định mất đi và dự đoán gần như hằng số. Nói chung k phải nhỏ hơn kích thước lớp nhỏ nhất (như ví dụ $k = 5$ với 3 sinh viên mỗi lớp ở mục 2.4).

5. Chênh lệch theo gam lớn gấp hàng nghìn lần so với theo mét, nên khoảng cách Euclid gần như chỉ phụ thuộc đặc trưng đo bằng gam; đặc trưng đo bằng mét bị bỏ qua. Cần chuẩn hóa (z-score) bằng trung bình và độ lệch chuẩn của **tập huấn luyện**, và dùng đúng các giá trị đó cho điểm truy vấn.

6. k-NN coi mọi đặc trưng đóng góp như nhau vào khoảng cách, nên nhiều đặc trưng nhiễu làm các láng giềng được chọn gần như ngẫu nhiên và khoảng cách mất khả năng phân biệt. Softmax *học trọng số*, nên gán trọng số gần 0 cho các đặc trưng không liên quan (nhất là khi có điều chuẩn hóa), vì vậy ít bị ảnh hưởng hơn.

3. k-NN hồi quy và hồi quy địa phương

Mục 2 đã biến ý tưởng "nhìn các mẫu lân cận" thành một bộ phân loại hoàn chỉnh. Nhưng nhìn kỹ công thức cốt lõi $\widehat{\mathbf{P}} = \frac{1}{k}\mathbf{NY}$, ta nhận ra một điều: mỗi **cột** của $\mathbf{Y}$ là một vector gồm các số 0 và 1 (mẫu có thuộc lớp đó hay không), và phép nhân $\frac{1}{k}\mathbf{N}$ chỉ đơn giản là **lấy trung bình các giá trị của k láng giềng**. Nếu gọi $\mathbf{y}^{(c)}$ là cột thứ c của $\mathbf{Y}$ thì

$$\widehat{\mathbf{P}} = \frac{1}{k}\mathbf{NY} = \left[\; 1/k\,\mathbf{N}\,\mathbf{y}^{(1)} \quad 1/k\,\mathbf{N}\,\mathbf{y}^{(2)} \quad \cdots \quad 1/k\,\mathbf{N}\,\mathbf{y}^{(K)} \;\right].$$

Nói cách khác, k-NN phân loại chính là **k-NN hồi quy** áp dụng cho K biến mục tiêu nhận giá trị $0/1$, rồi chọn lớp có giá trị trung bình lớn nhất. Điều này gợi ý rằng ta có thể dùng đúng bộ máy đó khi biến mục tiêu là một số thực bất kỳ, ví dụ điểm thi, giá nhà hay nhiệt độ. Đó là chủ đề của mục 3. Ở bài Softmax, ta cũng đã đi theo con đường tương tự: từ hồi quy tuyến tính (mục tiêu liên tục) sang Logistic và Softmax (mục tiêu là lớp). Ở đây ta đi theo hướng ngược lại, từ phân loại phi tham số về hồi quy phi tham số, và nhân dịp đó nâng cấp công cụ: thay trung bình *đơn giản* bằng trung bình *có trọng số trơn* (hồi quy nhân), rồi thay cả "trung bình" bằng việc *khớp một đường thẳng cục bộ* (hồi quy địa phương).

Trong mục 3, ta trả lời các câu hỏi sau:

- Trung bình của k láng giềng cho biến mục tiêu liên tục trông thế nào, và có nhược điểm gì? (mục 3.1)
- Làm sao để đường dự đoán **trơn** hơn bằng trọng số liên tục theo khoảng cách? Hàm nhân và băng thông h là gì? (mục 3.2)
- Có thể thay phép "lấy trung bình" bằng việc khớp một **mô hình tuyến tính cục bộ** quanh từng điểm truy vấn không? (mục 3.3)
- Chọn băng thông h thế nào, và vì sao các phương pháp trung bình bị **thiên lệch ở biên**? (mục 3.4)
- Các phương pháp này so với hồi quy tuyến tính thế nào, đặc biệt khi có mẫu ngoại lai? (mục 3.5)
- Cuối cùng là ví dụ tính tay một chiều (mục 3.6) và cài đặt từ đầu bằng Python (mục 3.7).

Quy ước ký hiệu

Ta giữ các quy ước của mục 1 và 2: **chữ thường in đậm là vector**, **chữ hoa in đậm là ma trận**, vector đặc trưng là vector hàng, và đặc trưng không có cột 1 trừ khi có dấu ngã ($\tilde{\mathbf{x}}$, $\tilde{\mathbf{X}}$). Biến mục tiêu bây giờ là số thực.

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Vector mục tiêu huấn luyện	$\mathbf{y}$	$n \times 1$	$y_i \in \mathbb{R}$ là giá trị của mẫu thứ i
Dự đoán tại một điểm truy vấn	$\hat{y}_0$	vô hướng	Dự đoán cho $\mathbf{x}_0$
Vector dự đoán cho m điểm	$\hat{\mathbf{y}}$	$m \times 1$	Ứng với các hàng của $\mathbf{X}_0$
Hàm nhân	$\kappa(u)$	vô hướng	Hàm không âm, đối xứng, giảm theo $\lvert u \rvert$
Băng thông	h	vô hướng	Thang đo "độ rộng" của lân cận
Trọng số nhân	$v_i(\mathbf{x}_0)$	vô hướng	$v_i = \kappa(d(\mathbf{x}_0, \mathbf{x}_i)/h)$
Ma trận nhân	$\mathbf{G}$	$m \times n$	$G_{ji} = \kappa\big(d(\mathbf{x}_{0,j}, \mathbf{x}_i)/h\big)$
Ma trận đường chéo trọng số	$\mathbf{V}$	$n \times n$	$\mathbf{V} = \mathrm{diag}(v_1, \dots, v_n)$ cho một điểm truy vấn
Hệ số hồi quy cục bộ	$\mathbf{w}(\mathbf{x}_0)$	$(d+1) \times 1$	Nghiệm của bài toán bình phương tối thiểu có trọng số tại $\mathbf{x}_0$
Trọng số hiệu dụng	$s_i(\mathbf{x}_0)$	vô hướng	$\hat{y}_0 = \sum_i s_i(\mathbf{x}_0)\, y_i$ (là $w_i(\mathbf{x}_0)$ của mục 1.3, đổi tên để dành $\mathbf{w}$ cho hệ số hồi quy)
Ma trận làm trơn	$\mathbf{S}$	$n \times n$	$\hat{\mathbf{y}} = \mathbf{S}\mathbf{y}$ trên các mẫu huấn luyện (mục 3.4)

3.1. k-NN hồi quy: trung bình của k giá trị lân cận

Công thức

Với bài toán hồi quy, bước **đo** và **chọn** giữ nguyên như mục 2.1: tính khoảng cách tới mọi mẫu huấn luyện và lấy tập chỉ số $\mathcal{N}_k(\mathbf{x}_0)$ của k mẫu gần nhất. Chỉ bước **quyết định** thay đổi: thay vì bỏ phiếu, ta lấy **trung bình** giá trị mục tiêu của các láng giềng:

$$\hat{y}_0 = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x}_0)} y_i.$$

Với m điểm truy vấn, dùng đúng ma trận láng giềng $\mathbf{N}$ ($m \times n$, mỗi hàng có k số 1) của mục 2.2:

$$\hat{\mathbf{y}} = \frac{1}{k}\, \mathbf{N}\, \mathbf{y}, \qquad \hat{\mathbf{y}} \in \mathbb{R}^{m \times 1}.$$

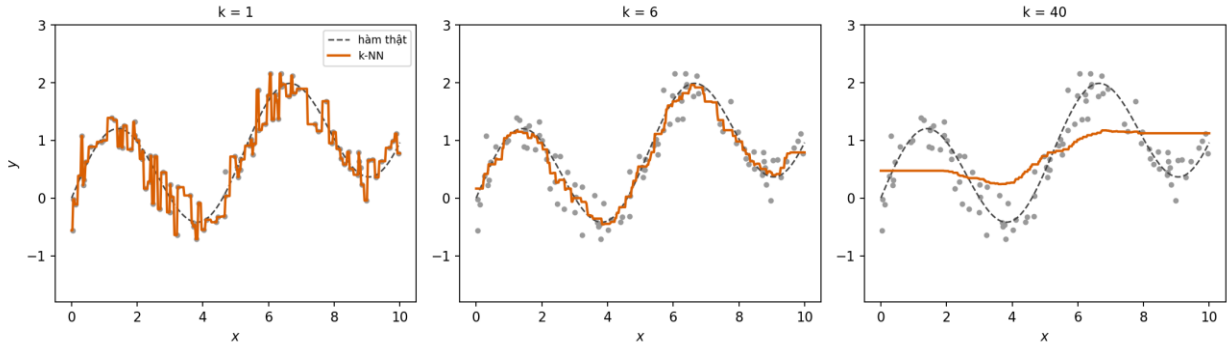
Đây là cùng một ma trận $\mathbf{N}$, chỉ khác là $\mathbf{Y}$ ($n \times K$) được thay bằng vector $\mathbf{y}$ ($n \times 1$). Nếu muốn trọng số theo khoảng cách, ta thay $\frac{1}{k}\mathbf{N}$ bằng $\mathbf{N}_w$ như ở mục 2.2. Công thức này cũng nằm trong dạng chung $\hat{y}_0 = \sum_i s_i(\mathbf{x}_0)\, y_i$ của mục 1.3 với $s_i = 1/k$ nếu $i \in \mathcal{N}_k$ và $s_i = 0$ nếu không.

Thí nghiệm: một hàm sin có nhiễu

Để quan sát bằng mắt, ta dùng một bài toán một chiều có hàm thật *biết trước*:

$$y = f(x) + \varepsilon, \qquad f(x) = \sin(1{,}2\,x) + 0{,}15\,x, \qquad \varepsilon \sim \mathcal{N}(0;\, 0, 3^2).$$

Ta sinh $n = 100$ điểm huấn luyện với x phân bố đều trong $[0{,}10]$ và 400 điểm kiểm tra theo cùng cách. Vì nhiễu có độ lệch chuẩn 0,3, **sai số căn quân phương (RMSE) trên điểm kiểm tra không thể xuống thấp hơn nhiều so với** 0,3, đây là "mức sàn" của bài toán. Hình 8 vẽ k-NN hồi quy với ba giá trị k.



Hình 8. k-NN hồi quy trên dữ liệu một chiều (chấm xám), hàm thật là đường nét đứt. Trái: $k = 1$, đường dự đoán đi qua mọi điểm và bám cả nhiễu. Giữa: $k = 6$, bám sát hàm thật. Phải: $k = 40$, đường dự đoán quá phẳng, mất các đỉnh và đáy.

k	RMSE huấn luyện	RMSE kiểm tra	Nhận xét
1	0,000	0,398	Học thuộc: bám nhiễu (phương sai cao)
6	0,262	0,311	Gần mức sàn 0,30
40	0,573	0,617	Quá mượt: bỏ lỡ cấu trúc (thiên lệch cao)

Hình ảnh và bảng này lặp lại đúng câu chuyện của mục 2.3: k nhỏ thì phương sai cao, k lớn thì thiên lệch cao, và có một vùng ở giữa vừa đủ. Với hồi quy, sự đánh đổi còn hiện ra rõ hơn vì ta so sánh được trực tiếp với hàm thật.

Hạn chế: đường dự đoán "bậc thang"

Nhìn kỹ Hình 8 (giữa và trái), đường dự đoán của k-NN là **hàm hằng từng khúc** (piecewise constant) có những bước nhảy đột ngột. Lý do: khi $\mathbf{x}_0$ dịch chuyển một chút, tập $\mathcal{N}_k(\mathbf{x}_0)$ thay đổi *rời rạc* (một láng giềng bị thay bằng một láng giềng khác) và trung bình nhảy một khoảng $\frac{1}{k}(y_{\text{mới}} - y_{\text{cũ}})$. Các bước nhảy đó không phản ánh cấu trúc thật mà chỉ là hệ quả của cách đặt trọng số "có hoặc không" ($\frac{1}{k}$ hoặc 0). Cách tự nhiên để khắc phục là cho trọng số giảm **liên tục** theo khoảng cách, đúng ý tưởng của mục 3.2.

3.2. Hồi quy nhân Nadaraya–Watson: trung bình có trọng số, hàm nhân và băng thông h

Từ trọng số "có hoặc không" đến trọng số liên tục

Thay vì chỉ lấy k láng giềng với trọng số bằng nhau, ta cho **mọi** mẫu tham gia, nhưng với trọng số v_i giảm dần theo khoảng cách tới $\mathbf{x}_0$. Chuẩn hóa để tổng bằng 1, ta được **ước lượng Nadaraya–Watson**:

$$\hat{y}_0 = \frac{\sum_{i=1}^{n} \kappa\left(\frac{d(\mathbf{x}_0, \mathbf{x}_i)}{h}\right) y_i}{\sum_{i=1}^{n} \kappa\left(\frac{d(\mathbf{x}_0, \mathbf{x}_i)}{h}\right)} = \sum_{i=1}^{n} s_i(\mathbf{x}_0)\, y_i, \qquad s_i(\mathbf{x}_0) = \frac{v_i}{\sum_{l=1}^{n} v_l}.$$

Trong đó κ là **hàm nhân** (kernel) và $h > 0$ là **băng thông** (bandwidth). Đây chính là dạng "trung bình có trọng số" của mục 1.3, với các trọng số không âm có tổng bằng 1. Với m điểm truy vấn, ta gom các trọng số vào ma trận nhân $\mathbf{G}$ $(m \times n)$ và chuẩn hóa từng hàng:

$$\hat{\mathbf{y}} = \operatorname{diag}(\mathbf{G}\mathbf{1}_n)^{-1}\, \mathbf{G}\, \mathbf{y}, \qquad G_{ji} = \kappa\left(\frac{d(\mathbf{x}_{0,j}, \mathbf{x}_i)}{h}\right).$$

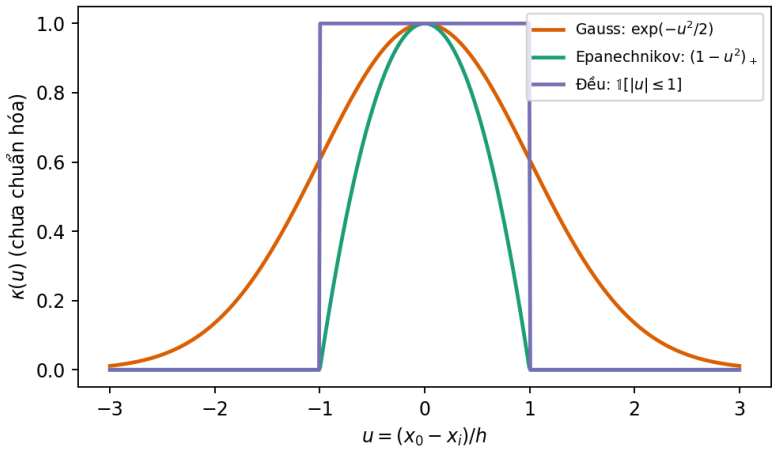
Ở đây $\mathbf{G}\mathbf{1}_n$ là vector các tổng hàng của $\mathbf{G}$ (kích thước $m \times 1$), và $\operatorname{diag}(\cdot)^{-1}$ là ma trận đường chéo dùng để chia từng hàng cho tổng của nó. So với k-NN hồi quy $\hat{\mathbf{y}} = \frac{1}{k}\mathbf{N}\mathbf{y}$, ta chỉ thay ma trận $0/1$ bằng ma trận nhân $\mathbf{G}$ có các phần tử liên tục, và hệ số $\frac{1}{k}$ bằng phép chuẩn hóa từng hàng.

Các hàm nhân thông dụng

Hàm nhân là một hàm $\kappa(u)$ không âm, đối xứng quanh 0 và giảm khi $|u|$ tăng, với $u = d/h$ là khoảng cách đã chia cho băng thông. Ba lựa chọn thường gặp (Hình 7):

$$\kappa_{\text{Gauss}}(u) = e^{-u^2/2}, \qquad \kappa_{\text{Epan}}(u) = (1 - u^2)_+, \qquad \kappa_{\text{đều}}(u) = \mathbb{I}[\,|u| \le 1\,],$$

với $(a)_+ = \max(a, 0)$ và $\mathbb{I}[\cdot]$ là hàm chỉ thị (bằng 1 nếu điều kiện đúng, ngược lại bằng 0). Các hằng số chuẩn hóa không cần thiết vì chúng triệt tiêu giữa tử và mẫu của công thức Nadaraya–Watson.



Hình 7. Ba hàm nhân thông dụng. Hàm Gauss không bao giờ bằng 0 *nên mọi mẫu đều đóng góp (dù rất ít khi ở xa); hàm Epanechnikov và hàm đều có giá đỡ compact (bằng* 0 *khi* $|u| > 1$*), nên chỉ các mẫu trong bán kính* h *mới tham gia.*

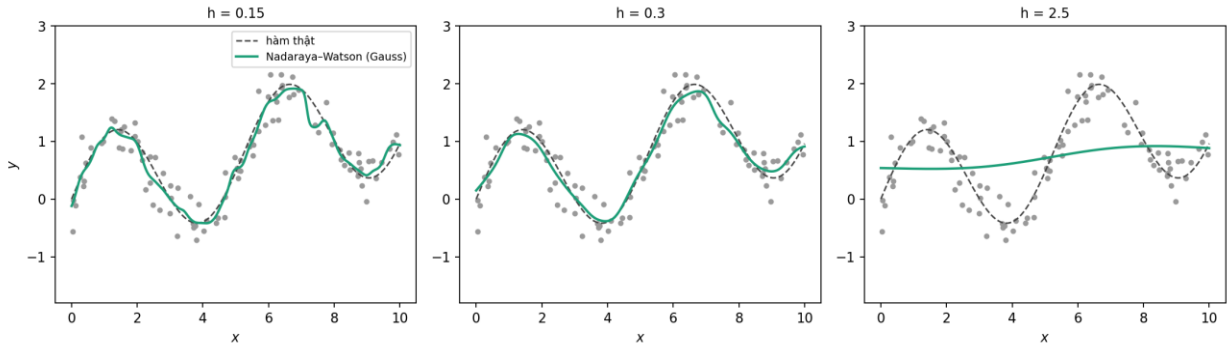
- **Nhân Gauss** cho đường dự đoán rất trơn (khả vi vô hạn), nhưng mọi mẫu đều có trọng số dương nên mỗi lần dự đoán đụng tới cả n mẫu.
- **Nhân Epanechnikov** và **nhân đều** chỉ dùng các mẫu trong bán kính h, nhanh hơn khi dùng cấu trúc tìm kiếm không gian, nhưng nhân đều cho lại đường dự đoán hằng từng khúc.
- Trong thực hành, lựa chọn hàm nhân ảnh hưởng ít hơn nhiều so với lựa chọn băng thông h. Với đặc trưng nhiều chiều, ta đặt $d(\mathbf{x}_0, \mathbf{x}_i)$ là khoảng cách Euclid *sau khi chuẩn hóa đặc trưng* (mục 2.5).

Quan hệ với k-NN: băng thông cố định và băng thông thích nghi

Hai họ phương pháp này thực chất cùng một ý tưởng, chỉ khác cách quy định "lân cận rộng bao nhiêu":

- Trong hồi quy nhân, độ rộng h **cố định** cho mọi điểm truy vấn. Với nhân đều, nó là trung bình của mọi mẫu nằm trong bán kính h, đúng như ý tưởng "quả cầu bán kính r" ở mục 1.2.
- Trong k-NN, số láng giềng k cố định, nên bán kính lân cận **tự điều chỉnh**: $h(\mathbf{x}_0)$ chính là khoảng cách từ $\mathbf{x}_0$ tới láng giềng thứ k. Vùng dữ liệu thưa thì lân cận rộng ra, vùng dày thì thu lại.

Mỗi cách có ưu điểm riêng: băng thông cố định cho đường dự đoán trơn nhưng vùng thưa có thể có rất ít mẫu (thậm chí mẫu số của công thức gần bằng 0), còn k-NN luôn có đúng k mẫu để lấy trung bình nhưng đường dự đoán "bậc thang". Hình 9 cho thấy tác động của h trên cùng bộ dữ liệu.



Hình 9. Hồi quy nhân Nadaraya–Watson (nhân Gauss) với ba băng thông. $h = 0{,}15$: đường dự đoán còn gợn theo nhiễu. $h = 0{,}3$ (giá trị chọn bằng kiểm định chéo ở mục 3.4): bám sát hàm thật. $h = 2{,}5$: quá mượt, gần như một đường cong thoải.

Băng thông h	RMSE huấn luyện	RMSE kiểm tra	Nhận xét
0,15	0,233	0,313	Hơi bám nhiễu
0,3	0,259	0,311	Cân bằng
2,5	0,652	0,709	Quá mượt, thiên lệch lớn

Hai cực của băng thông có thể hiểu trực tiếp từ công thức. Khi $h \to \infty$, mọi trọng số $\kappa(d/h) \to \kappa(0)$ như nhau, nên $\hat{y}_0 \to \bar{y}$ (trung bình toàn bộ dữ liệu), một đường nằm ngang. Khi $h \to 0$, chỉ mẫu gần $\mathbf{x}_0$ nhất còn trọng số đáng kể, và $\hat{y}_0 \to y_{\text{gần nhất}}$, tức k-NN với $k = 1$.

Một góc nhìn: ước lượng kỳ vọng có điều kiện

Hồi quy nhân là ước lượng của hàm hồi quy $m(\mathbf{x}) = \mathbb{E}[\,y \mid \mathbf{x}\,]$, tức giá trị trung bình của y khi biết $\mathbf{x}$. Theo định nghĩa kỳ vọng, $m(\mathbf{x}) = \int y\, p(y \mid \mathbf{x})\, dy = \frac{\int y\, p(\mathbf{x}, y)\, dy}{p(\mathbf{x})}$, tỉ số giữa một tích phân của mật độ đồng thời và mật độ biên. Nếu ta thay các mật độ này bằng các **ước lượng mật độ nhân** (chủ đề của mục 4), công thức Nadaraya–Watson xuất hiện đúng như một phép tính đại số. Vì vậy hồi quy nhân và ước lượng mật độ là hai mặt của cùng một ý tưởng, và mục 4 sẽ tiếp tục từ chỗ này. Tương tự, dùng nhân Gauss với toàn bộ n mẫu và ma trận $\mathbf{Y}$ thay cho $\mathbf{y}$ cho ta một **bộ phân loại**: $\widehat{\mathbf{P}} = \text{diag}(\mathbf{G}\mathbf{1}_n)^{-1}\mathbf{G}\mathbf{Y}$, một cách bỏ phiếu theo trọng số của mọi mẫu thay cho k láng giềng như ở mục 2.2.

3.3. Hồi quy tuyến tính trọng số địa phương (LOESS)

Nadaraya–Watson là khớp một hằng số cục bộ

Có một cách diễn đạt khác cho hồi quy nhân, mở ra cách cải tiến. Xét bài toán tìm **một hằng số** c khớp tốt nhất với các giá trị y_i quanh $\mathbf{x}_0$, trong đó mẫu i có độ quan trọng $v_i = \kappa(d_i/h)$:

$$\hat{c} = \underset{c}{\text{argmin}} \sum_{i=1}^{n} v_i\ (y_i - c)^2.$$

Đạo hàm theo c và cho bằng 0 cho $\sum_i v_i\,(y_i - c) = 0$, tức $\hat{c} = \sum_i v_i\,y_i / \sum_i v_i$, **chính là ước lượng Nadaraya–Watson**. Vậy Nadaraya–Watson xấp xỉ hàm hồi quy *trong lân cận của* $\mathbf{x}_0$ *bằng một hằng số*. Điều đó hợp lý nếu hàm gần như phẳng trong lân cận, nhưng khi hàm có độ dốc rõ rệt (như ở biên của dữ liệu, mục 3.4), một hằng số là xấp xỉ thô.

Khớp một hàm tuyến tính cục bộ

Ý tưởng của **hồi quy tuyến tính trọng số địa phương** (locally weighted linear regression, còn gọi là LOESS hay LOWESS khi dùng một họ hàm nhân cụ thể) là nâng bậc: tại mỗi điểm truy vấn $\mathbf{x}_0$, ta khớp **một hàm tuyến tính $\tilde{\mathbf{x}}\mathbf{w}$** thay vì một hằng số, nhưng *chỉ dùng các mẫu quanh* $\mathbf{x}_0$, mỗi mẫu có độ quan trọng v_i:

$$\mathbf{w}(\mathbf{x}_0) = \underset{\mathbf{w}}{\operatorname{argmin}}\, J_{\mathbf{x}_0}(\mathbf{w}), \qquad J_{\mathbf{x}_0}(\mathbf{w}) = \sum_{i=1}^{n} v_i\;(y_i - \tilde{\mathbf{x}}_i\mathbf{w})^2 = \big(\mathbf{y} - \tilde{\mathbf{X}}\mathbf{w}\big)^{\top}\mathbf{V}\,\big(\mathbf{y} - \tilde{\mathbf{X}}\mathbf{w}\big).$$

Dùng dấu ngã để nhắc rằng $\tilde{\mathbf{x}}_i = [\,1,\ \mathbf{x}_i\,]$ đã thêm cột 1 (như bài Softmax) và $\tilde{\mathbf{X}}$ có kích thước $n \times (d+1)$. Đạo hàm theo $\mathbf{w}$ và cho bằng $\mathbf{0}$ cho **phương trình chuẩn có trọng số**:

$$\tilde{\mathbf{X}}^{\top}\mathbf{V}\tilde{\mathbf{X}}\,\mathbf{w} = \tilde{\mathbf{X}}^{\top}\mathbf{V}\,\mathbf{y} \quad \Rightarrow \quad \mathbf{w}(\mathbf{x}_0) = (\tilde{\mathbf{X}}^{\top}\mathbf{V}\tilde{\mathbf{X}})^{-1}\tilde{\mathbf{X}}^{\top}\mathbf{V}\,\mathbf{y}.$$

Dự đoán tại $\mathbf{x}_0$ là giá trị của đường thẳng cục bộ đó tại chính $\mathbf{x}_0$:

$$\hat{y}_0 = \tilde{\mathbf{x}}_0\,\mathbf{w}(\mathbf{x}_0), \qquad \tilde{\mathbf{x}}_0 = [\,1,\ \mathbf{x}_0\,].$$

Công thức này rất quen thuộc: khi $\mathbf{V} = \mathbf{I}$ (mọi mẫu trọng số 1) nó trở thành $\big(\tilde{\mathbf{X}}^{\top}\tilde{\mathbf{X}}\big)^{-1}\tilde{\mathbf{X}}^{\top}\mathbf{y}$, đúng nghiệm hồi quy tuyến tính toàn cục đã học. Khác biệt là ở đây:

- ma trận trọng số $\mathbf{V}$ **phụ thuộc vào điểm truy vấn** $\mathbf{x}_0$ (mẫu gần $\mathbf{x}_0$ có v_i lớn, mẫu xa có $v_i \approx 0$);
- ta giải phương trình chuẩn **một lần cho mỗi điểm truy vấn**, thay vì một lần cho toàn bộ dữ liệu. Vì vậy ta nhận được vô số đường thẳng nhỏ, mỗi đường chỉ có hiệu lực quanh một điểm. Lưu ý quan trọng: **không có một vector w chung** cho cả mô hình; $\mathbf{w}(\mathbf{x}_0)$ là hàm của $\mathbf{x}_0$, và mô hình vẫn là phi tham số, vì để dự đoán ta phải lưu toàn bộ $(\mathbf{X}, \mathbf{y})$.

Nếu $\tilde{\mathbf{X}}^{\top}\mathbf{V}\tilde{\mathbf{X}}$ gần suy biến (khi lân cận quá ít mẫu, hoặc các mẫu thẳng hàng), ta cộng thêm một số nhỏ λ vào đường chéo: $\big(\tilde{\mathbf{X}}^{\top}\mathbf{V}\tilde{\mathbf{X}} + \lambda\mathbf{I}\big)^{-1}$, giống điều chuẩn hóa L2 ở bài Softmax.

Trọng số hiệu dụng và bậc của đa thức cục bộ

Thế $\mathbf{w}(\mathbf{x}_0)$ vào $\hat{y}_0$, ta thấy dự đoán vẫn có dạng trung bình có trọng số của y_i:

$$\hat{y}_0 = \underbrace{\tilde{\mathbf{x}}_0(\tilde{\mathbf{X}}^{\top}\mathbf{V}\tilde{\mathbf{X}})^{-1}\tilde{\mathbf{X}}^{\top}\mathbf{V}}_{\mathbf{s}(\mathbf{x}_0),\ 1\times n}\,\mathbf{y} = \sum_{i=1}^{n} s_i\,(\mathbf{x}_0)\,y_i, \qquad \sum_{i=1}^{n} s_i\,(\mathbf{x}_0) = 1.$$

Vậy LOESS cũng nằm trong dạng chung của mục 1.3; sự khác biệt so với Nadaraya–Watson nằm ở trọng số hiệu dụng $s_i(\mathbf{x}_0)$: chúng vẫn có tổng bằng 1 nhưng **có thể âm** (xem ví dụ ở mục 3.6). Nói theo bậc của đa thức cục bộ: khớp hằng số (bậc 0) cho Nadaraya–Watson, khớp đường thẳng (bậc 1) cho LOESS, khớp parabol (bậc 2) cho đa thức cục bộ bậc hai, và cứ thế. Bậc cao hơn giảm thiên lệch nhưng tăng phương sai và chi phí. Trong thực hành, bậc 1 là lựa chọn mặc định.

Trong thuật toán LOESS gốc, độ rộng lân cận được quy định bằng "span" (tỉ lệ số mẫu gần nhất được dùng, tức là băng thông thích nghi như k-NN) và nhân là hàm tricube $(1 - |u|^3)^3$ trong $|u| \le 1$. Ta giữ cách viết với hàm nhân và h để thống nhất với mục 3.2.

3.4. Chọn băng thông và thiên lệch ở biên

Chọn h bằng kiểm định chéo bỏ-một-ra

Giống k ở mục 2.4, băng thông h là một nút chỉnh độ phức tạp (h nhỏ thì phương sai cao, h lớn thì thiên lệch cao), và ta chọn nó bằng kiểm định chéo. Với các phương pháp trong dạng trung bình có trọng số, có một điều đặc biệt thuận lợi: **kiểm định chéo bỏ-một-ra không cần huấn luyện lại n lần**. Nhìn các dự đoán tại chính các mẫu huấn luyện: vì mỗi dự đoán là tổ hợp tuyến tính của $\mathbf{y}$, ta có

$$\hat{\mathbf{y}} = \mathbf{S}\,\mathbf{y}, \qquad \mathbf{S} \in \mathbb{R}^{n \times n}, \quad S_{ji} = s_i(\mathbf{x}_j),$$

trong đó $\mathbf{S}$ là **ma trận làm trơn** (hàng j gồm các trọng số hiệu dụng tại mẫu $\mathbf{x}_j$). Sai số bỏ-một-ra có công thức đóng:

$$\mathrm{LOOCV}(h) = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{y_i - \hat{y}_i}{1 - S_{ii}}\right)^2.$$

Vế phải chỉ cần một lần dựng $\mathbf{S}$ và đường chéo của nó. Ta đã kiểm tra công thức bằng cách so với việc thực sự bỏ từng mẫu và huấn luyện lại ($h = 0{,}6$): với Nadaraya–Watson, hai cách cho $0{,}114507$ (trùng nhau đến các chữ số thập phân cuối) và với LOESS cho $0{,}098554$ (cũng trùng). Với k-NN, công thức đóng này không áp dụng trực tiếp, và ta bỏ từng mẫu ra theo đúng định nghĩa.

Trên dữ liệu sin ở trên, ta quét h từ $0{,}1$ đến $3{,}0$ (Hình 10a). Kết quả:

Phương pháp	Tham số tốt nhất	Sai số bỏ-một-ra (MSE)	RMSE kiểm tra
k-NN hồi quy	$k = 6$	0,0935	0,311
Nadaraya–Watson (Gauss)	$h = 0{,}30$	0,0890	0,311
LOESS (Gauss)	$h = 0{,}35$	0,0829	0,312

Cả ba đạt RMSE kiểm tra xấp xỉ $0{,}31$, sát mức sàn $0{,}30$. Sai số bỏ-một-ra của LOESS thấp hơn một chút, nhưng chênh lệch này chưa lớn so với dao động ngẫu nhiên của một bộ dữ liệu. Đường cong LOOCV của cả hai phương pháp đều có hình chữ "U" lệch: giảm mạnh khi h tăng từ $0{,}1$, đạt đáy quanh $h \approx 0{,}3$ và tăng nhanh sau $h \approx 0{,}5$ vì hàm sin có chu kỳ chỉ khoảng 5 đơn vị, nên băng thông quá lớn sẽ làm phẳng các đỉnh.

Thiên lệch và phương sai theo h (mức trực giác)

Với một chiều và điểm x nằm *sâu trong miền dữ liệu*, phân tích điển hình cho hồi quy nhân cho các xấp xỉ sau, với $p(x)$ là mật độ của x và $\mu_2 = \int u^2 \kappa(u)\, du$, $R(\kappa) = \int \kappa(u)^2\, du$:

$$\text{Thiên lệch}[\hat{y}(x)] \approx \frac{h^2}{2}\,\mu_2 \left(f''(x) + \frac{2f'(x)\,p'(x)}{p(x)} \right), \qquad \text{Phương sai}[\hat{y}(x)] \approx \frac{\sigma^2\, R(\kappa)}{n\,h\,p(x)}.$$

Thiên lệch tăng như h^2 và phụ thuộc độ cong f'' của hàm thật (băng thông lớn làm phẳng các chỗ cong), còn phương sai giảm như $1/(nh)$ (băng thông lớn lấy trung bình trên nhiều mẫu hơn). Cân bằng hai số hạng cho băng thông tối ưu cỡ $h \propto n^{-1/5}$. Với d đặc trưng, số mũ trở thành $n^{-1/(d+4)}$ và sai số tối ưu giảm như $n^{-4/(d+4)}$, chậm đi rất nhiều khi d tăng, đối chiếu với tốc độ n^{-1} của mô hình tham số đúng dạng hàm. Đây là cách phát biểu định lượng cho lời nguyền chiều ở mục 2.5, và ta sẽ quay lại ở mục 6.3.

Thiên lệch ở biên

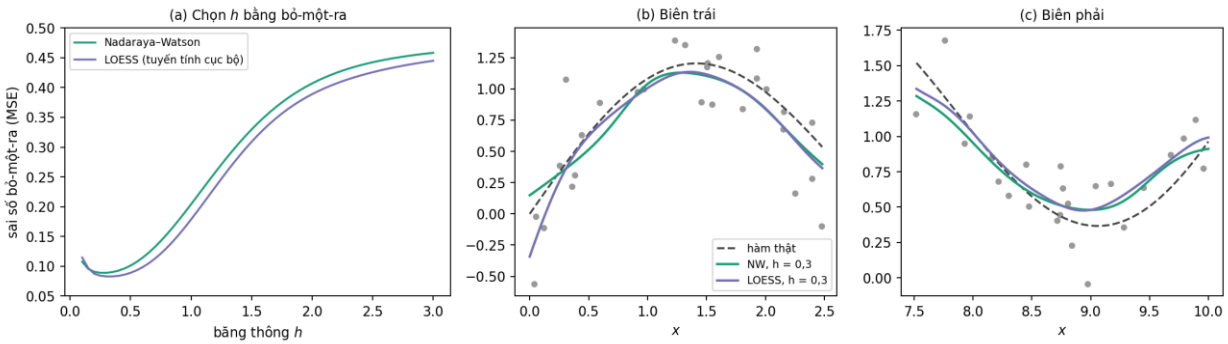
Công thức thiên lệch ở trên chỉ đúng cho điểm *sâu trong miền dữ liệu*. Ở **biên** (gần giá trị nhỏ nhất hoặc lớn nhất của x), lân cận của x_0 chỉ có mẫu về *một phía*. Nếu hàm thật đang đi lên thì trung bình các mẫu về phía bên phải sẽ cao hơn $f(x_0)$: ước lượng bị lệch lên. Thiên lệch này cỡ $h\,f'(x_0)$ (bậc nhất theo h), lớn hơn nhiều so với h^2 ở phần trong. Nadaraya–Watson, vốn chỉ khớp một hằng số, không có cách nào "biết" hàm đang có độ dốc.

LOESS khớp một *đường thẳng*, nên nó nhận ra độ dốc và nội suy theo đường thẳng đó thay vì chỉ lấy trung bình. Về mặt toán học, LOESS tái tạo chính xác mọi hàm tuyến tính, nên thiên lệch bậc nhất ở biên biến mất và chỉ còn thiên lệch bậc hai như ở phần trong. Để đo hiệu ứng này, ta lặp lại 300 lần: mỗi lần sinh một bộ 100 điểm mới, ước lượng tại $x = 0$ (biên trái), $x = 5$ (giữa) và $x = 10$ (biên phải), rồi tính thiên lệch và độ lệch chuẩn của ước lượng. Cả hai phương pháp dùng cùng băng thông $h = 0{,}3$:

Vị trí x	$f(x)$	**Phương pháp**	**Thiên lệch**	**Độ lệch chuẩn**
0 (biên trái)	0,000	Nadaraya–Watson	$+0{,}349$	0,178
		LOESS	$+0{,}045$	0,299
5 (giữa)	0,471	Nadaraya–Watson	$+0{,}013$	0,144
		LOESS	$+0{,}010$	0,107
10 (biên phải)	0,963	Nadaraya–Watson	$-0{,}231$	0,160
		LOESS	$-0{,}025$	0,306

Kết quả khẳng định cả hai phần: ở **giữa**, hai phương pháp đều có thiên lệch gần như bằng 0. Ở **biên**, Nadaraya–Watson lệch tới $+0{,}35$ và $-0{,}23$ (so với mức nhiễu 0,3 là rất lớn), còn LOESS giảm thiên lệch xuống còn 0,045 và 0,025. Nhưng LOESS phải trả bằng **phương sai ở biên cao hơn** (độ lệch chuẩn 0,30 so với 0,18): khi ngoại suy một đường thẳng ra mép dữ liệu, ta dựa vào ít mẫu và về một phía, nên ước lượng dao động nhiều hơn. Lần nữa, ta thấy đánh đổi giữa thiên lệch và phương sai, lần này ở cấp độ chọn bậc của đa thức cục bộ. Hình 10b và 10c vẽ đường

khớp trên *một* bộ dữ liệu cụ thể ở hai biên; vì bộ dữ liệu đó có nhiễu, chênh lệch giữa hai phương pháp ở đây không rõ bằng bảng thống kê trên 300 lần lặp.



Hình 10. (a) Sai số bỏ-một-ra theo băng thông h của Nadaraya–Watson và LOESS; cả hai đạt đáy quanh $h \approx 0{,}3$. (b, c) Đường khớp của hai phương pháp với $h = 0{,}3$ trên một bộ dữ liệu ở hai biên của miền. Đường nét đứt là hàm thật.

3.5. So sánh với hồi quy tuyến tính và với mẫu ngoại lai

Hồi quy tuyến tính toàn cục và đa thức

Ta so các phương pháp trên cùng bộ dữ liệu, với RMSE trên 400 điểm kiểm tra (mức sàn là 0,30):

Mô hình	Loại	RMSE kiểm tra
Hồi quy tuyến tính $y = \tilde{\mathbf{x}}\mathbf{w}$	Tham số, 2 tham số	0,738
Đa thức bậc 3	Tham số, 4 tham số	0,650
Đa thức bậc 7	Tham số, 8 tham số	0,298
k-NN hồi quy ($k = 6$)	Phi tham số	0,311
Nadaraya–Watson ($h = 0{,}3$)	Phi tham số	0,311
LOESS ($h = 0{,}35$)	Phi tham số	0,312

Có ba bài học.

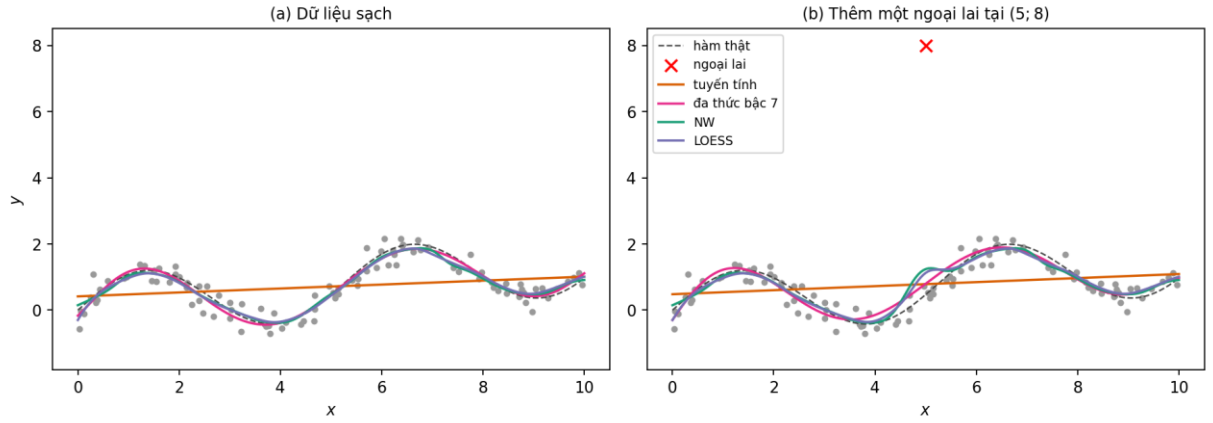
- **Hồi quy tuyến tính thất bại**, đúng như mục 1.1: một đường thẳng không thể mô tả hàm có hai chu kỳ dao động, và thiên lệch là nguyên nhân chính (RMSE 0,74, gấp hơn hai lần mức sàn).
- **Phương pháp phi tham số tiến sát mức sàn mà không cần biết hàm sin.** Ta không phải đoán dạng hàm; chỉ cần chọn một siêu tham số (k hoặc h) bằng kiểm định chéo.
- **Mô hình tham số cũng có thể thắng, nếu dạng hàm đủ linh hoạt.** Đa thức bậc 7 đạt 0,298, thậm chí nhỉnh hơn. Nhưng để biết cần *bậc* 7 thì ta đã phải thử, và đa

thức bậc cao có những rủi ro riêng (dao động mạnh ở biên, nhạy với từng mẫu). Điều k-NN và hồi quy nhân mang lại là tránh được việc chọn dạng hàm, đúng như mục 1 đã nêu, chứ không phải lúc nào cũng cho sai số thấp hơn.

Ảnh hưởng của một mẫu ngoại lai

Giờ ta thêm **một** điểm ngoại lai tại $(x, y) = (5; 8)$, cách rất xa đường cong thật ($f(5) \approx 0{,}47$), rồi huấn luyện lại và xem đường khớp thay đổi bao nhiêu (Hình 11). Bảng sau cho thay đổi lớn nhất của đường khớp trên $[0{,}10]$ và thay đổi tại ba vị trí:

Mô hình	**Thay đổi lớn nhất**	**Tại $x = 1$**	**Tại $x = 5$**	**Tại $x = 9$**
Hồi quy tuyến tính	0,079	0,067	0,072	0,078
Đa thức bậc 7	0,325	0,061	0,323	0,057
k-NN hồi quy ($k = 6$)	1,388	0,000	1,222	0,000
Nadaraya–Watson ($h = 0{,}3$)	0,768	0,000	0,757	0,000
LOESS ($h = 0{,}35$)	0,687	0,000	0,687	0,000



Hình 11. (a) Các đường khớp trên dữ liệu sạch. (b) Sau khi thêm một ngoại lai tại $(5; 8)$ *(dấu* $\times$ *đỏ): đường của các phương pháp địa phương chỉ bị kéo lên trong một khoảng hẹp quanh* $x = 5$*, còn các phần khác của đường giữ nguyên.*

Hai nhóm mô hình có hành vi khác hẳn. Ở các mô hình **toàn cục** (tuyến tính, đa thức), một điểm ngoại lai làm *toàn bộ* đường khớp dịch chuyển, kể cả ở nơi rất xa điểm đó ($x = 1$ và $x = 9$ đều thay đổi, hệ số góc của hồi quy tuyến tính đổi từ $0{,}0596$ sang $0{,}0610$ và hệ số chặn từ $0{,}415$ sang $0{,}481$). Ở các mô hình **địa phương**, ảnh hưởng *mạnh hơn nhiều* tại chỗ (k-NN thay đổi $1{,}2$ ngay tại $x = 5$) nhưng **bằng đúng** 0 **ở xa** (tại $x = 1$ và $x = 9$ thay đổi dưới 10^{-7} với Gauss, và đúng 0 với k-NN vì ngoại lai không nằm trong 6 láng giềng). Đây là tính chất **ảnh hưởng cục bộ** mà mục 6.2 sẽ phân tích kỹ hơn. Nó vừa là điểm mạnh (một lỗi nhập liệu chỉ làm hỏng dự đoán trong vùng nhỏ, và các phần khác của hàm không bị ô nhiễm) vừa là cảnh báo (một ngoại lai *không bị làm yếu* ngay tại vị trí của nó: dự đoán gần $x = 5$ bị kéo lên rất mạnh). Để chống ngoại lai trong vùng lân cận, người ta thay trung bình bằng **trung vị** hoặc dùng LOESS bền vững, tính lại trọng số lặp để giảm ảnh hưởng của các mẫu có phần dư lớn.

Ngoài miền dữ liệu

Một điểm khác biệt nữa so với mô hình tham số: các phương pháp địa phương **không ngoại suy được**. Với $x = 11$ (ngoài miền huấn luyện $[0{,}04;\ 9{,}96]$), hàm thật cho $f(11) \approx 2{,}24$ nhưng k-NN dự đoán 0,79, Nadaraya–Watson 0,89 và LOESS $-0{,}04$. Ở $x = -1$ (hàm thật $-1{,}08$), k-NN cho 0,16, Nadaraya–Watson $-0{,}24$ và LOESS cho $-3{,}66$, vì đường thẳng cục bộ bị kéo ra ngoài và dựa trên rất ít mẫu. Nguyên tắc: *chỉ dùng mô hình địa phương để dự đoán ở nơi có dữ liệu xung quanh*.

3.6. Ví dụ tính tay một chiều

Ta quay lại bối cảnh sinh viên: x là số giờ tự học mỗi tuần, y là điểm thi. Có 6 sinh viên:

Sinh viên	**1**	**2**	**3**	**4**	**5**	**6**
x_i (giờ tự học)	1	2	3	4	5	6
y_i (điểm thi)	4,0	5,2	5,8	7,1	8,0	9,1

Ta dự đoán điểm cho hai sinh viên mới: $x_0 = 3{,}7$ (nằm giữa dữ liệu) và $x_0 = 6$ (nằm ở **biên**, trùng sinh viên cuối cùng). Dùng $k = 3$ cho k-NN và nhân Gauss với $h = 1$ cho hai phương pháp còn lại. Để so sánh, hồi quy tuyến tính toàn cục trên 6 sinh viên là $\hat{y} = 3{,}0133 + 1{,}0057\,x$.

Điểm truy vấn $x_0 = 3{,}7$ (ở giữa)

k-NN ($k = 3$). Khoảng cách tới $x_i = 1, \ldots, 6$ là 2,7; 1,7; 0,7; 0,3; 1,3; 2,3. Ba giá trị nhỏ nhất ứng với sinh viên 4, 3, 5 nên

$$\hat{y}_0 = \frac{7{,}1 + 5{,}8 + 8{,}0}{3} = 6{,}967.$$

Nadaraya–Watson. Trọng số nhân $v_i = e^{-d_i^2/2}$ với d_i ở trên:

$$\mathbf{v} = [\,0{,}0261,\ \ 0{,}2357,\ \ 0{,}7827,\ \ 0{,}9560,\ \ 0{,}4296,\ \ 0{,}0710\,], \qquad \sum_i v_i = 2{,}5011,$$

$$\hat{y}_0 = \frac{\sum_i v_i\, y_i}{\sum_i v_i} = \frac{16{,}7402}{2{,}5011} = 6{,}693.$$

LOESS. Ta dựng $\tilde{\mathbf{X}}$ (cột 1 và cột x) và $\mathbf{V} = \mathrm{diag}(\mathbf{v})$, rồi tính hai ma trận của phương trình chuẩn:

$$\tilde{\mathbf{X}}^{\top}\mathbf{V}\tilde{\mathbf{X}} = \begin{pmatrix} \sum v_i & \sum v_i x_i \\ \sum v_i x_i & \sum v_i x_i^2 \end{pmatrix} = \begin{pmatrix} 2{,}5011 & 9{,}2435 \\ 9{,}2435 & 36{,}6045 \end{pmatrix}, \qquad \tilde{\mathbf{X}}^{\top}\mathbf{V}\mathbf{y} = \begin{pmatrix} 16{,}7402 \\ 64{,}3848 \end{pmatrix}.$$

Nghịch đảo của ma trận 2×2 rồi nhân với vế phải:

$$\mathbf{w}(\mathbf{x}_0) = \begin{pmatrix} 5{,}9911 & -1{,}5129 \\ -1{,}5129 & 0{,}4094 \end{pmatrix}\begin{pmatrix} 16{,}7402 \\ 64{,}3848 \end{pmatrix} \approx \begin{pmatrix} 2{,}885 \\ 1{,}031 \end{pmatrix}, \qquad \hat{y}_0 = \tilde{\mathbf{x}}_0\mathbf{w}$$
$$= 2{,}885 + 1{,}031 \times 3{,}7 \approx 6{,}698.$$

Đường thẳng cục bộ có hệ số góc $1{,}031$ gần với $1{,}006$ của đường toàn cục, vì dữ liệu gần tuyến tính. Trọng số hiệu dụng của LOESS tại điểm này là

$$\mathbf{s}(\mathbf{x}_0) = [\,0{,}010,\quad 0{,}094,\quad 0{,}312,\quad 0{,}383,\quad 0{,}173,\quad 0{,}029\,],$$

tổng bằng 1 và toàn dương, rất giống trọng số Nadaraya–Watson $v_i/\sum v_l$. Ở giữa dữ liệu, các phương pháp cho kết quả gần nhau.

Điểm truy vấn $x_0 = 6$ (ở biên)

k-NN ($k = 3$). Ba láng giềng gần nhất là sinh viên 6, 5, 4: $\hat{y}_0 = (9{,}1 + 8{,}0 + 7{,}1)/3 = 8{,}067$.

Nadaraya–Watson. Khoảng cách là $5,4,3,2,1,0$ nên $\mathbf{v} =$ $[\,0{,}000,\ 0{,}0003,\ 0{,}0111,\ 0{,}1353,\ 0{,}6065,\ 1{,}0000\,]$ và $\sum_i v_i = 1{,}7533$:

$$\hat{y}_0 = \frac{14{,}9793}{1{,}7533} = 8{,}543.$$

LOESS. Phương trình chuẩn cho

$$\tilde{\mathbf{X}}^{\top}\mathbf{V}\tilde{\mathbf{X}} = \begin{pmatrix} 1{,}7533 & 9{,}6080 \\ 9{,}6080 & 53{,}4300 \end{pmatrix}, \qquad \tilde{\mathbf{X}}^{\top}\mathbf{V}\mathbf{y} = \begin{pmatrix} 14{,}9793 \\ 82{,}9015 \end{pmatrix}, \qquad \mathbf{w}(\mathbf{x}_0) \approx \begin{pmatrix} 2{,}802 \\ 1{,}048 \end{pmatrix},$$

và $\hat{y}_0 = 2{,}802 + 1{,}048 \times 6 \approx 9{,}088$.

Phương pháp	$x_0 = 3{,}7$	$x_0 = 6$
k-NN ($k = 3$)	6,97	8,07
Nadaraya–Watson ($h = 1$)	6,69	8,54
LOESS ($h = 1$)	6,70	9,09
Hồi quy tuyến tính toàn cục	6,73	9,05
Giá trị quan sát tại $x = 6$		9,10

Nhận xét. Ở giữa dữ liệu, bốn phương pháp cho kết quả xấp xỉ nhau. Ở biên $x_0 = 6$, dữ liệu đang đi lên với độ dốc ≈ 1 điểm mỗi giờ, và hai phương pháp "trung bình" đều **thấp hơn rõ rệt** giá trị quan sát 9,10 (8,07 cho k-NN, 8,54 cho Nadaraya–Watson), vì mọi láng giềng đều nằm *phía bên trái* của x_0 và có điểm thấp hơn. LOESS cho 9,09, sát giá trị thật. Lý do thể hiện ở trọng số hiệu dụng tại $x_0 = 6$:

$$\mathbf{s}(\mathbf{x}_0) = [\,0{,}000,\quad -0{,}001,\quad -0{,}012,\quad -0{,}057,\quad 0{,}152,\quad 0{,}918\,], \qquad \sum_i s_i = 1.$$

Các trọng số của sinh viên 2, 3, 4 là **âm**: LOESS dùng chúng để "ngoại suy" theo đường thẳng cục bộ thay vì chỉ lấy trung bình, và tổng vẫn bằng 1. Đó chính là cơ chế giúp LOESS khắc phục thiên lệch bậc nhất ở biên, đúng như phân tích ở mục 3.4.

3.7. Mã hóa Nadaraya–Watson và LOESS từ đầu bằng Python

Cài đặt chỉ gồm vài hàm NumPy, dùng lại hàm tính bình phương khoảng cách bằng ma trận của mục 2.1.

```
import numpy as np

def binh_phuong_kc(X0, X):
    a = np.sum(X0**2, axis=1, keepdims=True)
    b = np.sum(X**2, axis=1)[None, :]
    return np.maximum(a - 2.0 * X0 @ X.T + b, 0.0)

def knn_hoi_quy(X0, X, y, k=3):
    D2 = binh_phuong_kc(X0, X)
    idx = np.argsort(D2, axis=1, kind="stable")[:, :k]
    return y[idx].mean(axis=1)                                  # (1/k) N y

def ham_nhan(u, loai="gauss"):
    if loai == "gauss":        return np.exp(-0.5 * u**2)
    if loai == "epanechnikov": return np.maximum(1.0 - u**2, 0.0)
    if loai == "deu":          return (np.abs(u) <= 1.0).astype(float)
    raise ValueError(loai)

def nadaraya_watson(X0, X, y, h, loai="gauss"):
    G = ham_nhan(np.sqrt(binh_phuong_kc(X0, X)) / h, loai)      # m x n
    return (G @ y) / G.sum(axis=1)                              # diag(G 1)^
{-1} G y

def loess(X0, X, y, h, loai="gauss", lam=1e-8):
    Xt = np.c_[np.ones(len(X)), X]                              # thêm cột 1
    Gv = ham_nhan(np.sqrt(binh_phuong_kc(X0, X)) / h, loai)     # m x n: v_i
(x0)
    out = np.empty(len(X0))
    for j in range(len(X0)):
        V = Gv[j][:, None]                                      # đường chéo
của V
        A = Xt.T @ (V * Xt) + lam * np.eye(Xt.shape[1])         # Xt^T V Xt
        w = np.linalg.solve(A, Xt.T @ (V[:, 0] * y))            # w(x0)
        out[j] = np.r_[1.0, X0[j]] @ w                          # x0_tilde w
    return out
```

Có hai điểm cài đặt đáng lưu ý. Trong *nadaraya_watson*, phép chia cho *G.sum(axis=1)* thực hiện đúng $\mathrm{diag}(\mathbf{G}\mathbf{1}_n)^{-1}$. Trong *loess*, ta không dựng ma trận đường chéo $\mathbf{V}$ kích thước $n \times n$ mà chỉ giữ vector đường chéo và nhân theo từng hàng (*V * Xt*), tiết kiệm bộ nhớ; ta giải hệ tuyến tính bằng *np.linalg.solve* thay vì tính nghịch đảo để ổn định hơn.

Kiểm tra trên 6 sinh viên

```
x = np.array([1, 2, 3, 4, 5, 6.]); y = np.array([4.0, 5.2, 5.8, 7.1, 8.0, 9.1
])
```

```
X, X0 = x[:, None], np.array([[3.7], [6.0]])

print(np.round(knn_hoi_quy(X0, X, y, k=3), 4))
print(np.round(nadaraya_watson(X0, X, y, h=1.0), 4))
print(np.round(loess(X0, X, y, h=1.0), 4))
```

```
[6.9667 8.0667]
[6.6931 8.5434]
[6.6975 9.0884]
```

Ba dòng kết quả trùng khớp với các số ta đã tính tay ở mục 3.6 (hai cột ứng với $x_0 = 3{,}7$ và $x_0 = 6$).

Đối chiếu với thư viện

Trên dữ liệu sin ($n = 100$), *knn_hoi_quy* với $k = 6$ cho dự đoán trùng hoàn toàn (sai khác lớn nhất bằng 0) với *KNeighborsRegressor(6)* của scikit-learn trên 50 điểm lưới. Để kiểm tra *loess*, ta so với *np.polyfit(x, y, 1, w=np.sqrt(v))*, hàm khớp đa thức bậc 1 có trọng số (w ở đây là *căn* của trọng số bình phương tối thiểu): tại $x_0 = 4{,}2$ với $h = 0{,}35$, cả hai cho $-0{,}2757$. Thư viện *statsmodels* có hàm *lowess* cho LOESS, nhưng nó dùng tham số *frac* (tỉ lệ số mẫu gần nhất) và nhân tricube cùng với một vòng lặp bền vững mặc định, nên con số sẽ không trùng với bản dùng nhân Gauss và băng thông cố định ở trên.

Chọn h bằng bỏ-một-ra

```
def ma_tran_lam_muot_nw(X, h):
    G = ham_nhan(np.sqrt(binh_phuong_kc(X, X)) / h)
    return G / G.sum(axis=1, keepdims=True)                    # S

def loocv(S, y):
    r = (y - S @ y) / (1.0 - np.diag(S))
    return np.mean(r**2)

hs = np.arange(0.1, 3.01, 0.05)
cv = [loocv(ma_tran_lam_muot_nw(X_sin, h), y_sin) for h in hs]
print(round(hs[int(np.argmin(cv))], 2), round(min(cv), 4))
```

```
0.3 0.089
```

Kết quả $h = 0{,}30$ với MSE bỏ-một-ra 0,089 khớp với bảng ở mục 3.4 (*X_sin*, *y_sin* là 100 điểm huấn luyện của bộ dữ liệu sin).

Chi phí tính toán

	Huấn luyện	**Dự đoán m điểm**	**Bộ nhớ**
k-NN hồi quy	$O(1)$ (chỉ lưu dữ liệu)	$O(mnd)$ + chọn k phần tử nhỏ nhất	$O(nd)$
Nadaraya–Watson	$O(1)$	$O(mnd)$ cho $\mathbf{G}$	$O(nd)$

	Huấn luyện	Dự đoán m điểm	Bộ nhớ
LOESS	$O(1)$	$O\big(m\,(nd^2 + d^3)\big)$: mỗi điểm giải một hệ $(d+1)\times(d+1)$	$O(nd)$
Hồi quy tuyến tính (so sánh)	$O(nd^2 + d^3)$ một lần	$O(md)$	$O(d)$

Tương tự k-NN phân loại, mọi chi phí nằm ở lúc dự đoán và tăng theo n. LOESS thêm một hệ số d^2 do phải giải một bài toán bình phương tối thiểu cho từng điểm truy vấn, nên thường dùng với số đặc trưng nhỏ. Với nhân có giá đỡ compact (Epanechnikov, đều), ta chỉ cần xét các mẫu trong bán kính h, và có thể dùng KD-tree để tìm chúng nhanh khi d nhỏ (mục 2.7).

Mối liên hệ với các mục trước

Mục 3 mở rộng mục 2 theo hai hướng, mà cả hai đều chỉ thay đổi *cách đặt trọng số* trong dạng chung $\hat{y}_0 = \sum_i s_i\,(\mathbf{x}_0)\,y_i$ của mục 1.3:

$$\underbrace{s_i = 1/k\;\mathbb{I}[i \in \mathcal{N}_k]}_{\text{k-NN (mục 2, 3.1)}} \;\rightarrow\; \underbrace{s_i = \kappa(d_i/h)\Big/\sum_l \kappa\,(d_l/h)}_{\text{Nadaraya–Watson (3.2)}} \;\rightarrow\; \underbrace{\mathbf{s} = \tilde{\mathbf{x}}_0\big(\tilde{\mathbf{X}}^\top \mathbf{V}\tilde{\mathbf{X}}\big)^{-1}\tilde{\mathbf{X}}^\top\mathbf{V}.}_{\text{LOESS (3.3)}}$$

Ba điều đã được thấy ở mục 2 vẫn đúng ở đây: k (hoặc h) là nút chỉnh thiên lệch–phương sai (mục 3.1, 3.2, 3.4); kiểm định chéo là cách chọn nút đó (mục 3.4, nay có công thức đóng cho bỏ-một-ra); và mọi tính toán đều nằm ở lúc dự đoán (mục 3.7). Hai điều mới: ở hồi quy ta *đo được* thiên lệch và phương sai trực tiếp vì biết hàm thật (mục 3.4), và ảnh hưởng của một mẫu ngoại lai là **cục bộ** (mục 3.5). Mục 4 sẽ đi tiếp một bước: thay vì ước lượng *giá trị trung bình* của y tại $\mathbf{x}_0$, ta ước lượng cả **hàm mật độ** $p(\mathbf{x})$. Như đã nhìn thấy ở mục 3.2, hồi quy nhân thực chất là tỉ số của hai ước lượng mật độ nhân, nên hàm nhân và băng thông h sẽ quay lại ở mục 4 với vai trò chính.

Tóm tắt mục 3

- k-NN phân loại là k-NN hồi quy áp dụng cho các cột one-hot của $\mathbf{Y}$. Với mục tiêu thực, k-NN hồi quy là trung bình của k giá trị lân cận: $\hat{\mathbf{y}} = \frac{1}{k}\mathbf{N}\mathbf{y}$. Đường dự đoán là hàm hằng từng khúc với các bước nhảy.
- **Hồi quy nhân Nadaraya–Watson** thay trọng số $0/\frac{1}{k}$ bằng trọng số liên tục $\kappa(d/h)$: $\hat{\mathbf{y}} = \mathrm{diag}(\mathbf{G}\mathbf{1}_n)^{-1}\mathbf{G}\mathbf{y}$. Băng thông h là nút chỉnh: $h \to 0$ cho k-NN với $k = 1$, $h \to \infty$ cho trung bình toàn cục. Đây là khớp một **hằng số** cục bộ.
- **LOESS** khớp một **đường thẳng** cục bộ bằng bình phương tối thiểu có trọng số: $\mathbf{w}(\mathbf{x}_0) = \big(\tilde{\mathbf{X}}^\top\mathbf{V}\tilde{\mathbf{X}}\big)^{-1}\tilde{\mathbf{X}}^\top\mathbf{V}\mathbf{y}$, $\hat{y}_0 = \tilde{\mathbf{x}}_0\mathbf{w}(\mathbf{x}_0)$. Trọng số hiệu dụng có tổng bằng 1 nhưng có thể âm.

- Chọn h (hoặc k) bằng kiểm định chéo; với các bộ làm trơn tuyến tính, sai số bỏ-một-ra tính được nhanh: $\frac{1}{n}\sum_i \left(\frac{y_i-\hat{y}_i}{1-S_{ii}}\right)^2$.
- Ở **biên**, hồi quy dạng trung bình bị thiên lệch bậc nhất theo h (trong thí nghiệm: $+0{,}349$ và $-0{,}231$ so với nhiễu 0,3); LOESS giảm thiên lệch này ($+0{,}045$ và $-0{,}025$) nhưng tăng phương sai ở biên.
- So với hồi quy tuyến tính, các phương pháp địa phương không cần chọn dạng hàm và tiến sát mức sàn nhiễu (0,31 so với 0,30), trong khi đường thẳng cho 0,74. Ảnh hưởng của một ngoại lai là **cục bộ**: mạnh tại chỗ, bằng 0 ở xa. Các phương pháp này **không ngoại suy được** ngoài miền dữ liệu.

Câu hỏi kiểm tra nhanh

1. Cho 4 mẫu một chiều $x = 1,2,4,7$ với $y = 2,4,5,9$. Dùng k-NN hồi quy với $k = 3$, hãy dự đoán y tại $x_0 = 3$.
2. Cho hai mẫu $x = 1,2$ với $y = 0,6$. Dùng Nadaraya–Watson với nhân Gauss và $h = 0,5$, hãy tính dự đoán tại $x_0 = 1,5$ và $x_0 = 1,2$.
3. Điều gì xảy ra với ước lượng Nadaraya–Watson khi $h \to \infty$ và khi $h \to 0$?
4. Trong công thức $\mathbf{w}(\mathbf{x}_0) = \left(\tilde{\mathbf{X}}^\top \mathbf{V} \tilde{\mathbf{X}}\right)^{-1} \tilde{\mathbf{X}}^\top \mathbf{V}\mathbf{y}$, hãy cho biết kích thước của từng ma trận khi dữ liệu có n mẫu và d đặc trưng. Công thức này khác gì hồi quy tuyến tính thông thường?
5. Vì sao Nadaraya–Watson bị thiên lệch ở biên còn LOESS thì ít hơn? Cái giá phải trả là gì?
6. Chứng minh rằng với $\mathbf{y}^{(c)}$ là cột thứ c của $\mathbf{Y}$, cột thứ c của $\widehat{\mathbf{P}} = \frac{1}{k}\mathbf{N}\mathbf{Y}$ chính là k-NN hồi quy áp dụng cho $\mathbf{y}^{(c)}$.

Gợi ý đáp án

1. Khoảng cách từ $x_0 = 3$ tới $x = 1,2,4,7$ là 2,1,1,4. Ba láng giềng gần nhất là $x = 2$ ($y = 4$), $x = 4$ ($y = 5$) và $x = 1$ ($y = 2$). Dự đoán là $\hat{y}_0 = (4 + 5 + 2)/3 = 11/3 \approx 3{,}667$.
2. Với $x_0 = 1,5$, hai mẫu cách đều ($d = 0,5$), trọng số bằng nhau nên $\hat{y}_0 = (0 + 6)/2 = 3$. Với $x_0 = 1,2$: $d = 0,2$ và 0,8, các trọng số là $e^{-0,5(0,2/0,5)^2} = e^{-0,08} \approx 0{,}923$ và $e^{-0,5(0,8/0,5)^2} = e^{-1,28} \approx 0{,}278$. Vậy $\hat{y}_0 = \frac{0,923\times 0 + 0,278\times 6}{0,923+0,278} \approx \frac{1,668}{1,201} \approx 1{,}39$, gần giá trị của mẫu gần hơn ($y = 0$) hơn là trung bình 3.

3. Khi $h \to \infty$, mọi trọng số $\kappa(d/h) \to \kappa(0)$ bằng nhau nên $\hat{y}_0 \to \bar{y}$ (đường nằm ngang, thiên lệch cao, phương sai thấp). Khi $h \to 0$, chỉ mẫu gần $\mathbf{x}_0$ nhất còn trọng số đáng kể nên $\hat{y}_0 \to y_{\text{gần nhất}}$, tức k-NN với $k = 1$ (bám nhiễu, phương sai cao).

4. $\widetilde{\mathbf{X}}$: $n \times (d+1)$; $\mathbf{V}$: $n \times n$ (đường chéo); $\mathbf{y}$: $n \times 1$; $\widetilde{\mathbf{X}}^\top \mathbf{V} \widetilde{\mathbf{X}}$: $(d+1) \times (d+1)$; $\widetilde{\mathbf{X}}^\top \mathbf{V}\mathbf{y}$: $(d+1) \times 1$; kết quả $\mathbf{w}(\mathbf{x}_0)$: $(d+1) \times 1$. Khác biệt so với hồi quy tuyến tính thông thường: $\mathbf{V}$ thay đổi theo từng điểm truy vấn $\mathbf{x}_0$ (mẫu gần có trọng số lớn) nên phải giải lại hệ cho mỗi điểm truy vấn; hồi quy thông thường là trường hợp $\mathbf{V} = \mathbf{I}$ và giải một lần.

5. Ở biên, các láng giềng chỉ nằm về một phía; Nadaraya–Watson chỉ lấy trung bình (khớp hằng số) nên bị kéo về phía có dữ liệu khi hàm có độ dốc, thiên lệch cỡ $h\,f'(x_0)$. LOESS khớp đường thẳng nên "nhận ra" độ dốc (và dùng một số trọng số âm), loại bỏ thiên lệch bậc nhất. Cái giá là phương sai ở biên cao hơn (trong thí nghiệm, độ lệch chuẩn $0{,}30$ so với $0{,}18$), vì ngoại suy một đường thẳng dựa trên dữ liệu về một phía.

6. Cột thứ c của $\widehat{\mathbf{P}}$ là $\frac{1}{k}\mathbf{N}\mathbf{Y}\mathbf{e}_c$, với $\mathbf{e}_c$ là vector đơn vị chọn cột thứ c. Vì $\mathbf{Y}\mathbf{e}_c = \mathbf{y}^{(c)}$, cột đó bằng $\frac{1}{k}\mathbf{N}\mathbf{y}^{(c)}$, đúng công thức k-NN hồi quy $\hat{\mathbf{y}} = \frac{1}{k}\mathbf{N}\mathbf{y}$ với $\mathbf{y} = \mathbf{y}^{(c)}$. Phần tử j của nó là $\frac{1}{k}\sum_{i \in \mathcal{N}_k(\mathbf{x}_{0,j})} y_i^{(c)}$, tức tỉ lệ láng giềng thuộc lớp c.

4. Ước lượng mật độ xác suất phi tham số

Đến hết mục 3, mọi mô hình ta gặp đều trả lời một câu hỏi dạng "**biết x, thì y thế nào?**". Softmax Regression ước lượng xác suất có điều kiện $P(y = c \mid \mathbf{x})$; k-NN phân loại ước lượng cùng đại lượng đó bằng cách đếm láng giềng; k-NN hồi quy, hồi quy nhân và LOESS ước lượng kỳ vọng có điều kiện $\mathbb{E}[\, y \mid \mathbf{x}\,]$. Khi kết thúc mục 3.2, ta đã thấy một manh mối: công thức Nadaraya–Watson là **tỉ số của hai ước lượng mật độ nhân**. Mục 4 đi theo manh mối đó và hỏi một câu hỏi khác, cơ bản hơn: **bản thân x được phân bố như thế nào?** Tức là ta muốn ước lượng hàm mật độ xác suất $p(\mathbf{x})$ mà không cần nhãn y và, theo tinh thần của cả bài, **không giả định trước p thuộc họ phân phối nào**.

Trong mục 4, ta trả lời các câu hỏi sau:

- Vì sao cần biết $p(\mathbf{x})$ khi ta đã có $P(y \mid \mathbf{x})$? Và có một nguyên lý chung nào cho mọi cách ước lượng mật độ phi tham số không? (mục 4.1)
- Cách đơn giản nhất, **histogram**, hoạt động ra sao và vì sao chưa đủ tốt? (mục 4.2)
- **Ước lượng mật độ nhân (KDE)** sửa các nhược điểm đó như thế nào? (mục 4.3), và chọn **băng thông** ra sao? (mục 4.4)
- **Ước lượng mật độ bằng k láng giềng** là gì, và vì sao nó giải thích được phép bỏ phiếu của k-NN ở mục 2? (mục 4.5)
- So với ước lượng **tham số** (Gauss, hợp lý cực đại), các phương pháp này tốt hơn khi nào, kém hơn khi nào, và lời nguyền chiều ảnh hưởng ra sao? (mục 4.6)
- Cuối cùng là ví dụ tính tay và cài đặt bằng Python (mục 4.7).

Quy ước ký hiệu

Ta giữ các quy ước trước đó: chữ thường in nghiêng là đại lượng vô hướng, **chữ thường in đậm là vector** (vector đặc trưng là vector hàng), **chữ hoa in đậm là ma trận**, và đặc trưng không có cột 1. Với ước lượng mật độ, ta chỉ cần các đặc trưng $\mathbf{X} \in \mathbb{R}^{n \times d}$, **không cần nhãn**. Ta ký hiệu mật độ *thật* (không biết) là $p(\mathbf{x})$ và mật độ *ước lượng* là $\hat{f}(\mathbf{x})$.

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Mật độ thật, mật độ ước lượng	$p(\mathbf{x})$, $\hat{f}(\mathbf{x})$	vô hướng	$p \geq 0$ và $\int p(\mathbf{x})\, d\mathbf{x} = 1$
Vector mật độ ước lượng	$\hat{\mathbf{f}}$	$m \times 1$	Giá trị $\hat{f}$ tại m điểm truy vấn (các hàng của $\mathbf{X}_0$)
Độ rộng ô của histogram	Δ	vô hướng	Độ rộng mỗi ô (bin)

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Số mẫu trong ô thứ j	n_j	vô hướng	Dùng trong histogram
Hàm nhân (đã chuẩn hóa)	$\kappa(u)$	vô hướng	$\kappa \geq 0$, đối xứng, $\int \kappa(u)\, du = 1$
Băng thông	h	vô hướng	Độ rộng của "gò" đặt trên mỗi mẫu
Ma trận nhân Gauss	$\mathbf{G}$	$m \times n$	$G_{ji} = \exp\left(-D_{ji}^{(2)}/(2h^2)\right)$, với $\mathbf{D}^{(2)}$ là ma trận bình phương khoảng cách (mục 2.1)
Bán kính tới láng giềng thứ k	$r_k(\mathbf{x})$	vô hướng	Khoảng cách từ $\mathbf{x}$ tới mẫu gần thứ k
Thể tích hình cầu	$V_k(\mathbf{x})$	vô hướng	$V_k = \omega_d\, r_k^d$, với ω_d là thể tích hình cầu đơn vị trong $\mathbb{R}^d$
Trung bình mẫu, hiệp phương sai mẫu	$\hat{\boldsymbol{\mu}}$, $\hat{\boldsymbol{\Sigma}}$	$1 \times d$, $d \times d$	Ước lượng hợp lý cực đại của phân phối Gauss
Xác suất tiên nghiệm của lớp	π_c	vô hướng	$\pi_c = P(y = c)$, dùng ở mục 4.5 và mục 5

4.1. Vì sao cần ước lượng mật độ $p(\mathbf{x})$: từ mô hình $p(y \mid \mathbf{x})$ sang mô hình sinh

Mô hình phân biệt và mô hình sinh

Softmax Regression là một **mô hình phân biệt** (discriminative): nó chỉ mô hình hóa ranh giới giữa các lớp, tức $P(y \mid \mathbf{x})$, và hoàn toàn *không nói gì* về việc dữ liệu $\mathbf{x}$ trông như thế nào. Một **mô hình sinh** (generative) làm nhiều hơn: nó mô hình hóa cách dữ liệu được tạo ra, tức phân phối đồng thời

$$p(\mathbf{x}, y) = p(\mathbf{x} \mid y)\, P(y) = P(y \mid \mathbf{x})\, p(\mathbf{x}).$$

Hai cách phân tích cùng một đại lượng cho ta cầu nối giữa hai thế giới. Nếu biết cách sinh dữ liệu của từng lớp, tức mật độ có điều kiện theo lớp $p(\mathbf{x} \mid y = c)$ và xác suất tiên nghiệm $\pi_c = P(y = c)$, thì **định lý Bayes** cho xác suất hậu nghiệm dùng để phân loại:

$$P(y = c \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid y = c)\, \pi_c}{\sum_{c'=1}^{K} p\,(\mathbf{x} \mid y = c')\, \pi_{c'}}.$$

Đây là nền tảng của mục 5 (Bayes ngây thơ). Việc còn lại là ước lượng các mật độ $p(\mathbf{x} \mid y = c)$, và đó là nhiệm vụ của mục 4: mỗi lớp cho ta một tập dữ liệu $\mathbf{X}_c$ gồm các mẫu của lớp đó, ta ước lượng mật độ của riêng lớp ấy.

Ngoài phân loại: những điều chỉ có mật độ mới trả lời được

Biết $p(\mathbf{x})$ cho phép trả lời những câu hỏi mà mô hình phân biệt không trả lời được:

- **Phát hiện bất thường.** Một mẫu có mật độ $\hat{f}(\mathbf{x})$ rất thấp là mẫu "lạ" so với dữ liệu đã thấy. Trong khi đó Softmax luôn đưa ra một phân phối xác suất có tổng bằng 1, kể cả với đầu vào nằm rất xa mọi dữ liệu huấn luyện: điểm số tuyến tính càng lớn thì xác suất của lớp thắng càng tiến về 1, tức là mô hình càng "tự tin" khi đi ra xa dữ liệu, điều ngược với trực giác.
- **Sinh dữ liệu mới** bằng cách lấy mẫu từ $\hat{f}$.
- **Khám phá cấu trúc**: các đỉnh (mode) của mật độ gợi ý các nhóm trong dữ liệu.
- **Hồi quy**: như đã thấy ở mục 3.2, kỳ vọng có điều kiện $\mathbb{E}[\,y \mid \mathbf{x}\,] = \int y\, p(\mathbf{x}, y)\, dy \,/\, p(\mathbf{x})$ là tỉ số của các mật độ.

Hai cách ước lượng: tham số và phi tham số

Cách **tham số** quen thuộc: chọn trước một họ phân phối $p(\mathbf{x}; \boldsymbol{\theta})$ và ước lượng $\boldsymbol{\theta}$ bằng hợp lý cực đại. Với họ Gauss nhiều chiều, nghiệm có công thức đóng:

$$\hat{\boldsymbol{\mu}} = \frac{1}{n}\,\mathbf{1}_n^\top \mathbf{X}, \qquad \hat{\boldsymbol{\Sigma}} = \frac{1}{n}\,(\mathbf{X} - \mathbf{1}_n\hat{\boldsymbol{\mu}})^\top(\mathbf{X} - \mathbf{1}_n\hat{\boldsymbol{\mu}}), \qquad \hat{f}(\mathbf{x}) = \frac{\exp\!\left(-\,1/2\,(\mathbf{x} - \hat{\boldsymbol{\mu}})\hat{\boldsymbol{\Sigma}}^{-1}(\mathbf{x} - \hat{\boldsymbol{\mu}})^\top\right)}{(2\pi)^{d/2}\,|\hat{\boldsymbol{\Sigma}}|^{1/2}}.$$

Cách này gặp lại đúng vấn đề của mục 1.1: nếu dạng phân phối đã chọn *sai*, kết quả bị thiên lệch mà thêm dữ liệu không sửa được. Một phân phối Gauss luôn có **một đỉnh duy nhất**; nó không thể mô tả dữ liệu có hai nhóm tách biệt, và sẽ đặt đỉnh ngay vào thung lũng giữa hai nhóm, đúng chỗ dữ liệu thưa nhất (xem mục 4.6). Cách **phi tham số** không chọn họ phân phối nào, và ta hỏi trực tiếp dữ liệu.

Nguyên lý chung: $p(\mathbf{x}) \approx \frac{k}{n\,V}$

Mọi phương pháp ước lượng mật độ phi tham số trong mục này đều xuất phát từ cùng một lập luận. Lấy một vùng nhỏ $\mathcal{R}$ bao quanh $\mathbf{x}$, có thể tích V. Xác suất để một mẫu rơi vào vùng đó là

$$P(\mathbf{X} \in \mathcal{R}) = \int_{\mathcal{R}} p\,(\mathbf{x}')\,d\mathbf{x}' \approx p(\mathbf{x})\,V,$$

nếu $\mathcal{R}$ đủ nhỏ để p gần như không đổi trong đó. Mặt khác, nếu có k trong n mẫu rơi vào $\mathcal{R}$ thì ta ước lượng xác suất đó bằng tần suất k/n. Cân bằng hai vế:

$$\hat{f}(\mathbf{x}) = \frac{k}{n\,V}$$

Công thức này gợi ý hai cách để biến nó thành thuật toán, và chính hai cách đó phân chia các phương pháp trong mục này:

- **Cố định V, đếm k.** Chọn trước kích thước vùng, đếm số mẫu rơi vào đó. Đây là histogram (mục 4.2) và ước lượng nhân (mục 4.3).
- **Cố định k, đo V.** Chọn trước số mẫu k, mở rộng vùng quanh $\mathbf{x}$ cho đến khi chứa đúng k mẫu và đo thể tích V. Đây là ước lượng bằng k láng giềng (mục 4.5), và nó chính là phép đối ngẫu của hai hướng ở mục 3.2 (băng thông cố định so với số láng giềng cố định).

4.2. Histogram và những nhược điểm

Định nghĩa

Chia trục số thành các **ô** (bin) liên tiếp có cùng độ rộng Δ, ví dụ ô thứ j là $B_j = [\,a_0 + (j-1)\Delta,\ a_0 + j\Delta\,)$ với a_0 là điểm bắt đầu. Gọi n_j là số mẫu rơi vào B_j. Áp dụng nguyên lý $k/(nV)$ với vùng $\mathcal{R}$ là chính ô chứa x (thể tích $V = \Delta$) và $k = n_j$:

$$\hat{f}(x) = \frac{n_j}{n\,\Delta} \qquad \text{nếu } x \in B_j.$$

Mật độ này không âm và có tích phân $\sum_j \frac{n_j}{n\Delta}\cdot\Delta = \frac{1}{n}\sum_j n_j = 1$, nên nó là một mật độ hợp lệ. Với d chiều, ô là các hình hộp có cạnh Δ, thể tích $V = \Delta^d$, và $\hat{f}(\mathbf{x}) = n_j/(n\Delta^d)$.

Một thí nghiệm có đáp án biết trước

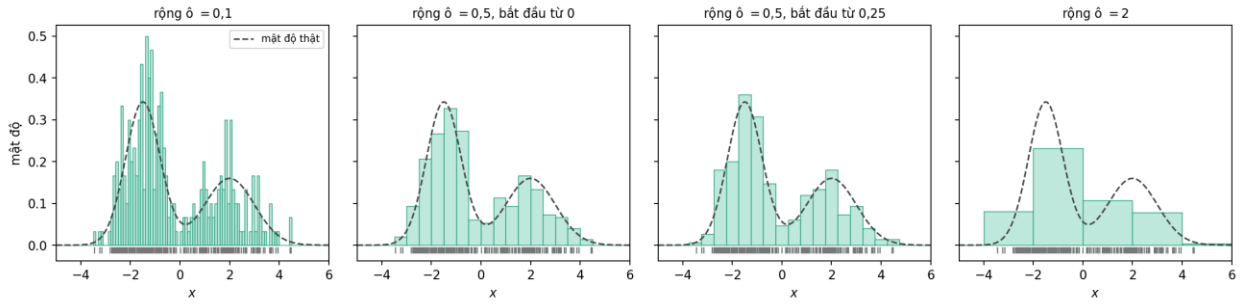
Để đánh giá ước lượng, ta cần một bài toán mà mật độ thật được biết. Ta sinh $n = 300$ mẫu một chiều từ một hỗn hợp hai thành phần Gauss:

$$p(x) = 0{,}6\,\mathcal{N}(x;\,-1{,}5,\,0{,}7^2) + 0{,}4\,\mathcal{N}(x;\,2{,}0,\,1{,}0^2),$$

cho hai đỉnh rõ rệt ở khoảng $x = -1{,}5$ và $x = 2$. Ta cũng sinh thêm $2\,000$ mẫu kiểm tra từ cùng phân phối. Hai thước đo dùng xuyên suốt mục này:

$$\mathrm{ISE} = \int (\hat{f}(x) - p(x))^2\,dx, \qquad \overline{\ell} = \frac{1}{m}\sum_{j=1}^{m} \ln \hat{f}\big(x_j^{\mathrm{kt}}\big).$$

ISE (sai số bình phương tích phân) đo khoảng cách giữa $\hat{f}$ và mật độ thật, chỉ tính được khi mô phỏng. Log-hợp lý trung bình $\overline{\ell}$ trên m mẫu kiểm tra không cần biết mật độ thật, càng lớn càng tốt; với mật độ thật, giá trị này (trung bình của $\ln p$) bằng $-1{,}839$ trong thí nghiệm này, và không ước lượng nào có kỳ vọng vượt qua mức đó.



Hình 12. Histogram của 300 *mẫu từ hỗn hợp hai Gauss (đường nét đứt là mật độ thật; vạch nhỏ dưới trục là các mẫu). Từ trái sang: ô rất hẹp (*$\Delta = 0{,}1$*) gồ ghề; ô vừa (*$\Delta = 0{,}5$*) bám khá sát; cùng* $\Delta = 0{,}5$ *nhưng bắt đầu từ* 0,25 *thay vì* 0 *cho hình dạng khác; ô rất rộng (*$\Delta = 2$*) gộp hai đỉnh làm một.*

Bốn nhược điểm

Thứ nhất, kết quả rất nhạy với độ rộng ô Δ (hai hình ngoài cùng trong Hình 12). Ô quá hẹp thì mỗi ô chỉ chứa vài mẫu, nên mật độ nhảy lên xuống theo ngẫu nhiên (phương sai cao); ô quá rộng thì làm mờ cấu trúc (thiên lệch cao), ví dụ gộp hai đỉnh thành một. Đây là đánh đổi thiên lệch–phương sai quen thuộc. Số liệu:

Độ rộng ô Δ	**ISE**	**Số mẫu kiểm tra rơi vào ô rỗng (trên** 2 000**)**	**Nhận xét**
0,1	0,0435	90	Quá gồ ghề, nhiều ô rỗng
0,5	0,0104	10	Cân bằng
2	0,0405	0	Quá mờ, mất cấu trúc hai đỉnh

Thứ hai, kết quả phụ thuộc điểm bắt đầu a_0**.** Cùng $\Delta = 0{,}5$, dịch điểm bắt đầu từ 0 sang 0,25 làm mật độ ước lượng thay đổi tới 0,127 ở một số điểm (so với đỉnh cao nhất của mật độ thật cỡ 0,34), tức là thay đổi cách ta "vẽ lưới" đã đủ làm hình dạng thay đổi, dù dữ liệu không đổi (hình giữa trong Hình 12).

Thứ ba, mật độ ước lượng không liên tục. Nó là một hàm bậc thang với các bước nhảy ở ranh giới ô. Mật độ thật thường trơn, và ta không có lý do gì để tin vào các bước nhảy đó. Nó cũng không có đạo hàm, nên không dùng được cho các thuật toán cần độ dốc.

Thứ tư, mật độ bằng đúng 0 **ở các ô rỗng.** Với $\Delta = 0{,}1$ có 90 trên 2 000 điểm kiểm tra rơi vào ô rỗng, nên $\hat{f} = 0$ tại đó và $\ln\hat{f} = -\infty$. Một mô hình phân loại dựa trên mật độ như vậy sẽ gán xác suất 0 cho một lớp chỉ vì ô đó tình cờ không có mẫu huấn luyện (ta sẽ gặp lại vấn đề này ở mục 5 và 7, nơi cần làm trơn Laplace). Khi tính log-hợp lý ở bảng sau, ta đặt sàn 10^{-6} cho mật độ.

Ngoài ra, histogram không mở rộng tốt lên nhiều chiều: số ô tăng như $(\text{độ dài miền}/\Delta)^d$, nên với d lớn hầu hết ô đều rỗng. Mọi nhược điểm đầu tiên đều xuất phát từ việc ranh giới các ô là *cứng* và *cố định*; ý tưởng của mục 4.3 là bỏ chúng đi.

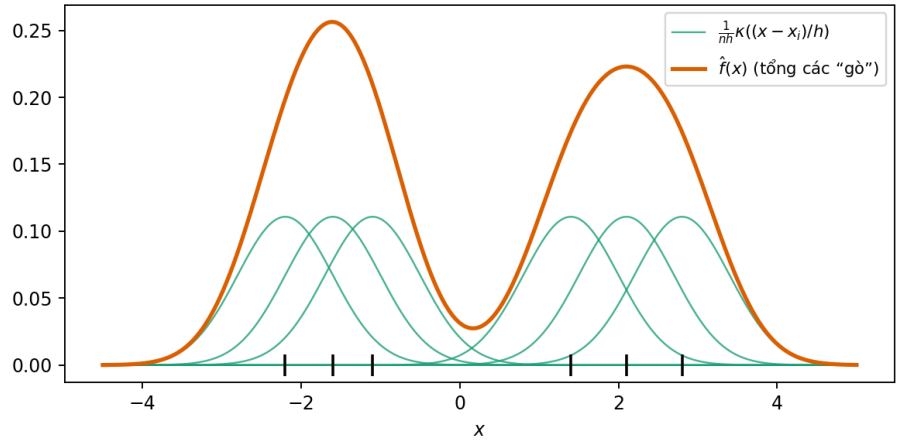
4.3. Ước lượng mật độ nhân (KDE)

Từ histogram đến KDE: đặt một "gò" trên mỗi mẫu

Histogram có thể diễn đạt lại như sau: mỗi mẫu đóng góp một "khối hộp" có chiều cao $\frac{1}{n\Delta}$ lên ô chứa nó. Hai điều gây vấn đề là ô cố định theo lưới và khối hộp có biên cứng. Ý tưởng của **ước lượng mật độ nhân** (kernel density estimation, KDE) là đặt **một gò trơn** có tâm *ngay tại từng mẫu* x_i rồi cộng chúng lại:

$$\hat{f}(x) = \frac{1}{n\,h}\sum_{i=1}^{n} \kappa\left(\frac{x - x_i}{h}\right),$$

trong đó κ là **hàm nhân** *đã chuẩn hóa* ($\int \kappa(u)\,du = 1$) và $h > 0$ là **băng thông**. Khác với mục 3.2, ở đây ta cần giữ hằng số chuẩn hóa, vì mục tiêu là một mật độ có tích phân bằng 1 chứ không chỉ một tỉ số. Hàm nhân Gauss chuẩn hóa là $\kappa(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}$. Hình 13 minh họa bằng 6 mẫu: mỗi mẫu tạo một gò Gauss có độ rộng h và chiều cao tỉ lệ $\frac{1}{n}$, tổng của chúng là $\hat{f}$.



Hình 13. KDE được xây dựng bằng cách đặt một gò Gauss ($\frac{1}{nh}\kappa\big((x - x_i)/h\big)$, nét xanh lục) tại mỗi trong 6 mẫu (vạch đen dưới trục) rồi cộng lại thành $\hat{f}(x)$ (nét cam). Những mẫu nằm gần nhau cho các gò chồng lên nhau và tạo một đỉnh cao; mẫu đơn lẻ chỉ cho một gò thấp.

Vài tính chất suy ra ngay từ định nghĩa. Vì $\kappa \geq 0$ nên $\hat{f} \geq 0$. Mỗi gò $\frac{1}{nh}\kappa\big((x - x_i)/h\big)$ có tích phân $\frac{1}{n}$ (đổi biến $u = (x - x_i)/h$), nên $\int \hat{f}(x)\,dx = n \cdot \frac{1}{n} = 1$: KDE là một mật độ hợp lệ. Hàm $\hat{f}$ trơn bằng hàm nhân: với nhân Gauss, nó khả vi vô hạn, nên *loại bỏ các bước nhảy* và *không còn phụ thuộc điểm bắt đầu* của histogram. Cũng nên để ý rằng với hàm nhân đều trên $[-1/2\,, 1/2]$, công thức trên trở thành $\hat{f}(x) = \frac{k_h(x)}{nh}$ với $k_h(x)$ là số mẫu trong khoảng $[x - h/2, x + h/2]$, đúng nguyên lý $k/(nV)$ của mục 4.1 với $V = h$ và vùng *đặt quanh từng điểm truy vấn* thay vì cố định theo lưới.

Nhiều chiều và dạng ma trận

Trong $\mathbb{R}^d$, dùng nhân Gauss $\kappa_d(\mathbf{u}) = (2\pi)^{-d/2}\exp(-1/2 \parallel \mathbf{u} \parallel^2)$ với cùng băng thông h cho mọi chiều:

$$\hat{f}(\mathbf{x}) = \frac{1}{n\,h^d}\sum_{i=1}^{n}\kappa_d\left(\frac{\mathbf{x}-\mathbf{x}_i}{h}\right) = \frac{1}{n\,(2\pi)^{d/2}\,h^d}\sum_{i=1}^{n}\exp\left(-\frac{\parallel \mathbf{x}-\mathbf{x}_i \parallel^2}{2h^2}\right).$$

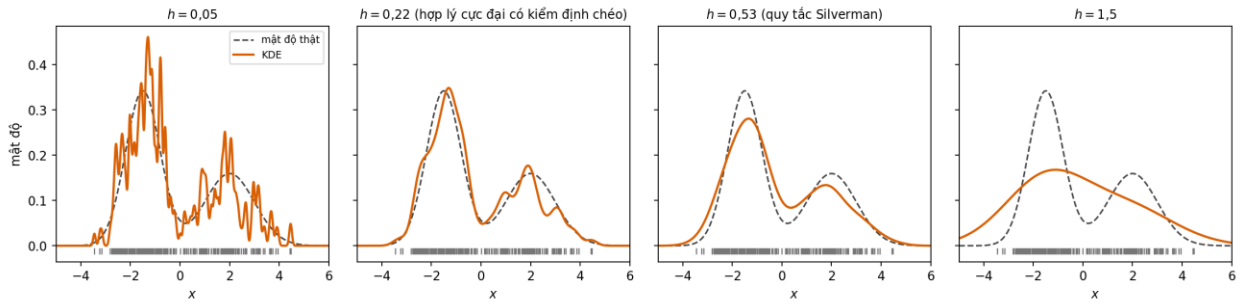
Với m điểm truy vấn xếp thành các hàng của $\mathbf{X}_0$, ta dùng lại ma trận bình phương khoảng cách $\mathbf{D}^{(2)}$ của mục 2.1 để tính toàn bộ cùng lúc:

$$G_{ji} = \exp\left(-\frac{D^{(2)}_{ji}}{2h^2}\right), \qquad \hat{\mathbf{f}} = \frac{1}{n\,(2\pi)^{d/2}\,h^d}\;\mathbf{G}\,\mathbf{1}_n, \qquad \hat{\mathbf{f}} \in \mathbb{R}^{m\times 1}.$$

Phép nhân $\mathbf{G}\mathbf{1}_n$ cộng từng hàng của $\mathbf{G}$, vậy KDE tại một điểm truy vấn chính là *tổng trọng số nhân của mọi mẫu*, cộng với một hằng số chuẩn hóa. Điều này chính xác là mẫu số của công thức Nadaraya–Watson ở mục 3.2: nếu viết $\widehat{m}(\mathbf{x}) = \frac{\frac{1}{nh^d}\sum_i \kappa_d(\cdot)\,y_i}{\hat{f}(\mathbf{x})}$ thì mẫu số là KDE của $p(\mathbf{x})$ còn tử số là ước lượng của $\int y\,p(\mathbf{x},y)\,dy$. Vậy ta đã khép được vòng tròn của mục 3.2: **hồi quy nhân là KDE được dùng để tính một kỳ vọng có điều kiện**. Cũng như k-NN, KDE nhiều chiều dựa trên khoảng cách nên cần *chuẩn hóa đặc trưng* trước (mục 2.5); nếu muốn băng thông khác nhau theo từng hướng, người ta thay h bằng một ma trận băng thông $\mathbf{H}$.

Kết quả trên dữ liệu hai đỉnh

Hình 14 vẽ KDE Gauss trên cùng 300 mẫu với bốn băng thông. Cách chọn hai giá trị ở giữa sẽ được giải thích ở mục 4.4.



Hình 14. KDE Gauss của 300 *mẫu với bốn băng thông (nét cam), so với mật độ thật (nét đứt).* $h = 0{,}05$*: quá gồ ghề, bám từng mẫu.* $h = 0{,}22$*: chọn bằng hợp lý cực đại có kiểm định chéo, bắt được cả hai đỉnh.* $h = 0{,}53$*: quy tắc Silverman, hơi mờ nhưng còn thấy hai đỉnh.* $h = 1{,}5$*: quá mờ, hai đỉnh biến thành một gò rộng.*

Băng thông h	**ISE**	**Log-hợp lý kiểm tra** $\overline{\ell}$
0,05	0,0229	−2,150
0,22 (hợp lý cực đại có kiểm định chéo)	**0,0062**	**−1,869**

Băng thông h	ISE	Log-hợp lý kiểm tra $\overline{\ell}$
0,53 (Silverman)	0,0078	$-1{,}880$
1,5	0,0464	$-2{,}083$
Mật độ thật	0	$-1{,}839$
Histogram $\Delta = 0{,}5$ (tốt nhất ở mục 4.2)	0,0104	$-1{,}917$ (có sàn 10^{-6})

Với băng thông tốt, KDE vượt histogram tốt nhất ở cả hai thước đo (ISE 0,0062 so với 0,0104) và tiến sát mức tối đa $-1{,}839$ của mật độ thật. Bốn hàng đầu cho cùng một hình chữ "U" như ở mọi mục trước: h quá nhỏ hoặc quá lớn đều cho sai số cao hơn rõ rệt.

4.4. Chọn băng thông: quy tắc Silverman, hợp lý cực đại có kiểm định chéo

Sai số theo băng thông (mức trực giác)

Ta lại gặp đánh đổi quen thuộc. Với một chiều, khai triển cho sai số bình phương tích phân *kỳ vọng và tiệm cận* (AMISE):

$$\mathrm{AMISE}(h) = \underbrace{\frac{R(\kappa)}{n\,h}}_{\text{phương sai}} + \underbrace{\frac{h^4}{4}\,\mu_2(\kappa)^2\,R(p'')}_{\text{thiên lệch}^2}, \qquad R(g) = \int g(u)^2\,du, \quad \mu_2(\kappa) = \int u^2\kappa(u)\,du.$$

Số hạng đầu giảm khi h tăng (lấy trung bình trên nhiều mẫu hơn); số hạng sau tăng theo h^4 và phụ thuộc độ cong $R(p'')$ của mật độ thật (gò rộng làm phẳng các đỉnh). Cho đạo hàm theo h bằng 0:

$$h_{\text{tối ưu}} = \left(\frac{R(\kappa)}{\mu_2(\kappa)^2\,R(p'')\;n}\right)^{1/5} \propto\; n^{-1/5}.$$

Công thức này không dùng trực tiếp được vì chứa $R(p'')$ của mật độ thật, nhưng nó cho thấy hai điều: băng thông tối ưu *giảm chậm* theo cỡ mẫu ($n^{-1/5}$), và nó nhỏ hơn khi mật độ thật có nhiều đỉnh hẹp (độ cong lớn).

Quy tắc Silverman (và Scott)

Một cách thực dụng là giả sử mật độ thật là Gauss với độ lệch chuẩn σ chỉ để *tính* $R(p'')$, cụ thể $R(p'') = \frac{3}{8\sqrt{\pi}\,\sigma^5}$. Thay vào công thức trên với nhân Gauss ta được $h \approx 1{,}06\,\sigma\,n^{-1/5}$. **Quy tắc Silverman** (quy tắc ngón tay cái) điều chỉnh để bền hơn khi dữ liệu không đúng Gauss:

$$h_{\text{Silverman}} = 0{,}9\,\min\left(\hat{\sigma},\ \frac{\mathrm{IQR}}{1{,}34}\right) n^{-1/5},$$

trong đó $\hat{\sigma}$ là độ lệch chuẩn mẫu và IQR là khoảng tứ phân vị (hiệu giữa phân vị 75% và 25%; với dữ liệu Gauss $\mathrm{IQR}/1{,}34 \approx \sigma$). Lấy min và hệ số 0,9 làm băng thông nhỏ lại khi dữ liệu có đuôi dài hoặc nhiều đỉnh. Với d chiều và các đặc trưng đã chuẩn hóa, **quy tắc Scott** cho

$$h_{\text{Scott}} = n^{-1/(d+4)}.$$

Trên dữ liệu hai đỉnh, độ lệch chuẩn mẫu là $1{,}839$ và quy tắc Silverman cho $h = 0{,}529$ (hình thứ ba của Hình 14). Quy tắc này nhanh, không cần tối ưu, và cho kết quả khá tốt (ISE $0{,}0078$). Nhưng vì được suy ra từ giả định "gần như Gauss một đỉnh", nó có xu hướng **làm mờ quá mức** khi mật độ thật có nhiều đỉnh, như ở đây: đỉnh bên trái trong hình bị thấp hơn và rộng hơn đỉnh thật.

Hợp lý cực đại có kiểm định chéo

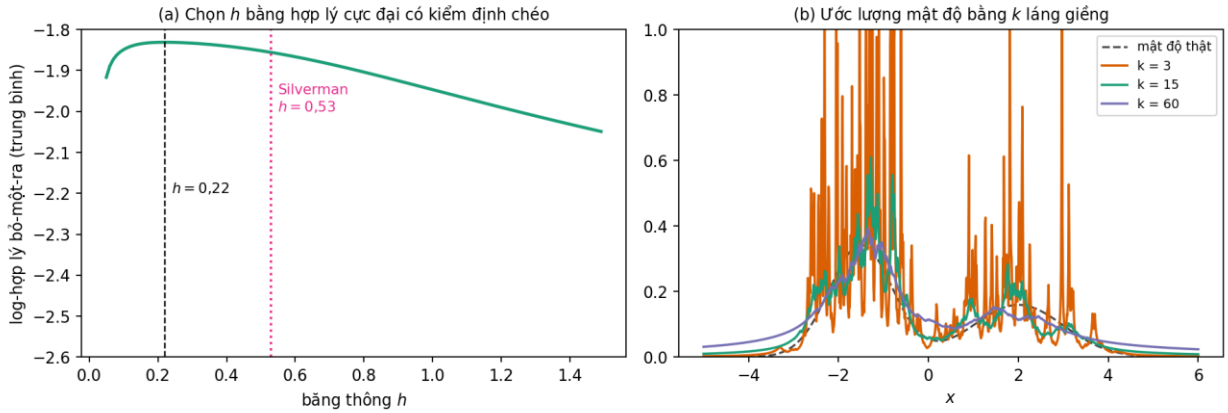
Cách chọn không dựa vào giả định nào về dạng của mật độ là **hợp lý cực đại có kiểm định chéo** (likelihood cross-validation). Ý tưởng: một ước lượng tốt phải gán mật độ cao cho những mẫu *chưa thấy*. Ta tối đa hóa log-hợp lý, nhưng tính tại mỗi mẫu $\mathbf{x}_i$ bằng ước lượng được xây dựng *không dùng chính nó* (bỏ-một-ra):

$$\begin{aligned} \mathrm{LCV}(h) &= \frac{1}{n}\sum_{i=1}^{n} \ln \hat{f}_{-i}(\mathbf{x}_i), \qquad \hat{f}_{-i}(\mathbf{x}_i) = \frac{1}{(n-1)\,h^d}\sum_{j\neq i} \kappa_d\left(\frac{\mathbf{x}_i - \mathbf{x}_j}{h}\right), \qquad \hat{h} \\ &= \operatorname*{argmax}_h \ \mathrm{LCV}(h). \end{aligned}$$

Việc bỏ chính mẫu $\mathbf{x}_i$ là **bắt buộc**. Nếu không bỏ, số hạng $j = i$ đóng góp $\frac{\kappa_d(\mathbf{0})}{nh^d}$ vào $\hat{f}(\mathbf{x}_i)$ và số hạng này tiến tới ∞ khi $h \to 0$, nên log-hợp lý huấn luyện không bị chặn và luôn chọn $h \to 0$ (một "gò" nhọn vô hạn trên mỗi mẫu, hiện tượng học thuộc cực đoan). Về mặt lý thuyết, tối đa hóa kỳ vọng log-hợp lý tương đương cực tiểu hóa **độ phân kỳ Kullback–Leibler**, vì

$$\mathbb{E}\big[\ln \hat{f}(\mathbf{X})\big] = \int p(\mathbf{x}) \ln p(\mathbf{x})\, d\mathbf{x} - \mathrm{KL}\big(p \,\|\, \hat{f}\big),$$

và số hạng đầu không phụ thuộc $\hat{f}$. Hình 15a vẽ $\mathrm{LCV}(h)$ cho h từ $0{,}05$ đến $1{,}5$: đường cong đạt cực đại tại $\hat{h} = 0{,}22$ (log-hợp lý bỏ-một-ra trung bình $-1{,}831$), khác khá nhiều so với $0{,}53$ của Silverman. Kết quả trên tập kiểm tra ủng hộ lựa chọn này: ISE $0{,}0062$ so với $0{,}0078$, $\overline{\ell} = -1{,}869$ so với $-1{,}880$ (bảng ở mục 4.3). Đổi lại, nó tốn tính toán hơn: mỗi giá trị h cần dựng ma trận $n \times n$, tức $O(n^2 d)$.



Hình 15. (a) Log-hợp lý bỏ-một-ra theo băng thông h; cực đại tại $h = 0{,}22$ (nét đứt đen), còn quy tắc Silverman cho $h = 0{,}53$ (nét chấm hồng). (b) Ước lượng mật độ bằng k láng giềng (mục 4.5) với $k = 3{,}15{,}60$, so với mật độ thật.

4.5. Ước lượng mật độ bằng k láng giềng

Công thức

Ta đi theo hướng thứ hai của nguyên lý $k/(nV)$: cố định k và để thể tích tự co giãn. Tại điểm $\mathbf{x}$, mở rộng một hình cầu quanh nó cho đến khi chứa đúng k mẫu. Bán kính lúc đó là $r_k(\mathbf{x})$, khoảng cách từ $\mathbf{x}$ tới láng giềng thứ k (chính là $h(\mathbf{x}_0)$ thích nghi ở mục 3.2). Thể tích hình cầu và ước lượng mật độ là

$$V_k(\mathbf{x}) = \omega_d \, r_k(\mathbf{x})^d, \quad \omega_d = \frac{\pi^{d/2}}{\Gamma(d/2+1)}, \qquad \hat{f}(\mathbf{x}) = \frac{k}{n\,V_k(\mathbf{x})}.$$

Trong đó ω_d là thể tích hình cầu bán kính 1 trong $\mathbb{R}^d$ ($\omega_1 = 2$, $\omega_2 = \pi$, $\omega_3 = 4\pi/3$). Ở một chiều, $V_k = 2\,r_k(x)$ và $\hat{f}(x) = \frac{k}{2n\,r_k(x)}$. Trực giác: vùng dữ liệu dày thì k mẫu nằm gần nhau, r_k nhỏ nên mật độ cao; vùng thưa thì phải đi xa mới gặp đủ k mẫu, r_k lớn nên mật độ thấp. Hình 15b cho kết quả trên dữ liệu hai đỉnh, và ta tính được hai thước đo sau:

k	ISE	Tích phân của $\hat{f}$ trên $[-200{,}200]$	$\overline{\ell}$ trên tập kiểm tra
3	0,3635	1,57	−1,695
15	0,0255	1,37	−1,783
60	0,0132	2,00	−1,774

Những điều cần lưu ý

- **Không phải là một mật độ hợp lệ.** Ở xa mọi mẫu, $r_k(x)$ lớn dần tỉ lệ với khoảng cách tới dữ liệu, nên $\hat{f}(x)$ giảm chậm như $1/|x|$, một đuôi quá nặng khiến $\int \hat{f}(x)\,dx = \infty$. Bảng cho thấy tích phân trên đoạn $[-200{,}200]$ đã vượt 1 (1,37 đến 2,00) và còn tăng nếu ta mở rộng đoạn.

- **Hệ quả: không được so log-hợp lý của ước lượng này với mật độ thật.** Cột cuối cho thấy $\overline{\ell} = -1{,}695$ khi $k = 3$, *cao hơn* cả mật độ thật ($-1{,}839$), điều không thể xảy ra với một mật độ chuẩn hóa. Đó là dấu hiệu của việc tích phân lớn hơn 1: ta "thổi phồng" log-hợp lý. Chỉ số so sánh đáng tin cho ước lượng này là ISE (đã tính trên miền $[-5{,}6]$ chứa hầu hết khối lượng xác suất), và ISE cho thấy $k = 60$ tốt nhất, còn $k = 3$ rất kém ($0{,}36$) do mật độ nhảy mạnh.
- **Không trơn.** $\hat{f}$ có các "góc nhọn" (đạo hàm không liên tục) ở những điểm mà láng giềng thứ k đổi từ mẫu này sang mẫu khác.
- k **là nút chỉnh**, giống h: k nhỏ thì nhiễu (phương sai cao), k lớn thì mờ (thiên lệch cao), và k cũng được chọn bằng kiểm định chéo.

Vì những hạn chế trên, ước lượng mật độ bằng k láng giềng ít được dùng độc lập để mô hình hóa mật độ; giá trị của nó nằm ở chỗ khác.

Giải thích bỏ phiếu đa số ở mục 2

Giá trị của ước lượng này là lời giải thích cho *phép bỏ phiếu đa số* của k-NN phân loại ở mục 2.2. Dùng định lý Bayes của mục 4.1 với mật độ theo lớp được ước lượng bằng k láng giềng. Tại điểm truy vấn $\mathbf{x}$, mở rộng hình cầu cho đến khi chứa đúng k mẫu *của toàn bộ tập huấn luyện* (thể tích V). Giả sử trong số đó có k_c mẫu thuộc lớp c, và toàn bộ tập huấn luyện có n_c mẫu lớp c. Khi đó, áp dụng nguyên lý $k/(nV)$ cho riêng lớp c và cho toàn bộ dữ liệu:

$$\hat{p}(\mathbf{x} \mid c) = \frac{k_c}{n_c V}, \qquad \hat{\pi}_c = \frac{n_c}{n}, \qquad \hat{p}(\mathbf{x}) = \frac{k}{nV}.$$

Thế vào định lý Bayes:

$$\hat{P}(y = c \mid \mathbf{x}) = \frac{\hat{p}(\mathbf{x} \mid c)\,\hat{\pi}_c}{\hat{p}(\mathbf{x})} = \frac{\dfrac{k_c}{n_c V} \cdot \dfrac{n_c}{n}}{\dfrac{k}{nV}} = \frac{k_c}{k}.$$

Đây **chính xác là công thức bỏ phiếu đa số** $\hat{p}_c = v_c/k$ của mục 2.2. Vậy k-NN phân loại *là* bộ phân loại Bayes dùng mật độ theo lớp ước lượng bằng k láng giềng, và thể tích V, cùng n_c và n triệt tiêu hết, chỉ còn lại tỉ lệ đếm. Đây là cầu nối giữa mục 2 (phân loại bằng láng giềng) và mục 5 (phân loại bằng mật độ): cùng một ý tưởng nhìn từ hai phía.

Tổng kết ba cách ước lượng

Phương pháp	Điều cố định	Điều thay đổi	Trơn?	Là mật độ hợp lệ?
Histogram	Thể tích $V = \Delta$ (theo lưới cố định)	Số mẫu $k = n_j$	Không (bậc thang)	Có
KDE	Thể tích (băng thông h, quanh từng điểm)	Số mẫu có trọng số	Có (bằng hàm nhân)	Có

Phương pháp	Điều cố định	Điều thay đổi	Trơn?	Là mật độ hợp lệ?
k láng giềng	Số mẫu k	Thể tích $V_k(\mathbf{x})$	Không (có góc nhọn)	Không (tích phân vô hạn)

4.6. So sánh với ước lượng tham số (Gauss, hợp lý cực đại) và lời nguyền chiều

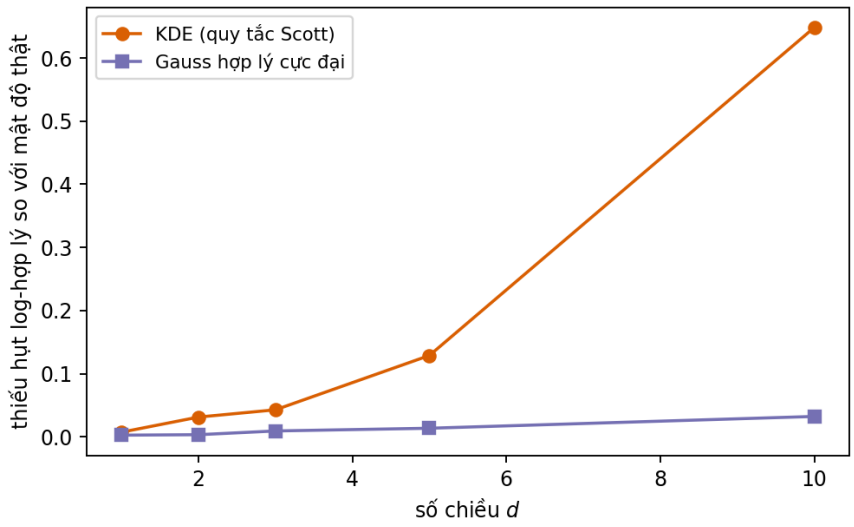
Khi dạng phân phối sai: Gauss trên dữ liệu hai đỉnh

Ta ước lượng một mật độ Gauss trên cùng 300 mẫu hai đỉnh, dùng công thức hợp lý cực đại của mục 4.1. Kết quả là $\hat{\mu} = -0{,}218$ và $\hat{\sigma} = 1{,}836$, tức một gò duy nhất có đỉnh gần $x \approx -0{,}2$, đúng nằm trong **thung lũng** giữa hai đỉnh thật ($-1{,}5$ và 2), nơi dữ liệu thưa nhất. Kết quả: ISE $= 0{,}0625$ (gấp khoảng 10 lần KDE tốt nhất, $0{,}0062$) và $\overline{\ell} = -2{,}099$ (so với $-1{,}869$ của KDE và $-1{,}839$ của mật độ thật). Đây lặp lại bài học của mục 1.1: chọn sai họ phân phối thì thiên lệch không giảm dù có thêm dữ liệu.

Khi dạng phân phối đúng: Gauss có lợi thế, và cái giá của KDE tăng theo số chiều

Ngược lại, giả sử dữ liệu *thật sự* là Gauss. Khi đó mô hình tham số chọn đúng họ hàm và chỉ phải ước lượng $d + \frac{d(d+1)}{2}$ tham số, trong khi KDE phải "học lại" hình dạng đó từ dữ liệu. Ta kiểm tra bằng cách lấy $n = 1\,000$ mẫu từ $\mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ với $d = 1,2,3,5,10$, ước lượng mật độ bằng hai cách (Gauss hợp lý cực đại; KDE nhân Gauss với băng thông Scott $n^{-1/(d+4)}$), rồi đo **thiếu hụt log-hợp lý** so với mật độ thật, tính trên $3\,000$ mẫu kiểm tra (con số càng nhỏ càng tốt, 0 là hoàn hảo):

Số chiều d	1	2	3	5	10
Gauss hợp lý cực đại	0,003	0,003	0,009	0,014	0,032
KDE (quy tắc Scott)	0,007	0,031	0,043	0,129	0,648



Hình 17. Thiếu hụt log-hợp lý (so với mật độ thật) của KDE và Gauss hợp lý cực đại khi dữ liệu thật sự là Gauss, theo số chiều d ($n = 1\,000$). KDE tụt nhanh khi d tăng, còn Gauss gần như không đổi.

Với $d = 1$ cả hai gần như ngang nhau; khi d tăng, thiếu hụt của KDE tăng nhanh (từ 0,007 lên 0,648) còn của Gauss chỉ tăng nhẹ. Cần nhớ rằng thí nghiệm này *thiên vị* mô hình Gauss vì dữ liệu được sinh ra từ đúng họ đó, nhưng nó cho thấy rõ cái giá của sự linh hoạt khi d lớn. Ở mức lý thuyết, đây là hệ quả của tốc độ hội tụ của KDE: sai số bình phương tích phân tối ưu giảm như

$$\text{AMISE}_{\text{tối ưu}} \propto n^{-4/(d+4)}, \qquad \text{so với mô hình tham số đúng dạng:} \quad \propto n^{-1}.$$

Số mũ $4/(d+4)$ giảm từ 0,8 ($d = 1$) xuống 0,29 ($d = 10$), nên để đạt cùng sai số, số mẫu cần tăng nhanh theo d. Đây là lời nguyền chiều trong ngôn ngữ ước lượng mật độ, cùng bản chất với điều ta thấy ở mục 2.5 (khoảng cách tập trung) và 3.4 (tốc độ $n^{-4/(d+4)}$ của hồi quy nhân). Nó cũng sẽ là động lực của mục 5: để phân loại bằng mật độ khi d lớn, ta cần một giả định đơn giản hóa như "ngây thơ" thay vì ước lượng mật độ d chiều nguyên vẹn.

Bảng so sánh

Tiêu chí	Ước lượng tham số (Gauss)	Ước lượng phi tham số (KDE)
Giả định dạng phân phối	Một đỉnh, đối xứng elip	Không
Số đại lượng lưu	$d + \frac{d(d+1)}{2}$	Toàn bộ dữ liệu (nd số)
"Huấn luyện"	Công thức đóng $O(nd^2)$	Chỉ lưu dữ liệu
Tính $\hat{f}$ tại một điểm	$O(d^2)$	$O(nd)$
Khi dạng sai (nhiều đỉnh)	Thiên lệch lớn	Bắt được nhiều đỉnh
Khi dạng đúng, d lớn	Rất hiệu quả	Phương sai lớn, cần nhiều mẫu
Nút chỉnh độ phức tạp	Họ phân phối, số thành phần	Băng thông h (hoặc k)

4.7. Ví dụ tính tay và mã hóa bằng Python

Ví dụ tính tay: số giờ tự học của 8 sinh viên

Có 8 sinh viên với số giờ tự học mỗi tuần (một chiều, $d = 1$): $x = 1;\ 2;\ 2{,}5;\ 3;\ 7;\ 8;\ 8{,}5;\ 9$. Dữ liệu rõ ràng chia thành hai nhóm: nhóm ít học (quanh 2 giờ) và nhóm học nhiều (quanh 8 giờ), khoảng 4 đến 6 giờ hầu như không có ai. Ta ước lượng mật độ tại hai điểm truy vấn: $x_0 = 2{,}5$ (giữa nhóm ít học) và $x_0 = 5$ (đúng khoảng trống). Dùng bốn phương pháp.

Histogram ($\Delta = 2$, điểm bắt đầu 0). Các ô $[0,2), [2,4), [4,6), [6,8), [8,10)$ chứa lần lượt 1,3,0,1,3 mẫu. Điểm 2,5 thuộc ô $[2,4)$ và điểm 5 thuộc ô $[4,6)$:

$$\hat{f}(2{,}5) = \frac{3}{8 \times 2} = 0{,}1875, \qquad \hat{f}(5) = \frac{0}{8 \times 2} = 0.$$

KDE Gauss ($h = 1$). Khoảng cách từ $x_0 = 2{,}5$ tới 8 mẫu là 1,5; 0,5; 0; 0,5; 4,5; 5,5; 6; 6,5 và từ $x_0 = 5$ là 4; 3; 2,5; 2; 2; 3; 3,5; 4. Ma trận nhân $\mathbf{G}$ (hai hàng ứng với hai điểm truy vấn, tám cột ứng với tám sinh viên, $G_{ji} = e^{-d_{ji}^2/2}$):

$$\mathbf{G} = \begin{pmatrix} 0{,}3247 & 0{,}8825 & 1{,}0000 & 0{,}8825 & 0{,}0000 & 0{,}0000 & 0{,}0000 & 0{,}0000 \\ 0{,}0003 & 0{,}0111 & 0{,}0439 & 0{,}1353 & 0{,}1353 & 0{,}0111 & 0{,}0022 & 0{,}0003 \end{pmatrix}.$$

Hằng số chuẩn hóa là $\frac{1}{n\sqrt{2\pi}\,h} = \frac{1}{8\sqrt{2\pi}} = 0{,}04987$, và $\hat{\mathbf{f}} = 0{,}04987\ \mathbf{G1}_8$:

$$\mathbf{G1}_8 = \begin{pmatrix} 3{,}0897 \\ 0{,}3397 \end{pmatrix}, \qquad \hat{\mathbf{f}} = \begin{pmatrix} 0{,}1541 \\ 0{,}0169 \end{pmatrix}.$$

k láng giềng ($k = 3$, một chiều nên $V = 2r_3$). Tại $x_0 = 2{,}5$, ba mẫu gần nhất cách 0, 0,5, 0,5 nên $r_3 = 0{,}5$, $V = 1$ và $\hat{f} = \frac{3}{8\times 1} = 0{,}375$. Tại $x_0 = 5$, ba mẫu gần nhất cách 2, 2, 2,5 nên $r_3 = 2{,}5$, $V = 5$ và $\hat{f} = \frac{3}{8\times 5} = 0{,}075$.

Gauss hợp lý cực đại. Trung bình $\hat{\mu} = 5{,}125$ và độ lệch chuẩn $\hat{\sigma} = 3{,}090$, cho $\hat{f}(2{,}5) = 0{,}090$ và $\hat{f}(5) = 0{,}129$.

Phương pháp	$\hat{f}(2{,}5)$	$\hat{f}(5)$
Histogram ($\Delta = 2$)	0,1875	0
KDE Gauss ($h = 1$)	0,1541	0,0169
k láng giềng ($k = 3$)	0,375	0,075
Gauss hợp lý cực đại	0,090	0,129

Nhận xét. Ba phương pháp phi tham số đều đồng ý rằng mật độ ở 2,5 cao hơn ở 5 (histogram cho 0 ở 5, KDE cho tỉ lệ khoảng 9 lần, k láng giềng khoảng 5 lần), phản ánh đúng việc có 3 sinh viên quanh 2,5 nhưng không có ai quanh 5. Phương pháp Gauss thì ngược lại: nó cho $\hat{f}(5) > \hat{f}(2{,}5)$ vì đặt đỉnh duy nhất ở $\hat{\mu} = 5{,}125$, ngay giữa khoảng trống. Histogram cho đúng 0 ở $x_0 = 5$, nghĩa là nếu một sinh viên mới học 5 giờ thì log-hợp lý là $-\infty$, còn KDE cho một giá trị nhỏ

nhưng dương. Hãy lưu ý các con số của k láng giềng ($0{,}375$ và $0{,}075$) khá khác KDE, vì với chỉ 8 mẫu, $k = 3$ là một lân cận rất rộng so với dữ liệu.

Chọn băng thông. Quy tắc Silverman với $\hat{\sigma} = 3{,}303$, IQR $= 8{,}125 - 2{,}375 = 5{,}75$ cho $h = 0{,}9 \times \min(3{,}303;\ 4{,}29) \times 8^{-1/5} = 1{,}961$, lớn hơn nhiều so với $h = 1$ ta dùng, vì dữ liệu có hai nhóm tách biệt (cùng lỗi "làm mờ" đã thấy ở mục 4.4). Với hợp lý cực đại có kiểm định chéo, log-hợp lý bỏ-một-ra trung bình là $-2{,}708$ ($h = 0{,}5$), $-2{,}358$ ($h = 1$) và $-2{,}592$ ($h = 2$), nên trong ba giá trị này, $h = 1$ là lựa chọn tốt nhất. Với chỉ 8 mẫu, các con số này còn rất nhiễu và chỉ mang tính minh họa.

Cài đặt từ đầu

```
import numpy as np
from math import gamma, pi

def binh_phuong_kc(X0, X):
    a = np.sum(X0**2, axis=1, keepdims=True); b = np.sum(X**2, axis=1)[None,
:]
    return np.maximum(a - 2.0 * X0 @ X.T + b, 0.0)

def histogram_mat_do(x0, x, w, goc=0.0):
    """Mật độ histogram một chiều: n_j / (n w)."""
    n = len(x)
    chi_so_x = np.floor((x - goc) / w).astype(int)
    chi_so_0 = np.floor((x0 - goc) / w).astype(int)
    dem = np.array([(chi_so_x == b).sum() for b in chi_so_0])
    return dem / (n * w)

def kde(X0, X, h):
    """KDE nhân Gauss: (1/n) (2 pi)^(-d/2) h^(-d) * G 1_n."""
    n, d = X.shape
    G = np.exp(-binh_phuong_kc(X0, X) / (2.0 * h**2))             # m x n
    return G.sum(axis=1) / (n * (2.0 * pi)**(d / 2) * h**d)

def the_tich_cau(d, r):
    return pi**(d / 2) / gamma(d / 2 + 1) * r**d

def knn_mat_do(X0, X, k):
    """f(x) = k / (n V_k(x))."""
    n, d = X.shape
    D2 = binh_phuong_kc(X0, X)
    r = np.sqrt(np.sort(D2, axis=1)[:, k - 1])                     # bán kính
tới láng giềng thứ k
    return k / (n * the_tich_cau(d, r))

def silverman_1d(x):
    q75, q25 = np.percentile(x, [75, 25])
    return 0.9 * min(x.std(ddof=1), (q75 - q25) / 1.34) * len(x)**(-0.2)

def log_kha_nang_bo_mot_ra(X, h):
```

```
    n, d = X.shape
    G = np.exp(-binh_phuong_kc(X, X) / (2.0 * h**2)); np.fill_diagonal(G, 0.0
)   # bỏ chính nó
    f = G.sum(axis=1) / ((n - 1) * (2.0 * pi)**(d / 2) * h**d)
    return np.mean(np.log(f + 1e-300))
```

Hàm *kde* bám sát công thức ma trận $\hat{\mathbf{f}} = \frac{1}{n(2\pi)^{d/2}h^d}\mathbf{G}\mathbf{1}_n$; việc *np.fill_diagonal(G, 0.0)* trong hàm cuối chính là "bỏ chính mẫu i" của $\hat{f}_{-i}$ (đường chéo của $\mathbf{G}$ ứng với $\mathbf{x}_i$ so với chính nó), và ta chia cho $n-1$ thay vì n.

Kiểm tra trên 8 sinh viên

```
x = np.array([1, 2, 2.5, 3, 7, 8, 8.5, 9.]); X = x[:, None]
X0 = np.array([[2.5], [5.0]])

print(np.round(histogram_mat_do(X0[:, 0], x, w=2.0), 4))
print(np.round(kde(X0, X, h=1.0), 4))
print(np.round(knn_mat_do(X0, X, k=3), 4))
print(round(silverman_1d(x), 3))
print([round(log_kha_nang_bo_mot_ra(X, h), 3) for h in (0.5, 1.0, 2.0)])
```

```
[0.1875 0.    ]
[0.1541 0.0169]
[0.375 0.075]
1.961
[-2.708, -2.358, -2.592]
```

Các kết quả trùng khớp với phép tính tay ở trên. Để kiểm tra tính đúng đắn của KDE, ta so với *KernelDensity* của scikit-learn:

```
from sklearn.neighbors import KernelDensity
kd = KernelDensity(kernel="gaussian", bandwidth=1.0).fit(X)
print(np.exp(kd.score_samples(X0)))
```

```
[0.15407583 0.01693928]
```

Kết quả trùng khớp với hàm *kde* của ta (cùng 0,1541 và 0,0169). Ta cũng kiểm tra rằng KDE là một mật độ hợp lệ bằng cách lấy tích phân số trên lưới $[-6,16]$ với bước 0,005: kết quả là 1,0000.

Chi phí tính toán và thực hành

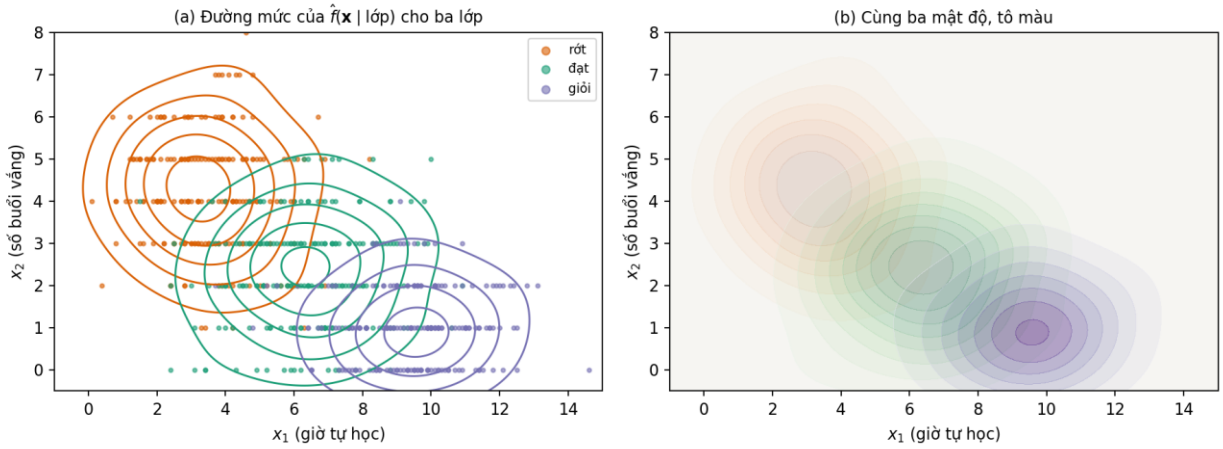
	"Huấn luyện"	**Đánh giá $\hat{f}$ tại m điểm**	**Bộ nhớ**
Histogram	$O(n)$ (đếm)	$O(m)$	$O\big((\text{số ô})\big)$, tăng theo d như Δ^{-d}
KDE	$O(1)$ (chỉ lưu dữ liệu)	$O(mnd)$	$O(nd)$

	"Huấn luyện"	Đánh giá $\hat{f}$ tại m điểm	Bộ nhớ
k láng giềng	$O(1)$	$O(mnd)$ cộng chọn k phần tử nhỏ nhất	$O(nd)$
Gauss hợp lý cực đại	$O(nd^2)$	$O(md^2)$	$O(d^2)$

Giống k-NN ở mục 2.7, KDE gần như không tốn gì khi huấn luyện nhưng tốn $O(n)$ cho mỗi lần đánh giá. Trong thực hành, người ta tăng tốc bằng cấu trúc cây (KD-tree, ball tree) cho phép bỏ qua các mẫu ở quá xa (đóng góp ≈ 0 khi $d(\mathbf{x}, \mathbf{x}_i) \gg h$), hoặc bằng cách đếm trước vào lưới (binning) rồi dùng phép nhân chập nhanh (FFT) trong trường hợp một chiều. Cũng nên nhớ rằng tối ưu băng thông bằng hợp lý cực đại có kiểm định chéo cần ma trận $n \times n$, tức $O(n^2 d)$ cho mỗi giá trị h, nên với dữ liệu lớn người ta dùng quy tắc Scott/Silverman hoặc kiểm định chéo trên một tập con.

Xem trước mục 5: mật độ theo lớp trên dữ liệu sinh viên

Để hình dung cách ước lượng mật độ sẽ phục vụ phân loại, Hình 16 vẽ KDE hai chiều *riêng cho từng lớp* trên 600 sinh viên mô phỏng của mục 2.3 (đặc trưng đã chuẩn hóa, băng thông theo quy tắc Scott $h = n_c^{-1/6}$ với $n_c = 200$ mẫu mỗi lớp).



Hình 16. KDE hai chiều $\hat{f}(\mathbf{x} \mid \text{lớp})$ cho từng lớp (rớt, đạt, giỏi) trên 600 sinh viên mô phỏng. (a) Các đường mức và các điểm dữ liệu. (b) Cùng ba mật độ, tô màu. Ba "quả đồi" chồng lấn ở các vùng giữa; phân loại bằng mật độ so sánh độ cao của ba quả đồi tại điểm cần dự đoán.

Nếu có một sinh viên mới ở một điểm $\mathbf{x}$ nào đó, ta so sánh $\hat{f}(\mathbf{x} \mid \text{rớt})\,\pi_{\text{rớt}}$, $\hat{f}(\mathbf{x} \mid \text{đạt})\,\pi_{\text{đạt}}$ và $\hat{f}(\mathbf{x} \mid \text{giỏi})\,\pi_{\text{giỏi}}$ rồi chọn lớp lớn nhất, đúng công thức Bayes ở mục 4.1. Với hai đặc trưng việc này làm tốt, nhưng với d lớn thì mật độ d chiều theo lớp trở nên không ước lượng được, đúng như lời nguyền chiều đã mô tả ở mục 4.6. Mục 5 sẽ cho thấy một giả định đơn giản (độc lập có điều kiện) cứu ta khỏi vấn đề này.

Mối liên hệ với các mục trước

Mục 4 hoàn thành chuỗi "từ dự đoán đến mật độ" đã nêu ở lộ trình mục 1.3. Mục 2 và mục 3 trả lời câu hỏi "y là gì khi biết $\mathbf{x}$" bằng cách nhìn các láng giềng; mục 4 trả lời "$\mathbf{x}$ phân bố thế nào" bằng cách *đếm các mẫu trong một vùng*, mà tất cả đều đến từ cùng một nguyên lý $\hat{f}(\mathbf{x}) = k/(nV)$. Ba mối liên hệ cụ thể:

- Hồi quy nhân (mục 3.2) có mẫu số là KDE; vì thế **băng thông h của mục 3 và mục 4 là cùng một đại lượng**, và cách chọn bằng kiểm định chéo (mục 3.4 cho sai số dự đoán, mục 4.4 cho log-hợp lý) là hai phiên bản của cùng một ý.
- Phân loại k-NN (mục 2.2) là phân loại Bayes với mật độ theo lớp ước lượng bằng k láng giềng: $\hat{P}(y = c \mid \mathbf{x}) = k_c/k$ (mục 4.5).
- Mọi khó khăn của mục 2 và 3 lặp lại: thiên lệch–phương sai theo h hoặc k, thiếu hụt ở số chiều lớn, và chi phí nằm ở lúc đánh giá.

Điều mới ở mục 4 là *đầu ra không còn phụ thuộc nhãn*: một ước lượng mật độ cho ta khả năng đánh giá độ "điển hình" của một mẫu và sinh dữ liệu. Mục 5 kết hợp mật độ theo lớp với định lý Bayes để phân loại; để chống lại lời nguyền chiều, nó thêm giả định "ngây thơ".

Tóm tắt mục 4

- Mô hình phân biệt (Softmax) ước lượng $P(y \mid \mathbf{x})$; mô hình sinh ước lượng $p(\mathbf{x} \mid y)$ và $P(y)$ rồi dùng định lý Bayes. Ước lượng mật độ $p(\mathbf{x})$ còn phục vụ phát hiện bất thường, sinh dữ liệu và hồi quy nhân.
- Nguyên lý chung của ước lượng mật độ phi tham số: $\hat{f}(\mathbf{x}) = \frac{k}{nV}$. **Cố định V, đếm k** cho histogram và KDE; **cố định k, đo V** cho ước lượng k láng giềng.
- **Histogram** $\hat{f} = n_j/(n\Delta)$ nhạy với độ rộng ô và điểm bắt đầu, không liên tục và cho mật độ 0 ở ô rỗng. **KDE** $\hat{f}(x) = \frac{1}{nh}\sum_i \kappa\left(\frac{x - x_i}{h}\right)$ đặt một gò trơn trên mỗi mẫu; ở dạng ma trận $\hat{\mathbf{f}} = \frac{1}{n(2\pi)^{d/2}h^d}\mathbf{G}\mathbf{1}_n$, và là mẫu số của hồi quy Nadaraya–Watson.
- Chọn h: quy tắc Silverman $h = 0{,}9\min(\hat{\sigma}, \text{IQR}/1{,}34)\, n^{-1/5}$ và Scott $h = n^{-1/(d+4)}$ nhanh nhưng làm mờ khi dữ liệu nhiều đỉnh; hợp lý cực đại có kiểm định chéo (bỏ-một-ra) không cần giả định, và trên dữ liệu thí nghiệm cho ISE $0{,}0062$ so với $0{,}0078$.
- Ước lượng k láng giềng $\hat{f} = k/(nV_k)$ không phải mật độ hợp lệ (tích phân vô hạn), nhưng nó giải thích bỏ phiếu đa số: $\hat{P}(y = c \mid \mathbf{x}) = k_c/k$.

- Ước lượng tham số (Gauss) thắng khi dạng đúng và d lớn, thua nặng khi dạng sai (ISE $0{,}0625$ so với $0{,}0062$ trên dữ liệu hai đỉnh); KDE có sai số giảm như $n^{-4/(d+4)}$ nên tụt nhanh khi d tăng.

Câu hỏi kiểm tra nhanh

1. Có 10 mẫu một chiều. Histogram với độ rộng ô $\Delta = 0{,}5$ có một ô chứa đúng 2 mẫu. Mật độ ước lượng trong ô đó là bao nhiêu?
2. Với hai mẫu $x_1 = 0$, $x_2 = 2$ và nhân Gauss chuẩn hóa $\kappa(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}$, $h = 1$, hãy tính $\hat{f}(1)$. Cho $\kappa(1) \approx 0{,}2420$ và $\kappa(0) \approx 0{,}3989$.
3. Vì sao khi chọn h bằng hợp lý cực đại phải dùng bỏ-một-ra? Điều gì xảy ra nếu tối đa hóa log-hợp lý *trên chính tập huấn luyện*?
4. Hai chiều ($d = 2$), $n = 100$ mẫu. Bán kính từ điểm $\mathbf{x}$ tới láng giềng thứ 5 là $r_5 = 0{,}2$. Tính $\hat{f}(\mathbf{x})$ theo ước lượng k láng giềng.
5. Chứng minh rằng nếu ước lượng $p(\mathbf{x} \mid c) = k_c/(n_c V)$ cho từng lớp và $\pi_c = n_c/n$ thì xác suất hậu nghiệm theo Bayes là k_c/k.
6. Với dữ liệu hai đỉnh tách biệt, ước lượng Gauss hợp lý cực đại và KDE khác nhau thế nào? Khi nào Gauss lại tốt hơn KDE?
7. Tính băng thông theo quy tắc Scott cho $n = 1\,000$ mẫu đã chuẩn hóa ở $d = 5$ chiều.

Gợi ý đáp án

1. $\hat{f} = \frac{n_j}{n\Delta} = \frac{2}{10 \times 0{,}5} = 0{,}4$.
2. $\hat{f}(1) = \frac{1}{2 \times 1}[\kappa(1 - 0/1) + \kappa(1 - 2/1)] = \frac{1}{2}\,(0{,}2420 + 0{,}2420) = 0{,}2420$. Điểm $x = 1$ cách đều hai mẫu một khoảng 1 nên mỗi mẫu đóng góp như nhau.
3. Nếu không bỏ chính mẫu $\mathbf{x}_i$ khỏi $\hat{f}$ khi đánh giá tại $\mathbf{x}_i$, thì số hạng $\kappa_d(\mathbf{0})/(nh^d)$ ở chính nó tiến tới ∞ khi $h \to 0$. Log-hợp lý huấn luyện khi đó không bị chặn và luôn chọn $h \to 0$ (một gò nhọn trên mỗi mẫu, học thuộc cực đoan). Bỏ-một-ra đo khả năng gán mật độ cao cho mẫu *chưa thấy*, nên hàm mục tiêu có cực đại thật sự ở h hữu hạn.
4. $V = \omega_2 r_5^2 = \pi \times 0{,}2^2 = 0{,}1257$. Vậy $\hat{f} = \frac{k}{nV} = \frac{5}{100 \times 0{,}1257} \approx 0{,}398$.

5. $\hat{P}(c \mid \mathbf{x}) = \frac{\hat{p}(\mathbf{x}|c)\,\hat{\pi}_c}{\hat{p}(\mathbf{x})} = \frac{\frac{k_c}{n_c V}\frac{n_c}{n}}{\frac{k}{nV}} = \frac{k_c}{nV} \cdot \frac{nV}{k} = \frac{k_c}{k}$; thể tích V và n_c, n đều triệt tiêu, chỉ còn tỉ lệ số láng giềng thuộc lớp c trong k láng giềng, đúng bỏ phiếu đa số.

6. Gauss chỉ có một đỉnh, nên với dữ liệu hai đỉnh nó đặt đỉnh vào thung lũng giữa hai nhóm (thiên lệch lớn: ISE $0{,}0625$ so với $0{,}0062$ của KDE trong thí nghiệm). KDE không giả định dạng nên bắt được cả hai đỉnh. Gauss tốt hơn khi dữ liệu thật sự gần Gauss, nhất là với số chiều lớn hoặc ít mẫu: nó chỉ ước lượng $d + d(d+1)/2$ tham số nên phương sai thấp, còn KDE có sai số giảm chậm như $n^{-4/(d+4)}$.

7. $h = n^{-1/(d+4)} = 1\,000^{-1/9} = e^{-\ln(1000)/9} = e^{-0{,}7675} \approx 0{,}464$.

5. Bayes ngây thơ: từ mật độ đến phân loại

Mục 4 kết thúc bằng một hình ảnh (Hình 16): ba "quả đồi" mật độ $\hat{f}(\mathbf{x} \mid \text{lớp})$ của ba nhóm sinh viên, chồng lấn nhau ở vùng giữa. Ta cũng đã viết ra công thức để biến chúng thành bộ phân loại, nhưng dừng lại ở hai câu hỏi chưa trả lời. **Thứ nhất**: công thức Bayes ở mục 4.1 thực sự cho ta quy tắc quyết định nào, và nó liên hệ thế nào với Softmax Regression mà ta đã học? **Thứ hai**: với hai đặc trưng, ước lượng mật độ theo lớp rất dễ, nhưng với d lớn thì lời nguyền chiều (mục 4.6) khiến mật độ d chiều của từng lớp gần như không ước lượng được. Làm sao tiếp tục?

Câu trả lời của mục 5 là một giả định đơn giản đến mức "ngây thơ": **khi đã biết lớp, các đặc trưng độc lập với nhau**. Giả định này hầu như luôn sai trong thực tế, nhưng nó biến bài toán ước lượng một mật độ d chiều thành d bài toán một chiều, mà ta đã giải ở mục 4, và vì vậy bộ phân loại **Bayes ngây thơ** (naive Bayes) thường hoạt động đáng ngạc nhiên. Trong mục 5, ta trả lời các câu hỏi sau:

- Quy tắc Bayes cho phân loại là gì, và mô hình sinh khác mô hình phân biệt ở đâu? (mục 5.1)
- Giả định "ngây thơ" là gì, và vì sao nó cứu ta khỏi lời nguyền chiều? (mục 5.2)
- Bayes ngây thơ có những biến thể nào cho các kiểu đặc trưng khác nhau (số thực, có/không, đếm từ) và cho cả KDE? (mục 5.3)
- "Huấn luyện" bằng công thức đóng nghĩa là gì, và vì sao cần làm trơn Laplace? (mục 5.4)
- Vì sao dự đoán phải tính trong không gian log, và nó liên hệ thế nào với softmax? (mục 5.5)
- Đường biên quyết định có dạng gì, và so với Logistic Regression thì sao? (mục 5.6)
- Cuối cùng là ví dụ tính tay và cài đặt từ đầu bằng Python (mục 5.7).

Quy ước ký hiệu

Ta giữ các quy ước của các mục trước: chữ thường in nghiêng là đại lượng vô hướng, **chữ thường in đậm là vector** (vector đặc trưng là vector hàng), **chữ hoa in đậm là ma trận**, đặc trưng không có cột 1 trừ khi có dấu ngã. Ma trận one-hot $\mathbf{Y}$ $(n \times K)$ quay lại giữ vai trò trung tâm, vì mọi thống kê theo lớp đều tính được bằng các phép nhân với $\mathbf{Y}^\top$.

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Số mẫu mỗi lớp	$\mathbf{t} = \mathbf{Y}^\top \mathbf{1}_n$	$K \times 1$	$t_c = n_c$ là số mẫu huấn luyện thuộc lớp c
Xác suất tiên nghiệm	$\boldsymbol{\pi}$	$K \times 1$	$\pi_c = P(y = c)$; ước lượng $\hat{\pi}_c = n_c/n$

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Mật độ theo lớp	$p(\mathbf{x} \mid c)$	vô hướng	Viết gọn của $p(\mathbf{x} \mid y = c)$
Mật độ một đặc trưng theo lớp	$p(x_j \mid c)$	vô hướng	Mật độ (hoặc xác suất) của đặc trưng j khi biết lớp c
Ma trận trung bình	$\mathbf{M}$	$K \times d$	$M_{cj} = \mu_{cj}$, trung bình của đặc trưng j ở lớp c
Ma trận phương sai	$\mathbf{Q}$	$K \times d$	$Q_{cj} = \sigma_{cj}^2$, phương sai của đặc trưng j ở lớp c
Ma trận đếm	$\mathbf{C}$	$K \times d$	C_{cj} là tổng giá trị (số lần đếm) của đặc trưng j trong lớp c
Ma trận xác suất theo lớp	$\mathbf{\Theta}$	$K \times d$	θ_{cj}: xác suất của đặc trưng (từ) j ở lớp c (mô hình Bernoulli, Multinomial)
Hệ số làm trơn	α	vô hướng	Số đếm giả cộng thêm (làm trơn Laplace)
Ma trận điểm số	$\mathbf{S}$	$m \times K$	$S_{jc} = s_c(\mathbf{x}_{0,j})$, với s_c định nghĩa ở mục 5.5
Ma trận xác suất hậu nghiệm	$\widehat{\mathbf{P}}$	$m \times K$	$\widehat{\mathbf{P}} = \mathrm{softmax}(\mathbf{S})$, mỗi hàng là một phân phối
Tích từng phần tử	$\odot$		$(\mathbf{A} \odot \mathbf{B})_{ij} = A_{ij}B_{ij}$

5.1. Định lý Bayes cho phân loại: mô hình sinh và mô hình phân biệt

Quy tắc Bayes

Ta muốn tính xác suất hậu nghiệm $P(y = c \mid \mathbf{x})$ cho từng lớp. Theo định lý Bayes (mục 4.1):

$$P(y = c \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid c)\,\pi_c}{p(\mathbf{x})}, \qquad p(\mathbf{x}) = \sum_{c'=1}^{K} p\,(\mathbf{x} \mid c')\,\pi_{c'}.$$

Mẫu số $p(\mathbf{x})$ là cùng một con số với mọi lớp, chỉ có tác dụng chuẩn hóa để các xác suất cộng lại bằng 1. Vì ta chỉ cần *so sánh* các lớp với nhau, có thể bỏ nó đi:

$$P(y = c \mid \mathbf{x}) \;\propto\; p(\mathbf{x} \mid c)\,\pi_c, \qquad \hat{y} = \operatorname*{argmax}_c\, p(\mathbf{x} \mid c)\,\pi_c.$$

Quy tắc chọn lớp có hậu nghiệm lớn nhất gọi là **quy tắc cực đại hậu nghiệm** (MAP). Nếu ta biết chính xác các mật độ và xác suất tiên nghiệm thì quy tắc này có sai số phân loại *thấp nhất có thể* trong mọi bộ phân loại, gọi là **bộ phân loại tối ưu Bayes**, và mục 6.4 sẽ nhắc lại nó như đích mà k-NN tiến tới khi $n \to \infty$. Trong thực tế ta không biết các đại lượng này, và phải *ước lượng* chúng từ dữ liệu:

- **Xác suất tiên nghiệm** π_c ước lượng bằng tỉ lệ các lớp trong tập huấn luyện: $\hat{\pi}_c = n_c/n$. Dạng ma trận: $\hat{\boldsymbol{\pi}} = \frac{1}{n}\mathbf{Y}^\top \mathbf{1}_n$, tức là trung bình các cột của $\mathbf{Y}$. (Nếu tập huấn luyện không phản ánh tỉ lệ thật ngoài đời thì ta có thể đặt π_c bằng tay.)
- **Mật độ theo lớp** $p(\mathbf{x} \mid c)$ ước lượng từ các mẫu $\mathbf{X}_c$ của lớp c bằng các phương pháp của mục 4: Gauss hợp lý cực đại, histogram, KDE, hay k láng giềng.

Mô hình sinh và mô hình phân biệt

Hai họ mô hình phân loại đối lập nhau ở chỗ chúng mô hình hóa đại lượng nào:

	Mô hình phân biệt (Logistic, Softmax)	**Mô hình sinh (Bayes ngây thơ, mục này)**
Mô hình hóa	$P(y \mid \mathbf{x})$ trực tiếp	$p(\mathbf{x} \mid y)$ và $P(y)$, rồi suy ra $P(y \mid \mathbf{x})$ bằng Bayes
Hàm mục tiêu khi học	Hợp lý có điều kiện $\sum_i \ln P(y_i \mid \mathbf{x}_i)$ (cross-entropy)	Hợp lý đồng thời $\sum_i \ln p(\mathbf{x}_i, y_i)$
Cách học	Lặp (Gradient Descent)	Thường là công thức đóng (đếm, lấy trung bình)
Biết thêm gì	Chỉ ranh giới giữa các lớp	Cả cách dữ liệu của từng lớp được sinh ra (có thể sinh mẫu mới, phát hiện bất thường)
Giả định	Dạng của $P(y \mid \mathbf{x})$	Dạng của $p(\mathbf{x} \mid y)$

Hai đại lượng cuối cùng là hai cách phân tích cùng một xác suất đồng thời $p(\mathbf{x}, y) = p(\mathbf{x} \mid y)P(y) = P(y \mid \mathbf{x})p(\mathbf{x})$. Hướng sinh "khó" hơn theo nghĩa phải ước lượng một đối tượng phức tạp hơn (mật độ của $\mathbf{x}$, không chỉ ranh giới), và điều đó chính là chỗ lời nguyền chiều chen vào. Phần còn lại của mục 5 là câu chuyện làm cho hướng sinh khả thi ở số chiều cao.

5.2. Giả định "ngây thơ" (độc lập có điều kiện) và vì sao nó cứu ta khỏi lời nguyền chiều

Giả định

Giả định của Bayes ngây thơ là: **biết lớp c rồi, các đặc trưng độc lập với nhau**. Khi đó mật độ đồng thời d chiều phân rã thành tích các mật độ một chiều:

$$p(\mathbf{x} \mid c) = p(x_1, x_2, \ldots, x_d \mid c) = \prod_{j=1}^{d} p\left(x_j \mid c\right).$$

Cần phân biệt kỹ hai khái niệm. Đây là độc lập **có điều kiện theo lớp**, *không* phải độc lập thông thường: số giờ tự học và kết quả học tập rõ ràng có liên quan, nhưng trong *riêng* nhóm sinh viên đạt, giả định nói rằng số giờ tự học và số buổi vắng không cho thêm thông tin về nhau. Giả định

này thường chỉ đúng gần đúng, ví dụ trong một văn bản thì sự xuất hiện của từ "bóng" làm tăng khả năng xuất hiện từ "đá" ngay cả khi đã biết chủ đề là thể thao. Vì thế nó được gọi là "ngây thơ". Thế vào quy tắc Bayes ta được bộ phân loại Bayes ngây thơ:

$$\hat{y} = \operatorname*{argmax}_c \left[\pi_c \prod_{j=1}^{d} p\left(x_j \mid c\right) \right].$$

Vì sao nó cứu ta khỏi lời nguyền chiều

Ở mục 4.6, ta thấy KDE d chiều có sai số giảm như $n^{-4/(d+4)}$, nên với d lớn gần như không dùng được. Với giả định ngây thơ, ta chỉ cần ước lượng d mật độ **một chiều** cho mỗi lớp, mỗi mật độ có sai số giảm như $n_c^{-4/5}$, *không phụ thuộc* d. Chi phí chỉ tăng *tuyến tính* theo d thay vì theo cấp số nhân. Ví dụ đếm số tham số cần học cho mỗi lớp:

Bài toán	Mô hình đồng thời đầy đủ	Bayes ngây thơ
d đặc trưng nhị phân	$2^d - 1$ xác suất (một cho mỗi tổ hợp)	d xác suất
$d = 20$ nhị phân	1 048 575	20
$d = 64$ nhị phân	$2^{64} - 1 \approx 1{,}8 \times 10^{19}$	64
d đặc trưng thực, mô hình Gauss	$d + \frac{d(d+1)}{2}$ (trung bình và hiệp phương sai đầy đủ)	$2d$ (trung bình và phương sai từng chiều)
$d = 64$ thực, Gauss	$64 + 2\,080 = 2\,144$	128

Với $d = 20$ đặc trưng nhị phân, mô hình đầy đủ cần hơn một triệu xác suất, và với vài nghìn mẫu huấn luyện thì hầu hết các tổ hợp không bao giờ xuất hiện; còn Bayes ngây thơ chỉ cần 20 con số mỗi lớp, mỗi con số ước lượng được từ toàn bộ n_c mẫu.

Đánh đổi: tăng thiên lệch để giảm phương sai

Nói theo ngôn ngữ của mục 1.3, giả định ngây thơ là một sự **đánh đổi**: nó đưa vào thiên lệch (vì giả định thường sai) nhưng giảm mạnh phương sai (vì có rất ít con số cần ước lượng). Khi d lớn hoặc dữ liệu ít, phương sai là kẻ thù chính, nên việc chấp nhận một chút thiên lệch thường đáng giá. Điều đáng chú ý là ngay cả khi giả định sai, *thứ tự* các xác suất hậu nghiệm vẫn thường đúng, nên việc chọn lớp có thể vẫn tốt dù các xác suất bị méo (ta sẽ thấy ví dụ ở mục 5.6).

5.3. Các biến thể: Gauss, Bernoulli, Multinomial và Bayes ngây thơ dùng KDE

Giả định ngây thơ chỉ cho biết $p(\mathbf{x} \mid c)$ phân rã thành tích; ta vẫn phải chọn dạng của từng mật độ một chiều $p\left(x_j \mid c\right)$, và lựa chọn đó phụ thuộc kiểu đặc trưng:

Biến thể	Kiểu đặc trưng x_j	$p(x_j \mid c)$	Tham số cho mỗi (c, j)
Gauss	Số thực (chiều cao, điểm, giờ học)	$\frac{1}{\sqrt{2\pi\sigma_{cj}^2}} \exp\left(-\frac{(x_j - \mu_{cj})^2}{2\sigma_{cj}^2}\right)$	μ_{cj}, σ_{cj}^2
Bernoulli	Nhị phân (0/1), ví dụ "từ có xuất hiện"	$\theta_{cj}^{x_j}(1-\theta_{cj})^{1-x_j}$	$\theta_{cj} = P(x_j = 1 \mid c)$
Multinomial	Số lần đếm (số lần từ j xuất hiện)	$\propto \theta_{cj}^{x_j}$, với $\sum_j \theta_{cj} = 1$	θ_{cj}
KDE	Số thực, không biết dạng	$\hat{f}_{cj}(x_j) = \frac{1}{n_c h_{cj}} \sum_{i \in c} \kappa\left(\frac{x_j - x_{ij}}{h_{cj}}\right)$	Dữ liệu và h_{cj}

Vài điểm cần làm rõ.

- **Gauss.** Là lựa chọn mặc định cho đặc trưng liên tục. Mỗi đặc trưng ở mỗi lớp có một Gauss một chiều riêng, nên mô hình có $2Kd$ tham số cộng với $K-1$ tham số của $\boldsymbol{\pi}$. Khác mô hình Gauss đầy đủ ở mục 4.6, ở đây hiệp phương sai theo lớp là **ma trận đường chéo** $\text{diag}(\sigma_{c1}^2, \ldots, \sigma_{cd}^2)$.
- **Bernoulli.** Mỗi đặc trưng là câu hỏi có/không (từ j có trong văn bản hay không). Mô hình này **phạt cả sự vắng mặt**: số hạng $(1-\theta_{cj})^{1-x_j}$ đóng góp khi từ không xuất hiện.
- **Multinomial.** Mô hình văn bản như một chuỗi $L = \sum_j x_j$ lần bốc từ độc lập từ một túi từ có xác suất θ_{cj}. Xác suất đầy đủ là $\frac{L!}{\prod_j x_j!} \prod_j \theta_{cj}^{x_j}$; hệ số đầu không phụ thuộc lớp c nên triệt tiêu khi so sánh các lớp và ta bỏ qua. Mô hình này chỉ dùng các từ *xuất hiện*, nên khác Bernoulli.
- **KDE.** Dùng ước lượng mật độ nhân một chiều của mục 4.3 cho từng cặp (lớp, đặc trưng). Đây là cách kết hợp trực tiếp mục 4 với mục 5: ta không còn giả định mỗi đặc trưng có dạng Gauss, nhưng vẫn giữ giả định độc lập giữa các đặc trưng.

Một nhận xét về cách phân loại các mô hình theo thuật ngữ của mục 1.3: Bayes ngây thơ Gauss, Bernoulli và Multinomial là mô hình **tham số** (số tham số $2Kd + K - 1$ hoặc $Kd + K - 1$ cố định, không tăng theo n), còn Bayes ngây thơ dùng KDE là **phi tham số** (phải giữ lại dữ liệu theo lớp). Đây là một ví dụ cho thấy ranh giới giữa hai họ mô hình không tách bạch, như đã báo trước ở mục 1.3.

5.4. "Huấn luyện" bằng công thức đóng (đếm và lấy trung bình), làm trơn Laplace

Nghiệm hợp lý cực đại là công thức đóng

Với Bayes ngây thơ, hợp lý đồng thời phân rã thành các số hạng *tách rời* cho từng (c, j), nên mỗi tham số được ước lượng riêng rẽ, và nghiệm hợp lý cực đại chỉ là các phép đếm và lấy trung bình quen thuộc. Với biến thể Gauss, tối đa hóa $\sum_{i \in c} \ln \mathcal{N}\left(x_{ij}; \mu_{cj}, \sigma_{cj}^2\right)$ cho trung bình mẫu và phương sai mẫu (như ở mục 4.1) của đặc trưng j trong các mẫu thuộc lớp c:

$$\hat{\mu}_{cj} = \frac{1}{n_c} \sum_{i:\, y_i = c} x_{ij}\,, \qquad \hat{\sigma}_{cj}^2 = \frac{1}{n_c} \sum_{i:\, y_i = c} (\,x_{ij} - \hat{\mu}_{cj})^2.$$

Nhờ ma trận one-hot $\mathbf{Y}$ (cột c chọn đúng các mẫu của lớp c), toàn bộ các thống kê này tính được bằng vài phép nhân ma trận mà không cần vòng lặp qua lớp:

$$\widehat{\boldsymbol{\pi}} = \frac{1}{n}\,\mathbf{t}, \qquad \widehat{\mathbf{M}} = \operatorname{diag}(\mathbf{t})^{-1}\,\mathbf{Y}^{\top}\mathbf{X}, \qquad \widehat{\mathbf{Q}} = \operatorname{diag}(\mathbf{t})^{-1}\,\mathbf{Y}^{\top}(\mathbf{X} \odot \mathbf{X}) - \widehat{\mathbf{M}} \odot \widehat{\mathbf{M}}.$$

Cụ thể $\mathbf{Y}^{\top}\mathbf{X}$ $(K \times d)$ có hàng c là tổng các vector đặc trưng của lớp c; chia cho n_c (phép nhân với $\operatorname{diag}(\mathbf{t})^{-1}$) cho trung bình. Số hạng thứ hai trong $\widehat{\mathbf{Q}}$ dùng công thức "trung bình của bình phương trừ bình phương của trung bình". Để tránh phương sai bằng 0 (khi một đặc trưng không đổi trong một lớp), người ta cộng thêm một sàn rất nhỏ, ví dụ 10^{-9} lần phương sai lớn nhất của dữ liệu.

Với các biến thể rời rạc, tham số là các tần suất tương đối. Gọi $\mathbf{C} = \mathbf{Y}^{\top}\mathbf{X}$ là **ma trận đếm** $(K \times d)$: C_{cj} là tổng số lần đặc trưng j xuất hiện trong các mẫu của lớp c (với Bernoulli, là số mẫu lớp c mà đặc trưng j bằng 1). Khi đó nghiệm hợp lý cực đại là

$$\hat{\theta}_{cj} = \frac{C_{cj}}{\sum_{l=1}^{d} C_{cl}} \quad \text{(Multinomial)}, \qquad \hat{\theta}_{cj} = \frac{C_{cj}}{n_c} \quad \text{(Bernoulli)}.$$

Cái tên "huấn luyện" nghe hơi quá: không có hàm mất mát, không có gradient, không có tốc độ học hay vòng lặp, chỉ có **một lượt đếm qua dữ liệu**, chi phí cỡ $O(nd)$. Đây là điểm khác biệt lớn với Softmax Regression, vốn phải lặp Gradient Descent hàng trăm bước (bài Softmax).

Vấn đề số 0 và làm trơn Laplace

Công thức tần suất ở trên có một lỗ hổng chết người: nếu từ j chưa bao giờ xuất hiện trong một lớp c trong tập huấn luyện thì $\hat{\theta}_{cj} = 0$, và với một văn bản mới *chứa từ đó* ta có $\prod_j \hat{\theta}_{cj}^{\,x_j} = 0$ bất kể các từ khác nói gì (hay $\ln 0 = -\infty$ trong không gian log). Một từ lạ duy nhất đủ để *loại hẳn* lớp đó. Đây chính là vấn đề "ô rỗng" của histogram ở mục 4.2 và xác suất 0 của k-NN ở mục 2.2, và cách chữa cũng giống nhau: **làm trơn Laplace**, thêm vào mỗi số đếm một lượng "đếm giả" $\alpha > 0$:

$$\hat{\theta}_{cj} = \frac{C_{cj} + \alpha}{\sum_{l=1}^{d} C_{cl} + \alpha\, d} \quad \text{(Multinomial)}, \qquad \hat{\theta}_{cj} = \frac{C_{cj} + \alpha}{n_c + 2\alpha} \quad \text{(Bernoulli)}.$$

Mẫu số được điều chỉnh đúng bằng tổng lượng đếm giả thêm vào để $\sum_j \hat{\theta}_{cj} = 1$ (Multinomial, d đặc trưng nhận α mỗi đặc trưng) hoặc để hai xác suất $x_j = 1$ và $x_j = 0$ cộng lại bằng 1 (Bernoulli, hai khả năng nhận α mỗi khả năng). Với $\alpha = 1$ ta có **làm trơn Laplace** (cộng một), với $0 < \alpha < 1$ là làm trơn Lidstone. Làm trơn đóng vai trò như điều chuẩn hóa ở các bài trước: α lớn kéo mọi $\hat{\theta}_{cj}$ về giá trị đồng đều $1/d$ (thiên lệch cao), $\alpha \to 0$ trở về tần suất thô (phương sai cao). Nó là siêu tham số thứ ba, cùng với k và h, và cũng được chọn bằng kiểm định chéo (mục 7.4).

5.5. Dự đoán trong không gian log và quan hệ với softmax

Vì sao phải dùng logarit

Công thức $\pi_c \prod_{j=1}^{d} p\left(x_j \mid c\right)$ là tích của $d + 1$ số, mỗi số thường nhỏ hơn 1. Với văn bản có $d = 500$ từ, mỗi xác suất cỡ 0,005, tích là cỡ $10^{-1\,150}$, nhỏ hơn rất nhiều so với số dương nhỏ nhất mà số thực 64 bit biểu diễn được (cỡ 10^{-308}): máy tính làm tròn thành 0 (hiện tượng **tràn số dưới**) và mọi lớp đều có điểm 0, không so sánh được. Vì hàm ln tăng nghiêm ngặt, việc so sánh các tích tương đương so sánh các **tổng logarit**, không bao giờ tràn số:

$$s_c(\mathbf{x}) = \ln \pi_c + \sum_{j=1}^{d} \ln p\left(x_j \mid c\right), \qquad \hat{y} = \operatorname*{argmax}_c \, s_c(\mathbf{x}).$$

Với ví dụ trên, tổng là $500 \times \ln 0{,}005 \approx -2\,649$, một con số bình thường. Một lợi ích nữa: phép nhân biến thành phép cộng, nên ta có thể dùng các phép nhân ma trận.

Xác suất hậu nghiệm chính là softmax của điểm số

Nếu muốn các xác suất $P(y = c \mid \mathbf{x})$ chứ không chỉ nhãn, ta chuẩn hóa tích $\pi_c \prod_j p\left(x_j \mid c\right) = e^{s_c}$ bằng cách chia cho tổng:

$$\hat{P}(y = c \mid \mathbf{x}) = \frac{e^{s_c(\mathbf{x})}}{\sum_{c'=1}^{K} e^{s_{c'}(\mathbf{x})}} = \operatorname{softmax}(s_1, \ldots, s_K)_c.$$

Đây chính xác là hàm softmax của bài Softmax Regression, với s_c đóng vai trò của điểm số (logit). Để tính ổn định, ta dùng đúng thủ thuật *trừ giá trị lớn nhất* của bài Softmax: $\ln \sum_{c'} e^{s_{c'}} = s_{\max} + \ln \sum_{c'} e^{s_{c'} - s_{\max}}$.

Dạng ma trận: Bayes ngây thơ là một mô hình softmax với trọng số tính bằng đếm

Với biến thể Multinomial (hoặc Bernoulli, thay phần dưới tương ứng), điểm số của m điểm truy vấn là

$$\mathbf{S} = \mathbf{1}_m \ln \hat{\boldsymbol{\pi}}^{\top} + \mathbf{X}_0 \left(\ln \hat{\boldsymbol{\Theta}}\right)^{\top}, \qquad \hat{\mathbf{P}} = \operatorname{softmax}(\mathbf{S}) \;\; \text{(theo từng hàng)},$$

trong đó ln lấy từng phần tử của ma trận. Viết lại bằng cách thêm cột 1 vào $\mathbf{X}_0$ (như bài Softmax), ta được:

$$\mathbf{S} = \tilde{\mathbf{X}}_0\,\mathbf{W}_{\text{NB}}, \qquad \tilde{\mathbf{X}}_0 = [\,\mathbf{1}_m \quad \mathbf{X}_0\,], \qquad \mathbf{W}_{\text{NB}} = \begin{pmatrix} \ln\hat{\boldsymbol{\pi}}^{\top} \\ (\ln\hat{\boldsymbol{\Theta}})^{\top} \end{pmatrix} \in \mathbb{R}^{(d+1)\times K}.$$

So sánh với Softmax Regression $\mathbf{P} = \text{softmax}(\tilde{\mathbf{X}}\mathbf{W})$: **cùng một công thức**, cùng một hình dạng $\mathbf{W}$ kích thước $(d+1)\times K$. Khác biệt duy nhất nằm ở cách có được $\mathbf{W}$: Softmax Regression tìm $\mathbf{W}$ bằng Gradient Descent để cực tiểu cross-entropy (hợp lý có điều kiện), còn Bayes ngây thơ *đặt luôn* $\mathbf{W}_{\text{NB}}$ bằng logarit của các tần suất đếm được (hợp lý đồng thời, công thức đóng). Hai mô hình cùng sống trong một họ hàm; chúng là hai cách ước lượng khác nhau của cùng một dạng.

Với biến thể Gauss, điểm số là

$$s_c(\mathbf{x}) = \ln\pi_c - \frac{1}{2}\sum_{j=1}^{d}\ln\left(2\pi\sigma_{cj}^2\right) - \frac{1}{2}\sum_{j=1}^{d}\frac{\left(x_j-\mu_{cj}\right)^2}{\sigma_{cj}^2},$$

là một hàm **bậc hai** của $\mathbf{x}$ (có x_j^2) nhưng *không có tích chéo* $x_j x_l$ vì các chiều độc lập. Có thể coi nó như một Softmax Regression trên các đặc trưng $[\,x_j,\, x_j^2\,]$, tức là đặc trưng đa thức bậc hai (không có tích chéo) quen thuộc từ mục 4.5 của bài Softmax.

5.6. Đường biên quyết định: tuyến tính hay bậc hai, so với Logistic Regression

Dạng của đường biên

Ranh giới giữa hai lớp a và b là tập các điểm có hai điểm số bằng nhau $s_a(\mathbf{x}) = s_b(\mathbf{x})$, tức hiệu số logarit của hai hậu nghiệm bằng 0:

$$\ln\frac{P(y=a\mid\mathbf{x})}{P(y=b\mid\mathbf{x})} = s_a(\mathbf{x}) - s_b(\mathbf{x}) = 0.$$

- **Bernoulli và Multinomial**: hiệu này **tuyến tính** theo $\mathbf{x}$,

  $$s_a - s_b = \ln\frac{\pi_a}{\pi_b} + \sum_{j=1}^{d} x_j\,\ln\frac{\theta_{aj}}{\theta_{bj}},$$

  nên ranh giới là một siêu phẳng, giống Logistic và Softmax. (Với Bernoulli, thêm số hạng hằng $\sum_j \ln\frac{1-\theta_{aj}}{1-\theta_{bj}}$ và hệ số của x_j là $\ln\frac{\theta_{aj}(1-\theta_{bj})}{\theta_{bj}(1-\theta_{aj})}$, vẫn tuyến tính.)

- **Gauss với phương sai riêng theo lớp** ($\sigma_{aj} \neq \sigma_{bj}$): hiệu là

  $$s_a - s_b = \ln\frac{\pi_a}{\pi_b} + \sum_{j=1}^{d}\left[\frac{1}{2}\ln\frac{\sigma_{bj}^2}{\sigma_{aj}^2} - \frac{\left(x_j-\mu_{aj}\right)^2}{2\sigma_{aj}^2} + \frac{\left(x_j-\mu_{bj}\right)^2}{2\sigma_{bj}^2}\right],$$

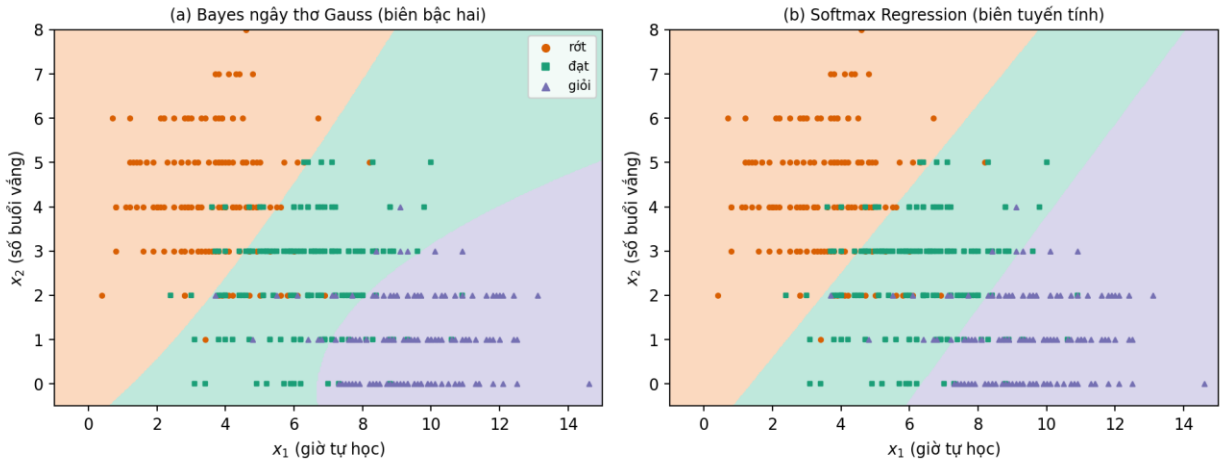
một đa thức **bậc hai** trong từng x_j nhưng không có tích chéo, nên ranh giới là một mặt bậc hai có các trục *song song với trục tọa độ*.

- **Gauss với phương sai chung** ($\sigma_{aj} = \sigma_{bj} = \sigma_j$): các số hạng x_j^2 triệt tiêu, còn lại

$$s_a - s_b = \ln\frac{\pi_a}{\pi_b} + \sum_{j=1}^{d} \frac{\mu_{aj} - \mu_{bj}}{\sigma_j^2}\, x_j - \sum_{j=1}^{d} \frac{\mu_{aj}^2 - \mu_{bj}^2}{2\sigma_j^2},$$

một hàm **tuyến tính**, hệ số của x_j là hiệu hai trung bình chia cho phương sai.

Hình 18 so sánh vùng quyết định của Bayes ngây thơ Gauss (phương sai riêng theo lớp) và Softmax Regression trên 600 sinh viên mô phỏng của mục 2.3. Biên của Bayes ngây thơ cong nhẹ (bậc hai), của Softmax thẳng.



Hình 18. Vùng quyết định trên tập huấn luyện của 600 *sinh viên mô phỏng. (a) Bayes ngây thơ Gauss: biên bậc hai (cong nhẹ), các trục song song với trục tọa độ. (b) Softmax Regression: biên là các đường thẳng.*

Mô hình	Độ chính xác kiểm tra	Log-loss kiểm tra
Bayes ngây thơ Gauss	82,2%	0,414
Softmax Regression	84,4%	0,423
k-NN ($k = 18$, mục 2.4)	82,2%	
Bayes ngây thơ KDE (Silverman theo từng cặp lớp, đặc trưng)	80,6%	0,439

Trên dữ liệu này (hai đặc trưng thực sự độc lập trong từng lớp, vì ta sinh chúng độc lập), Bayes ngây thơ ngang với Softmax: chênh 2,2 điểm phần trăm trên 180 mẫu kiểm tra chỉ là 4 sinh viên, và sai số chuẩn của độ chính xác ở cỡ mẫu này vào khoảng 2,8 điểm, nên chưa đủ để kết luận mô hình nào tốt hơn. Đáng chú ý là log-loss của Bayes ngây thơ (0,414) hơi *thấp hơn*

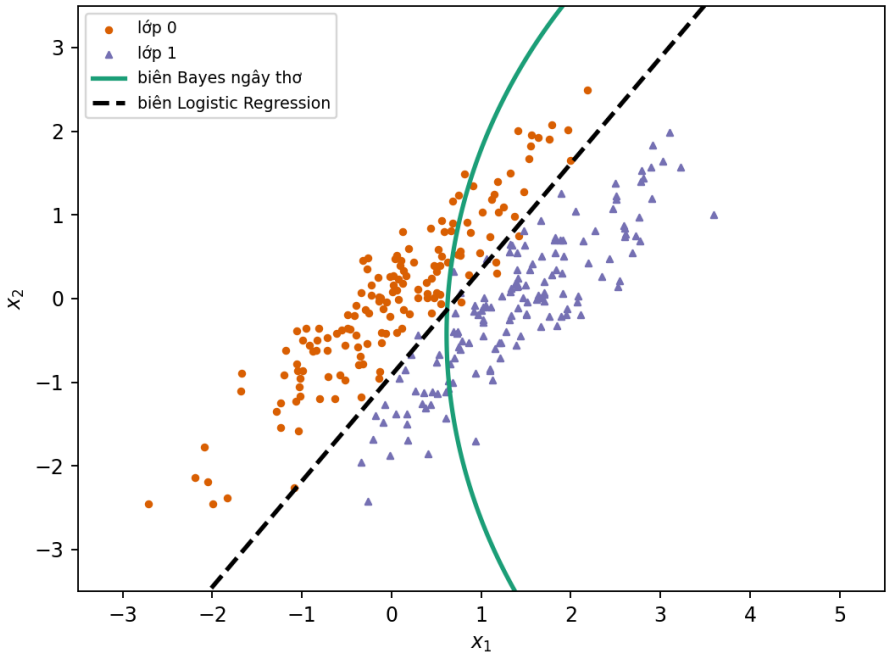
Softmax (0,423) dù độ chính xác thấp hơn một chút; và phiên bản dùng KDE (80,6%) không tốt hơn bản Gauss, vì dữ liệu gần Gauss nên không có lợi ích gì khi bỏ giả định đó.

So sánh với Logistic Regression: cùng dạng, khác cách ước lượng

Với hai lớp, Logistic Regression có log-odds tuyến tính $\ln\frac{P(1|\mathbf{x})}{P(0|\mathbf{x})} = \tilde{\mathbf{x}}\mathbf{w}$. Theo các công thức trên, Bayes ngây thơ (Bernoulli, Multinomial hoặc Gauss phương sai chung) cũng có log-odds tuyến tính. Chúng cùng họ ranh giới (siêu phẳng) và khác nhau ở cách chọn $\mathbf{w}$:

- **Bayes ngây thơ** ước lượng *từng hệ số* w_j *riêng rẽ* từ thống kê của riêng đặc trưng j (ví dụ $(\mu_{1j} - \mu_{0j})/\sigma_j^2$), không quan tâm các đặc trưng khác.
- **Logistic Regression** chọn *toàn bộ* $\mathbf{w}$ *cùng lúc* để cực tiểu cross-entropy, nên các hệ số điều chỉnh lẫn nhau, tính đến cả sự tương quan giữa các đặc trưng.

Sự khác biệt này có hệ quả rõ rệt khi giả định ngây thơ **bị vi phạm**. Hình 19 dùng một bài toán hai lớp hai đặc trưng, mỗi lớp là một Gauss với hiệp phương sai chung $\begin{pmatrix} 1 & 0{,}9 \\ 0{,}9 & 1 \end{pmatrix}$ (tương quan 0,9) và hai trung bình $(0{,}0)$ và $(1{,}5; 0)$; 300 mẫu mỗi lớp, một nửa để huấn luyện, một nửa để kiểm tra.



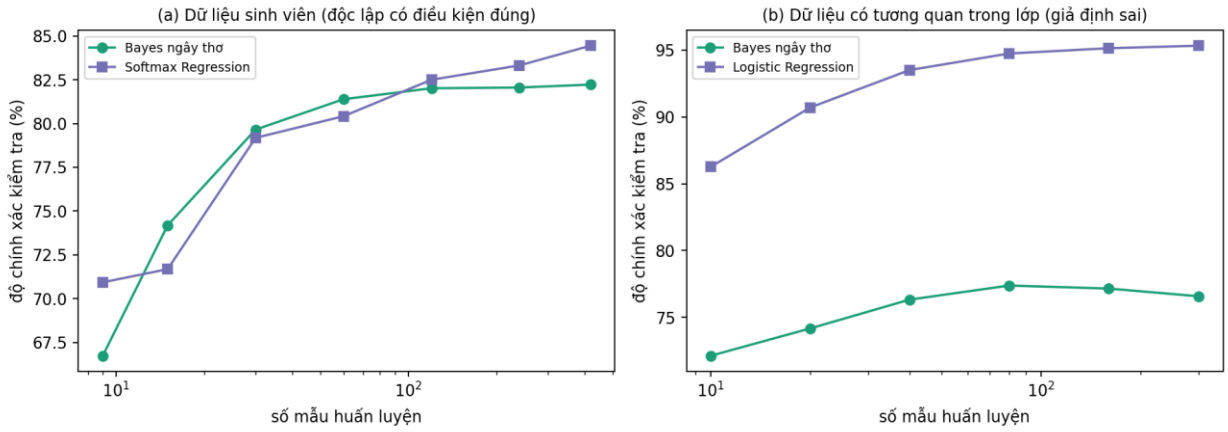
Hình 19. Hai lớp có đặc trưng tương quan mạnh (hệ số tương quan 0,9 *trong mỗi lớp). Biên của Bayes ngây thơ (nét xanh lục) gần như thẳng đứng, chỉ dùng* x_1 *vì trung bình của* x_2 *như nhau ở hai lớp. Biên của Logistic Regression (nét đứt đen) nghiêng theo hướng của sự tương quan và tách hai lớp tốt hơn nhiều.*

Mô hình	Độ chính xác (tập kiểm tra)	Log-loss	Hệ số trên (x_1, x_2)
Bayes ngây thơ Gauss	80,0%	0,429	$\approx (1{,}77;\ -0{,}11)$
Logistic Regression	95,0%	0,133	$(4{,}58;\ -3{,}62)$
Gauss đầy đủ, hiệp phương sai chung (LDA)	95,7%	0,118	

Bayes ngây thơ chỉ đạt 80,0% so với 95,0% của Logistic Regression. Nguyên nhân: trung bình của x_2 gần như bằng nhau ở hai lớp (0,014 so với $-0{,}077$), nên Bayes ngây thơ nhìn *riêng* x_2 thấy nó vô dụng và gán hệ số xấp xỉ 0 (hệ số $-0{,}11$ trong bảng). Nhưng trong thực tế x_2 rất hữu ích *khi kết hợp* với x_1: vì hai đặc trưng gần như cùng dao động, $x_1 - x_2$ có phương sai nhỏ nên tách hai lớp tốt. Logistic Regression học được điều này (hệ số $-3{,}62$ cho x_2); Bayes ngây thơ thì không. Kết quả này không phải do tập kiểm tra nhỏ: khi đánh giá cả hai mô hình (huấn luyện trên toàn bộ 600 mẫu) trên 200 000 mẫu mới, Bayes ngây thơ đạt 77,0% còn LDA đạt 95,7%. Và đường cong học ở Hình 21b (bên dưới) cho thấy Bayes ngây thơ gần như không cải thiện khi tăng số mẫu. Vậy sai số là **thiên lệch** do giả định sai, không phải phương sai do thiếu dữ liệu.

Sinh và phân biệt: ai cần bao nhiêu dữ liệu?

Hình 21 vẽ độ chính xác theo số mẫu huấn luyện (trung bình 40 lần lấy mẫu con) cho hai bài toán trên.



Hình 21. Độ chính xác kiểm tra theo số mẫu huấn luyện. (a) Dữ liệu sinh viên (giả định ngây thơ đúng): Bayes ngây thơ nhỉnh hơn Softmax khi mẫu ít (n từ 15 đến 60), ngang nhau ở khoảng $n = 120$, và Softmax nhỉnh hơn khi $n \geq 240$. (b) Dữ liệu có tương quan (giả định sai): Bayes ngây thơ chạm trần $\approx 77\%$ từ khoảng 80 mẫu, còn Logistic Regression liên tục tăng lên 95%.

Hai đồ thị minh họa một kết luận kinh điển (Ng và Jordan) về mô hình sinh và mô hình phân biệt: mô hình sinh với giả định đơn giản **đạt đến sai số tiệm cận của nó nhanh hơn** (ít tham số, ước lượng đếm nên ít phương sai), nhưng sai số tiệm cận đó **cao hơn** nếu giả định sai; mô hình phân biệt cần nhiều dữ liệu hơn nhưng tiến tới sai số tiệm cận thấp hơn. Ở đồ thị (a), Bayes ngây thơ dẫn đầu ở $n = 15$ (74,2% so với 71,7%), $n = 30$ (79,7% so với 79,2%) và $n = 60$ (81,4% so với 80,4%), nhưng ở $n = 9$ (chỉ 3 mẫu mỗi lớp) nó thua (66,7% so với 70,9%) vì ước lượng

phương sai từ 3 mẫu rất nhiễu, và ở $n = 420$ Softmax dẫn (84,4% so với 82,2%). Các chênh lệch này đều nhỏ so với dao động ngẫu nhiên nên chỉ nên đọc như xu hướng. Ở (b), xu hướng rõ ràng hơn nhiều: khoảng cách giữa hai đường (từ 14 điểm ở $n = 10$ lên 19 điểm ở $n = 300$) không thu hẹp.

Xác suất của Bayes ngây thơ thường "quá tự tin"

Dù độ chính xác phân loại có thể vẫn tốt, các xác suất hậu nghiệm của Bayes ngây thơ thường bị méo. Lý do: khi các đặc trưng tương quan, giả định ngây thơ **đếm cùng một bằng chứng nhiều lần**. Để thấy rõ, ta lấy dữ liệu sinh viên ở trên và sao chép đặc trưng x_1 thêm $m - 1$ lần (thêm $m - 1$ cột giống hệt nó). Với Softmax, thêm các cột trùng lặp không thêm thông tin mới, còn Bayes ngây thơ coi chúng là m bằng chứng độc lập:

Số bản sao của x_1	Bayes ngây thơ: độ chính xác	Log-loss	Độ tin cậy trung bình	Softmax: độ chính xác	Log-loss
1	82,2%	0,414	0,829	84,4%	0,423
2	81,7%	0,524	0,899	84,4%	0,422
5	78,9%	1,243	0,959	84,4%	0,422
10	78,9%	2,614	0,979	82,8%	0,422

Khi số bản sao tăng, độ tin cậy trung bình của Bayes ngây thơ tăng từ 0,83 lên 0,98 trong khi độ chính xác *giảm* từ 82,2% xuống 78,9%, nên log-loss tăng vọt từ 0,414 lên 2,614. Softmax gần như không đổi (0,422). Bài học: dùng Bayes ngây thơ để **xếp hạng** các lớp thì tốt, nhưng nếu cần xác suất đáng tin cậy (ví dụ để ra quyết định theo ngưỡng), phải hiệu chỉnh lại các xác suất (mục 7.2) hoặc dùng mô hình phân biệt.

5.7. Ví dụ tính tay và mã hóa Bayes ngây thơ từ đầu bằng Python

Ví dụ 1: Bayes ngây thơ Gauss trên 9 sinh viên

Ta dùng lại 9 sinh viên (rớt $= 0$, đạt $= 1$, giỏi $= 2$, mỗi lớp 3 người) và ba sinh viên mới $\mathbf{X}_0$ ở mục 2.6. **Huấn luyện**: tính trung bình và phương sai (chia cho $n_c = 3$) của hai đặc trưng trong từng lớp, và $\hat{\pi}_c = 3/9 = 1/3$:

$$\widehat{\mathbf{M}} = \begin{pmatrix} 3{,}000 & 4{,}000 \\ 6{,}333 & 3{,}000 \\ 9{,}000 & 0{,}333 \end{pmatrix}, \qquad \widehat{\mathbf{Q}} = \begin{pmatrix} 0{,}667 & 0{,}667 \\ 0{,}222 & 0{,}667 \\ 0{,}667 & 0{,}222 \end{pmatrix}.$$

Ví dụ lớp "rớt" gồm các sinh viên $(2,4), (3,3), (4,5)$: trung bình $(3; 4)$, phương sai x_1: $\frac{(2-3)^2+0+(4-3)^2}{3} = 0{,}667$, và phương sai x_2: $\frac{0+1+1}{3} = 0{,}667$.

Dự đoán cho sinh viên $\mathbf{x}_0 = [8,2]$. Điểm số $s_c = \ln\pi_c + \ln p(x_1 \mid c) + \ln p(x_2 \mid c)$, trong đó $\ln\pi_c = \ln\frac{1}{3} = -1{,}0986$ chung cho mọi lớp. Các số hạng logarit mật độ của từng đặc trưng:

Lớp	$\ln p(x_1 = 8 \mid c)$	$\ln p(x_2 = 2 \mid c)$	s_c
rớt	$-19{,}466$	$-3{,}716$	$-24{,}281$
đạt	$-6{,}417$	$-1{,}466$	$-8{,}982$
giỏi	$-1{,}466$	$-6{,}417$	$-8{,}982$

Chẳng hạn lớp "đạt": $x_1 = 8$ cách $\hat{\mu}_{11} = 6{,}333$ một khoảng $1{,}667$ với phương sai chỉ $0{,}222$ nên $\ln p = -\frac{1}{2}\ln(2\pi \times 0{,}222) - \frac{1{,}667^2}{2\times 0{,}222} = -0{,}168 - 6{,}250 = -6{,}417$. Ma trận điểm số và xác suất hậu nghiệm cho cả ba sinh viên mới (mỗi hàng một sinh viên, các cột là rớt, đạt, giỏi):

$$\mathbf{S} = \begin{pmatrix} -6{,}281 & -5{,}982 & -29{,}982 \\ -24{,}281 & -8{,}982 & -8{,}982 \\ -5{,}531 & -27{,}732 & -35{,}232 \end{pmatrix}, \qquad \widehat{\mathbf{P}} = \mathrm{softmax}(\mathbf{S}) \approx \begin{pmatrix} 0{,}426 & 0{,}574 & 0 \\ 0 & 0{,}500 & 0{,}500 \\ 1 & 0 & 0 \end{pmatrix}.$$

Dự đoán lần lượt là **đạt**, **hòa giữa đạt và giỏi**, **rớt**. Nhận xét: sinh viên thứ ba rất chắc chắn là rớt (xác suất gần 1). Sinh viên thứ nhất thuộc vùng biên rớt–đạt (xác suất 0,43 và 0,57), kết quả "đạt" cùng k-NN ở mục 2.6. Sinh viên thứ hai $[8,2]$ cho hai điểm số **bằng nhau chính xác** $(-8{,}982)$: đây là một sự trùng hợp số học: với lớp "đạt", đặc trưng x_1 lệch $1{,}667$ so với trung bình và có phương sai $0{,}222$, còn x_2 lệch 1 với phương sai $0{,}667$; với lớp "giỏi" hai cặp số này *hoán đổi vai trò* (x_1: lệch 1, phương sai $0{,}667$; x_2: lệch $1{,}667$, phương sai $0{,}222$), nên tổng hai số hạng bằng nhau. Bayes ngây thơ nói thật rằng nó không phân biệt được hai lớp. Cũng trong mục 2.6, k-NN bỏ phiếu đa số dự đoán "giỏi" còn k-NN có trọng số dự đoán "đạt", nên ba phương pháp bất đồng đúng ở điểm này, một điểm nằm sát biên. Khi hòa, ta dùng quy tắc xác định (chọn lớp có chỉ số nhỏ hơn, như mục 2.2).

Ví dụ 2: Bayes ngây thơ Multinomial và làm trơn Laplace

Một hệ thống lọc bình luận có từ vựng $d = 4$ từ: "thưởng", "miễn_phí", "họp", "bài". Có 5 bình luận huấn luyện; mỗi dòng là vector đếm từ $[\text{thưởng,miễn_phí,họp,bài}]$:

Bình luận	Đếm từ	Lớp
1	[2,1,0,0]	rác (spam)
2	[1,2,0,1]	rác
3	[0,0,1,2]	bình thường
4	[1,0,1,1]	bình thường
5	[0,0,2,1]	bình thường

Tiên nghiệm: $\hat{\pi}_{\text{rác}} = 2/5 = 0{,}4$, $\hat{\pi}_{\text{bt}} = 3/5 = 0{,}6$. Ma trận đếm $\mathbf{C} = \mathbf{Y}^{\top}\mathbf{X}$ (hàng: bình thường, rác; cột: bốn từ) và tổng số từ mỗi lớp:

$$\mathbf{C} = \begin{pmatrix} 1 & 0 & 4 & 4 \\ 3 & 3 & 0 & 1 \end{pmatrix}, \qquad \text{tổng từ: } 9 \text{ (bình thường)}, \quad 7 \text{ (rác)}.$$

Không làm trơn ($\alpha = 0$): $\hat{\theta}_{\text{bt}} = (1/9\,,0,4/9\,,4/9)$ và $\hat{\theta}_{\text{rác}} = (3/7\,,3/7\,,0,1/7)$. Xét bình luận mới có các từ "thưởng", "miễn_phí" và "họp" (đếm $\mathbf{x}_0 = [1,1,1,0]$). Lớp bình thường có

$\hat{\theta}_{\text{miễn_phí}} = 0$ nên điểm số là $-\infty$; lớp rác có $\hat{\theta}_{\text{họp}} = 0$ nên cũng $-\infty$. **Cả hai lớp đều bị loại**, và mô hình không đưa ra được kết luận nào.

Làm trơn Laplace ($\alpha = 1$, $d = 4$): mỗi tần suất thành $\frac{c_{cj}+1}{(\text{tổng từ})+4}$, nên mẫu số là 13 (bình thường) và 11 (rác):

$$\widehat{\boldsymbol{\Theta}} = \begin{pmatrix} 2/13 & 1/13 & 5/13 & 5/13 \\ 4/11 & 4/11 & 1/11 & 2/11 \end{pmatrix} \approx \begin{pmatrix} 0{,}154 & 0{,}077 & 0{,}385 & 0{,}385 \\ 0{,}364 & 0{,}364 & 0{,}091 & 0{,}182 \end{pmatrix}.$$

Điểm số của bình luận mới $\mathbf{x}_0 = [1,1,1,0]$ (từ "bài" không xuất hiện nên hệ số $\theta^0 = 1$ không đóng góp):

$$s_{\text{bt}} = \ln 0{,}6 + \ln 2/13 + \ln 1/13 + \ln 5/13 = -0{,}511 - 1{,}872 - 2{,}565 - 0{,}956 = -5{,}903,$$

$$s_{\text{rác}} = \ln 0{,}4 + \ln 4/11 + \ln 4/11 + \ln 1/11 = -0{,}916 - 1{,}012 - 1{,}012 - 2{,}398 = -5{,}337.$$

Hậu nghiệm: $\hat{P}(\text{rác} \mid \mathbf{x}_0) = \frac{e^{-5{,}337}}{e^{-5{,}337}+e^{-5{,}903}} = \frac{1}{1+e^{-0{,}566}} \approx 0{,}638$ và $\hat{P}(\text{bt} \mid \mathbf{x}_0) \approx 0{,}362$. Dự đoán **rác**: hai trong ba từ của bình luận ("thưởng", "miễn_phí") nghiêng về rác. Nhờ làm trơn, một từ chưa từng thấy trong một lớp chỉ *giảm* điểm của lớp đó chứ không *loại hẳn* nó.

Cài đặt từ đầu

Cài đặt bám sát công thức ma trận ở mục 5.4 và 5.5.

```
import numpy as np
from math import pi

def logsumexp(S):
    m = S.max(axis=1, keepdims=True)
    return m[:, 0] + np.log(np.exp(S - m).sum(axis=1))

class GaussNB:
    def __init__(self, lam=1e-9): self.lam = lam
    def fit(self, X, Y):
        n, d = X.shape
        t = Y.sum(axis=0)                                          # t_c = n_
c
        self.pi = t / n                                            # pi_hat
        self.mu = (Y.T @ X) / t[:, None]                           # M = diag
(t)^{-1} Y^T X
        E2 = (Y.T @ (X * X)) / t[:, None]
        self.var = E2 - self.mu**2 + self.lam * X.var(axis=0).max()   # Q (+
sàn nhỏ)
        return self
    def diem_so(self, X0):
        S = np.log(self.pi)[None, :] + np.zeros((len(X0), 1))
        for c in range(len(self.pi)):
            S[:, c] += np.sum(-0.5 * np.log(2 * pi * self.var[c])
                              - 0.5 * (X0 - self.mu[c])**2 / self.var[c], axi
```

```
s=1)
        return S
    def xac_suat(self, X0):
        S = self.diem_so(X0); return np.exp(S - logsumexp(S)[:, None])
    def du_doan(self, X0): return np.argmax(self.diem_so(X0), axis=1)

class MultiNB:
    def __init__(self, alpha=1.0): self.alpha = alpha
    def fit(self, X, Y):
        self.pi = Y.sum(axis=0) / len(X)
        C = Y.T @ X                                                  # ma trận
đếm K x d
        self.theta = (C + self.alpha) / (C.sum(axis=1, keepdims=True) + self.
alpha * X.shape[1])
        return self
    def diem_so(self, X0):                                           # S = 1 ln
(pi)^T + X0 ln(Theta)^T
        return np.log(self.pi)[None, :] + X0 @ np.log(self.theta).T
    def xac_suat(self, X0):
        S = self.diem_so(X0); return np.exp(S - logsumexp(S)[:, None])
    def du_doan(self, X0): return np.argmax(self.diem_so(X0), axis=1)
```

Có hai điều đáng lưu ý. Hàm *fit* của cả hai lớp hoàn toàn không có vòng lặp tối ưu: chỉ có phép nhân ma trận với $\mathbf{Y}^\top$ (đếm theo lớp). Và *diem_so* của *MultiNB* chính là công thức $\mathbf{S} = \mathbf{1}\ln\widehat{\boldsymbol{\pi}}^\top + \mathbf{X}_0\left(\ln\widehat{\boldsymbol{\Theta}}\right)^\top$, còn *xac_suat* áp dụng softmax ổn định bằng *logsumexp*.

Kiểm tra trên các ví dụ tay

```
X9 = np.array([[2,4],[3,3],[4,5],[6,3],[7,2],[6,4],[9,1],[10,0],[8,0]], float
)
y9 = np.array([0,0,0,1,1,1,2,2,2]); Y9 = np.eye(3)[y9]
X0 = np.array([[5, 3], [8, 2], [3, 2]], float)
g = GaussNB(lam=0).fit(X9, Y9)
print(np.round(g.diem_so(X0), 3)); print(np.round(g.xac_suat(X0), 3))

Xt = np.array([[2,1,0,0],[1,2,0,1],[0,0,1,2],[1,0,1,1],[0,0,2,1]], float)
Yt = np.eye(2)[[1,1,0,0,0]]                          # cột 0: bình thường, cột 1
: rác
m = MultiNB(alpha=1.0).fit(Xt, Yt)
print(np.round(m.diem_so(np.array([[1,1,1,0.]])), 3)); print(np.round(m.xac_s
uat(np.array([[1,1,1,0.]])), 3))
```

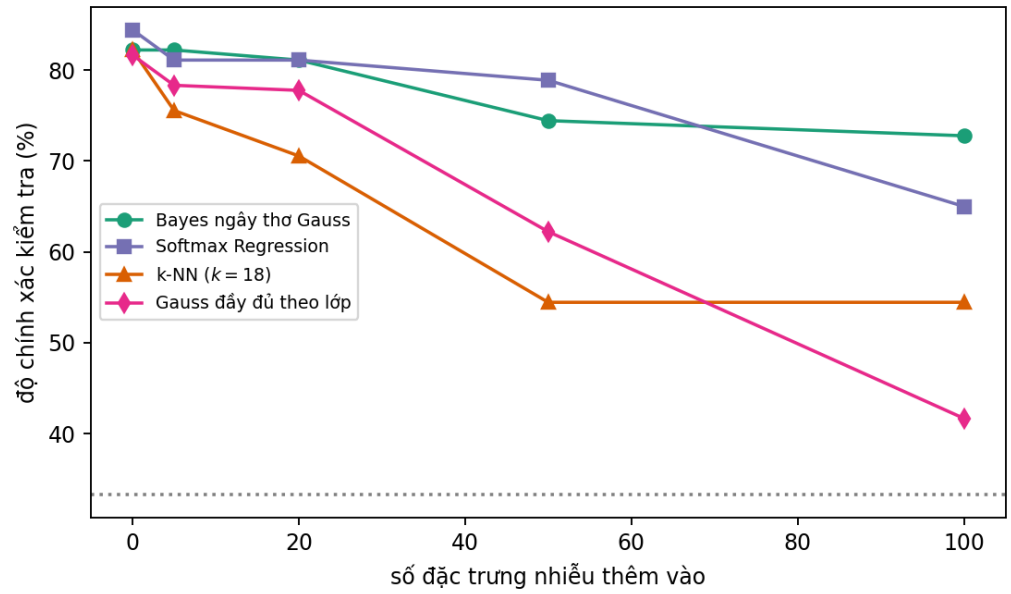
```
[[ -6.281  -5.982 -29.982]
 [-24.281  -8.982  -8.982]
 [ -5.531 -27.732 -35.232]]
[[0.426 0.574 0.   ]
 [0.    0.5   0.5  ]
 [1.    0.    0.   ]]
[[-5.903 -5.337]]
[[0.362 0.638]]
```

Cả bốn kết quả trùng khớp với phép tính tay. Ta cũng đối chiếu với scikit-learn: trên 600 sinh viên mô phỏng, `GaussNB` của ta và `GaussianNB` cho **cùng nhãn dự đoán trên** 100% **mẫu kiểm tra**, và ma trận xác suất khác nhau tối đa cỡ 6×10^{-15} (sai số làm tròn). Với `MultiNB` và `MultinomialNB` trên 200 văn bản ngẫu nhiên có 30 từ, sai khác xác suất tối đa là cỡ 4×10^{-14}; và trên ví dụ 5 bình luận, `MultinomialNB(alpha=1.0)` cho [0,3622; 0,6378], đúng như tay.

Đặc trưng nhiễu: Bayes ngây thơ ít bị ảnh hưởng hơn k-NN

Ở mục 2.5, ta đã thấy k-NN suy giảm nhanh khi thêm đặc trưng nhiễu, và mục 4.6 cho thấy ước lượng mật độ d chiều khó khăn hơn khi d lớn. Giờ ta thử lại cùng thí nghiệm với Bayes ngây thơ: thêm q đặc trưng nhiễu chuẩn tắc (không liên quan đến nhãn) vào dữ liệu 600 sinh viên và so bốn mô hình, trong đó "Gauss đầy đủ theo lớp" là bộ phân loại Bayes với mật độ Gauss có hiệp phương sai đầy đủ riêng cho từng lớp (tức là *không* có giả định ngây thơ):

Số đặc trưng nhiễu q	0	5	20	50	100
Bayes ngây thơ Gauss	82,2%	82,2%	81,1%	74,4%	72,8%
Softmax Regression	84,4%	81,1%	81,1%	78,9%	65,0%
k-NN ($k = 18$)	82,2%	75,6%	70,6%	54,4%	54,4%
Gauss đầy đủ theo lớp (không ngây thơ)	81,7%	78,3%	77,8%	62,2%	41,7%



Hình 20. Độ chính xác kiểm tra khi thêm các đặc trưng nhiễu thuần túy vào dữ liệu 600 *sinh viên mô phỏng. Bayes ngây thơ suy giảm chậm nhất; mô hình Gauss đầy đủ (không ngây thơ) và k-NN suy giảm nhanh nhất. Đường chấm là mức đoán mò (*33,3%*).*

Kết quả cho thấy giả định ngây thơ bảo vệ khá tốt. Với $q = 100$ (102 đặc trưng, gấp hơn hai lần số mẫu huấn luyện 420), Bayes ngây thơ vẫn đạt 72,8%, trong khi mô hình Gauss đầy đủ chỉ còn 41,7% (phải ước lượng $102 \times 103/2 = 5\,253$ phần tử hiệp phương sai cho mỗi lớp từ 140 mẫu) và k-NN còn 54,4%. Lý do với đặc trưng nhiễu: ở lớp nào cũng có cùng phân phối nên

$\ln p(x_j \mid c)$ gần như bằng nhau giữa các lớp và các số hạng nhiễu gần như triệt tiêu trong hiệu $s_a - s_b$, còn hai đặc trưng thật vẫn đóng góp. Cũng nên lưu ý: ở $q = 50$ Softmax (78,9%) tốt hơn Bayes ngây thơ (74,4%), và ở $q = 100$ ngược lại (65,0% so với 72,8%): khi số đặc trưng gần hoặc vượt số mẫu thì mô hình phân biệt không điều chuẩn cũng bị quá khớp, còn mô hình sinh đơn giản thì chịu tốt hơn. Đây là một thí nghiệm trên một lần chia dữ liệu nên có dao động.

Chi phí tính toán

	Huấn luyện	**Dự đoán m điểm**	**Bộ nhớ mô hình**
Bayes ngây thơ Gauss, Bernoulli, Multinomial	$O(nd)$ cho một lượt đếm	$O(mdK)$	$O(dK)$
Bayes ngây thơ KDE	$O(1)$ (lưu dữ liệu)	$O(mnd)$	$O(nd)$
Softmax Regression	$O(T\,ndK)$ cho T epoch	$O(mdK)$	$O(dK)$
k-NN	$O(1)$	$O(mnd)$	$O(nd)$

Biến thể tham số của Bayes ngây thơ có chi phí dự đoán giống Softmax, nhưng huấn luyện chỉ tốn một lượt qua dữ liệu thay vì T epoch. Với văn bản có d lên tới hàng chục nghìn từ, đặc tính này, cộng với ma trận đếm thưa, là lý do Bayes ngây thơ được dùng rộng rãi cho phân loại văn bản.

Mối liên hệ với các mục trước

Mục 5 là điểm hội tụ của các ý tưởng đã xây dựng:

- Từ **mục 4**: các mật độ theo lớp $p(x_j \mid c)$ là các ước lượng mật độ một chiều (Gauss hợp lý cực đại, hoặc KDE). Phép chia "theo lớp" dùng đúng các ma trận one-hot $\mathbf{Y}$ của bài Softmax. Mục 4.5 cho thấy k-NN là bộ phân loại Bayes với mật độ k láng giềng; mục 5 là bộ phân loại Bayes với mật độ *một chiều* và giả định ngây thơ, cùng một công thức Bayes với hai cách ước lượng mật độ khác nhau.
- Từ **bài Softmax**: hậu nghiệm của Bayes ngây thơ là softmax của một điểm số, và với Multinomial/Bernoulli, $\mathbf{S} = \tilde{\mathbf{X}}\mathbf{W}_{\text{NB}}$ cùng dạng $\tilde{\mathbf{X}}\mathbf{W}$ của Softmax Regression. Hai mô hình khác nhau ở *cách ước lượng* $\mathbf{W}$ (đếm, hợp lý đồng thời) hay (Gradient Descent, hợp lý có điều kiện).
- Từ **mục 1 đến 4**: lời nguyền chiều đã làm hỏng k-NN và ước lượng mật độ nhiều chiều, và giả định ngây thơ chữa nó bằng cách tăng thiên lệch để giảm phương sai. Làm trơn Laplace chữa vấn đề ô rỗng của mục 4.2, và α là một siêu tham số nữa bên cạnh k và h.

Phần còn lại của bài (mục 6) sẽ tổng hợp: phân rã sai số thành thiên lệch, phương sai và nhiễu cho mọi mô hình trên, phân tích ảnh hưởng của mẫu sai nhãn hay ngoại lai, và so sánh chi phí huấn luyện và dự đoán.

Tóm tắt mục 5

- Quy tắc Bayes: $P(y = c \mid \mathbf{x}) \propto p(\mathbf{x} \mid c)\,\pi_c$. Mô hình sinh ước lượng $p(\mathbf{x} \mid y)$ và $P(y)$; mô hình phân biệt (Softmax) ước lượng thẳng $P(y \mid \mathbf{x})$.
- **Giả định ngây thơ**: độc lập có điều kiện theo lớp, $p(\mathbf{x} \mid c) = \prod_j p\left(x_j \mid c\right)$. Nó biến một mật độ d chiều thành d mật độ một chiều, nên số tham số tăng tuyến tính theo d (ví dụ 20 thay vì 1 048 575 cho 20 đặc trưng nhị phân) và tránh được lời nguyền chiều, đổi lại thiên lệch.
- Các biến thể: Gauss (số thực), Bernoulli (có/không), Multinomial (đếm từ), và KDE (phi tham số, không giả định dạng từng chiều). Ba biến thể đầu là mô hình tham số.
- "Huấn luyện" là một lượt đếm và lấy trung bình bằng công thức đóng: $\widehat{\mathbf{M}} = \mathrm{diag}(\mathbf{t})^{-1}\mathbf{Y}^{\top}\mathbf{X}$, $\mathbf{C} = \mathbf{Y}^{\top}\mathbf{X}$. **Làm trơn Laplace** $\hat{\theta}_{cj} = \frac{C_{cj}+\alpha}{\sum_l C_{cl}+\alpha d}$ chặn xác suất 0.
- Dự đoán trong không gian log: $s_c = \ln\pi_c + \sum_j \ln p\left(x_j \mid c\right)$, hậu nghiệm $= \mathrm{softmax}(s)$. Với Multinomial, $\mathbf{S} = \tilde{\mathbf{X}}_0\mathbf{W}_{\mathrm{NB}}$ có cùng dạng Softmax Regression, chỉ khác cách ước lượng $\mathbf{W}$.
- Đường biên: tuyến tính (Bernoulli, Multinomial, Gauss phương sai chung) hoặc bậc hai không có tích chéo (Gauss phương sai riêng). Khi đặc trưng tương quan, Bayes ngây thơ kém hơn Logistic Regression nhiều (ví dụ 80,0% so với 95,0%) và xác suất bị "quá tự tin"; ngược lại, nó học nhanh khi mẫu ít và chịu đặc trưng nhiễu tốt hơn k-NN và Gauss đầy đủ.

Câu hỏi kiểm tra nhanh

1. Hai lớp có $\pi_0 = 0{,}7$, $\pi_1 = 0{,}3$. Tại một điểm $\mathbf{x}$, mật độ theo lớp là $p(\mathbf{x} \mid 0) = 0{,}2$ và $p(\mathbf{x} \mid 1) = 0{,}5$. Tính $P(y = 1 \mid \mathbf{x})$ và chọn lớp.
2. Có $d = 10$ đặc trưng nhị phân và $K = 3$ lớp. Đếm số xác suất cần ước lượng (không tính $\boldsymbol{\pi}$) cho mô hình đồng thời đầy đủ và cho Bayes ngây thơ.
3. Mô hình Multinomial với hai lớp A, B, tiên nghiệm bằng nhau, $\boldsymbol{\theta}_A = (0{,}5;\ 0{,}3;\ 0{,}2)$, $\boldsymbol{\theta}_B = (0{,}1;\ 0{,}3;\ 0{,}6)$. Tính điểm số và xác suất hậu nghiệm cho văn bản có đếm từ $\mathbf{x} = [2,0,1]$.
4. Vì sao khi phân loại văn bản 500 từ, ta phải cộng các logarit thay vì nhân các xác suất?

5. Trong mô hình Multinomial với từ vựng $d = 1\,000$, lớp "rác" có tổng cộng 100 từ trong tập huấn luyện và từ "thưởng" không xuất hiện lần nào. Tính $\hat{\theta}$ của từ "thưởng" ở lớp rác với $\alpha = 1$.

6. Chứng minh rằng với mô hình Gauss dùng phương sai chung σ_j^2 cho cả hai lớp, ranh giới của Bayes ngây thơ là tuyến tính. Điều đó cho ta biết gì về quan hệ với Logistic Regression?

7. Khi nào Bayes ngây thơ có thể tốt hơn Logistic Regression, và khi nào kém hơn?

Gợi ý đáp án

1. Tử số: $p(\mathbf{x} \mid 0)\pi_0 = 0{,}2 \times 0{,}7 = 0{,}14$ và $p(\mathbf{x} \mid 1)\pi_1 = 0{,}5 \times 0{,}3 = 0{,}15$. Chuẩn hóa: $P(y = 1 \mid \mathbf{x}) = \frac{0{,}15}{0{,}14+0{,}15} \approx 0{,}517$. Chọn lớp 1, dù lớp 0 có tiên nghiệm cao hơn.

2. Mô hình đầy đủ: mỗi lớp cần $2^{10} - 1 = 1\,023$ xác suất, tổng $3 \times 1\,023 = 3\,069$. Bayes ngây thơ: mỗi lớp 10 xác suất, tổng $3 \times 10 = 30$.

3. $s_A = \ln 0{,}5 + 2\ln 0{,}5 + 0 \cdot \ln 0{,}3 + \ln 0{,}2 = -0{,}693 - 1{,}386 - 1{,}609 = -3{,}689$ (bỏ qua tiên nghiệm bằng nhau, hoặc cộng thêm $\ln 0{,}5$ cho cả hai lớp, không đổi kết quả). $s_B = 2\ln 0{,}1 + \ln 0{,}6 = -4{,}605 - 0{,}511 = -5{,}116$. Hiệu $s_A - s_B = 1{,}427$, nên $P(A \mid \mathbf{x}) = \frac{1}{1+e^{-1{,}427}} \approx 0{,}806$ và chọn lớp A.

4. Tích 500 xác suất cỡ 0,005 là cỡ $10^{-1\,150}$, nhỏ hơn giá trị dương nhỏ nhất của số thực 64 bit (cỡ 10^{-308}), nên máy tính làm tròn thành 0 và mọi lớp đều bằng 0, không so sánh được. Tổng logarit là $500 \times \ln 0{,}005 \approx -2\,649$, biểu diễn bình thường. Vì ln tăng nghiêm ngặt nên argmax không đổi.

5. $\hat{\theta} = \frac{0+1}{100+1\times 1\,000} = \frac{1}{1\,100} \approx 0{,}00091$. Không có làm trơn thì bằng 0 và loại hẳn lớp rác với mọi văn bản chứa từ "thưởng".

6. Hiệu điểm số của hai lớp là $\ln\frac{\pi_a}{\pi_b} + \sum_j \left[\frac{(x_j-\mu_{bj})^2-(x_j-\mu_{aj})^2}{2\sigma_j^2}\right]$. Khai triển bình phương: $(x_j - \mu_{bj})^2 - (x_j - \mu_{aj})^2 = 2x_j(\mu_{aj} - \mu_{bj}) + \mu_{bj}^2 - \mu_{aj}^2$, số hạng x_j^2 triệt tiêu. Còn lại một hàm tuyến tính theo x_j với hệ số $(\mu_{aj} - \mu_{bj})/\sigma_j^2$. Vậy log-odds tuyến tính, cùng họ hàm với Logistic Regression; hai mô hình chỉ khác cách ước lượng các hệ số (Bayes ngây thơ dùng thống kê riêng của từng đặc trưng, Logistic tối ưu đồng thời toàn bộ).

7. Bayes ngây thơ tốt hơn khi dữ liệu ít, số đặc trưng lớn (có thể cả vượt số mẫu), hoặc các đặc trưng gần độc lập có điều kiện (ví dụ văn bản túi từ): ít tham số nên

phương sai thấp, ước lượng bằng đếm nhanh và ổn định, như đồ thị (a) của Hình 21 ở n nhỏ. Kém hơn khi các đặc trưng tương quan mạnh và có đủ dữ liệu để Logistic học được sự tương quan đó (đồ thị (b) của Hình 21), hoặc khi cần xác suất đáng tin cậy (Bayes ngây thơ thường quá tự tin).

6. Phân tích: thiên lệch và phương sai, lời nguyền chiều, chi phí tính toán

Năm mục đầu đã đưa ra một bộ sưu tập các mô hình phi tham số: k-NN phân loại và hồi quy, hồi quy nhân, LOESS, ước lượng mật độ, và Bayes ngây thơ. Mỗi mục đều có một "nút chỉnh" riêng (k, h, α), một hiện tượng suy giảm riêng (đặc trưng nhiễu, ô rỗng, đặc trưng tương quan), và một loại chi phí riêng. Bạn có thể đã nhận ra rằng *câu chuyện lặp lại*: nút chỉnh nhỏ thì mô hình bám nhiễu, nút chỉnh lớn thì mô hình quá mượt; số chiều lớn thì mọi thứ xấu đi; còn chi phí thì dồn sang lúc dự đoán. Mục 6 gom các quan sát rải rác đó lại và trả lời "vì sao" bằng một khung phân tích duy nhất. Ta trả lời năm câu hỏi:

- Sai số của một mô hình học máy gồm những phần nào, và các "nút chỉnh độ phức tạp" k, h, λ, C tác động lên từng phần ra sao? (mục 6.1)
- Điều gì xảy ra khi *một* mẫu bị gán nhãn sai hoặc là ngoại lai? Mô hình địa phương và mô hình toàn cục phản ứng khác nhau thế nào? (mục 6.2)
- Lời nguyền chiều có những mặt nào: khoảng cách mất ý nghĩa, đặc trưng nhiễu phá k-NN nhưng ít ảnh hưởng Bayes ngây thơ? (mục 6.3)
- Khi $n \to \infty$, mô hình nào tiến tới bộ phân loại tối ưu Bayes, mô hình nào không? (mục 6.4)
- Chi phí huấn luyện và dự đoán của các mô hình phi tham số so với tham số là bao nhiêu, và khi nào đáng dùng loại nào? (mục 6.5)

Quy ước ký hiệu

Ta giữ các quy ước trước đó (chữ thường in đậm là vector, chữ hoa in đậm là ma trận, đặc trưng không có cột 1 trừ khi có dấu ngã). Thêm vào:

Đối tượng	**Ký hiệu**	**Ý nghĩa**
Hàm thật	$f(x)$	Hàm mà dữ liệu được sinh ra từ đó: $y = f(x) + \varepsilon$
Nhiễu	ε, σ^2	$\mathbb{E}[\varepsilon] = 0$, $\mathrm{Var}[\varepsilon] = \sigma^2$
Tập huấn luyện ngẫu nhiên	$\mathcal{D}$	n mẫu được rút ngẫu nhiên; mô hình học được phụ thuộc vào $\mathcal{D}$
Mô hình học từ $\mathcal{D}$	$\hat{f}_{\mathcal{D}}(x)$	Dự đoán tại x của mô hình huấn luyện trên $\mathcal{D}$
Ma trận làm trơn	$\mathbf{S}$	$\hat{\mathbf{y}} = \mathbf{S}\mathbf{y}$ (mục 3.4), hàng j là trọng số hiệu dụng tại điểm j
Trọng số hiệu dụng	$\mathbf{s}(x) = [s_1(x), \dots, s_n(x)]$	$\hat{f}(x) = \sum_i s_i(x)\, y_i$

Đối tượng	Ký hiệu	Ý nghĩa
Bậc tự do hiệu dụng	$\mathrm{df} = \mathrm{tr}(\mathbf{S})$	Thước đo "độ phức tạp" của mô hình tuyến tính theo dữ liệu
Xác suất hậu nghiệm lớp 1	$\eta(\mathbf{x}) = P(y = 1 \mid \mathbf{x})$	Dùng cho phân loại hai lớp (mục 6.4)
Sai số Bayes	L^*	Sai số của bộ phân loại tối ưu Bayes

6.1. Phân rã sai số = thiên lệch^2 + phương sai + nhiễu

Phân rã cho hồi quy

Xét một điểm x cố định và một giá trị $y = f(x) + \varepsilon$ chưa thấy. Mô hình $\hat{f}_{\mathcal{D}}$ được huấn luyện trên một tập $\mathcal{D}$ ngẫu nhiên, nên bản thân $\hat{f}_{\mathcal{D}}(x)$ cũng là một biến ngẫu nhiên. Sai số bình phương *kỳ vọng* (theo cả $\mathcal{D}$ và ε) phân rã như sau, nhờ thêm và bớt $\mathbb{E}_{\mathcal{D}}\big[\hat{f}_{\mathcal{D}}(x)\big]$ rồi khai triển bình phương (các tích chéo có kỳ vọng 0 vì ε độc lập với $\mathcal{D}$ và $\hat{f}_{\mathcal{D}} - \mathbb{E}_{\mathcal{D}}\hat{f}_{\mathcal{D}}$ có kỳ vọng 0):

$$\mathbb{E}\big[(y - \hat{f}_{\mathcal{D}}(x))^2\big] = \underbrace{\sigma^2}_{\text{nhiễu}} + \underbrace{(\mathbb{E}_{\mathcal{D}}\big[\hat{f}_{\mathcal{D}}(x)\big] - f(x))^2}_{\text{thiên lệch}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[\big(\hat{f}_{\mathcal{D}}(x) - \mathbb{E}_{\mathcal{D}}\big[\hat{f}_{\mathcal{D}}(x)\big]\big)^2]}_{\text{phương sai}}.$$

Ba thành phần có ý nghĩa rất rõ:

- **Nhiễu** σ^2: phần ngẫu nhiên thật sự của y mà không mô hình nào dự đoán được. Đây là mức sàn, không giảm được (ở mục 3 và 4 ta đã dùng $\sigma = 0{,}3$ nên mức sàn của MSE là 0,09).
- **Thiên lệch**: trung bình của mô hình trên *mọi tập huấn luyện có thể* lệch bao xa so với hàm thật. Nó lớn khi mô hình *không đủ linh hoạt* để diễn tả f (đường thẳng cho hàm sin, k-NN với k quá lớn).
- **Phương sai**: mô hình dao động bao nhiêu khi đổi tập huấn luyện. Nó lớn khi mô hình *quá nhạy với từng mẫu* (k-NN với $k = 1$, đa thức bậc quá cao).

Công thức đóng cho các bộ làm trơn tuyến tính

Các mô hình ở mục 3 đều có dạng $\hat{f}(x) = \sum_i s_i(x)\, y_i$. Với thiết kế cố định (các x_i cho trước, chỉ nhiễu ε_i thay đổi giữa các lần lấy mẫu), hai thành phần còn lại có công thức *chính xác*:

$$\text{Thiên lệch}\big[\hat{f}(x)\big] = \sum_{i=1}^{n} s_i(x)\, f(x_i) - f(x), \qquad \text{Phương sai}\big[\hat{f}(x)\big] = \sigma^2 \sum_{i=1}^{n} s_i(x)^2 = \sigma^2 \,\|\, \mathbf{s}(x) \,\|^2.$$

Thiên lệch là sai khác giữa hàm thật *đã được làm trơn* và hàm thật; phương sai là σ^2 nhân với tổng bình phương các trọng số. Với k-NN hồi quy, $s_i = 1/k$ cho k láng giềng nên

$$\text{Phương sai} = \sigma^2 \cdot k \cdot \frac{1}{k^2} = \frac{\sigma^2}{k}, \qquad \text{Thiên lệch} = \frac{1}{k}\sum_{i \in \mathcal{N}_k(x)} f(x_i) - f(x).$$

Đây là một kết quả đẹp: **phương sai của k-NN hồi quy bằng đúng** σ^2/k, giảm đều khi k tăng, trong khi thiên lệch tăng vì các láng giềng thứ k ngày càng xa x nên $f(x_i)$ lệch dần khỏi $f(x)$. Một thước đo độ phức tạp chung cho mọi bộ làm trơn là **bậc tự do hiệu dụng** $\text{df} = \text{tr}(\mathbf{S})$, tổng các phần tử đường chéo của ma trận làm trơn. Với k-NN, mỗi mẫu có trọng số $1/k$ trên chính nó nên $\text{df} = n/k$; với đa thức bậc p, $\text{df} = p + 1$ (số tham số); với hồi quy nhân, df giảm khi h tăng.

Kiểm chứng bằng thí nghiệm

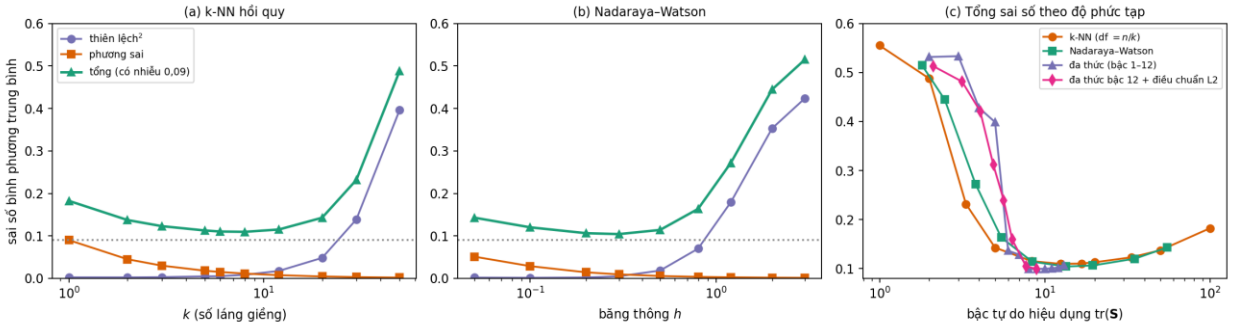
Ta dùng lại bài toán $y = \sin(1{,}2x) + 0{,}15x + \varepsilon$ ($\sigma = 0{,}3$) với $n = 100$ điểm cố định ở mục 3, và tính thiên lệch^2 và phương sai trung bình trên 201 điểm lưới đều từ 0 đến 10 bằng các công thức đóng ở trên. Để kiểm tra các công thức, ta cũng chạy mô phỏng Monte Carlo (2 000 lần sinh nhiễu mới) và đo trực tiếp sai số trên nhãn mới. Kết quả khớp: ví dụ với k-NN $k = 6$ công thức cho thiên lệch$^2 = 0{,}0053$ và phương sai $= 0{,}0150$ (đúng bằng $0{,}09/6$), nên tổng kỳ vọng là $0{,}09 + 0{,}0053 + 0{,}0150 = 0{,}1103$, còn mô phỏng đo được 0,1104. Với Nadaraya–Watson $h = 0{,}3$, công thức cho 0,1042 và mô phỏng 0,1040; với đa thức bậc 7, 0,0987 và 0,0985.

```
def phan_ra(S, f_tren_x, f_tren_luoi, sigma):
    """Thiên lệch^2 và phương sai trung bình của bộ làm trơn tuyến tính y_hat
= S y."""
    ky_vong = S @ f_tren_x                          # E[f_hat] tại các đi
ểm lưới
    bias2 = np.mean((ky_vong - f_tren_luoi)**2)
    var = sigma**2 * np.mean(np.sum(S**2, axis=1))      # sigma^2 ||s(x)||^2
    return bias2, var
```

Bảng sau cho thấy cách ba thành phần thay đổi theo độ phức tạp của từng họ mô hình (tổng = 0,09 + thiên lệch^2 + phương sai):

Mô hình	Nút chỉnh	Thiên lệch^2	Phương sai	Tổng sai số
k-NN hồi quy	$k = 1$	0,0024	0,0900	0,182
	$k = 6$	0,0053	0,0150	0,110
	$k = 8$	0,0083	0,0113	**0,110**
	$k = 20$	0,0482	0,0045	0,143
	$k = 50$	0,3958	0,0018	0,488
Nadaraya–Watson	$h = 0{,}05$	0,0018	0,0512	0,143
	$h = 0{,}1$	0,0014	0,0288	0,120
	$h = 0{,}3$	0,0049	0,0093	**0,104**
	$h = 0{,}8$	0,0700	0,0036	0,164
	$h = 2$	0,3530	0,0017	0,445
LOESS	$h = 0{,}1$	0,0001	0,0373	0,127

Mô hình	Nút chỉnh	Thiên lệch^2	Phương sai	Tổng sai số
	$h = 0{,}35$	0,0038	0,0090	**0,103**
	$h = 0{,}8$	0,0574	0,0045	0,152
Đa thức	bậc 1	0,4398	0,0018	0,532
	bậc 5	0,0411	0,0053	0,136
	bậc 7	0,0012	0,0075	**0,099**
	bậc 12	0,0000	0,0170	0,107
Đa thức bậc 12 + L2	$\lambda = 10^{-8}$	0,0004	0,0084	**0,099**
	$\lambda = 10^{-4}$	0,0634	0,0056	0,159
	$\lambda = 10$	0,4213	0,0015	0,513



Hình 22. Phân rã sai số cho dữ liệu sin ($n = 100$, $\sigma = 0{,}3$). (a) k-NN hồi quy theo k. (b) Nadaraya–Watson theo h. Thiên lệch^2 (tím) tăng và phương sai (cam) giảm khi mô hình mượt hơn; tổng (xanh lục) có hình chữ "U". Đường chấm là mức sàn nhiễu $\sigma^2 = 0{,}09$. (c) Cùng tổng sai số, vẽ theo bậc tự do hiệu dụng $\mathrm{tr}(\boldsymbol{S})$: bốn họ mô hình khác nhau cho các đường gần như trùng nhau.

Có ba bài học từ bảng và Hình 22.

- **Hình chữ "U" xuất hiện ở mọi họ mô hình.** Từ đầu bên trái (mô hình phức tạp: k nhỏ, h nhỏ, bậc cao, λ nhỏ), phương sai chiếm ưu thế (k-NN với $k = 1$: phương sai 0,090, gấp khoảng 38 lần thiên lệch^2 của chính nó là 0,0024). Từ đầu bên phải (mô hình đơn giản), thiên lệch chiếm ưu thế (đa thức bậc 1: 0,44, chiếm gần hết sai số 0,53). Điểm tối ưu nằm ở giữa, nơi hai thành phần cùng cỡ.
- **Cái quan trọng là độ phức tạp hiệu dụng, không phải họ mô hình.** Hình 22c cho thấy: khi vẽ theo bậc tự do $\mathrm{tr}(\boldsymbol{S})$, k-NN, Nadaraya–Watson, đa thức và đa thức có điều chuẩn L2 đều đạt đáy ở cùng một vùng (df ≈ 8 đến 13) với sai số tổng chỉ chênh nhau cỡ 0,01. Đa thức bậc 7 nhỉnh nhất (0,099) vì hàm thật nằm *đúng* trong họ đa thức bậc cao, còn k-NN thua một chút (0,110), có lẽ do các bước nhảy của hàm hằng từng khúc (mục 3.1). Điều này giải thích vì sao ở mục 3.5 các phương

pháp phi tham số tiến sát mức sàn: chúng chọn đúng độ phức tạp mà không cần biết dạng hàm.

- **Các "nút chỉnh" là các cách khác nhau để điều khiển cùng một độ phức tạp.** Bảng dưới liệt kê chúng, bao gồm cả λ (hệ số điều chuẩn L2 của Softmax Regression ở bài trước) và C (tham số của nhiều thư viện, ví dụ `LogisticRegression` của scikit-learn, với $C = 1/\lambda$ nên *đổi chiều*: C lớn nghĩa là điều chuẩn yếu).

Mô hình	Nút chỉnh	Tăng nút chỉnh thì	Hiệu ứng
k-NN	k	mượt hơn	thiên lệch ↑, phương sai ↓
Hồi quy nhân, KDE, LOESS	băng thông h	mượt hơn	thiên lệch ↑, phương sai ↓
Softmax, hồi quy tuyến tính	λ (hệ số L2)	đơn giản hơn	thiên lệch ↑, phương sai ↓
Các thư viện dùng $C = 1/\lambda$	C	*phức tạp hơn*	thiên lệch ↓, phương sai ↑
Bayes ngây thơ	làm trơn α	đồng đều hơn	thiên lệch ↑, phương sai ↓
Đa thức, mạng nơ-ron	bậc, số nơ-ron	phức tạp hơn	thiên lệch ↓, phương sai ↑

Phân loại và cách chẩn đoán trong thực hành

Với phân loại, sai số 0–1 không phân rã gọn thành tổng ba số hạng như trên, nhưng khung ý nghĩa vẫn y hệt. Giữ vai trò của σ^2 là **sai số Bayes** L^*, sai số nhỏ nhất của mọi bộ phân loại (do hai lớp chồng lấn, mục 6.4). Phần sai số vượt L^* cũng do thiên lệch (dạng mô hình sai) cộng phương sai (ước lượng từ ít dữ liệu). Cách đọc trong thực hành là so **độ chính xác trên tập huấn luyện** với **độ chính xác kiểm định chéo**:

Quan sát	Chẩn đoán	Cách xử lý
Huấn luyện cao, kiểm định chéo thấp, khoảng cách lớn	Phương sai cao (quá khớp)	Tăng k, h, λ, α; thêm dữ liệu; bỏ bớt đặc trưng
Cả hai đều thấp và gần nhau	Thiên lệch cao (thiếu khớp)	Giảm k, h, λ; thêm đặc trưng hoặc dùng mô hình linh hoạt hơn
Cả hai đều cao và gần nhau	Tốt	Dừng lại

Cả hai trường hợp đã xuất hiện trong bài: ở mục 2.4, k-NN với $k = 1$ có độ chính xác huấn luyện 92,9% so với kiểm định chéo 77,6% (phương sai cao), còn ở mục 1.1, Softmax trên ba vòng tròn đồng tâm chỉ đạt 43,3% huấn luyện và 37,8% kiểm tra (cả hai thấp và gần nhau: thiên lệch cao). Hai chẩn đoán đòi hỏi hai cách chữa ngược nhau, và nhầm lẫn giữa chúng (ví dụ tăng dữ liệu để chữa thiên lệch) là sai lầm phổ biến.

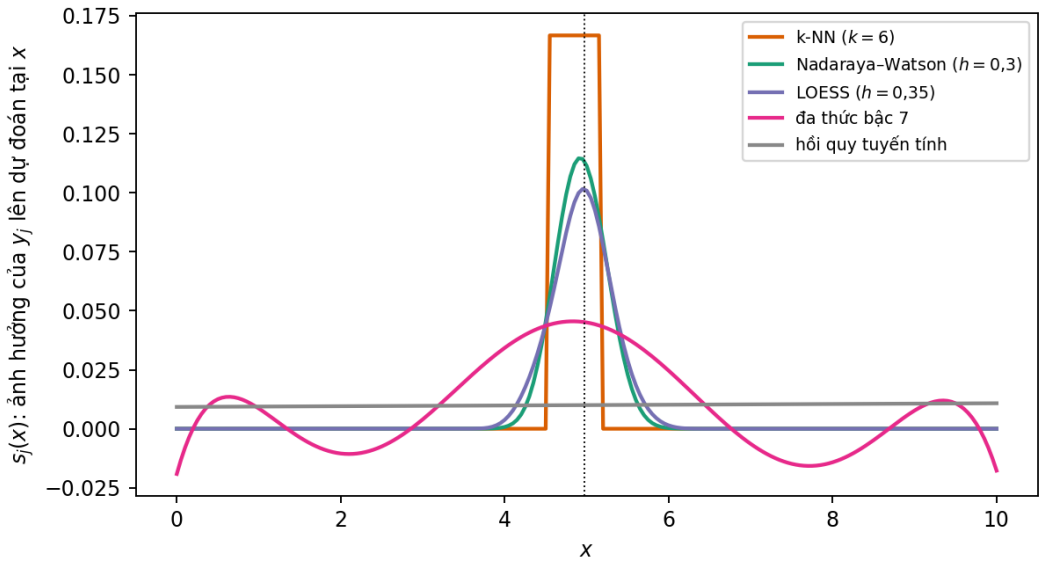
6.2. Một mẫu bị gán nhãn sai hoặc là ngoại lai: ảnh hưởng cục bộ

Hồi quy: ảnh hưởng là trọng số hiệu dụng

Với bộ làm trơn tuyến tính $\hat{f}(x) = \sum_i s_i(x)\, y_i$, việc thay đổi giá trị y_j của một mẫu đúng một lượng δ làm dự đoán tại x thay đổi

$$\Delta \hat{f}(x) = s_j(x)\, \delta.$$

Vậy **hàm ảnh hưởng của mẫu j chính là trọng số hiệu dụng** $s_j(x)$ của nó, và hình dạng của hàm này cho biết mẫu đó "nói" được bao xa. Hình 23 vẽ $s_j(x)$ cho mẫu j nằm ở $x_j \approx 5$ (mẫu ở giữa miền dữ liệu):



Hình 23. Ảnh hưởng $s_j(x)$ của một mẫu huấn luyện tại $x_j \approx 5$ lên dự đoán ở mọi điểm x. Các mô hình địa phương (k-NN, Nadaraya–Watson, LOESS) chỉ bị tác động trong một khoảng hẹp quanh x_j và bằng 0 ở xa. Hồi quy tuyến tính và đa thức bậc cao bị tác động ở mọi nơi, kể cả ở biên.

Mô hình	**Ảnh hưởng tại $x = 5$**	**Tại $x = 1$**	**Tại $x = 9$**	**Bề rộng vùng có ảnh hưởng $> 10^{-3}$**
k-NN ($k = 6$)	0,167	0	0	0,65
Nadaraya–Watson ($h = 0{,}3$)	0,111	≈ 0	≈ 0	1,85
LOESS ($h = 0{,}35$)	0,101	≈ 0	≈ 0	2,15
Đa thức bậc 7	0,045	0,0086	0,0077	9,7 (gần toàn miền)
Hồi quy tuyến tính	0,0100	0,0094	0,0106	10 (toàn miền)

Đây là cách diễn đạt định lượng của kết quả ở mục 3.5: các mô hình địa phương cho một mẫu ảnh hưởng *mạnh nhưng cục bộ* (ngay tại $x = 5$, mẫu này làm thay đổi dự đoán $0{,}17$ cho mỗi đơn vị sai lệch của y_j, gấp 17 lần ảnh hưởng $0{,}01$ trong hồi quy tuyến tính) và *bằng đúng* 0 *ở xa*. Các mô hình toàn cục cho mỗi mẫu một ảnh hưởng nhỏ nhưng *lan khắp nơi*. Cần chú ý rằng ảnh hưởng ở một điểm gần mẫu không *giảm* khi dùng mô hình địa phương, nên một ngoại lai đứng sát điểm truy vấn vẫn kéo dự đoán rất mạnh.

Phân loại: nhãn sai ngẫu nhiên

Để xem tác động của nhãn sai lên phân loại, ta lấy 420 sinh viên huấn luyện của mục 2.3, đổi nhãn một tỉ lệ p ngẫu nhiên sang một lớp khác, huấn luyện lại bốn mô hình và đo độ chính xác trên 180 mẫu kiểm tra *sạch*. Mỗi con số là trung bình của 30 lần đổi nhãn khác nhau:

Tỉ lệ nhãn sai p	0%	5%	10%	20%	30%
k-NN, $k = 1$	72,2%	69,4%	66,5%	59,8%	54,4%
k-NN, $k = 18$	82,2%	82,8%	81,9%	80,4%	78,8%
Softmax Regression	84,4%	83,5%	82,5%	80,0%	78,0%
Bayes ngây thơ Gauss	82,2%	82,4%	82,4%	81,7%	80,7%

Nhận xét:

- **k-NN với $k = 1$ rất nhạy.** Mỗi mẫu nhãn sai tạo một "đảo" nhãn sai quanh nó (ô Voronoi của nó, mục 2.3), nên độ chính xác rơi từ $72{,}2\%$ xuống $54{,}4\%$ khi 30% nhãn sai. Với k lớn hơn, một mẫu sai nhãn bị các láng giềng đúng lấn át: muốn đổi được kết quả của một điểm, cần ít nhất $(k + 1)/2$ láng giềng cùng sai, nên $k = 18$ chỉ giảm 3,4 điểm ở 30% nhãn sai.
- **Bayes ngây thơ bền nhất ở thí nghiệm này** (giảm 1,5 điểm): nó ước lượng trung bình và phương sai của từng lớp, tức là *tổng hợp* tất cả các mẫu, nên mỗi nhãn sai chỉ dịch các thống kê đi một lượng nhỏ cỡ $1/n_c$. Nhãn sai ngẫu nhiên đều sang các lớp khác nên các tâm lớp chỉ bị kéo nhẹ.
- **Softmax Regression** nằm giữa (giảm 6,4 điểm). Với dữ liệu này nó vẫn là mô hình tốt nhất khi nhãn sạch hoặc rất ít nhãn sai, nhưng bị tụt xuống sau Bayes ngây thơ và k-NN $k = 18$ ở mức nhiễu nhãn cao.

Phân loại: một nhãn bị đổi, và một ngoại lai trong không gian đặc trưng

Ta đo ảnh hưởng của *một* mẫu bằng cách đổi nhãn của nó (sang lớp "rớt") rồi so lại dự đoán trên một lưới 300×300 điểm phủ miền dữ liệu, ghi lại **tỉ lệ diện tích lưới đổi nhãn dự đoán**:

Mẫu bị đổi nhãn	k-NN $k = 1$	k-NN $k = 18$	Softmax	Bayes ngây thơ
Sinh viên giỏi sâu trong vùng "giỏi", tại $(9{,}5;\ 1)$	0,08%	0,00%	0,53%	1,34%
Sinh viên đạt ở vùng biên, tại $(8;\ 2)$	0,12%	0,10%	0,35%	0,72%

Với k-NN $k = 18$ và mẫu nằm sâu trong vùng của lớp mình, đổi một nhãn *không thể* thay đổi bất kỳ dự đoán nào (0,00%), vì không điểm nào có đa số láng giềng sai. Ngược lại, các mô hình tham số cập nhật *toàn bộ* các tham số ($\mathbf{W}$ hay $\widehat{\mathbf{M}}, \widehat{\mathbf{Q}}$) dựa trên mọi mẫu, nên đổi một nhãn dịch nhẹ các biên ở khắp nơi và làm đổi nhãn dự đoán trên một vùng rộng hơn (nhưng ở rìa các vùng chứ không gây đảo nhãn). Các con số đều nhỏ, và điều đáng nhớ là *bản chất* của ảnh hưởng: cục bộ (địa phương) hay lan rộng (toàn cục).

Cuối cùng là **ngoại lai trong không gian đặc trưng**. Ta thêm một mẫu "rớt" có đặc trưng cực đoan $(x_1, x_2) = (100;\ 4)$ (số giờ tự học vô lý) vào tập huấn luyện:

Mô hình	Độ chính xác kiểm tra trước	Sau khi thêm ngoại lai
k-NN, $k = 1$	72,2%	72,8%
k-NN, $k = 18$	82,2%	83,3%
Softmax Regression	84,4%	77,2%
Bayes ngây thơ Gauss	82,2%	74,4%

Một mẫu duy nhất làm Softmax mất 7,2 điểm và Bayes ngây thơ mất 7,8 điểm. Cơ chế với Bayes ngây thơ nhìn thấy được: phương sai của x_1 trong lớp "rớt" tăng từ 1,98 lên 67,5 (và trung bình từ 3,52 lên 4,20), vì ngoại lai nằm cách tâm lớp 96 đơn vị và bình phương độ lệch của nó áp đảo; mật độ Gauss của lớp "rớt" theo x_1 trở nên bẹt và đổi cách so sánh với hai lớp kia. Với Softmax, ngoại lai có đòn bẩy rất lớn vì $\| \mathbf{x} \|$ lớn làm gradient lớn. Còn k-NN (cả hai bản) gần như không bị ảnh hưởng, vì ngoại lai nằm xa mọi điểm truy vấn thông thường nên không bao giờ nằm trong tập láng giềng. Có hai cảnh báo: (i) đây là thí nghiệm một lần, các thay đổi $+0{,}6$ và $+1{,}1$ điểm của k-NN nằm trong dao động ngẫu nhiên; (ii) pipeline k-NN ở đây chuẩn hóa z-score bằng *chính tập huấn luyện đã có ngoại lai*, nên độ lệch chuẩn của x_1 bị phình ra; k-NN vẫn ổn ở ví dụ này nhưng nói chung bước chuẩn hóa là điểm yếu trước ngoại lai và nên thay bằng chuẩn hóa bền vững (dùng trung vị và khoảng tứ phân vị).

Tóm lại, **các mô hình địa phương cô lập tác hại của dữ liệu xấu trong một vùng nhỏ**, còn các mô hình tham số lan truyền nó qua các tham số chung. Đổi lại, mô hình địa phương không tận dụng được các mẫu ở xa để "sửa" một mẫu sai gần nó. Lựa chọn k lớn (hoặc h lớn) chính là cách cân bằng giữa hai thái cực.

6.3. Lời nguyền chiều: khoảng cách mất ý nghĩa, đặc trưng nhiễu phá k-NN nhưng ít ảnh hưởng Bayes ngây thơ

Lời nguyền chiều đã xuất hiện ở các mục 1.3, 2.5, 3.4 và 4.6, mỗi lần dưới một dạng khác nhau. Giờ ta gom chúng thành một bức tranh.

Bức tranh hình học

Vùng lân cận "nhỏ" không còn nhỏ. Giả sử dữ liệu phân bố đều trong hình lập phương đơn vị $[0,1]^d$ và ta muốn bắt được một tỉ lệ p của dữ liệu bằng một hình lập phương con có cạnh ℓ. Thể tích cần có $\ell^d = p$, nên

$$\ell = p^{1/d}.$$

Với $p = 1\%$, cạnh cần thiết là $\ell = 0{,}01$ ($d = 1$), $0{,}10$ ($d = 2$), $0{,}40$ ($d = 5$), $0{,}63$ ($d = 10$), $0{,}79$ ($d = 20$), $0{,}91$ ($d = 50$) và $0{,}955$ ($d = 100$). Ở $d = 100$, "lân cận chứa 1% dữ liệu" gần như phủ 95,5% độ dài của *mỗi* trục: không còn "địa phương" nữa.

Bán kính láng giềng tăng theo số chiều. Với n điểm trong khối đơn vị, láng giềng thứ k cách khoảng r_k mà quả cầu bán kính r_k chứa kỳ vọng k điểm, tức $n\,\omega_d\,r_k^d \approx k$ (với ω_d là thể tích hình cầu đơn vị như ở mục 4.5):

$$r_k \approx \left(\frac{k}{n\,\omega_d}\right)^{1/d}.$$

Với $n = 1\,000$ và $k = 5$: $r_5 \approx 0{,}0025$ ($d = 1$), $0{,}040$ ($d = 2$), $0{,}25$ ($d = 5$), $0{,}54$ ($d = 10$), $0{,}92$ ($d = 20$) và $1{,}6$ ($d = 50$). Ở $d \geq 20$, láng giềng thứ 5 đã nằm cách xa cỡ *toàn bộ độ rộng* của miền dữ liệu (công thức bỏ qua hiệu ứng biên, nhưng ý chính không đổi): k-NN không còn xem xét "các điểm gần" nữa.

Hầu hết thể tích nằm ở "góc". Tỉ lệ thể tích của quả cầu nội tiếp trong hình lập phương là $\omega_d / 2^d$, bằng $0{,}785$ ($d = 2$), $0{,}164$ ($d = 5$), $0{,}0025$ ($d = 10$) và $2{,}5 \times 10^{-8}$ ($d = 20$). Gần như toàn bộ khối lượng nằm ngoài quả cầu, tức ở các "góc" xa tâm; các điểm phân bố đều có xu hướng nằm gần như cùng xa nhau, đúng với hiện tượng tập trung khoảng cách đã đo ở mục 4.6 (độ chênh tương đối giữa điểm xa nhất và gần nhất giảm từ 269 ở $d = 2$ xuống $0{,}15$ ở $d = 1\,000$).

Số mẫu cần tăng nhanh theo d. Ở mục 4.6, sai số của KDE tối ưu giảm như $n^{-4/(d+4)}$. Để giảm sai số đi 10 lần, cần nhân số mẫu lên $10^{(d+4)/4}$ lần: $d = 1$: 18, $d = 2$: 32, $d = 5$: 178, $d = 10$: $3\,162$, $d = 20$: 10^6.

Tốc độ này nhanh theo cấp số nhân, nên ước lượng phi tham số nhiều chiều chỉ khả thi khi d nhỏ hoặc khi dữ liệu có cấu trúc đơn giản hơn số chiều của nó. (Với k-NN hồi quy trên hàm Lipschitz, tốc độ tương ứng là $n^{-2/(d+2)}$, cùng bản chất.)

Chiều thật và chiều hiệu dụng: cái gì mới là kẻ thù

Có một hiểu lầm cần tránh: lời nguyền chiều **không** nói rằng "nhiều đặc trưng thì k-NN tệ". Cái quyết định là số chiều *thông tin* (chiều hiệu dụng), không phải số chiều của vector biểu diễn. Để thấy điều đó, ta so hai cách làm tăng số chiều của cùng dữ liệu hai đặc trưng 600 sinh viên:

- **Nhúng đẳng cự (không thêm thông tin).** Nhân dữ liệu hai chiều (đã chuẩn hóa) với một ma trận $\mathbf{Q} \in \mathbb{R}^{2\times d}$ có hai hàng trực chuẩn ($\mathbf{Q}\mathbf{Q}^\top = \mathbf{I}_2$), tức là "đặt" mặt

phẳng dữ liệu vào $\mathbb{R}^d$ theo một hướng ngẫu nhiên. Phép nhúng này *bảo toàn mọi khoảng cách*, nên k-NN thấy đúng dữ liệu cũ.

- **Thêm đặc trưng nhiễu** (mục 2.5, 5.7): mỗi đặc trưng nhiễu thêm một chiều *có thật* vào khoảng cách.

Số chiều d (nhúng đẳng cự)	2	10	50	200
k-NN ($k = 18$)	82,2%	82,2%	83,3%	81,7%
Softmax Regression	84,4%	84,4%	84,4%	84,4%
Bayes ngây thơ Gauss	82,2%	81,7%	82,8%	82,8%

Dù đặt dữ liệu vào $\mathbb{R}^{200}$, độ chính xác của k-NN vẫn gần như không đổi (81,7% đến 83,3%, chênh lệch nhỏ là do các điểm hòa khoảng cách được xử lý khác nhau). Trái lại, khi thêm 50 đặc trưng nhiễu vào cùng dữ liệu (tổng 52 chiều), k-NN rơi xuống 54,4% (mục 2.5). Cùng một số chiều, kết quả khác hẳn, nên kết luận đúng là: **lời nguyền chiều nhắm vào số chiều hiệu dụng của dữ liệu và số chiều nhiễu, không phải số chiều ngoài**. Dữ liệu thực thường nằm gần một đa tạp có chiều thấp trong không gian chiều cao (ví dụ ảnh chữ số 8×8 nằm trong $\mathbb{R}^{64}$ nhưng chỉ có vài chục hướng biến thiên có ý nghĩa), đó là lý do k-NN vẫn dùng được trên nhiều bài toán chiều cao.

Vì sao Bayes ngây thơ ít bị đặc trưng nhiễu phá hơn k-NN

Ở mục 5.7 ta thấy khi thêm 100 đặc trưng nhiễu, Bayes ngây thơ còn 72,8% còn k-NN chỉ còn 54,4%. Có thể giải thích điều này bằng công thức. Với hai lớp a và b, điểm số chênh lệch là tổng các số hạng *theo từng đặc trưng*:

$$s_a(\mathbf{x}) - s_b(\mathbf{x}) = \ln\frac{\pi_a}{\pi_b} + \sum_{j=1}^{d} \ln\frac{p(x_j \mid a)}{p(x_j \mid b)}.$$

Một đặc trưng nhiễu j có *cùng phân phối ở mọi lớp*, tức $p(x_j \mid a) = p(x_j \mid b)$. Khi đó kỳ vọng đóng góp của nó vào tổng, lấy theo phân phối của lớp a, là độ phân kỳ Kullback–Leibler giữa hai phân phối (mục 4.4), và nó bằng 0:

$$\mathbb{E}_{x_j \sim p(\cdot \mid a)}\left[\ln\frac{p(x_j \mid a)}{p(x_j \mid b)}\right] = \mathrm{KL}\big(p(\cdot \mid a) \,\|\, p(\cdot \mid b)\big) = 0.$$

Vậy một đặc trưng nhiễu *không làm lệch* điểm số theo hướng nào; nó chỉ thêm một dao động ngẫu nhiên có trung bình 0 (từ sai số ước lượng các tham số). Hai đặc trưng thật tạo ra một khoảng cách điểm số *cố định* giữa hai lớp (kỳ vọng dương, bằng tổng các KL của các đặc trưng thật), còn các dao động nhiễu cộng dồn chậm (độ lệch chuẩn tăng cỡ $\sqrt{q}$ khi có q đặc trưng nhiễu), nên bộ phân loại *suy giảm dần dần*. Ngược lại, trong k-NN mỗi đặc trưng nhiễu đóng góp thêm một số hạng *bình phương* (luôn dương, không triệt tiêu) vào khoảng cách; khi q vượt xa số đặc trưng thật, phần nhiễu áp đảo khoảng cách và các láng giềng được chọn gần như ngẫu nhiên.

Các mô hình khác nhau có những cách đối phó khác nhau với nhiễu và chiều cao:

Mô hình	Cách nhìn các đặc trưng	Mức nhạy với đặc trưng nhiễu	Cách khắc phục
k-NN, hồi quy nhân, KDE	Mọi đặc trưng có trọng số như nhau trong khoảng cách	Cao	Chọn lọc đặc trưng, giảm chiều, chuẩn hóa, trọng số đặc trưng
Bayes ngây thơ	Mỗi đặc trưng đóng góp một số hạng log riêng	Thấp (nhiễu có kỳ vọng đóng góp 0)	Chọn lọc đặc trưng cho các biến thể nhiều đặc trưng tương quan
Gauss đầy đủ theo lớp	Ước lượng $\frac{d(d+1)}{2}$ phần tử hiệp phương sai mỗi lớp	Rất cao (suy biến khi $d \geq n_c$)	Điều chuẩn hiệp phương sai
Softmax Regression	Học trọng số cho từng đặc trưng	Trung bình (học trọng số ≈ 0 cho nhiễu nếu có điều chuẩn)	Điều chuẩn L1/L2

6.4. Tính nhất quán: khi $n \to \infty$, k-NN tiến tới bộ phân loại tối ưu Bayes

Bộ phân loại tối ưu Bayes và sai số Bayes

Xét bài toán hai lớp với $\eta(\mathbf{x}) = P(y = 1 \mid \mathbf{x})$. **Bộ phân loại tối ưu Bayes** (đã nhắc ở mục 5.1) chọn lớp có hậu nghiệm lớn hơn tại mỗi $\mathbf{x}$, và sai số của nó là

$$L^* = \mathbb{E}\big[\min\big(\eta(\mathbf{x}),\, 1 - \eta(\mathbf{x})\big)\big].$$

Đây là sai số thấp nhất mà *bất kỳ* bộ phân loại nào có thể đạt, kể cả khi biết hoàn hảo phân phối của dữ liệu; nó khác 0 khi hai lớp chồng lấn (tại điểm có $\eta = 0{,}8$, ta sai với xác suất 0,2 dù chọn đúng nhất). Một bộ phân loại được gọi là **nhất quán** (consistent) nếu sai số của nó tiến về L^* khi $n \to \infty$.

k-NN: trực giác

Với k cố định, đặc biệt $k = 1$. Khi $n \to \infty$, láng giềng gần nhất của $\mathbf{x}$ tiến sát $\mathbf{x}$, nên nhãn của nó là một mẫu từ phân phối $\eta(\mathbf{x})$, *độc lập* với nhãn thật của điểm truy vấn. Hai nhãn khác nhau với xác suất $\eta(1-\eta) + (1-\eta)\eta$, cho sai số tiệm cận của 1-NN:

$$L_{1\text{-NN}} = \mathbb{E}\big[\,2\,\eta(\mathbf{x})\big(1 - \eta(\mathbf{x})\big)\big], \qquad L^* \;\leq\; L_{1\text{-NN}} \;\leq\; 2L^*(1 - L^*) \;\leq\; 2L^*.$$

(Kết quả của Cover và Hart.) Bất đẳng thức phải nhớ: 1-NN không tiến tới L^* mà tới một giá trị nằm giữa L^* và *gấp đôi* L^*, vì nó dựa vào đúng một nhãn ngẫu nhiên. Với mọi k cố định, sai số tiệm cận vẫn lớn hơn L^*, và giảm dần khi k tăng.

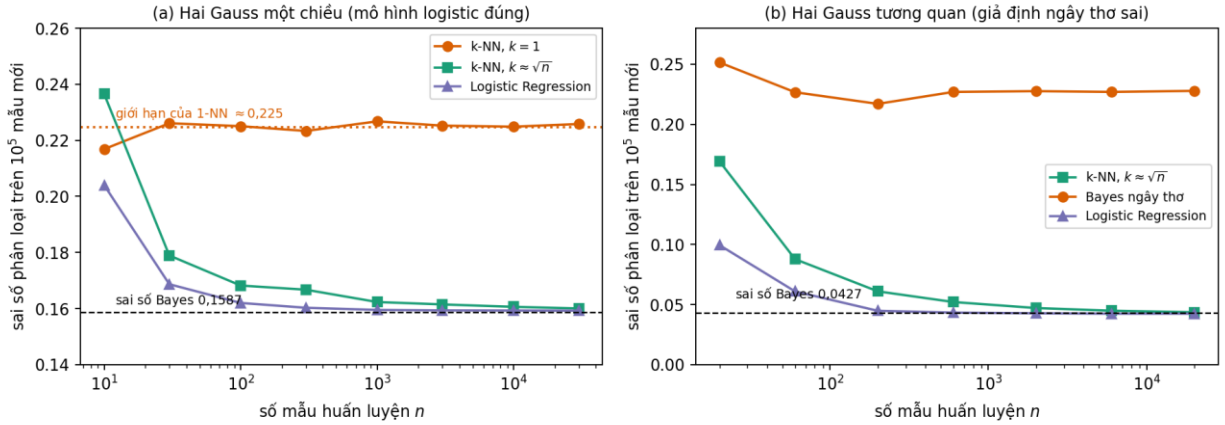
Với $k \to \infty$ nhưng $k/n \to 0$. Hai điều kiện đóng hai vai trò: $k \to \infty$ làm tần suất k_c/k trong lân cận hội tụ về xác suất thật (luật số lớn), còn $k/n \to 0$ làm lân cận *co lại* về điểm $\mathbf{x}$ nên xác suất thật trong lân cận chính là $\eta(\mathbf{x})$. Khi đó k-NN **nhất quán với mọi phân phối** (định lý của Stone, gọi là *nhất quán phổ quát*). Đây là điều mà *không* mô hình tham số cố định nào đạt được, vì mô

hình tham số chỉ nhất quán nếu dạng hàm đúng. Chọn $k \approx \sqrt{n}$ thoả cả hai điều kiện và là quy tắc kinh nghiệm hợp lý. Tương tự, KDE nhất quán (hội tụ về mật độ thật) khi $h \to 0$ và $nh^d \to \infty$, và hồi quy Nadaraya–Watson/LOESS nhất quán với các điều kiện tương tự.

Thí nghiệm: hai Gauss một chiều

Ta lấy hai lớp một chiều $\mathcal{N}(0,1)$ và $\mathcal{N}(2,1)$ với tiên nghiệm bằng nhau. Sai số Bayes tính được là $L^* = \Phi(-1) = 0{,}1587$, và sai số tiệm cận của 1-NN tính bằng tích phân số là 0,2248, nằm trong khoảng $[L^*,\ 2L^*(1-L^*)] = [0{,}1587;\ 0{,}2670]$ đúng như bất đẳng thức. Ta huấn luyện trên n mẫu từ 10 đến 30 000 và đo sai số trên 10^5 mẫu mới (trung bình 8 đến 30 lần chạy):

n	10	100	1 000	10 000	30 000
k-NN, $k = 1$	0,217	0,225	0,227	0,225	0,226
k-NN, $k \approx \sqrt{n}$	0,237	0,168	0,162	0,161	0,160
Logistic Regression	0,204	0,162	0,160	0,159	0,159



Hình 24. Sai số phân loại theo số mẫu huấn luyện n. (a) Hai Gauss một chiều, mô hình logistic đúng: k-NN với $k = 1$ bị kẹt ở $\approx 0{,}225$, tức gần 1,4 lần sai số Bayes; k-NN với $k \approx \sqrt{n}$ tiến chậm về sai số Bayes (0,1587); Logistic Regression đạt gần sai số Bayes rất nhanh. (b) Hai Gauss tương quan (cùng dữ liệu với Hình 19): k-NN và Logistic Regression tiến về sai số Bayes (0,0427), Bayes ngây thơ kẹt ở khoảng 0,228.

Kết quả minh họa đúng lý thuyết: 1-NN kẹt ở 0,225 (đúng bằng giá trị tiệm cận, không giảm theo n dù có 30 000 mẫu), còn k-NN với $k \approx \sqrt{n}$ tiến dần về 0,1587 (từ 0,168 ở $n = 100$ xuống 0,160 ở $n = 30\,000$). Logistic Regression ở đây nhanh hơn k-NN vì mô hình của nó *đúng*: với hai Gauss cùng phương sai, log-odds đúng là hàm tuyến tính của x.

Nhất quán và dạng mô hình: bức tranh đầy đủ

Hình 24b dùng bài toán hai Gauss tương quan 0,9 ở mục 5.6, có sai số Bayes 0,0427 (khoảng cách Mahalanobis 3,44).

n	20	200	2 000	20 000
k-NN, $k \approx \sqrt{n}$	0,169	0,061	0,047	0,0436
Bayes ngây thơ	0,251	0,217	0,228	0,228
Logistic Regression	0,099	0,045	0,043	0,0424

Ba mô hình, ba số phận: **k-NN** tiến về sai số Bayes (nhưng chậm: cần hàng nghìn mẫu cho một bài toán chỉ hai chiều); **Logistic Regression** cũng tiến về sai số Bayes (và nhanh hơn), vì với các Gauss cùng hiệp phương sai thì log-odds thật *tuyến tính*, nằm trong họ hàm của nó; **Bayes ngây thơ** kẹt ở 0,228 dù có bao nhiêu dữ liệu, vì giả định độc lập có điều kiện sai. Bảng tổng hợp:

Mô hình	**Nhất quán khi nào**	**Ví dụ không nhất quán**
k-NN, KDE, hồi quy nhân	Luôn nhất quán nếu $k \to \infty$, $k/n \to 0$ (hoặc $h \to 0$, $nh^d \to \infty$)	k cố định (1-NN kẹt ở khoảng L^* đến $2L^*$)
Softmax / Logistic Regression	Khi log-odds thật nằm trong họ hàm (tuyến tính theo đặc trưng đã chọn)	Ba vòng tròn đồng tâm trên đặc trưng gốc (37,8%, mục 1.1)
Bayes ngây thơ	Khi các đặc trưng thật sự độc lập có điều kiện (và dạng $p(x_j \mid c)$ đúng)	Đặc trưng tương quan (Hình 24b)
Gauss đầy đủ theo lớp	Khi dữ liệu mỗi lớp thật sự Gauss	Dữ liệu nhiều đỉnh (mục 4.6)

Đây là điểm cuối cùng của câu chuyện "tham số so với phi tham số" từ mục 1: mô hình phi tham số *đổi thiên lệch lấy phương sai*, và đảm bảo nhất quán (mọi dạng phân phối) với cái giá là cần nhiều dữ liệu hơn và chịu lời nguyền chiều. Lưu ý rằng nhất quán là lời hứa về $n \to \infty$, không bảo đảm gì cho n hữu hạn: ở $n = 20$ Logistic Regression (0,099) vượt k-NN (0,169) rất xa, kể cả ở bài toán mà k-NN cuối cùng sẽ tiệm cận Bayes.

6.5. Chi phí huấn luyện và dự đoán so với các mô hình tham số

Độ phức tạp tiệm cận

Tổng hợp các bảng chi phí ở các mục trước (với n mẫu, d đặc trưng, K lớp, m điểm truy vấn, T epoch):

Mô hình	**Huấn luyện**	**Dự đoán m điểm**	**Bộ nhớ**	**Thêm một mẫu mới**
k-NN (vét cạn)	$O(1)$	$O(mnd)$	$O(nd)$	$O(d)$ (chỉ lưu)
k-NN (KD-tree)	$O(ndlogn)$	$\sim O(mdlogn)$ khi d nhỏ; → $O(mnd)$ khi d lớn	$O(nd)$	dựng lại hoặc chèn
Nadaraya–Watson, KDE	$O(1)$	$O(mnd)$	$O(nd)$	$O(d)$
LOESS	$O(1)$	$O(m(nd^2 + d^3))$	$O(nd)$	$O(d)$

Mô hình	Huấn luyện	Dự đoán m điểm	Bộ nhớ	Thêm một mẫu mới
Bayes ngây thơ (tham số)	$O(nd)$	$O(mdK)$	$O(dK)$	$O(d)$ (cập nhật số đếm)
Softmax Regression	$O(T\,ndK)$	$O(mdK)$	$O(dK)$	huấn luyện lại hoặc SGD
Gauss đầy đủ theo lớp	$O(nd^2)$	$O(mKd^2)$	$O(Kd^2)$	$O(d^2)$

Có một sự **đối xứng** rõ ràng. Các mô hình phi tham số "lười": huấn luyện gần như tức thời, nhưng mỗi lần dự đoán phải duyệt (một phần) dữ liệu, và bộ nhớ tăng theo n. Các mô hình tham số "siêng": mất công huấn luyện một lần, rồi dự đoán cực nhanh với bộ nhớ không phụ thuộc n. Bayes ngây thơ có vị trí đặc biệt: huấn luyện chỉ một lượt đếm, và thêm một mẫu mới chỉ cần cập nhật số đếm.

Thời gian đo được

Để có con số cụ thể, ta đo thời gian trên dữ liệu tổng hợp ba lớp (ba đám Gauss), $d = 10$ và $d = 50$ đặc trưng, huấn luyện trên n từ 10^3 đến 10^5 mẫu, và dự đoán 1 000 điểm truy vấn (scikit-learn, $k = 5$). Đơn vị là mili-giây, **phụ thuộc máy** nên chỉ có ý nghĩa tương đối:

d	n	Softmax: huấn luyện / dự đoán	Bayes ngây thơ: huấn luyện / dự đoán	k-NN vét cạn: huấn luyện / dự đoán	k-NN KD-tree: huấn luyện / dự đoán
10	10^3	4,6 / 0,28	1,1 / 0,38	0,4 / 13,2	1,2 / 14,0
10	10^4	14,1 / 0,19	3,1 / 0,39	0,8 / 32,0	8,6 / 84,9
10	10^5	218 / 0,26	22,1 / 0,42	7,7 / 295	115 / 562
50	10^3	3,5 / 0,26	1,7 / 0,78	0,6 / 6,4	2,1 / 65,6
50	10^4	14,3 / 0,26	7,0 / 0,73	1,2 / 47,6	27,6 / 1 089
50	10^5	184 / 0,32	71,3 / 0,80	9,3 / 479	487 / 9 785

Quan sát:

- **Thời gian dự đoán** của k-NN tăng tuyến tính theo n (13,2 → 295 ms khi n tăng 100 lần ở $d = 10$), còn của Softmax và Bayes ngây thơ gần như không đổi (cỡ 0,3 đến 0,8 ms cho 1 000 điểm). Khi $n = 10^5$, k-NN dự đoán chậm hơn Softmax khoảng 1 000 lần.
- **Thời gian huấn luyện** theo chiều ngược lại: k-NN gần như bằng 0 (7,7 ms để lưu 10^5 mẫu), Softmax tốn 218 ms, Bayes ngây thơ 22 ms.
- **KD-tree không giúp ở số chiều này.** Ở $d = 10$ nó *chậm hơn* vét cạn (562 so với 295 ms ở $n = 10^5$), và ở $d = 50$ chậm hơn nhiều (9 785 so với 479 ms, gấp

khoảng 20 lần), đúng với lời cảnh báo ở mục 2.7: KD-tree chỉ nhanh khi d nhỏ (cỡ dưới 10). Còn vét cạn bằng phép nhân ma trận (mục 2.1) cực kỳ hiệu quả nhờ thư viện đại số tuyến tính tối ưu.

Khi nào đáng dùng loại nào: điểm hòa vốn

Tổng thời gian cho m lần dự đoán là $T(m) = T_{\text{huấn luyện}} + m\,T_{\text{dự đoán 1 mẫu}}$. Hai mô hình có tổng bằng nhau tại

$$m^* = \frac{T^{\text{tham số}}_{\text{huấn luyện}} - T^{\text{k-NN}}_{\text{huấn luyện}}}{T^{\text{k-NN}}_{\text{dự đoán 1 mẫu}} - T^{\text{tham số}}_{\text{dự đoán 1 mẫu}}}.$$

Với $d = 10$, $n = 10^5$: Softmax huấn luyện 218 ms và dự đoán 0,26 μs/mẫu; k-NN vét cạn huấn luyện 7,7 ms và dự đoán 295 μs/mẫu. Thế vào, $m^* = \frac{0{,}218 - 0{,}0077}{2{,}95\times10^{-4} - 2{,}6\times10^{-7}} \approx 710$. Nghĩa là **nếu chỉ cần dự đoán dưới khoảng 700 mẫu thì k-NN (tính cả huấn luyện) nhanh hơn Softmax; vượt qua đó Softmax thắng**. Với các hệ thống dự đoán hàng triệu yêu cầu, chi phí huấn luyện một lần của mô hình tham số gần như không đáng kể. Đặc điểm "thêm mẫu" cũng khác: thêm 100 mẫu vào Bayes ngây thơ ($n = 10^5$) bằng `partial_fit` tốn 0,5 ms, còn huấn luyện lại Softmax tốn 189 ms. Với dữ liệu đến liên tục, các mô hình lưu dữ liệu (k-NN) hoặc cập nhật số đếm (Bayes ngây thơ) có lợi thế lớn.

Bộ nhớ cũng khác biệt: với $n = 10^5$ và $d = 10$, k-NN phải giữ 10^6 số thực (khoảng 8 MB) và dữ liệu nhãn, còn Softmax chỉ giữ $(d + 1)K = 33$ số. Ở quy mô triệu mẫu hoặc nhiều chiều hơn, sự khác biệt này quyết định khả năng triển khai trên thiết bị nhỏ.

Tăng tốc cho k-NN: ở mức ý tưởng

Khi dữ liệu lớn và cần dự đoán nhiều, người ta không dùng vét cạn chính xác mà dùng **tìm láng giềng xấp xỉ** (approximate nearest neighbor): chấp nhận đôi khi bỏ sót một vài láng giềng thật để đổi lấy tốc độ, bằng cách băm các điểm gần nhau vào cùng một "giỏ" (băm nhạy vùng), hoặc dựng đồ thị các điểm gần nhau để đi theo. Một cách khác là *giảm dữ liệu*: giữ lại một tập con đại diện cho mỗi lớp, hoặc nén chiều bằng một phép chiếu (mục 6.3). Tất cả đều đánh đổi một chút độ chính xác lấy tốc độ, đúng tinh thần của toàn bộ chương này.

Bảng tổng hợp chọn mô hình

Hoàn cảnh	Nên cân nhắc	Lý do
Ít mẫu, ít chiều, hàm phức tạp chưa biết dạng	k-NN, hồi quy nhân, LOESS, KDE	Linh hoạt, không cần chọn dạng; chi phí không đáng ngại
Rất nhiều mẫu, cần dự đoán nhanh, số lượng lớn	Softmax, Bayes ngây thơ, mạng nơ-ron	Dự đoán không phụ thuộc n, bộ nhớ nhỏ
Số chiều lớn, nhiều đặc trưng nhiễu	Softmax có điều chuẩn, Bayes ngây thơ	Học trọng số hoặc cộng dồn bằng chứng, ít nhạy với nhiễu

Hoàn cảnh	Nên cân nhắc	Lý do
Văn bản (túi từ, số chiều hàng chục nghìn)	Bayes ngây thơ Multinomial	Huấn luyện một lượt, ma trận đếm thưa, hoạt động tốt với ít mẫu
Dữ liệu đến liên tục (cập nhật thường xuyên)	k-NN, Bayes ngây thơ	Thêm mẫu gần như miễn phí
Cần xác suất hiệu chỉnh tốt	Softmax	Bayes ngây thơ quá tự tin khi đặc trưng tương quan (mục 5.6)
Cần nhất quán với mọi phân phối, dữ liệu dồi dào	k-NN với $k \approx \sqrt{n}$	Nhất quán phổ quát (mục 6.4)
Có nhiều ngoại lai hoặc nhãn sai	k-NN với k lớn, Bayes ngây thơ	Cô lập hoặc pha loãng ảnh hưởng của mẫu xấu (mục 6.2)

Mối liên hệ với các mục trước

Mục 6 không giới thiệu mô hình mới mà *giải thích* các hiện tượng đã gặp:

- **Thiên lệch và phương sai (6.1)** giải thích hình chữ "U" của mọi đường cong theo k (mục 2.4), h (mục 3.4, 4.4) và α (mục 5.4). Công thức đóng Phương sai $= \sigma^2 \parallel \mathbf{s}(x) \parallel^2$ dùng lại các trọng số hiệu dụng $s_i(\mathbf{x}_0)$ của dạng chung ở mục 1.3 và ma trận làm trơn $\mathbf{S}$ ở mục 3.4.
- **Ảnh hưởng cục bộ (6.2)** trả lời điều đã báo trước ở mục 3.5: ngoại lai ảnh hưởng mạnh tại chỗ và bằng 0 ở xa, trong khi mô hình tham số lan truyền ảnh hưởng qua tham số chung.
- **Lời nguyền chiều (6.3)** gom các biểu hiện rải rác (khoảng cách tập trung ở mục 4.6, đặc trưng nhiễu ở mục 2.5 và 5.7), thêm chiều hình học và phân biệt chiều ngoài với chiều hiệu dụng.
- **Nhất quán (6.4)** khép lại câu chuyện của mục 1: mô hình phi tham số đổi lấy tính nhất quán phổ quát bằng phương sai và chi phí dữ liệu.
- **Chi phí (6.5)** hoàn thiện bảng so sánh "lười và siêng" đã hé ở mục 1.3 và đưa ra các con số đo được.

Mục 7 sẽ chuyển từ *phân tích* sang *đánh giá thực hành*: trực quan hóa, các độ đo (ma trận nhầm lẫn, precision, recall, F1, log-loss), cách chọn siêu tham số bằng kiểm định chéo và so sánh có hệ thống trên dữ liệu chữ số viết tay.

Tóm tắt mục 6

- Sai số kỳ vọng $= \sigma^2$ (nhiễu) + thiên lệch^2 + phương sai. Với bộ làm trơn tuyến tính $\hat{f} = \sum_i s_i(x) y_i$: thiên lệch $= \sum_i s_i f(x_i) - f(x)$ và phương sai $= \sigma^2 \parallel \mathbf{s}(x) \parallel^2$; với k-NN, phương sai bằng đúng σ^2/k.

- Mọi nút chỉnh (k, h, λ, α, bậc, số nơ-ron) điều khiển cùng một độ phức tạp hiệu dụng $\mathrm{df} = \mathrm{tr}(\mathbf{S})$; đường sai số có hình chữ "U", và các họ mô hình khác nhau đạt đáy ở cùng df (≈ 8 đến 13 trong thí nghiệm). Chẩn đoán bằng cách so độ chính xác huấn luyện với kiểm định chéo: khoảng cách lớn là phương sai cao, cả hai đều thấp là thiên lệch cao.
- Hàm ảnh hưởng của mẫu j là $s_j(x)$: mô hình địa phương cho ảnh hưởng mạnh nhưng chỉ trong một vùng hẹp (bằng 0 ở xa); mô hình toàn cục cho ảnh hưởng nhỏ nhưng lan khắp nơi. k-NN với k lớn bền trước nhãn sai ngẫu nhiên ($78{,}8\%$ ở 30% nhãn sai) còn $k = 1$ thì không ($54{,}4\%$); một ngoại lai đặc trưng cực đoan làm hỏng Softmax và Bayes ngây thơ ($-7{,}2$ và $-7{,}8$ điểm) nhưng không ảnh hưởng k-NN.
- Lời nguyền chiều có nhiều mặt: cạnh vùng lân cận $\ell = p^{1/d}$ tiến về 1, bán kính láng giềng tăng, khoảng cách tập trung, số mẫu cần tăng như $10^{(d+4)/4}$. Nó nhắm vào chiều hiệu dụng và chiều nhiễu chứ không phải số chiều ngoài: nhúng đẳng cự vào $\mathbb{R}^{200}$ không làm k-NN giảm, còn 50 đặc trưng nhiễu thì làm nó rơi xuống $54{,}4\%$. Bayes ngây thơ ít nhạy với nhiễu vì $\mathrm{KL} = 0$ cho đặc trưng nhiễu.
- k-NN với $k \to \infty$, $k/n \to 0$ nhất quán với mọi phân phối; 1-NN có sai số tiệm cận $\mathbb{E}[2\eta(1-\eta)] \in [L^*,\ 2L^*(1-L^*)]$. Mô hình tham số chỉ nhất quán khi dạng hàm đúng: Bayes ngây thơ kẹt ở $0{,}228$ trong khi sai số Bayes là $0{,}0427$. Nhất quán là lời hứa tiệm cận, không bảo đảm ở n nhỏ.
- Chi phí: mô hình phi tham số "lười" (huấn luyện ≈ 0, dự đoán $O(nd)$, bộ nhớ $O(nd)$), mô hình tham số "siêng" (huấn luyện nặng, dự đoán $O(dK)$, bộ nhớ nhỏ). Ở $n = 10^5$, $d = 10$, điểm hòa vốn giữa k-NN và Softmax khoảng 710 lần dự đoán. KD-tree chỉ có ích khi d nhỏ.

Câu hỏi kiểm tra nhanh

1. Hồi quy bằng k-NN với $k = 5$ tại một điểm x có $f(x) = 1{,}3$. Giá trị hàm thật tại 5 láng giềng là $1{,}0;\ 1{,}2;\ 0{,}8;\ 1{,}1;\ 0{,}9$. Nhiễu có $\sigma = 0{,}5$. Tính thiên lệch, phương sai và sai số bình phương kỳ vọng tại x.
2. Một mô hình đạt 99% trên tập huấn luyện nhưng chỉ 70% trong kiểm định chéo. Một mô hình khác đạt 60% và 58%. Chẩn đoán và đề xuất cách xử lý cho từng trường hợp.
3. Một bộ làm trơn có trọng số hiệu dụng $\mathbf{s}(x) = (0{,}5;\ 0{,}3;\ 0{,}2)$ (cộng bằng 1) tại một điểm. So sánh phương sai của nó (tính theo σ^2) với k-NN $k = 3$.

4. Với dữ liệu hai lớp có $\eta(\mathbf{x}) = 0{,}8$ tại một vùng, sai số của bộ phân loại Bayes ở vùng đó là bao nhiêu? Sai số tiệm cận của 1-NN ở vùng đó là bao nhiêu?

5. Với $d = 50$, hình lập phương con cần có cạnh bao nhiêu để chứa 1% dữ liệu phân bố đều trong $[0,1]^{50}$?

6. Vì sao đặt dữ liệu hai chiều vào $\mathbb{R}^{200}$ bằng một phép nhúng đẳng cự không làm k-NN tệ đi, còn thêm 198 đặc trưng nhiễu thì có?

7. Dự đoán bằng k-NN mất 5 ms mỗi mẫu và không cần huấn luyện; một mô hình tham số mất 2 giây huấn luyện và 0,01 ms mỗi mẫu. Với bao nhiêu lần dự đoán thì hai mô hình tốn thời gian bằng nhau?

Gợi ý đáp án

1. Trung bình các $f(x_i)$ là $(1{,}0 + 1{,}2 + 0{,}8 + 1{,}1 + 0{,}9)/5 = 1{,}0$, nên thiên lệch $= 1{,}0 - 1{,}3 = -0{,}3$ và thiên lệch$^2 = 0{,}09$. Phương sai $= \sigma^2/k = 0{,}25/5 = 0{,}05$. Sai số kỳ vọng $= \sigma^2 +$ thiên lệch$^2 +$ phương sai $= 0{,}25 + 0{,}09 + 0{,}05 = 0{,}39$.

2. Mô hình thứ nhất (99% so với 70%, khoảng cách lớn): phương sai cao (quá khớp). Xử lý: tăng k, h, λ hoặc α; thêm dữ liệu; bỏ bớt đặc trưng. Mô hình thứ hai (60% và 58%, thấp và gần nhau): thiên lệch cao (thiếu khớp). Xử lý: giảm k, h, λ; thêm đặc trưng hoặc dùng mô hình linh hoạt hơn. Thêm dữ liệu *không* giúp với trường hợp thứ hai.

3. Phương sai $= \sigma^2(0{,}5^2 + 0{,}3^2 + 0{,}2^2) = \sigma^2(0{,}25 + 0{,}09 + 0{,}04) = 0{,}38\,\sigma^2$. Với k-NN $k = 3$ mỗi trọng số bằng $1/3$, phương sai $= \sigma^2/3 \approx 0{,}333\,\sigma^2$. Trọng số không đều làm phương sai lớn hơn: với tổng trọng số cố định bằng 1, phương sai nhỏ nhất khi các trọng số bằng nhau.

4. Bayes chọn lớp có xác suất 0,8 nên sai với xác suất $\min(0{,}8;\ 0{,}2) = 0{,}2$. Sai số tiệm cận của 1-NN: $2\eta(1-\eta) = 2 \times 0{,}8 \times 0{,}2 = 0{,}32$, gấp 1,6 lần sai số Bayes (và nhỏ hơn cận $2L^*(1 - L^*) = 0{,}32$, trùng nhau vì η là hằng trong vùng).

5. $\ell = 0{,}01^{1/50} = e^{\ln(0{,}01)/50} = e^{-0{,}0921} \approx 0{,}912$: hình lập phương con phải dài 91% mỗi cạnh của miền dữ liệu mới chứa được 1% dữ liệu, nên không còn là "địa phương".

6. Phép nhúng đẳng cự ($\mathbf{Q}\mathbf{Q}^\top = \mathbf{I}$) *bảo toàn mọi khoảng cách giữa các điểm*, nên tập láng giềng của mọi điểm không đổi và k-NN đưa ra cùng dự đoán (số chiều hiệu dụng vẫn là 2). Còn 198 đặc trưng nhiễu là 198 chiều *có thật* cộng thêm các số hạng bình phương dương vào khoảng cách, áp đảo phần tín hiệu của 2 đặc trưng thật và làm các láng giềng được chọn gần như ngẫu nhiên.

7. Tổng thời gian k-NN: $5m$ ms; mô hình tham số: $2\,000 + 0{,}01m$ ms. Cho bằng nhau: $5m = 2\,000 + 0{,}01m \Rightarrow m = 2\,000/4{,}99 \approx 401$. Dưới khoảng 401 lần dự đoán thì k-NN nhanh hơn, từ đó trở lên mô hình tham số nhanh hơn.

7. Đánh giá, trực quan hóa và chọn siêu tham số

Sáu mục trước đã xây dựng một bộ công cụ và giải thích vì sao chúng hoạt động. Mục 7 chuyển từ *hiểu* sang *làm*: khi có một mô hình trong tay, ta **nhìn** nó như thế nào, **đo** nó bằng thước nào, **chọn** các nút chỉnh ra sao, và **so sánh** các mô hình công bằng như thế nào. Mỗi câu hỏi này có một cái bẫy riêng, và hầu hết các cái bẫy đã xuất hiện rải rác ở các mục trước (xác suất 0 làm log-loss bằng vô cùng ở mục 2.2, dữ liệu rò từ bước chuẩn hóa ở mục 2.4, xác suất quá tự tin ở mục 5.6). Mục 7 gom chúng lại thành một quy trình và áp dụng nó vào bài toán thực tế duy nhất của bài: nhận dạng **chữ số viết tay**.

Ta trả lời các câu hỏi sau:

- Làm sao để *nhìn thấy* một mô hình phi tham số: vùng quyết định, đường hồi quy và hàm mật độ, kể cả độ bất định của chúng? (mục 7.1)
- Đánh giá phân loại bằng những thước đo nào, và vì sao độ chính xác đơn thuần có thể đánh lừa? (mục 7.2)
- Đánh giá hồi quy và mật độ bằng gì? (mục 7.3)
- Chọn k, h, α bằng kiểm định chéo thế nào cho đúng? (mục 7.4)
- So sánh k-NN, Bayes ngây thơ, Logistic và Softmax trên dữ liệu chữ số viết tay cho thấy điều gì? (mục 7.5)
- Cuối cùng, huấn luyện và so sánh với `KNeighborsClassifier`, `KernelDensity`, `GaussianNB`, `MultinomialNB` của scikit-learn ra sao? (mục 7.6)

Quy ước ký hiệu

Ta giữ các quy ước trước đó: **chữ thường in đậm là vector**, **chữ hoa in đậm là ma trận**, đặc trưng không có cột 1. Ở mục này $\mathbf{Y} \in \mathbb{R}^{n \times K}$ là ma trận one-hot *thật* của tập đánh giá và $\widehat{\mathbf{Y}}$ là ma trận one-hot của *nhãn dự đoán* (cùng kích thước).

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Ma trận nhầm lẫn	$\mathbf{M} = \mathbf{Y}^\top \widehat{\mathbf{Y}}$	$K \times K$	M_{ab} là số mẫu có nhãn thật a và nhãn dự đoán b
Đúng dương, dương giả, âm giả	TP_c, FP_c, FN_c	vô hướng	Với lớp c: $\mathrm{TP}_c = M_{cc}$, $\mathrm{FP}_c = \sum_{a \neq c} M_{ac}$, $\mathrm{FN}_c = \sum_{b \neq c} M_{cb}$
Độ chính xác theo lớp, độ phủ theo lớp	P_c, R_c	vô hướng	Precision và recall của lớp c
Ma trận xác suất dự đoán	$\widehat{\mathbf{P}}$	$n \times K$	Hàng i là phân phối dự đoán cho mẫu i
Tập siêu tham số	$\boldsymbol{\theta}$	vector	Gồm k, h, α, C, …

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Điểm kiểm định chéo	$\mathrm{CV}(\boldsymbol{\theta})$	vô hướng	Trung bình điểm trên V phần
Số phần chia	V	vô hướng	Số phần của kiểm định chéo (dùng chữ V để không trùng với K)
Trung bình của biến mục tiêu	$\bar{y}$	vô hướng	Trung bình của y trên tập đánh giá

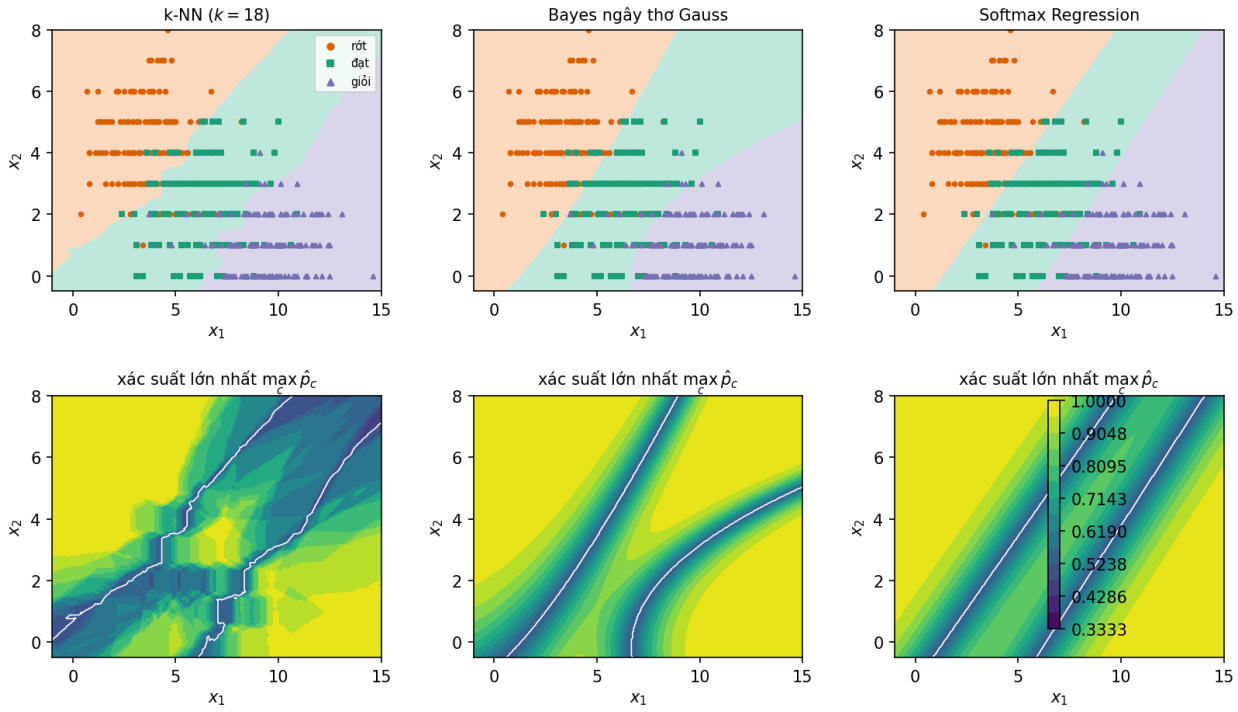
Về dữ liệu: ta dùng bộ **chữ số viết tay** của scikit-learn (`load_digits`): 1 797 ảnh xám 8×8 (tức $d = 64$ đặc trưng là độ đậm của từng điểm ảnh, nhận giá trị nguyên từ 0 đến 16), $K = 10$ lớp (chữ số 0 đến 9), mỗi lớp khoảng 180 ảnh. Ta chia phân tầng 70% huấn luyện (1 257 ảnh) và 30% kiểm tra (540 ảnh), cố định cho mọi thí nghiệm. Mỗi ảnh được trải phẳng thành một vector hàng, đúng như $\mathbf{x}_i \in \mathbb{R}^{1\times 64}$ của các mục trước.

7.1. Trực quan hóa vùng quyết định, đường hồi quy và hàm mật độ ước lượng

Trực quan hóa không phải là phần trang trí. Nó cho thấy những điều mà một con số không nói được: mô hình sai ở đâu, đường biên có hợp lý không, vùng nào mô hình "không biết". Giới hạn cơ bản là ta chỉ vẽ được hai (đôi khi ba) chiều, nên với dữ liệu nhiều chiều, ta vẽ trên hai chiều đã được chọn hoặc chiếu xuống (xem hình cuối của mục này).

Phân loại: vùng quyết định và bản đồ độ tin cậy

Với hai đặc trưng, ta vẽ nhãn dự đoán trên một lưới điểm phủ miền dữ liệu (như Hình 3, 4 và 18). Ngoài vùng quyết định, ta nên vẽ thêm **bản đồ xác suất lớn nhất** $\max_c \hat{p}_c(\mathbf{x})$ trên cùng lưới, vì nó cho biết mô hình *tự tin* ở đâu. Hình 25 làm cả hai cho 420 sinh viên huấn luyện (trong 600 sinh viên mô phỏng của mục 2.3).



Hình 25. Hàng trên: vùng quyết định của k-NN ($k = 18$), Bayes ngây thơ Gauss và Softmax Regression trên tập huấn luyện. Hàng dưới: xác suất lớn nhất $\max_c \hat{p}_c$ của chính mô hình đó trên lưới (đường trắng là biên quyết định). k-NN cho các "khối" xác suất rời rạc (bội của $1/18$) và gập ghềnh; Bayes ngây thơ cho các dải mượt hẹp quanh biên; Softmax cho các dải thẳng. Ở cả ba mô hình, xác suất tiến về 1 khi đi ra xa dữ liệu.

Có ba điều nhìn thấy được mà bảng số không cho:

- **Hình dạng biên khác nhau** (gập ghềnh, cong, thẳng) và phản ánh dạng mô hình (mục 2.3, 5.6, 1.1).
- **Độ tin cậy cao ở xa dữ liệu.** Ở góc phải dưới và góc trái trên, cả ba mô hình đều cho xác suất xấp xỉ 1 dù ở đó không có mẫu huấn luyện nào. Với Softmax và Bayes ngây thơ, đây là hệ quả của việc điểm số tuyến tính hay bậc hai tăng không giới hạn khi đi ra xa (mục 4.1). Với k-NN là vì mọi láng giềng ở đó cùng thuộc một lớp. *Xác suất cao không có nghĩa là mô hình hiểu vùng đó*; để biết một điểm có nằm trong vùng có dữ liệu hay không, ta cần mật độ $p(\mathbf{x})$ (mục 4).
- **Hình dạng của sự không chắc chắn.** Vùng xác suất thấp (màu tối) tập trung quanh các biên, hẹp với Bayes ngây thơ và Softmax, rộng và rách với k-NN (vì xác suất của k-NN chỉ nhận $k + 1$ mức, mục 2.2).

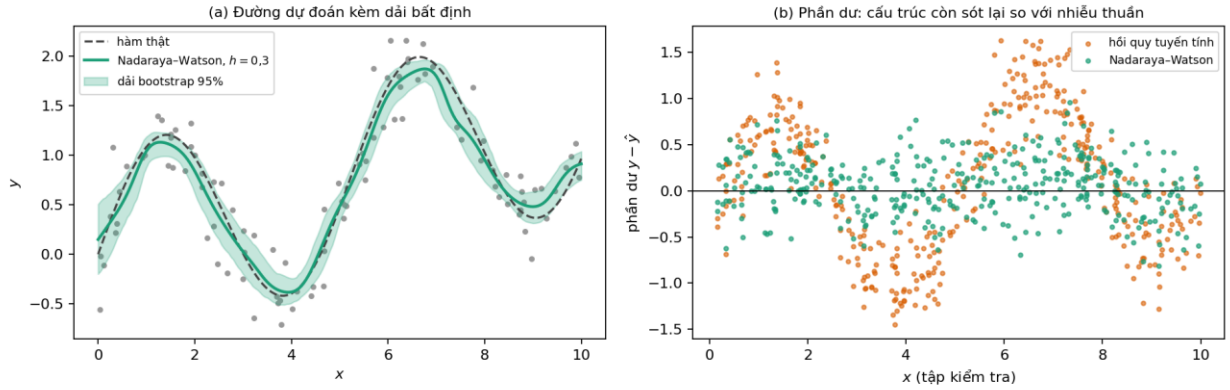
Hồi quy: đường dự đoán, dải bất định và phần dư

Với hồi quy một chiều, ta vẽ dữ liệu, đường dự đoán và (nếu biết) hàm thật. Chỉ vẽ một đường thì ta không biết có nên tin vào các "gợn" của nó hay không. Cách đơn giản để thêm **dải bất**

định là **bootstrap**: rút B bộ dữ liệu có hoàn lại từ tập huấn luyện (mỗi bộ cùng kích thước n), huấn luyện lại mô hình trên từng bộ để được $\hat{f}^{(1)}, \ldots, \hat{f}^{(B)}$, rồi tại mỗi điểm x lấy phân vị 2,5% và 97,5%:

$$\left[\hat{f}_{(2{,}5\%)}(x),\ \ \hat{f}_{(97{,}5\%)}(x)\right], \qquad \text{trong đó } \hat{f}_{(q)}(x) \text{ là phân vị cấp } q \text{ của } \{\hat{f}^{(b)}(x)\}_{b=1}^{B}.$$

Hình 26a vẽ Nadaraya–Watson ($h = 0{,}3$, dữ liệu sin của mục 3) với $B = 300$. Dải có độ rộng trung bình 0,36 ở phần trong miền $[1,9]$ và 0,50 ở hai biên, phản ánh việc phương sai lớn hơn ở biên (mục 3.4). Cần hiểu đúng ý nghĩa của dải: nó cho biết *phương sai* của ước lượng, **không** bao gồm thiên lệch. Vì vậy hàm thật nằm trong dải chỉ ở 91% số điểm lưới chứ không phải 95%; các chỗ ngoài dải là nơi thiên lệch (mô hình làm phẳng các đỉnh và đáy) lớn hơn độ rộng dải.

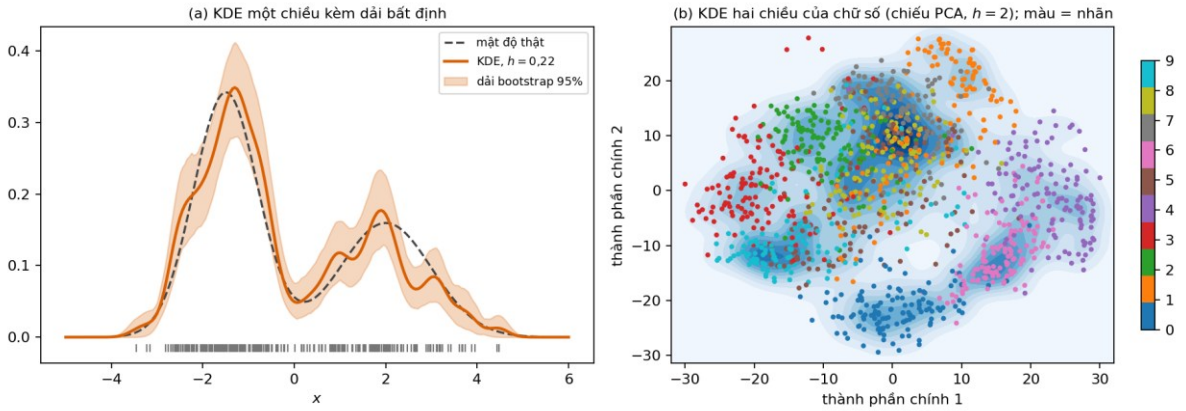


Hình 26. (a) Nadaraya–Watson ($h = 0{,}3$) trên dữ liệu sin, kèm dải bootstrap 95% ($B = 300$); đường nét đứt là hàm thật. (b) Phần dư $y - \hat{y}$ trên tập kiểm tra của hồi quy tuyến tính (cam) và Nadaraya–Watson (xanh lục). Phần dư của đường thẳng lặp lại hình sin (còn cấu trúc chưa được giải thích), còn phần dư của Nadaraya–Watson trông như nhiễu thuần.

Biểu đồ **phần dư** (Hình 26b) cho một phép thử khác: nếu mô hình đã nắm bắt hết cấu trúc, phần dư $y - \hat{y}$ phải trông như nhiễu ngẫu nhiên quanh 0, không có hình dạng. Phần dư của hồi quy tuyến tính chạy theo một đường sin rõ ràng (thiên lệch), trong khi của Nadaraya–Watson thì không.

Mật độ: ước lượng kèm dải và hình chiếu

Với ước lượng mật độ một chiều, ta cũng dùng dải bootstrap (Hình 27a, KDE với $h = 0{,}22$ trên 300 mẫu hai đỉnh của mục 4). Với dữ liệu nhiều chiều, ta chiếu lên hai chiều rồi vẽ đường mức. Hình 27b chiếu các ảnh chữ số 64 chiều xuống hai thành phần chính (PCA, phép chiếu tuyến tính giữ lại hai hướng có phương sai lớn nhất) và vẽ KDE hai chiều với $h = 2$, tô màu điểm theo nhãn.



Hình 27. (a) KDE một chiều kèm dải bootstrap 95% cho 300 mẫu hai đỉnh (đường nét đứt là mật độ thật). (b) KDE hai chiều của các chữ số huấn luyện sau khi chiếu xuống hai thành phần chính (PCA); màu là nhãn thật.

Hình 27b cho thấy một điều quan trọng về dữ liệu chữ số: các chữ số 0, 6, 4 và 1 chiếm các vùng khá riêng biệt ngay trong hai chiều đầu tiên, còn các chữ số 2, 5, 7, 8 chồng lấn dày đặc ở giữa (và 3, 9 ở bên trái chồng lấn một phần với các chữ số khác). Đây là dấu hiệu cho thấy chiều hiệu dụng của dữ liệu thấp hơn nhiều 64 (mục 6.3), điều giải thích vì sao k-NN vẫn hoạt động tốt ở mục 7.5.

Một số nguyên tắc khi trực quan hóa. (i) Luôn vẽ *dữ liệu* cùng với mô hình. (ii) Khi biết hàm thật (dữ liệu mô phỏng), vẽ nó. (iii) Vẽ ít nhất ba giá trị của siêu tham số chính (quá nhỏ, vừa, quá lớn) để thấy sự đánh đổi (như Hình 8, 9, 14). (iv) Không kết luận chỉ từ một hình hai chiều khi dữ liệu thật có nhiều chiều hơn.

7.2. Đánh giá phân loại: ma trận nhầm lẫn, precision, recall, F1, log-loss

Ma trận nhầm lẫn và độ chính xác

Với một tập đánh giá có n mẫu, gọi $\mathbf{Y}$ là ma trận one-hot nhãn thật và $\widehat{\mathbf{Y}}$ là ma trận one-hot nhãn dự đoán (hàng i của $\widehat{\mathbf{Y}}$ có một số 1 ở vị trí $\hat{y}_i$). **Ma trận nhầm lẫn** tính được bằng một phép nhân ma trận:

$$\mathbf{M} = \mathbf{Y}^{\top}\widehat{\mathbf{Y}} \in \mathbb{R}^{K\times K}, \qquad M_{ab} = \#\{\, i : y_i = a,\ \hat{y}_i = b \,\}.$$

Hàng a liệt kê mọi mẫu có nhãn thật a và chúng được dự đoán vào đâu; cột b liệt kê mọi mẫu được dự đoán là b và chúng thật sự thuộc lớp nào. Đường chéo là các dự đoán đúng, nên **độ chính xác** (accuracy) là

$$\text{accuracy} = \frac{\operatorname{tr}(\mathbf{M})}{n} = \frac{1}{n}\sum_{c=1}^{K} M_{cc}.$$

Precision, recall và F1

Độ chính xác tổng gộp mọi lớp lại, che đi việc mô hình đối xử với từng lớp thế nào. Với mỗi lớp c, định nghĩa $\mathrm{TP}_c = M_{cc}$ (đúng dương), $\mathrm{FP}_c = \sum_{a \neq c} M_{ac}$ (dương giả: mẫu lớp khác bị dự đoán là c, tức phần còn lại của *cột* c) và $\mathrm{FN}_c = \sum_{b \neq c} M_{cb}$ (âm giả: mẫu lớp c bị dự đoán là lớp khác, tức phần còn lại của *hàng* c). Khi đó:

$$\mathrm{P}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c} = \frac{M_{cc}}{\sum_a M_{ac}}, \qquad \mathrm{R}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c} = \frac{M_{cc}}{\sum_b M_{cb}}, \qquad \mathrm{F1}_c = \frac{2\,\mathrm{P}_c\,\mathrm{R}_c}{\mathrm{P}_c + \mathrm{R}_c}$$

$$= \frac{2\,\mathrm{TP}_c}{2\,\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}.$$

- **Precision** P_c trả lời: *trong những mẫu mô hình bảo là lớp c, bao nhiêu phần trăm đúng?* (chia cho tổng *cột* c). Quan trọng khi "báo động giả" tốn kém.
- **Recall** R_c trả lời: *trong những mẫu thật sự thuộc lớp c, mô hình bắt được bao nhiêu phần trăm?* (chia cho tổng *hàng* c). Quan trọng khi bỏ sót tốn kém.
- **F1** là trung bình điều hòa của hai số trên, đạt giá trị cao chỉ khi *cả hai* cùng cao (một trong hai thấp là F1 thấp).

Để tóm tắt K giá trị F1 thành một số, có ba cách: **macro** (trung bình đơn giản của $\mathrm{F1}_c$, coi các lớp ngang nhau), **có trọng số** (trung bình có trọng số theo số mẫu mỗi lớp) và **micro** (gộp TP, FP, FN của mọi lớp trước khi tính; với phân loại mỗi mẫu một nhãn, micro-F1 bằng đúng độ chính xác).

Ví dụ: 180 sinh viên kiểm tra

Với k-NN ($k = 18$) trên tập kiểm tra 180 sinh viên của mục 2.3 (đã đạt 82,2%), ma trận nhầm lẫn (hàng: nhãn thật; cột: nhãn dự đoán; thứ tự rớt, đạt, giỏi) là

$$\mathbf{M} = \begin{pmatrix} 49 & 11 & 0 \\ 3 & 49 & 8 \\ 0 & 10 & 50 \end{pmatrix}.$$

Độ chính xác $= (49 + 49 + 50)/180 = 148/180 = 82{,}2\%$. Theo từng lớp:

Lớp	TP	Tổng cột (dự đoán là lớp đó)	Tổng hàng (thật sự là lớp đó)	Precision	Recall	F1
rớt	49	52	60	0,942	0,817	0,875
đạt	49	70	60	0,700	0,817	0,754
giỏi	50	58	60	0,862	0,833	0,847

Macro-F1 $= (0{,}875 + 0{,}754 + 0{,}847)/3 = 0{,}825$. Số liệu cho thấy rõ cái mà độ chính xác 82,2% che đi: lớp "đạt" là lớp bị dự đoán nhầm nhiều nhất (precision chỉ 0,70) vì nó là lớp *ở giữa* và hấp thụ các sinh viên rớt (11) và giỏi (10) bị đoán nhầm; lỗi hầu như không bao giờ

nhảy giữa hai lớp đầu và cuối (chỉ 0 sinh viên giỏi bị đoán là rớt và 0 ngược lại), đúng với cấu trúc có thứ tự của dữ liệu.

Cái bẫy của độ chính xác với dữ liệu lệch lớp

Giả sử trong 1 000 mẫu có 50 mẫu dương (ví dụ thư rác) và một "mô hình" luôn dự đoán âm. Nó có $\mathrm{TP} = 0$, $\mathrm{FN} = 50$, $\mathrm{FP} = 0$, $\mathrm{TN} = 950$: độ chính xác $= 950/1\,000 = 95\%$ nhưng $\mathrm{R} = 0/50 = 0$ và $\mathrm{F1} = 0$. Độ chính xác cao mà mô hình vô dụng. Với dữ liệu lệch lớp, hãy dùng precision, recall, F1 (hay ma trận nhầm lẫn) thay vì chỉ độ chính xác, và chọn thước đo theo chi phí của từng loại lỗi.

Log-loss: đánh giá cả xác suất

Precision và recall chỉ dùng nhãn dự đoán. Nếu mô hình trả về cả xác suất $\widehat{\mathbf{P}}$, ta có thể đánh giá xác suất đó bằng **log-loss** (cross-entropy trung bình, cũng chính là hàm mất mát của Softmax Regression):

$$\mathrm{LogLoss} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{c=1}^{K} y_{ic}\,\ln\hat{p}_{ic} = -\frac{1}{n}\,\mathbf{1}_n^{\top}\big(\mathbf{Y}\odot\ln\widehat{\mathbf{P}}\big)\mathbf{1}_K,$$

trong đó $\odot$ là tích từng phần tử và ln lấy theo từng phần tử. Mỗi mẫu đóng góp $-\ln$ của xác suất mô hình gán cho **lớp thật**. Log-loss phạt rất nặng việc *tự tin mà sai*: xác suất 0,9 cho lớp thật đóng góp 0,105, xác suất 0,1 đóng góp 2,303, còn xác suất 0 đóng góp $+\infty$.

Với k-NN và Bayes ngây thơ, xác suất 0 cho lớp thật *xảy ra thật*. Với k-NN trên chữ số, nếu tất cả k láng giềng đều thuộc lớp khác thì $\hat{p}_{\text{thật}} = 0$. Khi $k = 3$, 4 trong 540 mẫu kiểm tra có tính chất này; log-loss "thô" (với sàn 10^{-15} của scikit-learn) là 0,277, trong đó chỉ 4 mẫu đóng góp $4 \times 34{,}5/540 = 0{,}256$, tức 92% tổng, vì mỗi mẫu như vậy đóng góp $-\ln 10^{-15} = 34{,}5$. Con số này phụ thuộc vào sàn tùy ý, không có ý nghĩa. Cách chữa là **làm trơn xác suất**, đúng ý tưởng làm trơn Laplace của mục 5.4 áp dụng lên số phiếu v_c của mục 2.2:

$$\tilde{p}_c = \frac{v_c + \alpha}{k + K\alpha}, \qquad \sum_{c=1}^{K}\tilde{p}_c = 1.$$

Làm trơn α	**Log-loss kiểm tra, k-NN $k = 3$**	$k = 5$	$k = 10$
0 (thô, sàn 10^{-15})	0,277	0,105	0,136
0,01	0,092	0,071	0,093
0,1	0,300	0,210	0,164

Hai điều rút ra: (i) một lượng làm trơn **nhỏ** ($\alpha = 0{,}01$) giảm log-loss từ 0,277 xuống 0,092 vì loại bỏ các điểm đóng góp vô cùng; (ii) làm trơn **quá mạnh** ($\alpha = 0{,}1$) lại làm xấu đi (0,300), vì khi mô hình đúng 98% số lần, làm trơn kéo xác suất đúng từ 1 xuống thấp hơn trên cả 98% số mẫu đúng. Giống mọi siêu tham số, α cần được chọn bằng kiểm định chéo (mục 7.4).

Độ tin cậy có khớp với độ chính xác không?

Một cách đơn giản để kiểm tra "hiệu chỉnh xác suất" (calibration) là so **độ tin cậy trung bình** $\overline{c} = \frac{1}{n}\sum_i \max_c \hat{p}_{ic}$ với độ chính xác. Mô hình hiệu chỉnh tốt khi hai số xấp xỉ bằng nhau; $\overline{c} \gg$ accuracy là quá tự tin (như Bayes ngây thơ ở mục 5.6), $\overline{c} <$ accuracy là chưa đủ tự tin. Mục 7.5 báo cáo số này cho từng mô hình trên chữ số.

7.3. Đánh giá hồi quy (MAE, RMSE, R^2) và đánh giá mật độ

Hồi quy

Với n mẫu đánh giá, giá trị thật y_i và dự đoán $\hat{y}_i$, ba thước đo thông dụng là

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |\, y_i - \hat{y}_i|, \qquad \text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}\,, \qquad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

- **MAE** (sai số tuyệt đối trung bình): cùng đơn vị với y, mỗi sai số được tính tỉ lệ thuận, nên ít bị một vài sai số lớn chi phối.
- **RMSE** (căn sai số bình phương trung bình): cũng cùng đơn vị với y, nhưng bình phương làm các sai số lớn có trọng lượng lớn hơn nhiều. Đây là thước đo tự nhiên của phân rã thiên lệch–phương sai ở mục 6.1.
- R^2: tỉ lệ phương sai của y mà mô hình giải thích được, so với mô hình hằng $\hat{y} = \bar{y}$ (cho $R^2 = 0$). Có thể *âm* nếu mô hình tệ hơn việc luôn dự đoán trung bình.

Bảng sau cho kết quả trên 400 điểm kiểm tra của bài toán $y = \sin(1{,}2x) + 0{,}15x + \varepsilon$ ($\sigma = 0{,}3$) ở mục 3:

Mô hình	**MAE**	**RMSE**	R^2	**RMSE/MAE**
Hồi quy tuyến tính	0,626	0,738	0,029	1,18
Đa thức bậc 7	0,237	0,298	0,841	1,26
k-NN ($k = 6$)	0,247	0,311	0,827	1,26
Nadaraya–Watson ($h = 0{,}3$)	0,247	0,311	0,828	1,26
LOESS ($h = 0{,}35$)	0,248	0,312	0,826	1,26
Mức sàn (nhiễu)	0,239	0,300	≈ 0,84 đến 0,85	1,25

Cách đọc đúng:

- **Biết mức sàn trước khi đánh giá.** Nhiễu có $\sigma = 0{,}3$ nên RMSE $\geq \sigma = 0{,}30$, và với nhiễu Gauss thì MAE $\geq \sigma\sqrt{2/\pi} = 0{,}239$. Khi đó, R^2 cũng không thể đến 1: $R^2_{\max} = 1 - \sigma^2/\text{Var}(y) = 1 - 0{,}09/0{,}560 = 0{,}839$ (kỳ vọng; với riêng tập kiểm

tra này, dùng chính hàm thật cho $R^2 = 0{,}850$). Vì vậy $R^2 = 0{,}83$ của các phương pháp phi tham số không phải là "chỉ 83%" mà là **gần tối đa có thể**.

- **Hồi quy tuyến tính có** $R^2 = 0{,}029$: gần như không giải thích được gì, vì một đường thẳng không thể mô tả hàm có hai chu kỳ dao động (mục 1.1).
- **Tỉ số RMSE/MAE là một chẩn đoán.** Với phần dư Gauss, tỉ số này xấp xỉ $\sqrt{\pi/2} \approx 1{,}253$. Các mô hình tốt ở bảng đều ra 1,26, đúng như vậy. Nếu tỉ số này lớn hơn đáng kể (ví dụ 2), phần dư có đuôi nặng hoặc có ngoại lai (mục 6.2).

Mật độ

Ở mục 4, ta đã dùng ISE (chỉ tính được khi biết mật độ thật) và **log-hợp lý trung bình trên tập kiểm tra** $\overline{\ell} = \frac{1}{m}\sum_{j=1}^{m} \ln \hat{f}\left(\mathbf{x}_j^{\text{kt}}\right)$ (không cần biết mật độ thật). Đây là thước đo đánh giá mật độ chính vì (mục 4.4)

$$\mathbb{E}\left[\ln \hat{f}(\mathbf{X})\right] = \int p(\mathbf{x}) \ln p(\mathbf{x})\, d\mathbf{x} - \mathrm{KL}\left(p \,\|\, \hat{f}\right),$$

nên $\overline{\ell}$ cao hơn nghĩa là $\hat{f}$ gần mật độ thật hơn theo độ phân kỳ KL. Có ba lưu ý. **(i)** Chỉ so sánh các $\overline{\ell}$ trên *cùng* tập kiểm tra và cùng đơn vị (nats khi dùng ln). **(ii)** Chỉ áp dụng với ước lượng *đã chuẩn hóa* (tích phân bằng 1): ước lượng k láng giềng ở mục 4.5 cho $\overline{\ell}$ cao hơn cả mật độ thật chỉ vì tích phân của nó vượt 1. **(iii)** Mật độ bằng 0 tại một điểm kiểm tra làm $\overline{\ell} = -\infty$ (như histogram ở mục 4.2). Ví dụ:

Bài toán	**Mô hình**	$\overline{\ell}$ **(nats/mẫu)**
Hai đỉnh một chiều (mục 4)	Mật độ thật	$-1{,}839$
	KDE, $h = 0{,}22$ (hợp lý cực đại có kiểm định chéo)	$-1{,}869$
	KDE, $h = 0{,}53$ (Silverman)	$-1{,}880$
	Gauss hợp lý cực đại	$-2{,}099$
Chữ số chiếu PCA hai chiều (Hình 27b)	KDE, $h = 2$ (chọn bằng kiểm định chéo)	$-7{,}673$
	Gauss hợp lý cực đại	$-7{,}989$

Cả hai bài toán cho cùng kết luận: KDE thắng Gauss, vì dữ liệu không phải một đỉnh Gauss. Với dữ liệu chữ số hai chiều, chênh lệch 0,32 nats mỗi mẫu tương ứng một mật độ trung bình lớn hơn $e^{0{,}32} \approx 1{,}37$ lần ở các điểm kiểm tra.

7.4. Chọn k, h, hệ số làm trơn α bằng kiểm định chéo

Nguyên tắc: chia ba, và không bao giờ nhìn tập kiểm tra

Quy trình chuẩn chia dữ liệu thành ba phần có vai trò khác nhau: **huấn luyện** (để mô hình học; với phi tham số, để lưu), **kiểm định** (để chọn siêu tham số) và **kiểm tra** (chỉ dùng *một lần ở cuối* để báo cáo kết quả). Kiểm định chéo V phần (mục 2.4) là cách dùng dữ liệu hiệu quả để thay phần kiểm định: với mỗi bộ siêu tham số $\boldsymbol{\theta}$, ta huấn luyện trên $V-1$ phần và đo điểm trên phần còn lại, lặp V lần, rồi lấy trung bình và chọn bộ tốt nhất:

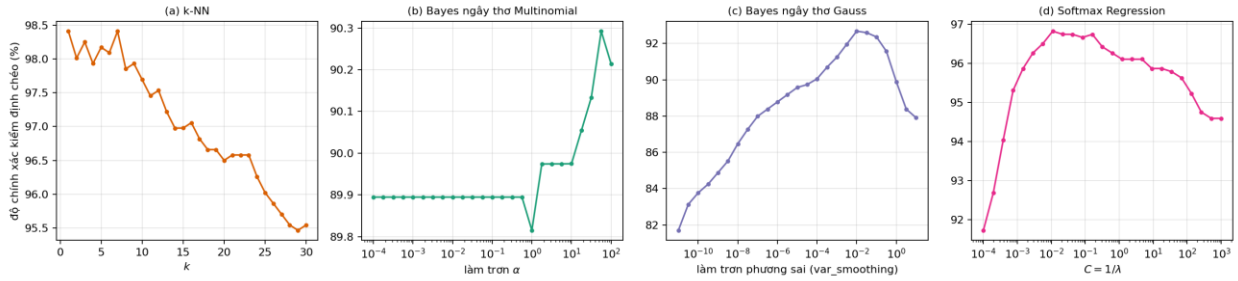
$$\mathrm{CV}(\boldsymbol{\theta}) = \frac{1}{V}\sum_{v=1}^{V} \text{Điểm}\left(\boldsymbol{\theta};\ \text{phần } v\right), \qquad \widehat{\boldsymbol{\theta}} = \operatorname*{argmax}_{\boldsymbol{\theta}} \mathrm{CV}(\boldsymbol{\theta}).$$

Sau khi chọn $\widehat{\boldsymbol{\theta}}$, ta huấn luyện lại *một lần nữa* trên toàn bộ tập huấn luyện rồi mới đo trên tập kiểm tra. Có bốn quy tắc thực hành:

- **Chia phân tầng** (mỗi phần có tỉ lệ lớp giống toàn bộ), đặc biệt khi lớp lệch.
- **Lưới theo thang phù hợp.** Tham số nguyên như k dùng lưới tuyến tính $(1,2,3,\ldots)$; tham số thang đo như h, α, C, `var_smoothing` dùng lưới **logarit** (nhân lên đều: $10^{-4}, 10^{-3}, \ldots$). Nếu giá trị tốt nhất nằm *ở rìa* lưới thì phải mở rộng lưới (khi tìm băng thông cho KDE phân loại chữ số, lưới ban đầu $0{,}5$ đến 6 cho kết quả tốt nhất ở 6, đúng rìa; ta mở rộng đến 16 để kiểm tra và thấy $h = 6$ thật sự là cực đại bên trong).
- **Mọi bước tiền xử lý nằm trong vòng lặp.** Chuẩn hóa z-score, PCA hay chọn đặc trưng phải được *học lại bên trong mỗi phần* chỉ từ $V-1$ phần huấn luyện. Nếu tính trên toàn bộ dữ liệu trước rồi mới chia, thông tin của phần kiểm định "rò" vào bước huấn luyện và điểm bị lạc quan (mục 2.4). Trong scikit-learn, dùng `Pipeline` để đảm bảo điều này.
- **Chọn thước đo phù hợp mục đích.** Độ chính xác, F1, log-loss cho các kết quả khác nhau (ví dụ dưới đây).

Ví dụ trên chữ số viết tay

Ta chạy kiểm định chéo 5 phần trên $1\,257$ ảnh huấn luyện cho bốn mô hình (Hình 29).



Hình 29. Độ chính xác kiểm định chéo 5 phần trên 1 257 ảnh chữ số huấn luyện theo từng siêu tham số. (a) k-NN theo k. (b) Bayes ngây thơ Multinomial theo α (đường gần như phẳng: thay đổi chưa đến 0,5 điểm). (c) Bayes ngây thơ Gauss theo mức làm trơn phương sai. (d) Softmax Regression theo $C = 1/\lambda$.

Mô hình	Siêu tham số	Giá trị tốt nhất	Độ chính xác kiểm định chéo	Độ chính xác kiểm tra
k-NN	$k \in \{1, \dots, 30\}$	$k = 1$ ($k = 7$ ngang bằng)	98,41%	98,33%
Bayes ngây thơ Gauss	$var_smoothing \in [10^{-11}, 10]$	10^{-2}	92,68%	90,74%
Bayes ngây thơ Gauss	mặc định (10^{-9})		84,88%	84,81%
Bayes ngây thơ Multinomial	$\alpha \in [10^{-4}, 10^{2}]$	$\alpha \approx 56$	90,29%	89,81%
Softmax Regression	$C \in [10^{-4}, 10^{3}]$	$C = 0{,}011$	96,82%	97,59%

Các quan sát đáng chú ý:

- **k-NN muốn k nhỏ** trên dữ liệu này (98,4% ở $k = 1$ và $k = 7$, giảm dần về 95,5% ở $k = 30$). Lý do: các ảnh chữ số cùng lớp tạo thành các cụm khá gọn với khoảng 125 mẫu mỗi lớp, nên láng giềng gần nhất rất đáng tin, và k lớn bắt đầu lấy láng giềng từ lớp khác (như quy tắc "k nhỏ hơn kích thước lớp" ở mục 2.4, ở đây dữ liệu còn dày hơn nhiều).
- **Làm trơn phương sai quyết định Bayes ngây thơ Gauss.** Với mặc định (10^{-9}) độ chính xác kiểm định chéo chỉ 84,9%; tăng lên 10^{-2} cho 92,7% (tăng 7,8 điểm). Lý do: tập huấn luyện có 4 điểm ảnh luôn bằng 0 và mỗi lớp có từ 10 đến 16 điểm ảnh có phương sai bằng 0 *trong lớp đó*; mật độ Gauss với phương sai gần 0 cực kỳ nhọn, nên một điểm ảnh khác 0 trên ảnh kiểm tra bị mật độ phạt cực nặng và quyết định bị chi phối bởi những điểm ảnh ít quan trọng. Làm trơn cộng thêm vào *mọi* phương sai một sàn $\varepsilon \cdot \max_j \mathrm{Var}(x_j)$ (ở đây $\max_j \mathrm{Var} = 42{,}3$, nên

sàn là 0,42 khi $\varepsilon = 10^{-2}$) và chữa vấn đề đó. Đây là phiên bản liên tục của làm trơn Laplace.

- **Bayes ngây thơ Multinomial gần như không nhạy với** α (đường ở Hình 29b gần như phẳng, chênh lệch chưa tới 0,5 điểm). Với dữ liệu này mỗi điểm ảnh (như "số lần xuất hiện") có số đếm lớn trong mọi lớp, nên ít khi có số đếm 0 và làm trơn ít quan trọng; chênh lệch giữa $\alpha = 1$ và $\alpha = 56$ nằm trong dao động ngẫu nhiên, nên lựa chọn $\alpha \approx 56$ ở bảng chỉ là giá trị cao nhất tình cờ, không nên đọc quá nhiều.
- **Softmax** có cực đại rõ ràng ở $C \approx 0{,}01$: C quá nhỏ (điều chuẩn quá mạnh, λ lớn) làm thiếu khớp (91,7% ở $C = 10^{-4}$), C quá lớn làm quá khớp nhẹ (94,6% ở $C = 10^{3}$).

Thước đo quyết định lựa chọn

Cùng một mô hình k-NN, hai tiêu chí cho hai lựa chọn khác nhau. Bảng dưới là **log-loss kiểm định chéo** (đã làm trơn bằng α) theo (k, α); hàng cuối ($\alpha = 0$) dùng sàn 10^{-15}:

k	$\alpha = 0$	$\alpha = 0{,}001$	$\alpha = 0{,}01$	$\alpha = 0{,}1$	$\alpha = 1$	**Độ chính xác kiểm định chéo**
1	0,549	0,119	0,159	0,636	1,716	**98,41%**
3	0,245	**0,0795**	0,091	0,301	1,205	98,25%
5	0,205	0,083	0,088	0,220	0,956	98,17%
12	0,189	0,110	0,109	0,165	0,604	97,53%
30	0,184	0,185	0,187	0,213	0,429	95,54%

Chọn theo **độ chính xác**, ta được $k = 1$ (98,41%). Chọn theo **log-loss đã làm trơn**, ta được $k = 3$ và $\alpha = 0{,}001$ (log-loss 0,0795). Với $k = 1$, xác suất của k-NN chỉ là 0 hoặc 1, nên log-loss chỉ "biết" đúng hay sai, không phân biệt được mức độ tự tin. Trên tập kiểm tra, lựa chọn theo log-loss ($k = 3$, $\alpha = 0{,}001$) cho độ chính xác 98,52%, log-loss 0,083 và độ tin cậy trung bình 0,983, gần khớp độ chính xác. **Chọn thước đo theo điều bạn cần**: chỉ cần nhãn thì độ chính xác hoặc F1; cần xác suất đáng tin thì log-loss.

Quy tắc một sai số chuẩn

Khi đường kiểm định chéo gần phẳng quanh cực đại (như Hình 29a ở k từ 1 đến 5, hay Hình 29b), chênh lệch giữa các giá trị siêu tham số nằm trong dao động ngẫu nhiên. **Quy tắc một sai số chuẩn** khuyên chọn mô hình *đơn giản nhất* có điểm trong khoảng một sai số chuẩn của điểm cao nhất:

$$\mathrm{SE}(\boldsymbol{\theta}) = \frac{\mathrm{sd}_v\big(\text{Điểm}_v(\boldsymbol{\theta})\big)}{\sqrt{V}}, \qquad \text{chọn } \boldsymbol{\theta} \text{ đơn giản nhất thỏa } \mathrm{CV}(\boldsymbol{\theta}) \geq \mathrm{CV}\big(\widehat{\boldsymbol{\theta}}\big) - \mathrm{SE}\big(\widehat{\boldsymbol{\theta}}\big),$$

trong đó "đơn giản" nghĩa là k lớn hơn, h lớn hơn, α hoặc λ lớn hơn (mô hình mượt hơn, ít phương sai hơn).

Kiểm định chéo lồng nhau: tránh điểm số lạc quan

Điểm CV$(\hat{\boldsymbol{\theta}})$ là *điểm cao nhất trong nhiều điểm*, nên nó có xu hướng lạc quan: ta đã chọn tham số *vì* nó cho điểm cao trên chính các phần này. Để ước lượng khả năng tổng quát hóa của **cả quy trình** (tức là cả bước chọn siêu tham số), dùng **kiểm định chéo lồng nhau**: vòng ngoài chia $V_{\text{ngoài}}$ phần; với mỗi phần ngoài, chạy toàn bộ việc chọn tham số bằng kiểm định chéo *bên trong* phần huấn luyện của nó, huấn luyện lại, rồi đo trên phần ngoài (chưa dùng cho chọn tham số). Điểm cuối là trung bình các điểm ngoài. Trên chữ số:

Mô hình	CV **tốt nhất (chọn rồi đo trên cùng các phần)**	Kiểm định chéo lồng nhau
k-NN	98,41%	98,49%
Softmax Regression	96,82%	96,58%
Bayes ngây thơ Multinomial	90,29%	89,58%

Với Softmax và Bayes ngây thơ Multinomial, điểm lồng nhau thấp hơn một chút (0,2 và 0,7 điểm), đúng hướng lạc quan; với k-NN nó cao hơn 0,1 điểm do dao động ngẫu nhiên. Độ lạc quan ở đây nhỏ vì mỗi lưới chỉ có *một* siêu tham số và đường cong khá phẳng; nó trở nên đáng kể khi lưới lớn (nhiều siêu tham số), dữ liệu ít, hoặc khi so sánh nhiều mô hình rồi chọn mô hình thắng.

7.5. So sánh k-NN, Bayes ngây thơ với Logistic, Softmax trên dữ liệu chữ số viết tay

Với các siêu tham số đã chọn ở mục 7.4 (bằng kiểm định chéo trên tập huấn luyện), ta huấn luyện lại trên 1 257 ảnh và đánh giá một lần trên 540 ảnh kiểm tra. “Logistic” ở đây là Logistic Regression nhị phân áp dụng theo kiểu *một-so-với-còn-lại* (mỗi chữ số một bộ phân loại riêng, chọn lớp có điểm cao nhất), còn Softmax là mô hình đa lớp ở các bài trước. Hai cột cuối là thời gian máy chạy (mili-giây; phụ thuộc máy nên chỉ có ý nghĩa tương đối), 540 điểm dự đoán.

Mô hình (siêu tham số)	Độ chính xác	Macro-F1	Log-loss	Độ tin cậy trung bình	Huấn luyện	Dự đoán
k-NN ($k = 1$)	98,33%	0,983	$0,576^{*}$	1,000	0,6	4,5
k-NN ($k = 3, \alpha = 0{,}001$)	98,52%		0,083	0,983		
Bayes ngây thơ Gauss (mặc định)	84,81%	0,848	3,216	0,987	1,2	1,4
Bayes ngây thơ Gauss (làm trơn 10^{-2})	90,74%	0,908	1,213	0,988	1,4	3,5
Bayes ngây thơ Multinomial ($\alpha \approx 56$)	89,81%	0,899	1,820	0,994	1,1	0,4

Mô hình (siêu tham số)	Độ chính xác	Macro-F1	Log-loss	Độ tin cậy trung bình	Huấn luyện	Dự đoán
Bayes ngây thơ Bernoulli (nhị phân hóa ở 8)	89,63%	0,897	0,553	0,954	1,9	0,8
Logistic một-so-với-còn-lại ($C = 0{,}0215$)	96,85%	0,969	0,147	0,902	90,3	1,0
Softmax Regression ($C = 0{,}011$)	97,59%	0,976	0,114	0,933	145,1	0,5
Bayes (KDE) trên 20 thành phần PCA ($h = 6$)	98,15%	0,981	0,048			60,7

* Log-loss của k-NN $k = 1$ là sàn 10^{-15} tại 9 lỗi, nên chỉ có giá trị tương đối (mục 7.2).

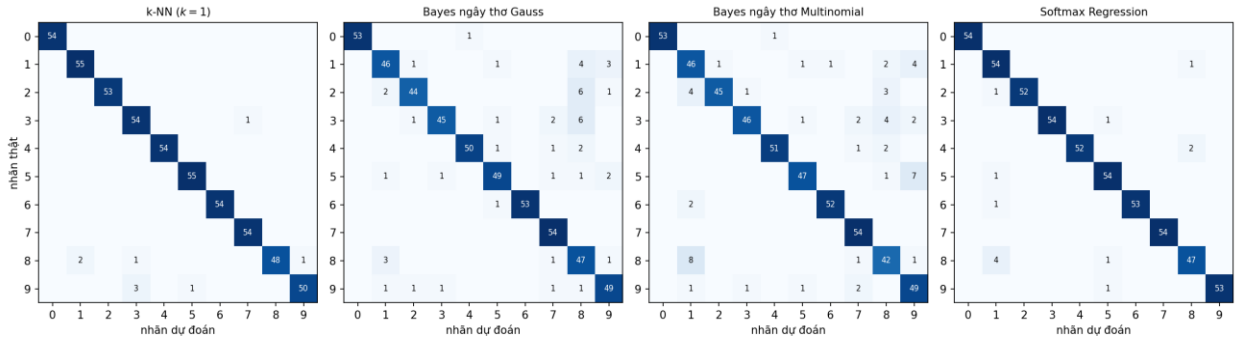
Tập kiểm tra có 540 mẫu, nên sai số chuẩn của độ chính xác quanh 98% là $\sqrt{0{,}98 \times 0{,}02/540} \approx 0{,}6$ điểm: sự chênh giữa k-NN (98,3% đến 98,5%), KDE trên PCA (98,2%) và Softmax (97,6%) tương ứng chỉ khoảng 3 đến 5 ảnh, **chưa đủ để xếp hạng**. Các chênh lệch lớn hơn nhiều mới có ý nghĩa: nhóm đầu (k-NN, KDE, Softmax, Logistic, $\geq$ 96,8%) hơn hẳn nhóm Bayes ngây thơ (85% đến 91%).

Vì sao có thứ hạng đó

- **k-NN (và Bayes với KDE nhiều chiều) đứng đầu.** Các ảnh chữ số nằm trong $\mathbb{R}^{64}$ nhưng gần một không gian có số chiều hiệu dụng thấp (Hình 27b: hai thành phần đầu đã tách nhiều lớp; và 20 thành phần PCA đã đủ để KDE đạt 98,1%), nên lời nguyền chiều nhẹ (mục 6.3). Dữ liệu cũng dày (≈ 125 mẫu mỗi lớp) nên láng giềng gần nhất đáng tin.
- **Softmax và Logistic đứng ngay sau** (97,6% và 96,9%): ranh giới tuyến tính đã gần đủ để tách các chữ số, và chúng học trọng số theo từng điểm ảnh. Softmax hơn Logistic một-so-với-còn-lại 0,7 điểm (khoảng 4 ảnh), ủng hộ việc dùng mô hình đa lớp thống nhất, và có log-loss thấp hơn (0,114 so với 0,147).
- **Bayes ngây thơ ở cuối** (89,6% đến 90,7%): đúng như phân tích ở mục 5.6, giả định độc lập có điều kiện *sai* nặng với ảnh, vì các điểm ảnh lân cận rất tương quan (một nét bút chiếm nhiều điểm ảnh liền kề), nên mô hình đếm cùng một bằng chứng nhiều lần. Dấu hiệu rõ nhất là độ tin cậy: Bayes ngây thơ Gauss tin 0,988 nhưng đúng chỉ 0,907 (quá tự tin 8 điểm), Multinomial tin 0,994 nhưng đúng 0,898 (quá tự tin 10 điểm) và log-loss của chúng (1,2 đến 1,8) tệ hơn Softmax khoảng 10 đến 16 lần. Bernoulli, vốn "chỉ nhìn có hay không", cho độ tự tin vừa phải hơn (0,954, log-loss 0,553). Ngược lại, k-NN đã làm trơn có độ tin cậy 0,983 gần khớp độ chính xác 0,985, còn Softmax *chưa đủ tự tin* (0,933 so với 0,976) vì điều chuẩn mạnh ($C = 0{,}011$ chọn theo độ chính xác) làm các xác suất bị kéo về phía đồng đều.

- **Gauss mặc định thấp nhất** (84,8%, log-loss 3,2) vì làm trơn phương sai quá nhỏ (mục 7.4): việc điều chỉnh một siêu tham số đã tăng 6 điểm. Bài học: khi so sánh mô hình, **phải điều chỉnh siêu tham số của từng mô hình một cách công bằng**; so k-NN đã chỉnh với Bayes ngây thơ chưa chỉnh sẽ cho kết luận sai.

Ma trận nhầm lẫn: ai nhầm cái gì



Hình 28. Ma trận nhầm lẫn trên 540 *ảnh kiểm tra (hàng: nhãn thật, cột: nhãn dự đoán): k-NN* ($k = 1$)*, Bayes ngây thơ Gauss, Bayes ngây thơ Multinomial và Softmax Regression. Các ô ngoài đường chéo là lỗi.*

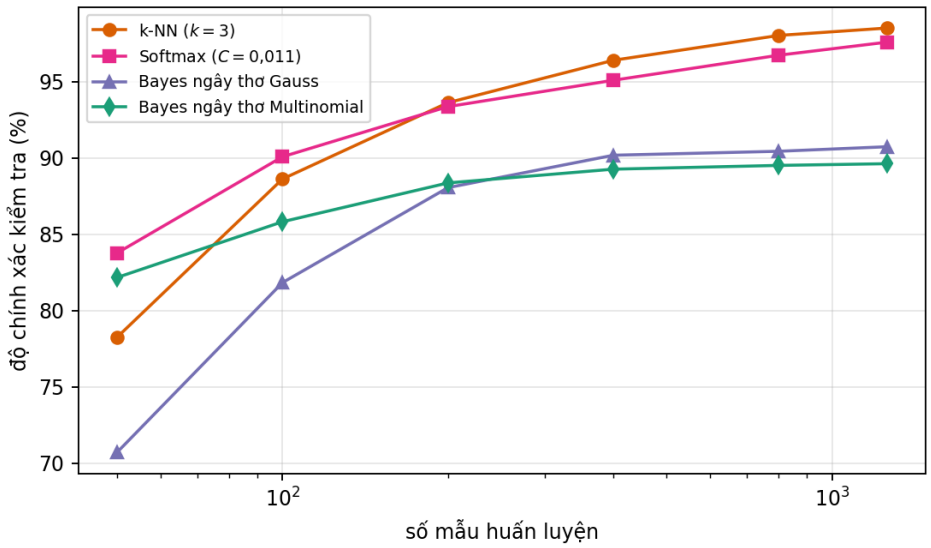
Các cặp nhầm thường gặp nhất (nhãn thật → nhãn dự đoán, số ảnh):

Mô hình	Các lỗi lớn nhất
k-NN ($k = 1$)	9 → 3 (3 ảnh), 8 → 1 (2 ảnh), còn lại mỗi cặp 1 ảnh
Bayes ngây thơ Gauss	3 → 8 (6), 2 → 8 (6), 1 → 8 (4)
Bayes ngây thơ Multinomial	8 → 1 (8), 5 → 9 (7), 3 → 8 (4), 2 → 1 (4)
Softmax Regression	8 → 1 (4), 4 → 8 (2)

Chữ số 8 là kẻ gây rắc rối nhất ở mọi mô hình, đúng với trực giác: nó "chứa" nét của 1, 3 và 9. Với k-NN $k = 1$ theo từng lớp, recall thấp nhất là chữ số 8 (0,923) và 9 (0,926), precision thấp nhất là chữ số 3 (0,931) vì chữ số 3 hấp thụ các ảnh 9 và 8 bị đoán nhầm; các chữ số 0, 2, 4, 6, 8 (về precision) đạt 1,000. Bayes ngây thơ Gauss lại nhầm *ngược chiều* (3, 2, 1 đều bị đoán thành 8): có lẽ vì chữ số 8 có nhiều điểm ảnh "bật" ở nhiều vị trí nên mật độ Gauss của lớp 8 có phương sai lớn và "hút" nhiều ảnh (đây là giả thuyết, tôi không kiểm chứng riêng), một lỗi hệ thống không có ở k-NN.

Đường cong học trên chữ số

Hình 30 vẽ độ chính xác theo số mẫu huấn luyện (trung bình 15 lần lấy mẫu con, siêu tham số giữ cố định).



Hình 30. Độ chính xác kiểm tra trên chữ số theo số mẫu huấn luyện (trung bình 15 lần lấy mẫu con; siêu tham số cố định như ở mục 7.4, riêng k-NN dùng $k = 3$ và Multinomial dùng $\alpha = 1$).

Số mẫu huấn luyện	50	100	200	400	800	1 257
k-NN ($k = 3$)	78,3%	88,6%	93,6%	96,4%	98,0%	98,5%
Softmax	83,8%	90,1%	93,4%	95,1%	96,7%	97,6%
Bayes ngây thơ Multinomial	82,2%	85,8%	88,4%	89,3%	89,5%	89,6%
Bayes ngây thơ Gauss	70,7%	81,8%	88,1%	90,2%	90,4%	90,7%

Đường cong tái hiện đúng câu chuyện của mục 5.6 và 6.4: ở số mẫu rất ít (50 ảnh, tức 5 ảnh mỗi chữ số), mô hình tham số (Softmax, Multinomial) hơn k-NN khoảng 4 đến 5,5 điểm vì có ít con số cần ước lượng; k-NN đuổi kịp Softmax ở khoảng 200 ảnh và vượt dần từ 400 ảnh trở đi (96,4% so với 95,1%) vì càng có nhiều dữ liệu thì "hàng xóm gần" càng gần; cả hai loại Bayes ngây thơ **chạm trần ở khoảng** 90% (từ 400 ảnh) và gần như không cải thiện khi tăng dữ liệu, dấu hiệu rõ ràng của thiên lệch do giả định sai, chứ không phải thiếu dữ liệu.

7.6. Huấn luyện và so sánh với *KNeighborsClassifier*, *KernelDensity*, *GaussianNB*, *MultinomialNB* của scikit-learn

Bản đồ giữa công thức và thư viện

Khái niệm trong bài	**Lớp của scikit-learn**	**Siêu tham số chính**
k-NN phân loại (mục 2)	*KNeighborsClassifier*	*n_neighbors* (k), *weights* (*"uniform"* hoặc *"distance"*), p (bậc Minkowski), *algorithm*
Ước lượng mật độ nhân (mục 4.3)	*KernelDensity*	*bandwidth* (h), *kernel* (Gauss, Epanechnikov, …)

Khái niệm trong bài	Lớp của scikit-learn	Siêu tham số chính
Bayes ngây thơ Gauss (mục 5.3)	`GaussianNB`	`var_smoothing` (ε ở mục 7.4)
Bayes ngây thơ Multinomial	`MultinomialNB`	`alpha` (α ở mục 5.4)
Bayes ngây thơ Bernoulli	`BernoulliNB`	`alpha`, `binarize`
Softmax Regression	`LogisticRegression`	C ($1/\lambda$)
Chọn siêu tham số (mục 7.4)	`GridSearchCV`, `StratifiedKFold`, `Pipeline`	lưới giá trị, số phần V

Có hai lưu ý hay gây nhầm lẫn: `LogisticRegression` dùng $C = 1/\lambda$ nên C *lớn nghĩa là điều chuẩn yếu* (mục 6.1); và `KernelDensity.score(X)` trả về **tổng** log-hợp lý trên **X**, còn `score_samples(X)` trả về log-mật độ *từng mẫu* ($\ln \hat{f}$). Khi chọn băng thông bằng `GridSearchCV`, hàm `score` mặc định là tổng log-hợp lý, tức hợp lý cực đại có kiểm định chéo ở mục 4.4.

Chọn siêu tham số: bốn mô hình, một vòng lặp

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, GridSearchCV, Stratifie
dKFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB, MultinomialNB
from sklearn.linear_model import LogisticRegression

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, st
ratify=y)
cv = StratifiedKFold(5, shuffle=True, random_state=0)

cac_mo_hinh = {
    "knn":     (KNeighborsClassifier(),           {"n_neighbors": list(range
(1, 31))}),
    "gnb":     (GaussianNB(),                     {"var_smoothing": np.logsp
ace(-11, 1, 25)}),
    "mnb":     (MultinomialNB(),                  {"alpha": np.logspace(-4,
2, 25)}),
    "softmax": (LogisticRegression(max_iter=3000), {"C": np.logspace(-4, 3, 2
5)}),
}
for ten, (mo_hinh, luoi) in cac_mo_hinh.items():
    gs = GridSearchCV(mo_hinh, luoi, cv=cv).fit(Xtr, ytr)
    tham_so = {k: float(np.round(v, 4)) for k, v in gs.best_params_.items()}
```

```
    print(ten, tham_so, round(gs.best_score_, 4), round(gs.score(Xte, yte), 4
))
```

```
knn {'n_neighbors': 1.0} 0.9841 0.9833
gnb {'var_smoothing': 0.01} 0.9268 0.9074
mnb {'alpha': 56.2341} 0.9029 0.8981
softmax {'C': 0.011} 0.9682 0.9759
```

Bốn dòng kết quả trùng khớp với bảng ở mục 7.4 (cột "kiểm định chéo" và "kiểm tra"). Nhờ *random_state* cố định, kết quả lặp lại được.

Mô hình sinh với *KernelDensity*: bộ phân loại Bayes bằng KDE

KernelDensity ước lượng *một* mật độ; để phân loại, ta cần một mật độ cho mỗi lớp (mục 5.3). Ta bọc nó thành một bộ ước lượng tương thích scikit-learn, để dùng được với *Pipeline* và *GridSearchCV*:

```
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.neighbors import KernelDensity
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline

class KDEClassifier(BaseEstimator, ClassifierMixin):
    def __init__(self, bandwidth=1.0):
        self.bandwidth = bandwidth
    def fit(self, X, y):
        self.classes_ = np.unique(y)
        self.pi_ = np.array([(y == c).mean() for c in self.classes_])
# pi_c
        self.kdes_ = [KernelDensity(bandwidth=self.bandwidth).fit(X[y == c])
# f_c(x)
                      for c in self.classes_]
        return self
    def _diem_so(self, X):
# s_c = ln pi_c + ln f_c(x)
        return np.log(self.pi_)[None, :] + np.column_stack(
            [k.score_samples(X) for k in self.kdes_])
    def predict(self, X):
        return self.classes_[np.argmax(self._diem_so(X), axis=1)]
    def predict_proba(self, X):
        S = self._diem_so(X); S = S - S.max(axis=1, keepdims=True)
# ổn định số
        P = np.exp(S); return P / P.sum(axis=1, keepdims=True)

pipe = Pipeline([("pca", PCA(20, random_state=0)), ("kde", KDEClassifier())])
gs = GridSearchCV(pipe, {"kde__bandwidth": [2, 3, 4, 6, 8, 10, 12, 16]}, cv=c
v).fit(Xtr, ytr)
print(gs.best_params_, round(gs.best_score_, 4), round(gs.score(Xte, yte), 4)
)
```

```
{'kde__bandwidth': 6} 0.9849 0.9815
```

Ở đây hàm *_diem_so* chính là $s_c(\mathbf{x}) = \ln\pi_c + \ln\hat{f}_c(\mathbf{x})$ ở mục 5.5, và *predict_proba* là softmax của nó với thủ thuật trừ giá trị lớn nhất. Ta *không* chạy KDE trên toàn bộ 64 chiều: với 64 chiều và chỉ khoảng 125 ảnh mỗi lớp, lời nguyền chiều (mục 4.6) làm mật độ không đáng tin; ta chiếu xuống 20 thành phần PCA trước (PCA nằm trong *Pipeline*, nên được học lại trong từng phần kiểm định chéo, tránh rò thông tin). Với 5 thành phần, độ chính xác chỉ đạt 93,5% (mất thông tin); 10 thành phần cho 97,6%; 20 thành phần cho 98,1%. Đây là cách hiện thực hóa ý tưởng ở mục 6.3 và 7.5: dữ liệu có chiều hiệu dụng thấp, nên giảm chiều rồi dùng ước lượng mật độ phi tham số.

Đối chiếu với cài đặt từ đầu

Các cài đặt từ đầu ở các mục trước (hàm *knn_xac_suat* ở mục 2.7, *kde* ở mục 4.7, các lớp *GaussNB* và *MultiNB* ở mục 5.7) được đối chiếu với scikit-learn trên bộ chữ số (540 ảnh kiểm tra):

Cài đặt từ đầu	So với scikit-learn	Nhãn dự đoán trùng nhau	Sai khác lớn nhất
k-NN, $k = 1$	*KNeighborsClassifier(1)*	100%	
k-NN, $k = 5$	*KNeighborsClassifier(5)*	100%	
GaussNB (làm trơn 10^{-9})	*GaussianNB()*	100%	$2{,}4 \times 10^{-11}$ trên xác suất
MultiNB ($\alpha = 1$)	*MultinomialNB(alpha=1.0)*	100%	$1{,}6 \times 10^{-13}$ trên xác suất
kde (nhân Gauss, $h = 2$)	*KernelDensity(bandwidth=2.0)*		$5{,}1 \times 10^{-13}$ trên $\ln\hat{f}$

Mọi cài đặt trùng khớp đến sai số làm tròn. Điều này vừa kiểm chứng các công thức ma trận của bài (ví dụ $\widehat{\mathbf{P}} = \frac{1}{k}\mathbf{NY}$ và $\mathbf{S} = \mathbf{1}\ln\widehat{\boldsymbol{\pi}}^{\top} + \mathbf{X}_0\left(\ln\widehat{\boldsymbol{\Theta}}\right)^{\top}$), vừa cho thấy thư viện không có "phép màu" nào ngoài các công thức đó cộng với kỹ thuật tăng tốc (cấu trúc cây cho k-NN và KDE, ổn định số). Khác với mục 2.7 (nơi vài điểm khác nhau do hòa khoảng cách), ở bộ chữ số này nhãn trùng 100% kể cả với $k = 1$: trên 540 ảnh kiểm tra không xuất hiện sai khác nào do cách phá hòa.

Quy trình đề xuất

Gộp các bài học của cả mục 7 thành một quy trình:

1. Chia phân tầng thành huấn luyện và kiểm tra; *cất tập kiểm tra đi*.
2. Dựng *Pipeline* gồm mọi bước tiền xử lý (chuẩn hóa, PCA) và mô hình.
3. Với mỗi mô hình, chọn siêu tham số bằng *GridSearchCV* (lưới logarit cho thang đo, kiểm tra không ở rìa lưới), bằng thước đo phù hợp mục đích (độ chính xác, F1, log-loss).

4. Dùng kiểm định chéo lồng nhau nếu cần ước lượng trung thực kết quả của cả quy trình, đặc biệt khi lưới lớn hoặc dữ liệu ít.

5. Huấn luyện lại trên toàn bộ tập huấn luyện với $\widehat{\boldsymbol{\theta}}$, đo **một lần** trên tập kiểm tra, báo cáo ma trận nhầm lẫn, F1 theo lớp, log-loss và độ tin cậy trung bình, kèm sai số chuẩn.

6. So sánh các mô hình *sau khi* đã chỉnh từng cái công bằng; chỉ kết luận khi chênh lệch lớn hơn dao động ngẫu nhiên.

Mối liên hệ với các mục trước

Mục 7 thu lại các sợi chỉ đã được kéo từ các mục trước:

- Từ **mục 2**: $\widehat{\mathbf{P}} = \frac{1}{k}\mathbf{NY}$ có thể đem vào log-loss, nhưng cần làm trơn $\tilde{p}_c = \frac{v_c+\alpha}{k+K\alpha}$ vì xác suất 0 (mục 2.2); và chuẩn hóa phải nằm trong vòng kiểm định chéo (mục 2.4).

- Từ **mục 3 và 4**: các thước đo hồi quy phản ánh phân rã $\sigma^2 +$ thiên lệch$^2 +$ phương sai (mục 6.1), bootstrap mô phỏng phương sai của ước lượng, và log-hợp lý đánh giá mật độ theo độ phân kỳ KL (mục 4.4).

- Từ **mục 5**: làm trơn Laplace ở mục 5.4 và *var_smoothing* ở mục 7.4 là cùng một ý tưởng (một sàn nhỏ chống số 0); độ tin cậy quá cao của Bayes ngây thơ (mục 5.6) hiện ra trên chữ số ở mục 7.5.

- Từ **mục 6**: lời nguyền chiều (mục 6.3) giải thích vì sao KDE trên 64 chiều hỏng nhưng trên 20 thành phần PCA lại tốt; kết quả "phi tham số thắng khi dữ liệu nhiều" (mục 6.4) hiện ở Hình 30; chi phí (mục 6.5) hiện ở cột thời gian của bảng ở mục 7.5.

Mục 8 sẽ tổng kết: ưu và nhược điểm của các mô hình phi tham số, so sánh bốn họ mô hình (tham số, kernel, mạng nơ-ron, phi tham số), các hướng mở rộng (cây quyết định, rừng ngẫu nhiên, mô hình hỗn hợp Gauss) và bài tập thực hành.

Tóm tắt mục 7

- **Trực quan hóa**: vẽ vùng quyết định *và* bản đồ xác suất lớn nhất (mô hình tự tin ở xa dữ liệu là chuyện bình thường); vẽ đường hồi quy kèm dải bootstrap (chỉ đo phương sai, không đo thiên lệch) và phần dư; vẽ KDE (một chiều kèm dải, nhiều chiều qua phép chiếu).

- **Phân loại**: ma trận nhầm lẫn $\mathbf{M} = \mathbf{Y}^\top\widehat{\mathbf{Y}}$; $\mathrm{P}_c = M_{cc}/\sum_a M_{ac}$, $\mathrm{R}_c = M_{cc}/\sum_b M_{cb}$, $\mathrm{F1}_c = \frac{2\mathrm{P}_c\mathrm{R}_c}{\mathrm{P}_c+\mathrm{R}_c}$; macro-F1 coi các lớp ngang nhau. Độ chính xác có thể đánh lừa khi

lớp lệch (95% nhưng recall 0). Log-loss phạt nặng việc tự tin mà sai và cần làm trơn $\tilde{p}_c = \frac{v_c+\alpha}{k+K\alpha}$ cho k-NN (với $k = 3$ trên chữ số, từ 0,277 xuống 0,092).

- **Hồi quy và mật độ**: MAE, RMSE, R^2; phải biết mức sàn của nhiễu (RMSE $\geq \sigma$; $R^2_{\max} = 1 - \sigma^2/\mathrm{Var}(y)$); RMSE/MAE $\approx 1{,}25$ cho phần dư Gauss. Mật độ đánh giá bằng log-hợp lý kiểm tra (chỉ với ước lượng đã chuẩn hóa).
- **Chọn siêu tham số**: kiểm định chéo V phần phân tầng, lưới logarit, mọi tiền xử lý nằm trong vòng lặp, tập kiểm tra dùng một lần. Thước đo quyết định lựa chọn (độ chính xác chọn $k = 1$, log-loss đã làm trơn chọn $k = 3$). Kiểm định chéo lồng nhau cho điểm trung thực hơn (ở đây độ lạc quan 0,2 đến 0,7 điểm).
- **Chữ số viết tay** (540 ảnh kiểm tra): k-NN 98,3% đến 98,5%, Bayes (KDE) trên 20 thành phần PCA 98,2%, Softmax 97,6%, Logistic một-so-với-còn-lại 96,9%, Bayes ngây thơ 89,6% đến 90,7% (Gauss mặc định 84,8%). Bayes ngây thơ chạm trần $\approx 90\%$ vì các điểm ảnh tương quan và quá tự tin (0,99 tin nhưng 0,90 đúng); chênh lệch dưới ≈ 1 điểm giữa nhóm đầu chưa đủ để xếp hạng.
- **scikit-learn**: các cài đặt từ đầu trùng khớp đến sai số làm tròn; `KernelDensity.score` là *tổng* log-hợp lý; bộ phân loại Bayes bằng KDE nên dùng sau PCA.

Câu hỏi kiểm tra nhanh

1. Một bộ phát hiện thư rác chạy trên 1 000 thư và cho TP $= 40$, FP $= 10$, FN $= 20$, TN $= 930$. Tính accuracy, precision, recall và F1 của lớp "rác".
2. Một mô hình gán xác suất cho lớp thật ở ba mẫu lần lượt là 0,9, 0,5 và 0,1. Tính log-loss.
3. k-NN với $k = 3$, $K = 3$ lớp cho số phiếu $\mathbf{v} = (2,1,0)$. Tính xác suất đã làm trơn với $\alpha = 0{,}5$.
4. Một mô hình hồi quy có RMSE $= 0{,}31$ trên tập đánh giá có $\mathrm{Var}(y) = 0{,}56$. Ước tính R^2. Mô hình hằng $\hat{y} = \bar{y}$ có R^2 bằng bao nhiêu?
5. Vì sao không nên chọn siêu tham số bằng tập kiểm tra, và kiểm định chéo lồng nhau giải quyết vấn đề gì?
6. Vì sao việc chuẩn hóa z-score phải được thực hiện *bên trong* từng phần của kiểm định chéo?
7. Khi nào nên báo cáo macro-F1 thay vì độ chính xác?

8. Khác biệt giữa `KernelDensity.score` và `KernelDensity.score_samples` là gì? Điều đó ảnh hưởng thế nào khi dùng `GridSearchCV` để chọn băng thông?

Gợi ý đáp án

1. Accuracy $= (40 + 930)/1\,000 = 0{,}97$. Precision $= 40/(40 + 10) = 0{,}80$. Recall $= 40/(40 + 20) = 0{,}667$. F1 $= \frac{2\times 40}{2\times 40+10+20} = \frac{80}{110} = 0{,}727$. Độ chính xác 97% che đi việc mô hình bỏ sót $1/3$ thư rác.

2. $\text{LogLoss} = -\frac{\ln 0{,}9+\ln 0{,}5+\ln 0{,}1}{3} = \frac{0{,}105+0{,}693+2{,}303}{3} = \frac{3{,}101}{3} \approx 1{,}034$. Mẫu có xác suất 0,1 đóng góp phần lớn.

3. $\tilde{p}_c = \frac{v_c+0{,}5}{3+3\times 0{,}5} = \frac{v_c+0{,}5}{4{,}5}$, nên $\tilde{\mathbf{p}} = (2{,}5/4{,}5;\ 1{,}5/4{,}5;\ 0{,}5/4{,}5) = (0{,}556;\ 0{,}333;\ 0{,}111)$, cộng bằng 1. Lớp 3 không còn bị gán xác suất 0.

4. Phần dư có MSE $\approx 0,31^2 = 0{,}0961$, nên $R^2 \approx 1 - 0{,}0961/0{,}56 = 1 - 0{,}172 = 0{,}828$. Mô hình hằng $\hat{y} = \bar{y}$ có $\sum(y - \hat{y})^2 = \sum(y - \bar{y})^2$ nên $R^2 = 0$ (còn chính xác 0 khi $\bar{y}$ tính trên chính tập đánh giá).

5. Nếu chọn siêu tham số bằng tập kiểm tra thì tập đó đã tham gia vào việc huấn luyện (gián tiếp) và con số báo cáo bị lạc quan, không còn là phép thử khách quan. Kiểm định chéo lồng nhau cho vòng ngoài ước lượng điểm của *toàn bộ quy trình* (gồm cả bước chọn siêu tham số) trên dữ liệu chưa dùng để chọn, nên bớt thiên lệch lạc quan của điểm "tốt nhất trong nhiều lựa chọn".

6. Nếu chuẩn hóa trên toàn bộ dữ liệu trước khi chia, trung bình và độ lệch chuẩn đã chứa thông tin của phần kiểm định (rò thông tin), nên điểm kiểm định chéo lạc quan. Chuẩn hóa phải *học* ($\boldsymbol{\mu}$, $\boldsymbol{\sigma}$) chỉ từ $V - 1$ phần huấn luyện rồi *áp dụng* cho phần còn lại, đúng như điểm truy vấn mới ngoài đời thật; trong scikit-learn, đặt nó trong `Pipeline`.

7. Khi các lớp lệch hoặc khi muốn mọi lớp có tầm quan trọng ngang nhau (ví dụ chẩn đoán bệnh hiếm). Độ chính xác bị chi phối bởi lớp lớn; macro-F1 (trung bình đơn giản của F1 từng lớp) phạt việc làm tệ một lớp nhỏ.

8. `score` trả về **tổng** log-hợp lý trên toàn bộ $\mathbf{X}$ ($\sum_i \ln \hat{f}(\mathbf{x}_i)$), còn `score_samples` trả về vector các log-mật độ $\ln \hat{f}(\mathbf{x}_i)$ riêng từng mẫu. `GridSearchCV` dùng `score` nên chọn băng thông có log-hợp lý (tổng) lớn nhất trên các phần kiểm định, đúng hợp lý cực đại có kiểm định chéo ở mục 4.4. Cần chia cho số mẫu nếu muốn báo cáo log-hợp lý trung bình mỗi mẫu.

8. Tổng kết và mở rộng

Hành trình của bài này bắt đầu từ một câu hỏi: *có thể dự đoán mà không chọn trước dạng hàm không?* Mục 1 đặt câu hỏi và đưa ra ý tưởng "nhìn các mẫu lân cận". Các mục 2 và 3 biến ý tưởng đó thành k-NN phân loại và hồi quy (nhân, địa phương); mục 4 chuyển sang ước lượng mật độ; mục 5 dùng mật độ để phân loại bằng Bayes ngây thơ; mục 6 giải thích vì sao tất cả hoạt động (và khi nào thất bại); mục 7 đưa vào thực hành trên chữ số viết tay. Mục 8 khép lại bằng bốn việc: nhìn lại toàn cảnh (ưu điểm và hạn chế), đặt các phương pháp của bài vào bản đồ chung với các họ mô hình khác, giới thiệu ba mở rộng tự nhiên (cây quyết định, rừng ngẫu nhiên và mô hình hỗn hợp Gauss), và cuối cùng là bài tập thực hành.

Ta trả lời các câu hỏi sau:

- Các mô hình phi tham số mạnh ở đâu, yếu ở đâu, và bằng chứng nào trong bài ủng hộ từng kết luận? (mục 8.1)
- Bốn họ mô hình (tham số, kernel, mạng nơ-ron, phi tham số) khác nhau thế nào về dạng hàm, số tham số, chi phí và khi nào dùng cái nào? (mục 8.2)
- Cây quyết định và rừng ngẫu nhiên liên hệ thế nào với k-NN, và mô hình hỗn hợp Gauss nằm ở đâu giữa Gauss đơn và KDE? (mục 8.3)
- Làm thế nào để củng cố những điều đã học bằng bài tập? (mục 8.4)

Quy ước ký hiệu

Ta giữ các quy ước của các mục trước: chữ thường in đậm là vector (vector đặc trưng là vector hàng), chữ hoa in đậm là ma trận, đặc trưng không có cột 1 trừ khi có dấu ngã. Các ký hiệu mới của mục này:

Đối tượng	**Ký hiệu**	**Ý nghĩa**
Hàm nhân (kernel) của phương pháp nhân	$\mathcal{K}(\mathbf{x}, \mathbf{x}')$	Độ tương đồng giữa hai điểm; dùng chữ $\mathcal{K}$ để không trùng với k (số láng giềng) và K (số lớp)
Hệ số của mẫu i trong mô hình nhân	β_i	Trọng số được *học* của hàm nhân đặt tại $\mathbf{x}_i$
Các lá của cây	$\mathcal{R}_1, \dots, \mathcal{R}_L$	Phân hoạch không gian đặc trưng thành L vùng (hình hộp)
Độ tạp Gini của vùng	$G(\mathcal{R})$	$G = 1 - \sum_c \hat{p}_c\,(\mathcal{R})^2$
Số cây trong rừng, hệ số tương quan	B, ρ	Số cây và tương quan trung bình giữa dự đoán của hai cây
Số thành phần của hỗn hợp	J	Mô hình hỗn hợp Gauss có J thành phần

Đối tượng	Ký hiệu	Ý nghĩa
Trọng số, trung bình, hiệp phương sai của thành phần j	π_j, $\boldsymbol{\mu}_j$, $\boldsymbol{\Sigma}_j$	$\sum_j \pi_j = 1$
Ma trận trách nhiệm	$\boldsymbol{\Gamma}$	$n \times J$, γ_{ij} là xác suất mẫu i thuộc thành phần j

8.1. Ưu điểm và hạn chế của các mô hình phi tham số

Một công thức cho tất cả

Trước khi liệt kê, ta nhắc lại điều thống nhất các phương pháp của bài, được nêu lần đầu ở mục 1.3: gần như mọi dự đoán đều là **trung bình có trọng số** của các giá trị đã biết,

$$\widehat{\mathbf{p}}(\mathbf{x}_0) = \sum_{i=1}^{n} s_i(\mathbf{x}_0)\, \mathbf{y}_i, \qquad \sum_{i=1}^{n} s_i(\mathbf{x}_0) = 1,$$

và các phương pháp chỉ khác nhau ở cách đặt trọng số hiệu dụng $s_i(\mathbf{x}_0)$. Bảng dưới gom lại các trọng số đã gặp, cộng thêm hai hàng sẽ được chứng minh ở mục 8.3:

Phương pháp	Trọng số hiệu dụng $s_i(\mathbf{x}_0)$	Mục
k-NN	$\frac{1}{k}\, \mathbb{I}[\, i \in \mathcal{N}_k(\mathbf{x}_0)\,]$	2, 3.1
Nadaraya–Watson	$\frac{\kappa(d_i/h)}{\sum_l \kappa\,(d_l/h)}$	3.2
LOESS	$\tilde{\mathbf{x}}_0(\tilde{\mathbf{X}}^{\top}\mathbf{V}\tilde{\mathbf{X}})^{-1}\tilde{\mathbf{X}}^{\top}\mathbf{V}$ (hàng i), có thể âm	3.3
Cây quyết định	$\frac{1}{\lvert L(\mathbf{x}_0)\rvert}\, \mathbb{I}[\, \mathbf{x}_i \in L(\mathbf{x}_0)\,]$, với $L(\mathbf{x}_0)$ là lá chứa $\mathbf{x}_0$	8.3
Rừng ngẫu nhiên	Trung bình của các trọng số trên B cây	8.3

Mỗi trọng số trả lời cùng một câu hỏi: *mẫu i được "nghe" bao nhiêu khi dự đoán tại $\mathbf{x}_0$?* Và mục 6.1 cho thấy mọi hệ quả về sai số đều đọc được từ các trọng số này: thiên lệch là $\sum_i s_i\, f(\mathbf{x}_i) - f(\mathbf{x}_0)$, phương sai là $\sigma^2 \,\|\, \mathbf{s} \,\|^2 = \sigma^2 / n_{\text{hiệu dụng}}$ với $n_{\text{hiệu dụng}} = 1/\sum_i s_i^2$.

Ưu điểm, kèm bằng chứng trong bài

Ưu điểm	Bằng chứng trong bài
Không phải chọn trước dạng hàm. Chỉ cần một khái niệm "gần" và giả thiết "gần thì giống".	Ba vòng tròn đồng tâm: Softmax trên đặc trưng gốc chỉ đạt 37,8%, k-NN đạt 99,4% đến 100% (mục 1.1, 8.2).
Huấn luyện gần như không tốn gì: không hàm mất mát, không gradient.	Lưu 10^5 mẫu mất 7,7 ms, so với 218 ms của Softmax (mục 6.5).
Nhất quán với mọi phân phối khi $k \to \infty$, $k/n \to 0$ (hoặc $h \to 0$, $nh^d \to \infty$).	$k \approx \sqrt{n}$ tiến về sai số Bayes 0,1587; 1-NN kẹt ở 0,225 (mục 6.4).

Ưu điểm	Bằng chứng trong bài
Cô lập ảnh hưởng của dữ liệu xấu trong một vùng nhỏ.	Một ngoại lai làm Softmax mất $7{,}2$ điểm nhưng không ảnh hưởng k-NN; $k = 18$ chịu được 30% nhãn sai với mức giảm $3{,}4$ điểm (mục 6.2).
Thêm dữ liệu mới gần như miễn phí.	Thêm 100 mẫu: $0{,}5$ ms (Bayes ngây thơ, cập nhật số đếm) so với 189 ms huấn luyện lại Softmax (mục 6.5).
Dễ giải thích từng dự đoán bằng ví dụ ("bạn giống các sinh viên này") và cho xác suất tự nhiên $\hat{p}_c = k_c/k$.	Mục 1.2, 2.6.
Một bộ máy cho nhiều bài toán: phân loại, hồi quy, mật độ, và cầu nối sang mô hình sinh.	Mục 3 (hồi quy), 4 (mật độ), 4.5 (k-NN là Bayes với mật độ k láng giềng).
Bayes ngây thơ có công thức đóng, chịu đặc trưng nhiễu tốt, rất hợp với dữ liệu ít chiều cao như văn bản.	$72{,}8\%$ với 100 đặc trưng nhiễu so với k-NN $54{,}4\%$ (mục 5.7); học nhanh khi mẫu ít (mục 5.6, 7.5).

Hạn chế, kèm bằng chứng

Hạn chế	Bằng chứng trong bài
Lời nguyền chiều. Lân cận "nhỏ" không còn nhỏ, khoảng cách tập trung, số mẫu cần tăng như $10^{(d+4)/4}$ khi muốn giảm sai số 10 lần.	Mục 4.6, 6.3: KDE tụt từ $0{,}007$ lên $0{,}648$ thiếu hụt khi d từ 1 đến 10.
Chi phí dự đoán và bộ nhớ $O(nd)$, không phụ thuộc vào độ phức tạp của bài toán.	Mục 6.5: chậm hơn Softmax cỡ $1\,000$ lần ở $n = 10^5$. KD-tree vô dụng khi d lớn.
Nhạy với thang đo và đặc trưng nhiễu, vì mọi đặc trưng đóng góp như nhau vào khoảng cách.	Mục 2.5: 50 đặc trưng nhiễu đưa k-NN từ $82{,}2\%$ xuống $54{,}4\%$.
Không ngoại suy được ra ngoài miền dữ liệu.	Mục 3.5: ở $x = 11$ hàm thật là $2{,}24$ nhưng các phương pháp địa phương cho $-0{,}04$ đến $0{,}89$.
Thiên lệch ở biên của các phương pháp trung bình.	Mục 3.4: Nadaraya–Watson lệch $+0{,}35$ và $-0{,}23$ (nhiễu $0{,}3$).
Xác suất thô hoặc không hiệu chỉnh: k-NN chỉ có $k + 1$ mức xác suất; Bayes ngây thơ quá tự tin khi đặc trưng tương quan.	Mục 7.2, 7.5: Bayes ngây thơ tin $0{,}99$ nhưng đúng $0{,}90$ trên chữ số.
Giả định ngây thơ có thể sai nặng, và khi đó Bayes ngây thơ không nhất quán.	Mục 5.6, 6.4: $80{,}0\%$ so với $95{,}0\%$ khi hai đặc trưng tương quan $0{,}9$; kẹt ở $0{,}228$ sai số khi sai số Bayes là $0{,}0427$.

Hạn chế	Bằng chứng trong bài
Cần chọn siêu tham số và đo đạc cẩn thận; kết quả phụ thuộc độ đo khoảng cách.	Mục 7.4: *var_smoothing* làm Bayes ngây thơ Gauss đổi 84,8% → 90,7%.
Không có mô hình toàn cục để đọc: không có "hệ số" hay tỉ lệ odds.	Mục 5.6: Softmax có trọng số **W** đọc được, k-NN thì không.

Ba bài học xuyên suốt

Thứ nhất, mọi mô hình trong bài đều là một cách điều khiển cùng một độ phức tạp. Nút chỉnh là k, h, λ (C), α, bậc đa thức hay số nơ-ron, nhưng tất cả đều đo được bằng bậc tự do hiệu dụng $\mathrm{tr}(\mathbf{S})$, và sai số luôn có hình chữ "U" theo nút chỉnh (Hình 22). Chẩn đoán thiên lệch cao hay phương sai cao (mục 6.1) quyết định phải *tăng* hay *giảm* độ phức tạp.

Thứ hai, không có mô hình nào thắng ở mọi nơi. Trên ba vòng tròn, Softmax tệ hại (37,8%) nhưng cùng Softmax với đặc trưng bậc hai đạt 100%. Trên dữ liệu sinh viên (đám Gauss gần tuyến tính), Softmax (84,4%) nhỉnh hơn k-NN (82,2%). Trên chữ số, k-NN (98,3%) hơn hẳn Bayes ngây thơ (90,7%). *Dạng hàm khớp với dữ liệu thì mô hình tham số rất tốt; dạng hàm không khớp (hoặc không biết) thì phi tham số an toàn hơn, miễn là đủ dữ liệu và số chiều hiệu dụng thấp.*

Thứ ba, đánh giá công bằng quan trọng ngang chọn mô hình. Chọn siêu tham số bằng kiểm định chéo (không bằng tập kiểm tra), đặt mọi tiền xử lý trong vòng lặp, chọn thước đo theo mục đích, và cho *mọi* mô hình được chỉnh công bằng trước khi so sánh (mục 7.4). Chênh lệch dưới một sai số chuẩn (khoảng 0,6 điểm với 540 mẫu) không đủ để xếp hạng.

8.2. So sánh các họ mô hình: tham số, kernel, mạng nơ-ron và phi tham số

Bốn họ, bốn dạng hàm

Ta xếp các mô hình thành bốn họ theo *cách chúng biểu diễn hàm dự đoán*.

Họ tham số tuyến tính (Softmax Regression, Bayes ngây thơ Multinomial): dạng hàm cố định, một ma trận trọng số $\mathbf{W} \in \mathbb{R}^{(d+1)\times K}$,

$$\hat{\mathbf{p}}(\mathbf{x}) = \mathrm{softmax}(\tilde{\mathbf{x}}\mathbf{W}).$$

Họ mạng nơ-ron: vẫn tham số (kiến trúc cố định, số tham số không phụ thuộc n), nhưng *đặc trưng được học*. Với một lớp ẩn gồm m nơ-ron (mục 6.3 của bài Softmax),

$$\hat{\mathbf{p}}(\mathbf{x}) = \mathrm{softmax}\big(\tilde{\mathbf{h}}\,\mathbf{W}_2\big), \qquad \mathbf{h} = \tanh(\tilde{\mathbf{x}}\mathbf{W}_1).$$

Họ phương pháp nhân (kernel): chọn một hàm nhân $\mathcal{K}(\mathbf{x}, \mathbf{x}')$ đo độ tương đồng giữa hai điểm (ví dụ hàm Gauss $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \parallel \mathbf{x} - \mathbf{x}' \parallel^2)$). Dự đoán có dạng một tổ hợp các "gò" nhân đặt tại các mẫu huấn luyện:

$$f(\mathbf{x}) = \sum_{i=1}^{n} \beta_i \, \mathcal{K}(\mathbf{x}_i, \mathbf{x}) + b.$$

Máy vector hỗ trợ (SVM) với nhân Gauss là ví dụ điển hình. Công thức này giống hệt dạng của Nadaraya–Watson ở mục 3.2 (một tổng các hàm nhân đặt tại dữ liệu), nhưng hệ số β_i ở đây được **học bằng tối ưu hóa** (bài toán lồi), thay vì được ấn định bởi khoảng cách. Hệ quả: nhiều β_i bằng 0 (chỉ các "vector hỗ trợ" mới có $\beta_i \neq 0$) nên mô hình *lớn dần theo n nhưng thưa*. Về mặt khái niệm, $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})\boldsymbol{\phi}(\mathbf{x}')^\top$ với $\boldsymbol{\phi}$ là một ánh xạ đặc trưng (có thể vô hạn chiều), tức là một mô hình *tuyến tính trong không gian đặc trưng ẩn*, và tham số γ đóng vai trò nghịch đảo của băng thông ($\gamma \sim 1/h^2$) còn C đóng vai trò $1/\lambda$.

Họ phi tham số (k-NN, hồi quy nhân, KDE, cây, rừng): hàm dự đoán là trung bình có trọng số của chính dữ liệu, $\hat{\mathbf{p}} = \sum_i s_i(\mathbf{x})\mathbf{y}_i$, với các trọng số xác định bởi dữ liệu; kích thước mô hình tăng theo n.

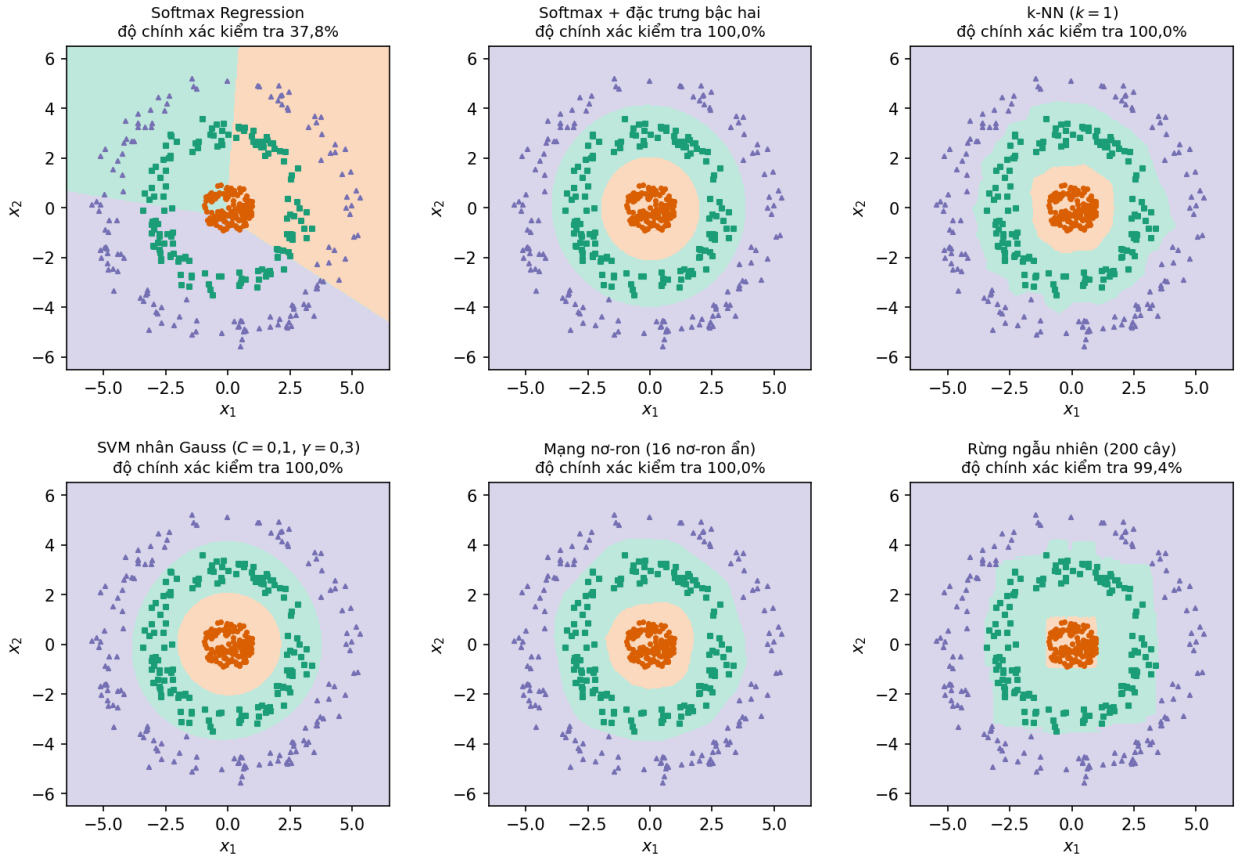
Đặt cạnh nhau

Tiêu chí	Tham số tuyến tính	Mạng nơ-ron	Phương pháp nhân (SVM)	Phi tham số (k-NN, nhân)
Dạng hàm	$\text{softmax}(\tilde{\mathbf{x}}\mathbf{W})$	Hợp của các phép tuyến tính và phi tuyến	$\sum_i \beta_i \, \mathcal{K}(\mathbf{x}_i, \mathbf{x}) + b$	$\sum_i s_i(\mathbf{x})\mathbf{y}_i$
Đặc trưng	Chọn tay	**Học** (lớp ẩn)	Ngầm định qua $\mathcal{K}$	Không (chỉ khoảng cách)
Số đại lượng lưu	$(d+1)K$, cố định	Cố định (theo kiến trúc)	Số vector hỗ trợ ($\leq n$), thưa	nd, tăng theo n
Huấn luyện	Tối ưu **lồi**	Tối ưu **không lồi** (nhiều cực tiểu địa phương)	Tối ưu lồi, cỡ $O(n^2)$ đến $O(n^3)$	Gần như không có
Dự đoán một mẫu	$O(dK)$	$O(dm + mK)$	$O\left(n_{\text{vector hỗ trợ}}\, d\right)$	$O(nd)$
Nút chỉnh	λ (C)	Số nơ-ron, λ, dừng sớm	C, γ	k, h
Tính nhất quán	Chỉ khi dạng hàm đúng	Xấp xỉ vạn năng khi $m \to \infty$	Có (với nhân phù hợp)	Có
Khi dữ liệu ít	Tốt nếu dạng đúng	Khó (nhiều tham số)	Khá	Kém (phương sai cao)
Số chiều lớn	Khá ổn	Tốt (học đặc trưng)	Khá	Kém (lời nguyền chiều)
Diễn giải	Cao ($\mathbf{W}$)	Thấp	Thấp	Từng ví dụ

Thí nghiệm 1: ba vòng tròn đồng tâm

Ta quay lại bài toán minh họa của mục 1.1 (600 điểm, 420 điểm huấn luyện, mỗi họ chọn siêu tham số bằng kiểm định chéo 5 phần):

Mô hình	**Họ**	**Độ chính xác kiểm tra**
Softmax trên (x_1, x_2)	tham số tuyến tính	37,8%
Softmax trên đặc trưng bậc hai	tham số, đặc trưng chọn tay	100,0%
Bayes ngây thơ Gauss	tham số	100,0%
Mạng nơ-ron (16 nơ-ron ẩn)	mạng nơ-ron	100,0%
SVM nhân Gauss ($C = 0{,}1, \gamma = 0{,}3$)	nhân	100,0%
k-NN ($k = 1$, chọn bằng kiểm định chéo)	phi tham số	100,0%
Cây quyết định (độ sâu 6)	phi tham số	99,4%
Rừng ngẫu nhiên (200 cây)	phi tham số	99,4%



Hình 31. Vùng quyết định của sáu mô hình thuộc bốn họ trên ba vòng tròn đồng tâm (đặc trưng đã chuẩn hóa). Softmax tuyến tính thất bại; mọi mô hình có đủ độ linh hoạt (đặc trưng bậc hai,

k-NN, SVM nhân, mạng nơ-ron, rừng ngẫu nhiên) đều bám đúng các vòng, nhưng biên khác nhau về hình dạng: trơn (SVM, mạng nơ-ron, đặc trưng bậc hai), gập ghềnh (k-NN), bậc thang hình chữ nhật (rừng).

Bài học: để giải bài toán này, **bất kỳ cách nào cho phép biên cong đều được**, dù đi bằng đường tham số (thêm đặc trưng bậc hai, hoặc học đặc trưng bằng mạng nơ-ron), nhân hay phi tham số. Khác biệt không nằm ở khả năng giải mà ở *điều ta phải biết trước*: với đặc trưng bậc hai, ta phải *biết* các lớp là các vòng tròn; k-NN và SVM không đòi hỏi điều đó. Bayes ngây thơ Gauss đạt 100% vì khác phương sai theo lớp tạo ra biên bậc hai (mục 5.6). Ghi chú: ở mục 1.1 ta báo 99,4% cho k-NN với $k = 15$; ở đây k chọn bằng kiểm định chéo là 1 và cho 100%, cho thấy cách chọn k có ảnh hưởng.

Thí nghiệm 2: chữ số viết tay

Trên bộ chữ số của mục 7.5 (1 257 ảnh huấn luyện, 540 ảnh kiểm tra), thêm các họ còn thiếu. Siêu tham số chọn bằng kiểm định chéo 5 phần trong lưới nhỏ; thời gian phụ thuộc máy, tính bằng mili-giây (540 điểm dự đoán):

Mô hình	Họ	Kích thước mô hình	Độ chính xác kiểm tra	Huấn luyện	Dự đoán
Softmax ($C = 0{,}01$)	tham số tuyến tính	650 tham số	97,59%	132	0,3
Mạng nơ-ron (64 nơ-ron ẩn, $\alpha = 0{,}01$)	mạng nơ-ron	4 810 tham số	97,04%	485	0,8
SVM nhân Gauss ($C = 1, \gamma = 0{,}001$)	nhân	643 vector hỗ trợ	**99,26%**	45	21,8
k-NN ($k = 1$)	phi tham số	1 257 × 64 số	98,33%	0,5	3,8
Bayes ngây thơ Gauss (làm trơn 10^{-2})	tham số	1 290 số	90,74%	2,5	4,8
Cây quyết định (độ sâu 13, 136 lá)	phi tham số	136 lá	84,26%	19	0,4
Rừng ngẫu nhiên (300 cây)	phi tham số	94 266 nút	97,78%	1 015	44,7

Cần đọc bảng này một cách thận trọng. Với 540 ảnh kiểm tra, sai số chuẩn của độ chính xác quanh 98% là cỡ 0,6 điểm: SVM (99,3%) hơn k-NN (98,3%) khoảng 5 ảnh và hơn Softmax (97,6%) khoảng 9 ảnh, có thể kết luận SVM thuộc nhóm đầu nhưng khoảng cách với k-NN còn mỏng. Có bốn điều đáng rút ra:

- **Mô hình nhân (SVM) tốt nhất ở đây** nhờ kết hợp điều tốt của hai thế giới: nó cũng đặt hàm nhân tại dữ liệu như k-NN (nên biên cong, linh hoạt) nhưng *học* các hệ số để chỉ giữ 643 trong 1 257 mẫu và đặt trọng số tối ưu cho chúng.

- **Mạng nơ-ron (97,0%) không hơn Softmax (97,6%)** ở đây: dữ liệu chỉ có 1 257 mẫu và mạng (4 810 tham số) chỉ được chỉnh qua một siêu tham số, nên không thể kết luận gì về năng lực tiềm tàng của mạng nơ-ron. Lợi thế của chúng thường lộ ra với dữ liệu lớn hơn rất nhiều.
- **Cây đơn lẻ kém (84,3%) nhưng rừng 300 cây tốt (97,8%)**, cho thấy tác dụng của việc lấy trung bình để giảm phương sai (mục 8.3).
- **Chi phí khác nhau theo họ**: Softmax và Bayes ngây thơ dự đoán nhanh nhất; k-NN huấn luyện nhanh nhất; rừng và mạng nơ-ron tốn nhiều nhất khi huấn luyện; SVM dự đoán chậm hơn Softmax cỡ 70 lần vì phải tính hàm nhân với 643 vector hỗ trợ.

Chọn họ nào?

Hoàn cảnh	Hướng ưu tiên	Lý do
Dạng hàm biết trước hoặc đoán được; cần diễn giải; ít dữ liệu	Tham số (Softmax, Bayes ngây thơ)	Ít phương sai; diễn giải được; huấn luyện lồi
Dữ liệu vừa phải (10^2 đến 10^5), chiều hiệu dụng thấp, hàm phức tạp chưa rõ dạng	Nhân (SVM) hoặc phi tham số (k-NN, rừng)	Linh hoạt, ít giả định; tinh chỉnh hai ba siêu tham số
Dữ liệu rất lớn, cấu trúc phức tạp (ảnh, âm thanh, văn bản)	Mạng nơ-ron	Học đặc trưng; chi phí dự đoán không phụ thuộc n
Cần dự đoán rất nhiều, độ trễ thấp	Tham số, mạng nơ-ron	Dự đoán $O(dK)$, bộ nhớ nhỏ
Dữ liệu đến liên tục	k-NN, Bayes ngây thơ	Thêm mẫu gần như miễn phí
Nhiều đặc trưng nhiễu	Bayes ngây thơ, rừng, mô hình có điều chuẩn L1/L2	Không dồn mọi đặc trưng vào khoảng cách
Văn bản túi từ	Bayes ngây thơ Multinomial, Softmax	Huấn luyện một lượt, ma trận thưa
Cần xác suất tin cậy	Softmax (đã hiệu chỉnh), k-NN đã làm trơn	Bayes ngây thơ thường quá tự tin

Quy tắc thực dụng đáng nhớ: **bắt đầu bằng mô hình đơn giản nhất** (Softmax, Bayes ngây thơ) làm mốc so sánh, rồi thêm độ phức tạp (đặc trưng, k-NN, SVM, rừng) nếu mốc đó có thiên lệch cao; **luôn so sánh công bằng** theo quy trình ở mục 7.6.

8.3. Mở rộng: cây quyết định, rừng ngẫu nhiên, mô hình hỗn hợp Gauss

Ba mở rộng sau đều gắn rất chặt với các ý tưởng của bài: cây quyết định là một dạng "láng giềng thích nghi" mà lân cận được học từ nhãn; rừng ngẫu nhiên là cách làm giảm phương sai của cây;

và mô hình hỗn hợp Gauss là chiếc cầu bắc giữa ước lượng Gauss đơn (mục 4.6) và KDE (mục 4.3).

Cây quyết định

Ý tưởng. Thay vì định nghĩa "gần" bằng khoảng cách, cây **chia không gian đặc trưng thành các hình hộp** bằng một chuỗi câu hỏi có/không dạng "đặc trưng $x_j \le t$?". Mỗi hình hộp (lá) $\mathcal{R}_l$ nhận một dự đoán hằng số, nên cây là một hàm hằng từng mảnh:

$$\hat{f}(\mathbf{x}) = \sum_{l=1}^{L} c_l \, \mathbb{I}[\,\mathbf{x} \in \mathcal{R}_l\,], \qquad c_l = \text{trung bình của } y_i \text{ (hoặc lớp phổ biến nhất) trong } \mathcal{R}_l.$$

Đây đúng là dạng trung bình có trọng số $\sum_i s_i(\mathbf{x}) y_i$ với $s_i(\mathbf{x}) = \mathbb{I}[\mathbf{x}_i \in L(\mathbf{x})] \,/\, |L(\mathbf{x})|$ ($L(\mathbf{x})$ là lá chứa $\mathbf{x}$), giống k-NN ở chỗ lấy trung bình của một nhóm láng giềng, **khác ở chỗ nhóm đó do nhãn quyết định**: vùng nhỏ chỗ hàm biến thiên nhanh, vùng lớn chỗ hàm phẳng, và các đặc trưng vô ích không bao giờ được chọn để chia.

Xây cây bằng cách chia tham lam. Với bài toán phân loại, đo độ tạp của một vùng $\mathcal{R}$ bằng **độ tạp Gini**, trong đó $\hat{p}_c(\mathcal{R})$ là tỉ lệ các mẫu lớp c trong vùng:

$$G(\mathcal{R}) = 1 - \sum_{c=1}^{K} \hat{p}_c\,(\mathcal{R})^2.$$

$G = 0$ khi vùng thuần (một lớp), và lớn nhất khi các lớp trộn đều. Tại mỗi nút, thuật toán thử mọi cặp (đặc trưng j, ngưỡng t), chia các mẫu thành $\mathcal{R}_{\text{trái}} = \{x_j \le t\}$ và $\mathcal{R}_{\text{phải}} = \{x_j > t\}$, rồi chọn cặp cực tiểu độ tạp trung bình có trọng số:

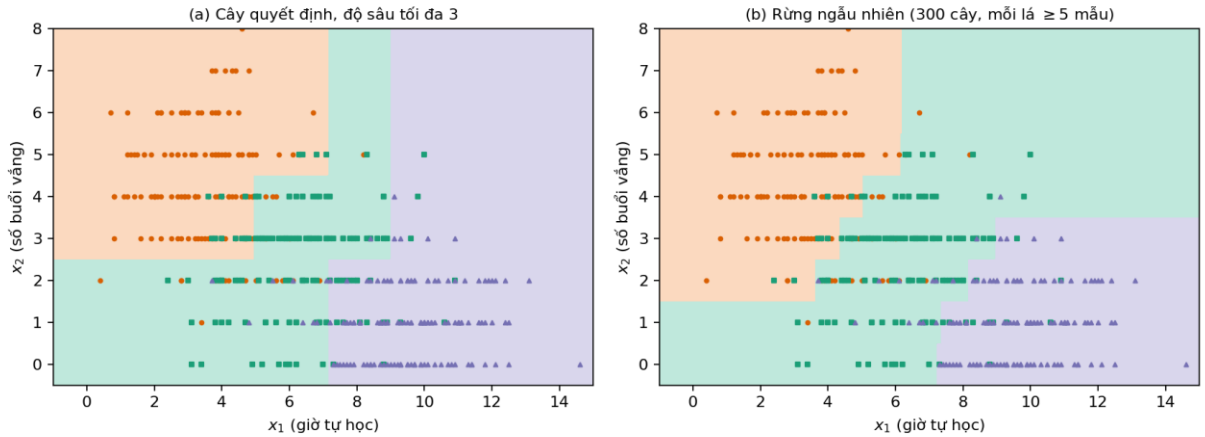
$$(\hat{\jmath}, \hat{t}) = \underset{j,t}{\operatorname{argmin}} \left[\frac{n_{\text{trái}}}{n} G(\mathcal{R}_{\text{trái}}) + \frac{n_{\text{phải}}}{n} G\big(\mathcal{R}_{\text{phải}}\big)\right].$$

(Với hồi quy, thay G bằng tổng bình phương sai khác so với trung bình của vùng.) Quá trình lặp lại đệ quy trên từng vùng mới cho đến khi gặp điều kiện dừng (độ sâu tối đa, số mẫu tối thiểu trong lá). Việc chia là *tham lam*: mỗi bước chọn cách chia tốt nhất ngay lúc đó, không đảm bảo tối ưu toàn cục.

Ví dụ tay: 9 sinh viên. Ba lớp có 3 sinh viên mỗi lớp nên độ tạp gốc là $G = 1 - 3 \times (1/3)^2 = 0{,}667$. Thử chia theo $x_1 \le 5$ (ngưỡng giữa 4 và 6): vùng trái gồm sinh viên 1, 2, 3 (cả ba rớt), thuần, $G = 0$; vùng phải gồm 6 sinh viên (3 đạt, 3 giỏi), $G = 1 - (1/2)^2 - (1/2)^2 = 0{,}5$. Độ tạp có trọng số là $\frac{3}{9} \times 0 + \frac{6}{9} \times 0{,}5 = 0{,}333$, giảm từ 0,667 xuống 0,333. Kiểm tra mọi ngưỡng cho thấy có **ba cách chia hòa nhau** ở 0,333 ($x_1 \le 5$, $x_1 \le 7{,}5$ và $x_2 \le 1{,}5$); một cài đặt cụ thể chọn theo thứ tự duyệt (đặc trưng đầu, ngưỡng nhỏ trước), tức $x_1 \le 5$. Sau đó vùng phải ($x_1 > 5$) tách hoàn toàn bằng một ngưỡng nữa ($x_1 \le 7{,}5$): đạt ở giữa, giỏi ở phải. Cây cuối cùng chỉ có ba lá và hai câu hỏi, và phân loại đúng cả 9 sinh viên.

Kết quả thực nghiệm. Cây rất dễ quá khớp nếu để mọc hết. Trên 420 sinh viên huấn luyện, cây đầy đủ chỉ đạt 76,1% trên tập kiểm tra, còn cây giới hạn độ sâu 3 đạt 81,1% (Hình 32a; mức của k-NN là 82,2%). Với hồi quy trên dữ liệu sin ($n = 100$), RMSE kiểm tra của cây theo độ sâu là

0,427 (4 lá), 0,373 (8 lá), 0,357 (16 lá), và 0,398 khi mọc đầy đủ (100 lá, mỗi lá một mẫu): hình chữ "U" quen thuộc của mục 6.1, với mức tốt nhất (0,324, ứng với lá tối thiểu 3 mẫu) vẫn kém các phương pháp trơn (0,311).



Hình 32. Phân loại 420 *sinh viên huấn luyện. (a) Cây quyết định độ sâu tối đa* 3*: các vùng là hình chữ nhật có cạnh song song trục, độ chính xác kiểm tra* 81,1%*. (b) Rừng ngẫu nhiên* 300 *cây: biên mượt hơn nhiều,* 81,7%.

Điểm mạnh và yếu. Cây dễ giải thích (đọc được như một chuỗi câu hỏi), không cần chuẩn hóa đặc trưng, tự động chọn đặc trưng. Nhưng cây **phương sai cao** (đổi vài mẫu có thể đổi hẳn cấu trúc cây) và biên bậc thang hình chữ nhật không tự nhiên. Chính điểm yếu thứ hai dẫn đến rừng ngẫu nhiên.

Rừng ngẫu nhiên

Ý tưởng: lấy trung bình để giảm phương sai. Một cây có phương sai cao nhưng thiên lệch thấp (khi mọc sâu). Ta trồng B cây trên B tập huấn luyện khác nhau (rút có hoàn lại từ dữ liệu, gọi là **bootstrap**) và lấy trung bình dự đoán (hồi quy) hoặc trung bình xác suất (phân loại). Nếu mỗi cây có phương sai σ^2 và hai cây bất kỳ có tương quan trung bình ρ, thì phương sai của trung bình là

$$\operatorname{Var}\left[\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(\mathbf{x})\right] = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2.$$

Số hạng thứ hai biến mất khi B lớn, nhưng số hạng đầu $\rho\sigma^2$ không giảm: **muốn rừng tốt, các cây phải ít tương quan**. Vì thế, tại mỗi nút, mỗi cây chỉ được xét một tập con ngẫu nhiên các đặc trưng (thường $\sqrt{d}$ trong d đặc trưng) để chia, làm các cây khác nhau hơn, giảm ρ. Thiên lệch của trung bình xấp xỉ thiên lệch của một cây, nên rừng giảm *phương sai* mà gần như không tăng *thiên lệch*.

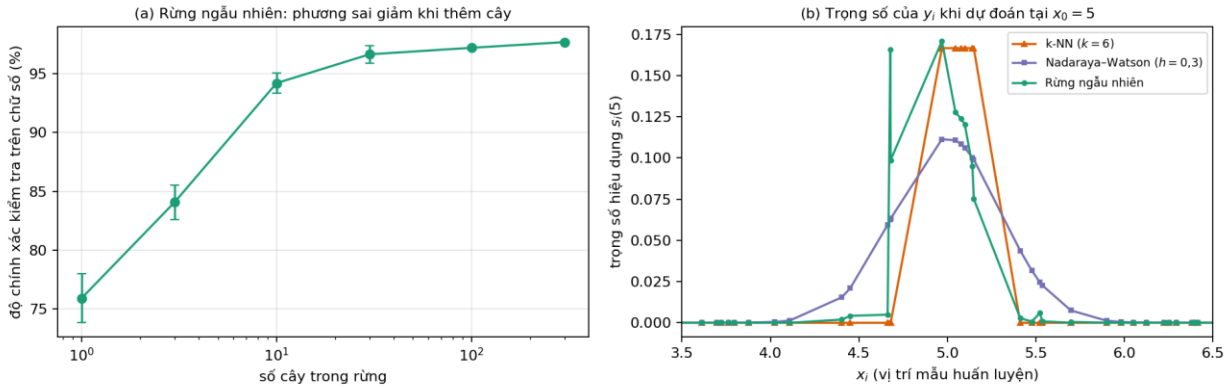
Thực nghiệm trên chữ số (Hình 33a): một "rừng" chỉ gồm 1 cây (bootstrap, đặc trưng ngẫu nhiên) đạt 75,9%; 3 cây 84,1%; 10 cây 94,2%; 30 cây 96,6%; 100 cây 97,2%; 300 cây 97,7% (trung bình 5 lần chạy). Độ chính xác tăng mạnh rồi bão hòa, đúng như công thức: khi B lớn,

phương sai còn lại là $\rho\sigma^2$. Cùng bộ dữ liệu, một cây mọc đầy đủ trên toàn bộ dữ liệu chỉ đạt 84,3%.

Rừng cũng là một "láng giềng thích nghi". Vì mỗi cây là một trung bình có trọng số của y_i, trung bình của chúng cũng vậy. Với rừng huấn luyện bằng bootstrap, gọi c_{ib} là số lần mẫu i xuất hiện trong mẫu bootstrap của cây b và $L_b(\mathbf{x})$ là lá chứa $\mathbf{x}$ trong cây đó, thì

$$\hat{f}(\mathbf{x}) = \sum_{i=1}^{n} s_i(\mathbf{x})\, y_i, \qquad s_i(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} \frac{c_{ib}\, \mathbb{I}[\mathbf{x}_i \in L_b(\mathbf{x})]}{\sum_l c_{lb}\, \mathbb{I}[\mathbf{x}_l \in L_b(\mathbf{x})]}, \qquad \sum_i s_i(\mathbf{x}) = 1.$$

Ta đã kiểm tra công thức này trên dữ liệu sin với rừng 300 cây: tại $x = 5$, $\sum_i s_i\, y_i = 0{,}47887$ và `RandomForestRegressor.predict` cho 0,47887 (trùng đến 10^{-15}); các trọng số cộng bằng 1. Hình 33b so sánh trọng số của ba phương pháp tại $x_0 = 5$: k-NN ($k = 6$) cho $1/6$ cho đúng 6 mẫu gần nhất; Nadaraya–Watson cho trọng số hình chuông trơn; rừng cho trọng số *rời rạc, không đều và không đối xứng* (mẫu nào hay rơi cùng lá với x_0 qua nhiều cây thì được nghe nhiều). Với rừng có 16 mẫu có trọng số khác 0 tại $x_0 = 5$, số mẫu hiệu dụng $n_{\text{hiệu dụng}} = 1/\sum_i s_i^2$ là 7,8 (so với 6 của k-NN và 11,9 của Nadaraya–Watson), tức phương sai của dự đoán xấp xỉ $\sigma^2/7{,}8$ theo mục 6.1.



Hình 33. (a) Độ chính xác kiểm tra của rừng ngẫu nhiên trên chữ số theo số cây (trung bình và độ lệch chuẩn của 5 lần chạy). (b) Trọng số hiệu dụng $s_i(5)$ của các mẫu huấn luyện khi dự đoán tại $x_0 = 5$ trên dữ liệu sin: k-NN, Nadaraya–Watson và rừng ngẫu nhiên đều là trung bình có trọng số, khác nhau ở cách đặt trọng số.

Cách nhìn này cho thấy rừng ngẫu nhiên và k-NN cùng họ, và khác ở chỗ **khái niệm "gần" của rừng được học từ nhãn**: hai điểm "gần nhau" nếu chúng thường rơi vào cùng một lá. Điều đó giải thích một ưu điểm lớn: rừng bỏ qua đặc trưng nhiễu vì các cây hiếm khi chia theo chúng. Thí nghiệm trên 600 sinh viên cộng q đặc trưng nhiễu cho thấy rừng (300 cây, lá tối thiểu 5 mẫu) giữ được 81,7% ($q = 0$), 82,2% ($q = 20$), 80,0% ($q = 50$) và 78,3% ($q = 100$), trong khi k-NN rơi xuống 82,2%, 70,6%, 54,4%, 54,4% (mục 5.7) và Softmax xuống còn 65,0% ở $q = 100$. Rừng giải quyết vấn đề mà mục 6.3 nêu ra cho phương pháp dựa trên khoảng cách. Cái giá là chi phí: rừng 300 cây trên chữ số có 94 266 nút, huấn luyện 1,0 giây và dự đoán 45 ms (mục 8.2), nhiều hơn k-NN, và nó mất đi tính dễ giải thích của một cây.

Mô hình hỗn hợp Gauss

Ý tưởng: giữa một Gauss và n Gauss. Ở mục 4.6, ta thấy một phân phối Gauss đơn không thể mô tả dữ liệu hai đỉnh (đặt đỉnh vào thung lũng), còn KDE thì đặt một gò trên *mỗi* mẫu và phải giữ cả dữ liệu. **Mô hình hỗn hợp Gauss** (GMM) là điểm giữa: tổ hợp J thành phần Gauss với J cố định, $J \ll n$:

$$p(\mathbf{x}) = \sum_{j=1}^{J} \pi_j \, \mathcal{N}\big(\mathbf{x};\, \boldsymbol{\mu}_j,\, \boldsymbol{\Sigma}_j\big), \qquad \pi_j \geq 0, \;\; \sum_{j=1}^{J} \pi_j = 1.$$

Mỗi thành phần j là một "cụm" có tâm $\boldsymbol{\mu}_j$, hình dạng $\boldsymbol{\Sigma}_j$ và trọng số π_j. Các trường hợp đặc biệt cho ta bản đồ: $J = 1$ là Gauss hợp lý cực đại (mục 4.1); và KDE Gauss chính là một GMM với $J = n$ thành phần có tâm *đặt sẵn tại các mẫu*, trọng số bằng nhau $\pi_j = 1/n$ và hiệp phương sai chung $h^2\mathbf{I}$. GMM là mô hình **tham số** (số tham số $J - 1 + Jd + J\frac{d(d+1)}{2}$ cố định, không tăng theo n) nhưng *rất linh hoạt* khi J đủ lớn, vì vậy hay được coi là bán tham số: nó thừa hưởng khả năng bắt nhiều đỉnh của KDE mà chỉ lưu vài con số.

Thuật toán EM. Khác với Gauss đơn, hợp lý cực đại của GMM không có công thức đóng, vì ta không biết mẫu nào thuộc thành phần nào. Thuật toán **kỳ vọng–cực đại** (EM) lặp hai bước. **Bước E**: với tham số hiện tại, tính **trách nhiệm** γ_{ij}, xác suất mẫu i được sinh bởi thành phần j (hậu nghiệm theo Bayes, chính là công thức của mục 5.1 với "lớp" là thành phần):

$$\gamma_{ij} = \frac{\pi_j \, \mathcal{N}\big(\mathbf{x}_i;\, \boldsymbol{\mu}_j,\, \boldsymbol{\Sigma}_j\big)}{\sum_{l=1}^{J} \pi_l \, \mathcal{N}\big(\mathbf{x}_i;\, \boldsymbol{\mu}_l,\, \boldsymbol{\Sigma}_l\big)}, \qquad \boldsymbol{\Gamma} = \big[\gamma_{ij}\big] \in \mathbb{R}^{n \times J}.$$

Bước M: cập nhật tham số như trong mục 5.4, nhưng thay ma trận one-hot $\mathbf{Y}$ bằng **ma trận trách nhiệm** $\boldsymbol{\Gamma}$ (một "one-hot mềm"). Với $\mathbf{t} = \boldsymbol{\Gamma}^{\top}\mathbf{1}_n$ là số mẫu "mềm" của từng thành phần:

$$\widehat{\boldsymbol{\pi}} = \frac{1}{n}\,\mathbf{t}, \qquad \widehat{\mathbf{M}} = \operatorname{diag}(\mathbf{t})^{-1}\,\boldsymbol{\Gamma}^{\top}\mathbf{X}, \qquad \widehat{\boldsymbol{\Sigma}}_j = \frac{1}{t_j}\sum_{i=1}^{n} \gamma_{ij}\,(\mathbf{x}_i - \widehat{\boldsymbol{\mu}}_j)^{\top}\big(\mathbf{x}_i - \widehat{\boldsymbol{\mu}}_j\big),$$

với hàng j của $\widehat{\mathbf{M}}$ là $\widehat{\boldsymbol{\mu}}_j$. Đây *chính xác* là công thức Bayes ngây thơ ở mục 5.4 (trung bình theo lớp $= \operatorname{diag}(\mathbf{t})^{-1}\mathbf{Y}^{\top}\mathbf{X}$) với $\mathbf{Y}$ thay bằng $\boldsymbol{\Gamma}$; nếu nhãn lớp được biết thì bước E là không cần thiết. Mỗi vòng lặp không làm giảm log-hợp lý, nhưng EM chỉ đảm bảo hội tụ đến một **cực đại địa phương**, nên thực hành chạy nhiều lần với khởi tạo khác nhau và giữ kết quả tốt nhất.

Ví dụ tay một bước. Có ba mẫu một chiều $x = (0{,}1{,}5)$, $J = 2$, khởi tạo $\pi_1 = \pi_2 = 0{,}5$, $\mu_1 = 0$, $\mu_2 = 5$, $\sigma_1^2 = \sigma_2^2 = 1$. Bước E cho

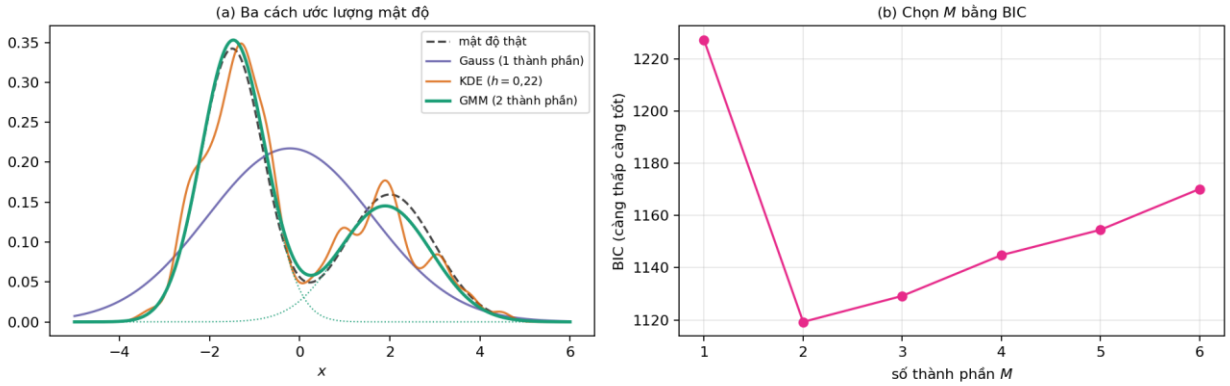
$$\boldsymbol{\Gamma} \approx \begin{pmatrix} 1{,}000 & 0{,}000 \\ 0{,}999 & 0{,}001 \\ 0{,}000 & 1{,}000 \end{pmatrix}, \qquad \mathbf{t} = \boldsymbol{\Gamma}^{\top}\mathbf{1} \approx \begin{pmatrix} 1{,}999 \\ 1{,}001 \end{pmatrix}.$$

Bước M cho $\widehat{\boldsymbol{\pi}} = \mathbf{t}/3 \approx (0{,}667;\, 0{,}333)$ và $\hat{\mu}_1 \approx (0 \times 1 + 1 \times 0{,}999)/1{,}999 \approx 0{,}500$, $\hat{\mu}_2 \approx 4{,}998 \approx 5$: hai thành phần đã tìm đúng hai nhóm $\{0{,}1\}$ và $\{5\}$.

Chọn số thành phần J. J là nút chỉnh độ phức tạp (như k, h ở các mục trước). Vì GMM là mô hình tham số có hợp lý, ta dùng **tiêu chuẩn thông tin Bayes** (BIC), phạt số tham số p:

$$\mathrm{BIC}(J) = -2\ln\hat{L}_J + p_J \ln n, \qquad \hat{J} = \operatorname*{argmin}_J \mathrm{BIC}(J),$$

trong đó $\hat{L}_J$ là hợp lý cực đại của mô hình có J thành phần. Số hạng đầu thưởng độ khớp, số hạng sau phạt độ phức tạp (cùng tinh thần với điều chuẩn ở các mục trước). Với dữ liệu hai đỉnh 300 mẫu của mục 4 (một chiều, $p_J = 3J - 1$), BIC cho 1 227,3 ($J = 1$), $\mathbf{1\,119,3}$ ($J = 2$), 1 129,2 ($J = 3$), 1 144,8 ($J = 4$): chọn đúng $J = 2$ (Hình 34b). Ước lượng EM cho $\hat{\boldsymbol{\pi}} = (0{,}377;\ 0{,}623)$, $\hat{\boldsymbol{\mu}} = (1{,}874;\ -1{,}485)$, $\hat{\boldsymbol{\sigma}} = (1{,}043;\ 0{,}703)$, rất sát giá trị thật $\pi = (0{,}4;\ 0{,}6)$, $\mu = (2;\ -1{,}5)$, $\sigma = (1;\ 0{,}7)$. Bản EM cài đặt từ đầu (hơn 20 dòng NumPy) cần 28 vòng lặp và cho log-hợp lý trên tập kiểm tra $-1{,}8464$, trùng với `GaussianMixture` của scikit-learn.



Hình 34. (a) Ba cách ước lượng mật độ từ 300 mẫu hai đỉnh: Gauss đơn (1 thành phần, đỉnh nằm ở thung lũng), KDE ($h = 0{,}22$, phải lưu 300 mẫu), và hỗn hợp Gauss 2 thành phần (nét đứt chấm là hai thành phần; chỉ 5 tham số). (b) BIC theo số thành phần J, nhỏ nhất ở $J = 2$.

Mô hình mật độ	Số đại lượng lưu	Log-hợp lý kiểm tra (nats/mẫu)
Mật độ thật		$-1{,}839$
GMM, $J = 2$	5 tham số	$-1{,}846$
KDE, $h = 0{,}22$	300 mẫu	$-1{,}869$
Gauss đơn ($J = 1$)	2 tham số	$-2{,}099$

GMM với $J = 2$ thắng cả KDE ($-1{,}846$ so với $-1{,}869$) bằng 5 tham số thay vì 300 mẫu. Điều này không có gì lạ: dữ liệu *được sinh ra* từ hỗn hợp hai Gauss nên GMM chọn đúng họ hàm; đây là minh họa đúng cho bài học của mục 6.4: **khi dạng hàm đúng, mô hình tham số hiệu quả hơn mô hình phi tham số**. Nếu dữ liệu không phải hỗn hợp Gauss, KDE sẽ an toàn hơn.

GMM làm bộ phân loại. Dùng GMM cho mật độ theo lớp $p(\mathbf{x} \mid c)$ rồi áp dụng định lý Bayes (mục 5.1) ta được bộ phân loại tổng quát hóa cả Gauss đầy đủ theo lớp ($J = 1$) và Bayes bằng KDE ($J = n_c$). Trên chữ số (chiếu xuống 20 thành phần PCA, siêu tham số chọn bằng kiểm định chéo), hiệp phương sai đầy đủ với $J = 1$ cho kiểm định chéo 97,9% và độ chính xác kiểm tra 98,0%; tăng J lại làm *giảm* (95,5% với $J = 2$ và 92,5% với $J = 3$) vì với khoảng 125 ảnh mỗi lớp, mỗi thành phần có hiệp phương sai đầy đủ 20×20 tốn 230 tham số và gây quá khớp; còn

với hiệp phương sai *đường chéo* thì tăng J giúp (93,8% → 96,4% khi J từ 1 đến 3). So với Bayes bằng KDE trên cùng không gian PCA (98,1%), GMM với $J = 1$ ngang bằng nhưng chỉ lưu 230 số mỗi lớp thay vì dữ liệu. Đây là thêm một ví dụ của đánh đổi thiên lệch–phương sai: số thành phần và dạng hiệp phương sai phải khớp với lượng dữ liệu có.

Bản đồ các phương pháp: từ ít đến nhiều giả định

Để kết thúc phần mở rộng, ta xếp các mô hình mật độ và phân loại của bài thành một thang liên tục theo mức giả định (từ nhiều giả định, ít tham số, đến ít giả định, nhiều tham số):

$$\underset{J=1}{\text{Gauss đơn}} \;\rightarrow\; \underset{J \text{ vừa}}{\text{GMM}} \;\rightarrow\; \underset{J=n,\,\boldsymbol{\Sigma}_j=h^2\mathbf{I}}{\text{KDE}} \qquad\qquad \underset{\text{biên tuyến tính}}{\text{Softmax}} \;\rightarrow\; \underset{\text{biên tùy ý}}{\text{Rừng / SVM / k-NN.}}$$

Càng sang phải, thiên lệch càng thấp, phương sai càng cao, và đòi hỏi càng nhiều dữ liệu; nút chỉnh (J, h, k, độ sâu) quyết định ta đứng ở đâu trên thang.

8.4. Bài tập thực hành

Các bài tập chia theo mức: **(CB)** cơ bản, **(TB)** trung bình, **(NC)** nâng cao. Bài tính tay có đáp án ở cuối mục; bài lập trình dùng bộ dữ liệu có sẵn của scikit-learn (`load_digits`, `load_iris`, `load_wine`, `load_breast_cancer`) và nên làm theo quy trình ở mục 7.6.

A. Tính tay

Bài 1 (CB). k-NN. Có sáu mẫu huấn luyện hai chiều: lớp 0 gồm $(1,1)$, $(2,3)$, $(3,1)$; lớp 1 gồm $(6,5)$, $(7,3)$, $(5,6)$. Dự đoán lớp của $\mathbf{x}_0 = (4,3)$ bằng khoảng cách Euclid trong ba trường hợp: bỏ phiếu đa số với $k = 3$, với $k = 5$, và bỏ phiếu theo trọng số $w_i = 1/d_i^2$ với $k = 3$. Nhận xét về sự phụ thuộc vào k.

Bài 2 (CB). Nadaraya–Watson và LOESS. Ba mẫu một chiều $(x_i, y_i) = (0,1), (1,3), (2,2)$. Với nhân Gauss và $h = 1$, tính dự đoán tại $x_0 = 0{,}5$ bằng Nadaraya–Watson và bằng LOESS (đường thẳng cục bộ). Giải thích vì sao hai kết quả khác nhau.

Bài 3 (CB). KDE. Mẫu một chiều $\{1,2,4\}$, nhân Gauss chuẩn hóa, $h = 0{,}5$. Tính $\hat{f}(2)$ và $\hat{f}(3)$. Vì sao $\hat{f}(3)$ nhỏ dù 3 nằm giữa 2 và 4?

Bài 4 (TB). Bayes ngây thơ Bernoulli. Ba từ nhị phân (thưởng, miễn_phí, họp). Thư rác: $[1,1,0]$, $[1,0,0]$, $[1,1,1]$. Thư bình thường: $[0,1,1]$, $[0,0,1]$. Dùng làm trơn Laplace $\alpha = 1$ (mẫu số $n_c + 2\alpha$), tính xác suất thư mới $[1,0,1]$ là rác.

Bài 5 (TB). Cây quyết định. Với 9 sinh viên của các mục trước, kiểm tra rằng cách chia $x_1 \le 5$ cho độ tạp Gini có trọng số $0{,}333$, và cách chia $x_2 \le 2{,}5$ cho $0{,}433$.

Bài 6 (TB). Một bước EM. Ba mẫu $x = (0,1,5)$, $J = 2$, khởi tạo $\pi = (0{,}5;\ 0{,}5)$, $\mu = (0,5)$, $\sigma^2 = (1,1)$. Tính ma trận trách nhiệm $\boldsymbol{\Gamma}$ và $\hat{\boldsymbol{\pi}}$, $\hat{\boldsymbol{\mu}}$ mới.

B. Chứng minh và lý thuyết

Bài 7 (TB). Chứng minh rằng với bộ làm trơn $\hat{f}(x) = \sum_i s_i(x) y_i$ có $\sum_i s_i = 1$ và nhiễu độc lập có phương sai σ^2, phương sai của $\hat{f}(x)$ bằng $\sigma^2 \sum_i s_i^2$, và rằng $\sum_i s_i^2 \geq 1/n$ với dấu bằng khi mọi trọng số bằng nhau. Từ đó suy ra vì sao, với cùng tổng trọng số bằng 1, k-NN có phương sai nhỏ nhất trong số các bộ làm trơn dùng k mẫu.

Bài 8 (TB). Chứng minh rằng nếu một đặc trưng được sao chép thành m bản giống hệt, log-odds của Bayes ngây thơ nhận đóng góp của đặc trưng đó nhân lên m lần, còn của Logistic Regression thì có thể giữ nguyên. Giải thích mối liên hệ với hiện tượng quá tự tin ở mục 5.6.

Bài 9 (NC). Chứng minh KDE Gauss với băng thông h là GMM với $J = n$ thành phần ($\pi_j = 1/n$, $\boldsymbol{\mu}_j = \mathbf{x}_j$, $\boldsymbol{\Sigma}_j = h^2\mathbf{I}$). Từ đó chỉ ra điều gì xảy ra với hợp lý huấn luyện khi cho EM chạy và để $\boldsymbol{\Sigma}_j$ tự do với $J = n$, và vì sao cần bỏ-một-ra (mục 4.4).

C. Lập trình và thực nghiệm

Bài 10 (CB). Cài đặt từ đầu k-NN hồi quy với chọn k bằng bỏ-một-ra, rồi so với `KNeighborsRegressor`. Vẽ đường dự đoán và dải bootstrap (mục 7.1).

Bài 11 (TB). Trên `load_wine` hoặc `load_breast_cancer`, lặp lại thí nghiệm đặc trưng nhiễu (mục 2.5, 5.7, 8.3): thêm $q \in \{0,5,20,50,100\}$ đặc trưng nhiễu chuẩn tắc và so sánh k-NN (có chuẩn hóa), Bayes ngây thơ Gauss, Softmax và rừng ngẫu nhiên. Mô hình nào bền nhất? Giải thích bằng KL ở mục 6.3.

Bài 12 (TB). Trên `load_digits`, tái hiện Hình 29 (đường kiểm định chéo theo k, α, `var_smoothing`, C), rồi chọn k theo hai tiêu chí (độ chính xác, log-loss đã làm trơn) và so sánh hai lựa chọn.

Bài 13 (TB). Cài đặt KDE một chiều và hai cách chọn băng thông: quy tắc Silverman và hợp lý cực đại có kiểm định chéo. Sinh dữ liệu từ hỗn hợp ba Gauss do bạn thiết kế, so ISE và log-hợp lý kiểm tra, và kiểm tra thực nghiệm rằng h tối ưu giảm như $n^{-1/5}$ khi tăng n.

Bài 14 (NC). Cài đặt EM cho GMM nhiều chiều ở dạng ma trận (dùng $\boldsymbol{\Gamma}$ như trên), so với `GaussianMixture`, chọn J bằng BIC, và dùng nó làm bộ phân loại Bayes trên `load_digits` sau PCA. Khảo sát ảnh hưởng của số thành phần PCA và dạng hiệp phương sai.

Bài 15 (NC). Dự án tổng hợp. Trên `load_digits` (hoặc một bộ dữ liệu khác), xây dựng quy trình hoàn chỉnh theo mục 7.6 với ít nhất năm mô hình thuộc ít nhất ba họ: chọn siêu tham số bằng kiểm định chéo (có `Pipeline`), kiểm định chéo lồng nhau cho ít nhất một mô hình, báo cáo độ chính xác, macro-F1, log-loss, độ tin cậy trung bình, ma trận nhầm lẫn và đường cong học, kèm một đoạn phân tích: mô hình nào thắng, chênh lệch có vượt sai số chuẩn không, và vì sao.

D. Thảo luận

Bài 16 (TB). Với mỗi tình huống, chọn họ mô hình và giải thích bằng các bảng ở mục 6.5, 8.1 và 8.2: (a) lọc thư rác trên 10^6 thư, hàng chục nghìn từ; (b) chẩn đoán dựa trên 200 bệnh nhân với 30 xét nghiệm, cần giải thích cho bác sĩ; (c) hệ thống gợi ý trực tuyến, dữ liệu mới đến liên tục, mỗi yêu cầu phải trả lời trong vài mili-giây.

Đáp án cho các bài tính tay

Bài 1. Khoảng cách từ $(4,3)$ tới các mẫu theo thứ tự đã cho là $\sqrt{13} \approx 3{,}606$, 2, $\sqrt{5} \approx 2{,}236$, $\sqrt{8} \approx 2{,}828$, 3, $\sqrt{10} \approx 3{,}162$. Xếp tăng dần: $(2,3)$ [lớp 0, 2,000], $(3,1)$ [lớp 0, 2,236], $(6,5)$ [lớp 1, 2,828], $(7,3)$ [lớp 1, 3,000], $(5,6)$ [lớp 1, 3,162], $(1,1)$ [lớp 0, 3,606]. Với $k = 3$: hai phiếu lớp 0, một phiếu lớp 1, dự đoán **lớp** 0 ($\hat{p}_0 = 2/3$). Với $k = 5$: hai phiếu lớp 0, ba phiếu lớp 1, dự đoán **lớp** 1 ($\hat{p}_1 = 3/5$). Có trọng số $w = 1/d^2 = 0{,}25;\ 0{,}2;\ 0{,}125$ cho ba láng giềng gần nhất ($k = 3$): $\hat{p}_0 = \frac{0{,}25+0{,}2}{0{,}25+0{,}2+0{,}125} = \frac{0{,}45}{0{,}575} \approx 0{,}783$, dự đoán **lớp** 0. Điểm $\mathbf{x}_0$ nằm gần biên giữa hai lớp nên kết quả đổi theo k (từ 3 sang 5); trọng số theo khoảng cách làm kết quả ổn định hơn vì ưu tiên các láng giềng gần.

Bài 2. Các khoảng cách là $0{,}5;\ 0{,}5;\ 1{,}5$, trọng số nhân $e^{-d^2/2} = 0{,}8825;\ 0{,}8825;\ 0{,}3247$ (tổng 2,0896). Nadaraya–Watson: $\hat{y}_0 = \frac{0{,}8825\times1+0{,}8825\times3+0{,}3247\times2}{2{,}0896} = \frac{4{,}1793}{2{,}0896} = 2{,}000$. LOESS: $\tilde{\mathbf{X}}^\top\mathbf{W}\tilde{\mathbf{X}} = \begin{pmatrix}2{,}0896 & 1{,}5318\\ 1{,}5318 & 2{,}1811\end{pmatrix}$, $\tilde{\mathbf{X}}^\top\mathbf{W}\mathbf{y} = \begin{pmatrix}4{,}1793\\ 3{,}9461\end{pmatrix}$, nghiệm $\mathbf{w} \approx (1{,}389;\ 0{,}834)$ và $\hat{y}_0 = 1{,}389 + 0{,}834 \times 0{,}5 \approx 1{,}806$. Hai kết quả khác nhau vì Nadaraya–Watson chỉ lấy trung bình có trọng số (coi hàm gần hằng), còn LOESS khớp một đường thẳng và nhận ra xu hướng tăng từ $y = 1$ ở $x = 0$ lên $y = 3$ ở $x = 1$, nên giá trị tại $x = 0{,}5$ (gần $x = 0$ hơn) thấp hơn trung bình có trọng số.

Bài 3. $\hat{f}(x) = \frac{1}{3\times0{,}5}\sum_i \kappa\,(x - x_i/0{,}5)$ với $\kappa(u) = \frac{1}{\sqrt{2\pi}}e^{-u^2/2}$. Tại $x = 2$: các $u = 2;\ 0;\ -4$, nên $\kappa = 0{,}0540;\ 0{,}3989;\ \approx 0$, tổng $0{,}4529$ và $\hat{f}(2) = 0{,}4529/1{,}5 = 0{,}302$. Tại $x = 3$: các $u = 4;\ 2;\ -2$, nên $\kappa \approx 0;\ 0{,}0540;\ 0{,}0540$, tổng $0{,}1080$ và $\hat{f}(3) = 0{,}1080/1{,}5 = 0{,}0721$. $\hat{f}(3)$ nhỏ vì điểm 3 cách các mẫu gần nhất (2 và 4) một khoảng $1 = 2h$, và hàm Gauss giảm rất nhanh sau 2 lần băng thông; $h = 0{,}5$ nhỏ nên giữa hai mẫu có một "thung lũng".

Bài 4. Lớp bình thường ($n_c = 2$): số đếm từng từ $(0,1,2)$, $\hat{\theta} = (0 + 1/4\,,\ 1 + 1/4\,,\ 2 + 1/4) = (0{,}25;\ 0{,}5;\ 0{,}75)$, $\pi = 0{,}4$. Lớp rác ($n_c = 3$): số đếm $(3,2,1)$, $\hat{\theta} = (4/5\,,\ 3/5\,,\ 2/5) = (0{,}8;\ 0{,}6;\ 0{,}4)$, $\pi = 0{,}6$. Với $\mathbf{x}_0 = [1,0,1]$: $s_{\text{rác}} = \ln0{,}6 + \ln0{,}8 + \ln(1 - 0{,}6) + \ln0{,}4 = -0{,}511 - 0{,}223 - 0{,}916 - 0{,}916 = -2{,}567$; $s_{\text{bt}} = \ln0{,}4 + \ln0{,}25 + \ln(1 - 0{,}5) + \ln0{,}75 = -0{,}916 - 1{,}386 - 0{,}693 - 0{,}288 = -3{,}283$. Vậy $P(\text{rác}) = 1/(1 + e^{-3{,}283+2{,}567}) = 1/(1 + e^{-0{,}716}) \approx \mathbf{0{,}672}$.

Bài 5. Chia $x_1 \le 5$: trái gồm sinh viên 1,2,3 (cùng lớp rớt), $G = 0$; phải gồm 6 sinh viên (3 đạt, 3 giỏi), $G = 0{,}5$; độ tạp có trọng số $\frac{3}{9}\cdot 0 + \frac{6}{9}\cdot 0{,}5 = 0{,}333$. Chia $x_2 \le 2{,}5$: trái gồm sinh viên 5,7,8,9 (một đạt, ba giỏi), $G = 1 - \left(\frac{1}{4}\right)^2 - \left(\frac{3}{4}\right)^2 = 0{,}375$; phải gồm sinh viên 1,2,3,4,6 (ba rớt, hai đạt), $G = 1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2 = 0{,}48$; có trọng số $\frac{4}{9}\times 0{,}375 + \frac{5}{9}\times 0{,}48 = 0{,}1667 + 0{,}2667 = 0{,}433$. Cách chia $x_1 \le 5$ tốt hơn ($0{,}333 < 0{,}433$).

Bài 6. Mật độ Gauss của từng mẫu theo từng thành phần cho $\boldsymbol{\Gamma} \approx \begin{pmatrix}1{,}0000 & 0{,}0000\\ 0{,}9994 & 0{,}0006\\ 0{,}0000 & 1{,}0000\end{pmatrix}$ (mẫu $x = 1$ cách $\mu_1 = 0$ một khoảng 1 và cách $\mu_2 = 5$ một khoảng 4, nên tỉ số mật độ là $e^{-0{,}5}/e^{-8} \approx$

1 800). Số mẫu mềm $\mathbf{t} = (1{,}9994;\ 1{,}0006)$, nên $\hat{\boldsymbol{\pi}} = (0{,}6665;\ 0{,}3335)$ và $\hat{\boldsymbol{\mu}} = \left(\frac{0+0{,}9994}{1{,}9994}, \frac{0{,}0006+5}{1{,}0006}\right) \approx (0{,}4999;\ 4{,}9978)$.

Gợi ý cho bài 7. $\mathrm{Var}[\sum_i s_i\,\varepsilon_i] = \sum_i s_i^2\,\sigma^2$ vì các ε_i độc lập. Theo bất đẳng thức Cauchy–Schwarz, $1 = (\sum_i s_i)^2 \le n\sum_i s_i^2$, nên $\sum_i s_i^2 \ge 1/n$, dấu bằng khi các s_i bằng nhau. Với k mẫu có trọng số khác 0, bất đẳng thức cho $\sum_i s_i^2 \ge 1/k$, đạt được bởi trọng số đều $1/k$: đó là k-NN, nên k-NN thuần đạt phương sai nhỏ nhất trong số các bộ làm trơn chỉ dùng k mẫu (còn thiên lệch thì khác).

Gợi ý cho bài 8. Với đặc trưng j sao chép m lần, tổng $\sum_j \ln\frac{p(x_j|a)}{p(x_j|b)}$ của Bayes ngây thơ có số hạng đó xuất hiện m lần, nên đóng góp của nó vào log-odds nhân với m (bằng chứng được đếm lặp). Logistic Regression học trọng số *chung* cho m cột giống nhau: tổng $w_1 + \cdots + w_m$ là thứ quyết định nên mô hình có thể (với điều chuẩn L2, tổng đó chia đều) giữ nguyên ảnh hưởng của đặc trưng, vì thế log-loss gần như không đổi ở thí nghiệm của mục 5.6.

Mối liên hệ với các mục trước

Mục 8 không thêm kiến thức nền mới mà **đặt toàn bài vào một bức tranh**. Bảng trọng số hiệu dụng ở đầu mục 8.1 gom k-NN (mục 2, 3.1), Nadaraya–Watson (mục 3.2), LOESS (mục 3.3) và hai phương pháp mở rộng (cây, rừng) thành cùng một dạng trung bình có trọng số từ mục 1.3. Các ưu và nhược điểm ở mục 8.1 đều trỏ về một thí nghiệm cụ thể trong bài (mục 1.1, 2.5, 3.4, 3.5, 5.6, 6.2 đến 6.5, 7.4, 7.5). Mục 8.2 đặt các họ mô hình cạnh nhau nhờ cùng một ngôn ngữ thiên lệch–phương sai của mục 6.1; mục 8.3 cho thấy rừng giảm phương sai theo đúng công thức $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, và GMM nối ước lượng Gauss đơn (mục 4.1, 4.6) với KDE (mục 4.3) qua bước M dùng lại đúng công thức của mục 5.4. Còn bài tập ở mục 8.4 bám sát các công cụ của mục 2 đến 7.

Tóm tắt mục 8 và kết luận của bài

- Mọi phương pháp của bài đều là **trung bình có trọng số** $\hat{\mathbf{p}}(\mathbf{x}_0) = \sum_i s_i\,(\mathbf{x}_0)\mathbf{y}_i$; sai số đọc được từ các trọng số: thiên lệch $= \sum_i s_i\,f(\mathbf{x}_i) - f(\mathbf{x}_0)$, phương sai $= \sigma^2 \parallel \mathbf{s} \parallel^2 = \sigma^2/n_{\text{hiệu dụng}}$.
- **Ưu điểm** của mô hình phi tham số: không chọn trước dạng hàm, huấn luyện gần như bằng không, nhất quán với mọi phân phối, ảnh hưởng của dữ liệu xấu mang tính cục bộ, thêm dữ liệu miễn phí. **Hạn chế**: lời nguyền chiều, chi phí dự đoán và bộ nhớ $O(nd)$, nhạy với đặc trưng nhiễu và thang đo, không ngoại suy, thiên lệch ở biên, xác suất thô hoặc quá tự tin.
- Bốn họ (tham số, mạng nơ-ron, nhân, phi tham số) giải được cùng một bài toán nếu đủ linh hoạt; khác nhau ở *điều phải biết trước*, số đại lượng lưu, chi phí và diễn giải. Không có họ nào thắng ở mọi nơi: Softmax 37,8% trên ba vòng tròn nhưng 84,4% nhỉnh hơn k-NN trên đám Gauss; trên chữ số, SVM nhân 99,3%, k-NN 98,3%, Softmax 97,6%, Bayes ngây thơ 90,7%.

- **Cây quyết định** chia không gian bằng cách tối thiểu độ tạp $G = 1 - \sum_c \hat{p}_c^2$, là "láng giềng thích nghi" có lân cận do nhãn quyết định, nhưng phương sai cao. **Rừng ngẫu nhiên** lấy trung bình B cây ít tương quan: phương sai $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$; dự đoán của rừng *chính xác bằng* một trung bình có trọng số; bền với đặc trưng nhiễu (78,3% ở $q = 100$, so với k-NN 54,4%).
- **Mô hình hỗn hợp Gauss** $p(\mathbf{x}) = \sum_j \pi_j \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$ nằm giữa Gauss đơn ($J = 1$) và KDE ($J = n$); học bằng EM, trong đó bước M là công thức mục 5.4 với $\mathbf{Y}$ thay bằng ma trận trách nhiệm $\boldsymbol{\Gamma}$; chọn J bằng BIC (chọn đúng $J = 2$ trên dữ liệu hai đỉnh và đạt $-1{,}846$ nats so với KDE $-1{,}869$ với 5 tham số).

Kết luận của bài. Từ Perceptron đến Softmax Regression, ta quen với cách làm *chọn một dạng hàm rồi tìm tham số*. Bài này cho thấy một cách làm khác, *để dữ liệu tự nói*, và đưa ra một bức tranh thống nhất: mọi mô hình đều là một cách làm trung bình có trọng số các quan sát, với một nút chỉnh điều khiển độ phức tạp. Chọn mô hình không phải là chọn "tốt nhất" mà là chọn **đúng mức giả định cho lượng dữ liệu và cấu trúc bài toán đang có**, rồi *đo công bằng* để biết mình chọn đúng hay không.